

软件工程知识体系指南（2004 版）

蒋遂平 翻译

蒋遂平，计算机应用专业博士，国家系统分析员，CSAI 专业顾问。曾从事过数据库、虚拟现实和人脸识别等方面的研究工作，先后参与和主持了多个系统的软件开发，主要感兴趣的领域包括软件工程，图象处理和数据库。

Guide to the Software Engineering Body of Knowledge 2004 Version

软件工程知识体系指南是IEEE计算机学会（IEEE Computer Society）职业实践委员会（Professional Practices Committee）主持的一个项目。®SWEBOK是IEEE的官方服务标记。
<http://www.swebok.org>

目录

- 第1章 引言
- 第2章 软件需求
- 第3章 软件设计
- 第4章 软件构造
- 第5章 软件测试
- 第6章 软件维护
- 第7章 软件配置管理
- 第8章 软件工程管理
- 第9章 软件工程过程
- 第10章 软件工程工具与方法
- 第11章 软件质量
- 第12章 相关学科知识域
- 附录A 2004年版软件工程知识体系指南的知识域描述规范
- 附录B 指南演化过程
- 附录C IEEE和ISO软件工程标准到SWEBOK知识域的分配
- 附录D 根据Bloom分类学的主题分类

////////////////////////////////////

第一章 指南简介

尽管全世界有数百万软件开发人员，软件在我们的社会中无处不在，软件工程在最近才达到了合理的工程学科和被认可的职业的状态。

一个职业在核心知识体系上达成一致，是所有学科的关键里程碑，IEEE计算机学会认为这是软件工程向职业状态演化的关键。本指南是在职业实践委员会的主持赞助下编写成的，它是一个被设计为达到这个一致的跨越数年的项目的一部分。

什么是“软件工程”？

IEEE计算机学会将“软件工程”定义为：“（1）应用系统化的、学科化的、量的方法，来开发、运行和维护软件，即，将工程应用到软件。（2）对（1）中各种方法的研究”。

（参见：IEEE Standard Glossary of Software Engineering Terminology。IEEE, Piscataway, NJ std 610.12-1990, 1990）

什么是被认可的职业？

软件工程要成为合理的工程学科和一个被认可的职业，在一个核心知识体系上达成一致就非常重要。Starr在定义什么将被认为是一个合理的学科和一个被认可的职业时，清楚地展示了这点。他在获得普利策奖的关于美国医学职业历史的书中，写道：

“专业人员威信的合法化涉及3个不同的需要：首先，专业人员的知识和能力能被其同行所确认；第二，这些被一致确认的知识依靠理性的、科学的基础，第三，专业人员的判断和建议要面向真实的价值，例如健康。这些合法性的各个方面对应于体现在术语“职业”上的各类属性：学院的、认知的和道德的。（参见：P. Starr, The Social Transformation of American Medicine: Basic Books, 1982. p15）。

什么是一个职业的特征？

Gary Ford和Norman Gibbs研究了几个被认可的职业，包括医学、法律、工程和会计等（参见：G Ford and N E Gibbs, “A Mature Profession of Software Engineering,” Software Engineering Institute, Carnegie Mellon University, Pittsburgh, Pennsylvania, Technical CMU/SEI-96-TR-004, January 1996）。他们的结论是，一个工程职业由下列几个特征刻画：（1）由团体通过认证而确认的课程表的初始职业教育；（2）通过自愿认证或强制许可的适应实践的注册；（3）专门的技术培养和继续职业教育；（4）有职业团体的公共支持；（5）承诺遵从以伦理准则形式形成的规范。

本指南包括了这些成分的前面3个。清晰地指出知识体系是发展一个职业关键的一步，因为它代表了对于软件工程专业人员应该知道什么的一个广泛的一致意见。没有这样的一致，就不能确认任何职业许可的考试，就不能为专业人员参与考试准备课程表，也就不能形成一个认证一个课程表的准则。达成一致也是一个组织中采纳发展连贯技能和继续职业教育程序的前提。

什么是SWEBOK项目的目标？

不应当将指南与知识体系本身混淆，知识体系已经存在与发表的文献中，指南的目的是描述知识体系的哪些部分已经被普遍接受，将这些部分组织起来，提供一个使用它们的主题。对于“普遍接受”包含的附加信息可以在下面或附录A中找到。

建立软件工程知识体系（SWEBOK）指南有下面5个目的：（1）促进世界范围内对软件工程的一致观点；（2）阐明软件工程相对其它学科（如计算机科学、项目管理、计算机工程

和数学等)的位置,并确立它们的分界;(3)刻画软件工程学科的内容;(4)提供使用知识体系的主题;(5)为开发课程表和个人认证与许可材料,提供一个基础。

第一个目标,促进世界范围内对软件工程的一致观点,由指南的开发过程支持,来自42个国家的大约500名评审人员参与了石人阶段(1998年—2001年),产生了试用版本,来自21个国家120多位评审人员参与了铁人阶段(2003年),产生了2004年版本。关于指南开发过程的更多信息可以在前言看到,或者在www.swebok.org 网站上看到。我们与涉及软件工程的职业和学术团体、公共机构等进行了接触,向它们告知了本项目,邀请它们参与评审过程。我们从北美、太平洋周边地区和欧洲,招募了指南的副编辑,在多个国际会议场合,做了本项目的演示报告,在以后,我们还安排了一些关于本项目的报告。

第二个目标,为软件工程确立边界,激发了本指南的基本组织。被认为是本学科的材料按照表1,组织为10个知识域(Knowledge Areas, KA),本指南对每个知识域分别用一章的篇幅来描述。

表1: SWEBOK知识域

软件需求	Software Requirements
软件设计	Software Design
软件构造	Software Construction
软件测试	Software Testing
软件维护	Software Maintenance
软件配置管理	Software Configuration Management
软件工程管理	Software Engineering Management
软件工程过程	Software Engineering Process
软件工程工具和方法	Software Engineering Tools and Methods
软件质量	Software Quality

在建立边界时,识别哪些学科与软件工程共享边界并有公共的交集,非常重要。最后,本指南识别出8个相关的学科,如表2所示(参见第12章,软件工程相关学科)。当然,软件工程具有来自这些学科的知识材料(知识域将参考它们),但是,SWEBOK指南的目标不是刻画相关学科的知识,而是刻画什么知识被认为对软件工程特别有用。

表2 相关学科

计算机工程	Computer Engineering
计算机科学	Computer Science
管理	Management
数学	Mathematics
项目管理	Project Management
质量管理	Quality Management
软件人类工程学	Software Ergonomics
系统工程	Systems Engineering

层次结构组织

知识域描述或章节Z的组织支持项目的第3个目标——刻画软件工程的内容。项目编辑组给各个副编辑提供的关于知识域描述的内容的详细规格说明,可以在附录A中找到。

本指南采用层次组织,将每个知识域分解为具有可识别标签的主题的集合。两级或三级

的结构分解，为找到感兴趣的主题提供了合理的方法。指南处理主题的方式与主流的思想学派的结构分解兼容，也与工业界和软件工程文献和标准的结构分解兼容。没有为主题的结构分解假设特定的应用领域、商业使用、管理哲学、开发方法等。每个主题的描述程度仅仅为主题的普遍接受特性需要的，并能让读者容易找到参考文献。总之，知识体系可以在参考材料中，而不是在指南中找到。

参考材料和矩阵

项目的第4个目标是提供按主题使用知识，本指南为每个知识域标明了参考材料，包括书记章节、论文和其它被认可的权威信息来源。每个知识域的描述也包括一个矩阵，将参考材料与主题联系起来。引用的文献的总量适合于大学本科毕业、具有4年工作经验的人员掌握。

在本版指南中，为所有知识域都分配了约500页的参考文献，这是对副编辑的规格说明。有人指出，一些知识域（如软件设计）需要更多的参考文献。本指南的以后版本将考虑这一点。

应该指出，指南并不打算在引用时包罗万象，没有包括很多适当的优秀材料，选择材料部分或主要是由于它覆盖了需要的主题。

处理的深度

从一开始，指南的深度问题就有了。项目组采纳了一个方法，以支持项目的第5个目标：为开发课程表、认证和许可提供一个基础。编辑组使用“普遍接受”的知识的准则，与高级研究知识（根据成熟性）和专门知识（基于应用的普遍性）进行区分，如图1所示。这个定义来自项目管理协会：“普遍接受的知识在大多数时间应用于大多数项目，广泛的一致意见确认其价值和效力”。（参见：*A Guide to the Project Management Body of Knowledge*, 2000 Edition, Project Management Institute, Newport Square, PA. www.pmi.org。）

但是，术语“普遍接受”并不意味着，指定的知识应该一律地应用于所有软件工程任务（每个项目需要确定需要的知识），但它意味着，有能力的软件工程师，为了胜任潜在的应用，应该具有这些知识。更确切地说，普遍接受的知识应该包括在软件工程师的许可考试的学习材料中，本这是本科毕业并工作4年后应该参加的考试。尽管这个准则是针对美国风格的教育，并不适合于其它国家，但我们认为它很有用。但是，普遍接受的知识两个定义应该被视为互补的。

专门的	普遍接受的
仅用于某些特定类型软件的实践	许多组织推荐的已经建立的传统实践
	高级的研究性的 被某些组织测试和使用的新的实践、研究组织正在发展和测试的概念

图 1 知识的分类

与本书格式有关的局限

本版本采用的书记格式有其局限。如果采用超文本结构，内容的本质将会更好，因为主题可以相互链接，而不是按顺序排列。

有时，知识域之间、子知识域之间等的边界多少有些武断。没有过多强调这些边界的重要性，我们尽可能地在相关或有用的地方，使用指示器和链接。

知识域之间的链接并不是输入/输出类型，知识域是对一个人应该拥有的与知识域有关的软件工程知识的观点。知识域内的学科结构分解，以及描述知识域的顺序安排，都没有吸收任何特定的方法和模型。指南中适当的知识域中描述了一些方法，指南本身并不是任何一种方法。

知识域

图2和图3从总体上描述了11章，以及各章中包括的主题。首先按传统的瀑布生命周期顺序描述前5个知识域，但是，这并不表示指南采纳或鼓励瀑布方法，或任何其它模型。随后按字母顺序描述各个知识域，最后一章描述相关的学科。这相当于它们出现在本指南中的顺序。

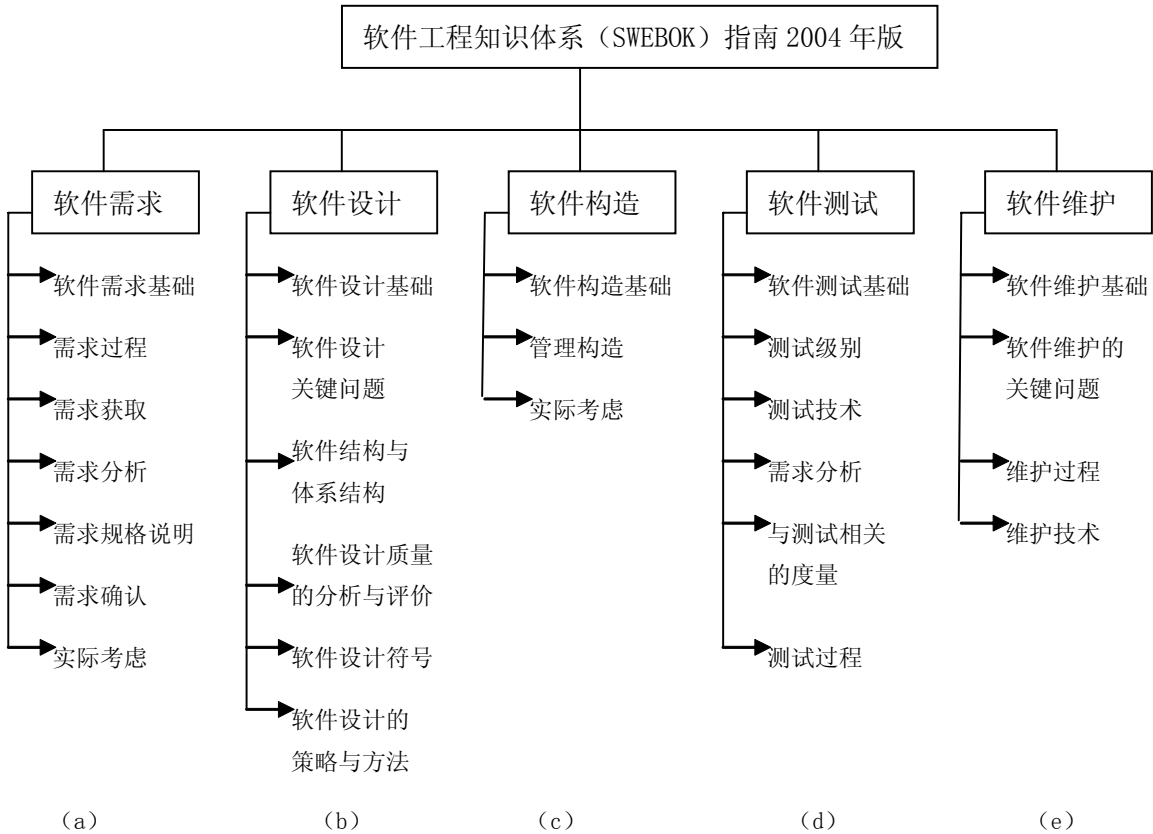


图 2 前 5 个知识域

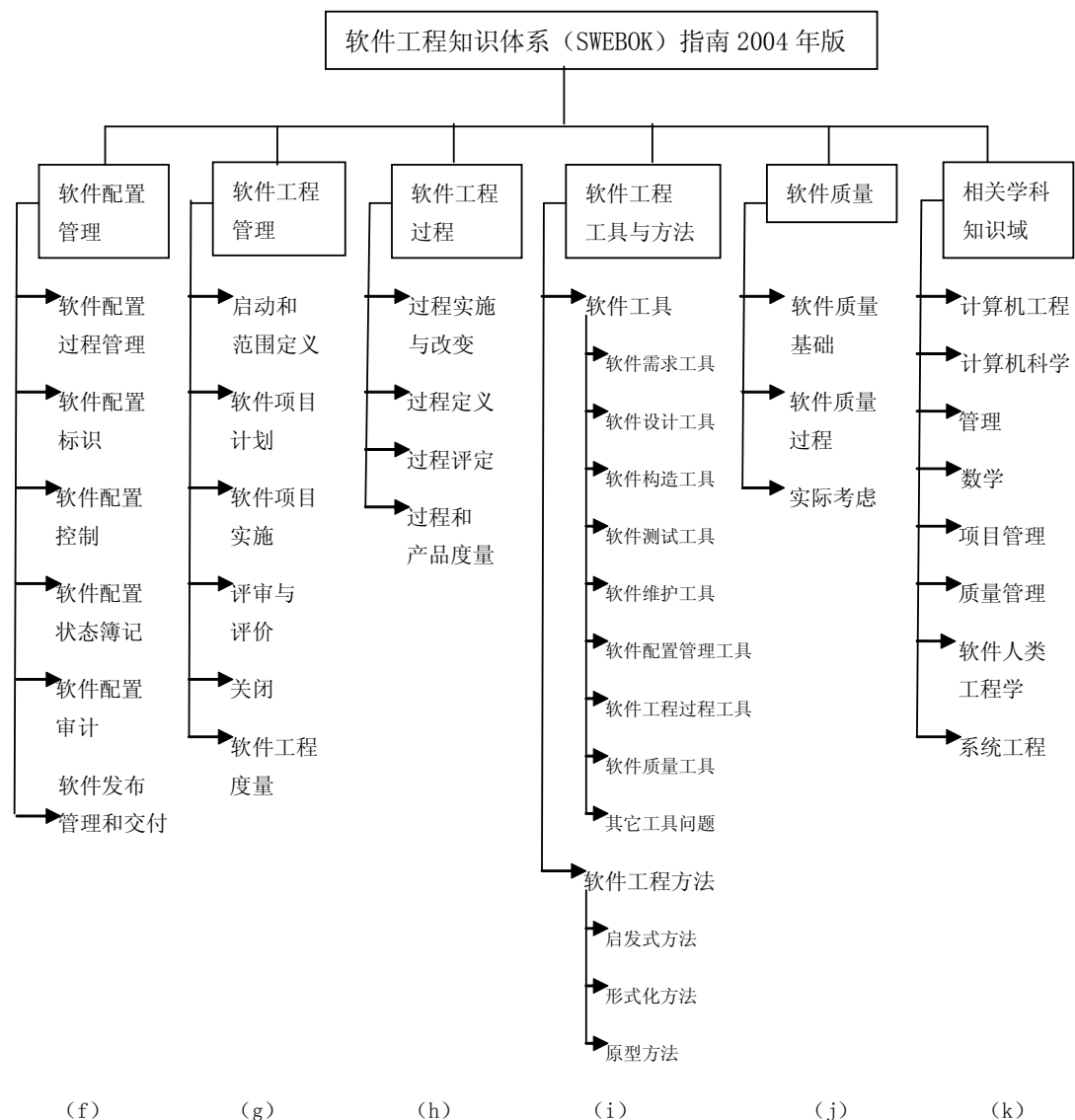


图 3 后 6 个知识域

知识域描述结构

知识域描述的结构如下所述：

在简介中，给出知识域的简要定义、其范围的总体视图、与其它知识域的关系。

主题的结构分解组成每个知识域描述的核心，它描述了将知识域分解为子域、主题和子主题。对于每个主题或子主题，给出简要描述，然后是一篇或多篇参考文献。

选择一个参考材料主要是因为认为它构成了与主题相关的知识的最佳表述，并考虑了对选择参考文献的限制（见前）。我们使用一个矩阵来联系主题和参考材料。

知识域描述的最后部分是推荐的参考文献列表。每个知识域的附录A为希望了解知识域主题更多内容的读者，列出了深入读物，附录B列出与知识域最相关的标准。注意，方括号[]中的引用表示推荐的参考文献，圆括号()中的引用表示对于编写或验证文本有用的参考文献，前者可以在对应的知识域章节中找到，后者可以在知识域的附录A中找到。

最后给出了知识域描述的简要总结和附录。

软件需求知识域(图2, 列a)

需求定义为解决真实世界问题而必须展示的特性。

第一个知识子域是软件需求基础,它包括软件需求本身的定义、主要的需求类型的定义,如产品与过程、功能与非功能、突发性(emergent)特性等。子域也描述了可量化需求的重要性,并区分了系统的和软件的需求。

第二个子域是需求过程,它介绍过程本身,面向余下的5个子域,说明需求工程如何与其它软件工程过程吻合。它描述了过程模型、过程参与者、过程支持与管理、过程质量与改进。

第三个子域是需求获取,它涉及软件需求来自何方?软件工程师如何收集这些需求,它包括需求来源和收集技术。

第四个子域是需求分析,涉及分析需求的过程:(1)检测 and 解决需求之间的冲突;(2)发现软件的边界,以及软件如何与外界交互;(3)详细描述系统需求和软件需求。需求分析包括需求分类、概念建模、体系结构设计 with 需求分配、需求协商。

第五个子域是需求规格说明,一般是指产生一份(电子)文档,这样可以系统地评审、评价和批准需求。对于复杂系统,特别是涉及大量非软件组件的系统,至少需要产生3类不同的文档:系统定义、系统需求规格说明、软件需求规格说明。子域描述了这3类文档,以及产生它们的活动。

第六个子域是需求确认,目标是在分配资源给需求之前,发现任何潜在的问题。需求确认涉及检查需求文档,以保证它们定义了正确的系统(即,这是用户期望的系统)。这个子域进一步划分为需求评审的引导、快速原型、模型确认和接收测试的描述。

第七个和最后一个子域是实际考虑,它描述在实践中需要理解的主题。第一个主题是需求过程的迭代本质,后面3个主题是关于处于实际反映了要建造或已经建造的软件的的状态的需求的变更管理和维护。它包括变更管理、需求属性、需求追踪。最后一个的主题是需求度量。

软件设计知识域(图2, 列b)

根据IEEE的定义[IEEE610.12--90],设计既是“定义一个系统或组件的体系结构、组件、接口和其它特征的过程”,又是“这个过程的结果”。软件设计的知识域分为6个子域。

第一个子域是软件设计基础,它是理解软件设计作用和范围的基础,这些是:一般的软件概念、软件设计上下文和软件设计的使能(enabling)技术。

第二个子域将软件设计的关键问题聚集在一起,它们包括:并发性、事件的控制和处理、组件的分布、错误和异常处理、容错、交互与表现、数据持久性。

第三个子域是软件结构与体系结构,它的主题是体系结构与视点、体系结构风格、设计模式、程序与构架族。

第四个子域描述软件设计质量的分析与评价。虽然有一个完整的软件质量知识域,这个子域描述与软件设计质量特别有关的主题。这些方面包括:质量属性、质量分析和评价技术与度量。

第五个子域是软件设计符号,它分为结构与行为描述两部分。

最后一个子域是软件设计策略与方法。首先描述一般策略,然后是面向功能的设计方法、面向对象的设计方法、以数据结构为中心的设计、基于组件的设计和其它方法。

软件构造(图2, 列c)

软件构造指通过编码、验证、单元测试、集成测试和排错的组合,详细创建一个可以工作的、有意义的软件,其知识域分为3个子域。

第一个子域是软件构造的基础，前3个主题是：复杂性最小化、变更预见和为验证进行构造。最后一个主题讨论软件构造的标准。

第二个子域描述构造的管理，主题包括：构造的模型、构造的计划、构造的度量。

第三个子域覆盖实践考虑，主题包括：构造的设计、构造的语言、编码、构造的测试、复用、构造的质量和集成。

软件测试（图2，列d）

软件测试由在有限测试用例集合上，根据期望的行为，对程序的行为进行的动态验证组成，测试用例是从实际上是无限的执行域中适当的选择出来的。软件测试包括5个子域。

第一个子域是软件测试基础，首先介绍与测试有关的术语，然后描述测试的关键问题，最后是测试与其它活动的联系。

第二个子域是测试级别，这些是根据测试对象（target）和测试目标来划分的。

第三个子域是测试技术。第一个范畴包括基于测试人员直觉和经验的测试，第二组是基于规格说明的技术组成，然后是基于代码的技术、基于错误（fault）的技术、基于使用的技术和与应用本质有关的技术。最后讨论如何选择和组合适当的技术。

第四个子域是测试相关的度量，度量划分为：与被测试的程序的评价有关的度量、与测试本身的评价有关的度量。

最后一个子域是测试过程，包括了测试时的实际考虑和测试活动。

软件维护（图2，列e）

软件一旦投入运行，就可能出现异常，运行环境可能发生改变，用户回提出新的需求。生命周期的维护阶段从软件交付时开始，但维护活动出现得还要早。软件维护知识域划分为4个子域。

第一个子域是软件维护基础：定义和术语、维护的本质特征、维护的必要性、维护成本的大份额性、软件的进化、维护的分类。

第二个子域将软件维护中的关键问题聚集在一起，这些是：技术问题、管理问题、维护成本估算和软件维护度量。

第三个子域是维护过程，其中的主题包括各种维护过程和维护活动。

第四个子域是维护技术，包括程序的理解、再工程和逆向工程。

软件配置管理（图3，列f）

软件配置管理（Software Configuration Management，SCM）是为了系统地控制配置的变更和维护配置在整个系统的生命周期中的完整性和可追踪性，而标识软件在时间上不同点的配置的学科。这个知识域包括6个子域。

第一个子域是SCM过程管理，包括的主题有：SCM的组织结构上下文、SCM的约束和指导、SCM计划、SCM计划本身和SCM的监管。

第二个子域是软件配置标识，它识别要控制的项目，为各个项目及其版本建立标识方案，确定在获取和管理被控制项目中要使用的工具和技术。子域中第一个主题是识别要控制的项目和软件库。

第三个子域是软件配置控制，它管理软件生命周期中的变更。其中的主题包括：（1）软件变更的请求、评价和批准；（2）实现软件变更；（3）偏离和放弃（deviation and waiver）。

第四个子域是软件配置状态簿记，其主题有软件配置状态信息和软件配置状态报告生成。

第五个子域是软件配置审计，包括：软件功能配置审计、软件物理配置审计、软件基线

的过程内部（in-process）审计。

最后一个子域是软件发布管理和交付，覆盖软件建造和软件发布管理。

软件工程管理（图3，列g）

软件工程管理知识域处理软件工程的管理与度量，虽然度量是所有知识域的一个重要方面，但在这里涉及的是度量程序的主题。软件工程管理游个子域，前5个覆盖软件工程管理，第六个描述软件度量的程序。

第一个子域是启动和范围定义，它由需求的确定与协商、可行性分析、需求的评审和修订过程组成。

第二个子域是软件工程计划，包括过程计划、确定可交付成果、工作量、进度与成本估算、资源分配、风险管理、质量管理、计划本身的管理。

第三个子域是软件项目实施，它的主题是：计划的实现、供应商合同管理、度量过程的实现、过程的监理、过程的控制、报告生成。

第四个主题是评审与评价，主题有：确定需求的满足程度、评审和评价项目性能。

第五个主题描述项目的关闭：确定关闭项目、关闭涉及的活动。

第六个子域描述软件工程度量，特别是度量本身的程序。产品和过程的度量在软件工程过程知识域中描述，许多其它知识域也描述其特定的度量。这个子域的主题包括：建立和维护度量工作、度量过程的计划、进行度量过程和评估度量。

软件工程过程（图3，列h）

软件工程过程的知识域涉及软件工程过程本身的定义、实现、评定、度量、管理、变更和改进。它分为4个子域。

第一个子域是过程实现与变更，其主题有：构成基础结构、软件过程管理周期、过程实现与变更的模型、实际考虑。

第二个子域处理多成定义，主题有：软件生命周期模型、软件生命周期过程、过程定义符号、过程适配（adaptation）和自动化。

第三个子域是过程评定，主题有：过程评定模型和过程评定方法。

第四个子域描述过程与产品度量。软件工程过程覆盖一般的产品度量，以及过程度量。特定于各知识域的度量在相关的知识域描述。子域的主题有：过程度量、软件产品度量、度量结果的质量、软件信息模型和过程度量技术。

软件工程工具和方法（图3，列i）

软件工程工具和方法知识域包括软件工程工具、软件工程方法。

软件工程工具子域使用了与指南相同的结构，为其它9个软件工程知识域各分配一个主题，附加一个主题是其它工具问题，例如工具集成技术，这些问题可能应用于所有类型的工具。

软件工程方法子域分为4个小节：处理形式化途径的启发式方法、处理基于数学的途径的形式化方法、处理基于各种原型的软件开发途径的原型方法、??。

软件质量（图3，列j）

软件质量知识域处理跨越软件生命周期过程的软件质量的考虑，由于软件质量在软件工程中无处不在，其它知识域也涉及质量问题，读者可以注意到本知识域到其它知识域的指示器。本知识域覆盖4个子域。

第一个子域描述软件质量基础，例如软件工程文化和伦理学、质量的价值与成本、模型

和质量特征和质量改进。

第二个子域覆盖软件质量管理过程，主题有：软件质量保证、验证和确认、评审和审计。

第三个即最后一个子域描述与软件质量有关的实际考虑，主题包括：软件质量需求、缺陷特征、软件质量管理技术、软件质量度量。

软件工程相关学科（图3，列k）

最后一章是软件工程相关学科。为确定软件工程的范围，有必要鉴别与软件工程有公共边界的学科，这一章按字母顺序，鉴别这些相关学科。对每个相关学科，使用我们找到的基于一致认可的来源，进行以下内容的鉴别：（1）资料性定义（可行时）；（2）知识域列表。

相关学科包括：计算机工程、计算机科学、管理、数学、项目管理、质量管理、软件人类工程学、系统工程。

附录

附录A 知识域描述规格说明

这个附录描述了编辑组向副编辑提供的规格说明：内容、推荐的参考文献、格式、知识域的描述风格。

附录B 指南的演化

第二个附录描述项目建议的指南进化。2004年版指南只是指南的当前版本，指南将继续演化，以适应软件工程团体的需要。演化的计划还没有制订，但本指南附录提供了一个完成暂时的过程轮廓。在本次编写时，这个过程已经被项目的工业界顾问委员会认可，并向IEEE计算机学会的主管委员会发送了简报，但还没有得到资助或实施。

附录C 标准到知识域的分配

第三个附录是大多数相关标准的注释表，这些标准主要来自IEEE和ISO，注释表将标准分配到SWEBOK指南的各个知识域。

附录D Bloom级别

作为支持项目的第五个目标的辅助手段，特别是对于课程表开发人员（和其它用户），第五个附录对每个主题按照一个教育学目录集，划分等级，这个目录主要由Benjamin Bloom提出。其概念是，教育目标可以分类6类，分别表示递增的深度：知识、理解、应用、分析、综合和评价。对所有知识域的等级分类结构在附录D中。但是，这个附录不能认为是确定性的分类，而应该被认为是一个起点。

////////////////////////////////////

第2章 软件需求

术语与缩写

- DAG: Directed Acyclic Graph, 无环有向图
- FSM: Functional Size Measurement, 功能规模估算
- INCOSE: International Council on Systems Engineering, 国际系统工程理事会
- SADT: Structured Analysis and Design Technique, 结构化分析与设计技术
- UML: Unified Modeling Language, 统一建模语言

引言

软件需求知识域涉及软件需求的获取、分析、规格说明和确认。在软件工业界，人们广泛认为，如果这些活动完成得不好，软件工程项目很容易上失败。

软件需求表达了施加在要解决真实世界问题的软件产品上的要求和约束[Kot00]。

术语“需求工程”在需求中被广泛使用，表示系统地处理需求。但是出于一致性，本指南将不使用它，正如本指南要避免使用术语“工程”来表示除了软件工程活动以外的活动一样。

基于同样原因，本指南也不使用出现在一些文献中的术语“需求工程师”，我们使用“软件工程师”，有时也使用“需求专家”，后一个术语表示问题中的角色通常由个人完成，而不是软件工程师完成。但是，这并不表示软件工程师不能去完成这个任务。

知识域的结构分解与IEEE12207中关于需求活动一节广泛兼容(IEEE12207.1-96)。

在我们提出的结构分解中有一个固有风险：隐含了类似瀑布模型的过程。为澄清这一点，我们设计了子域2（需求过程）来给出需求过程的高层视图，它确定过程进行需要的资源和约束，以及使其具有一定形式而需要的行动。

另外一个可能的分解是使用基于产品的结构（系统需求、软件需求、原型、用况等）。基于过程的分解结构反映了一个事实：需求过程要成功，就必须作为一个涉及复杂、紧密耦合的多个活动组成的过程考虑，而不是作为在软件开发项目开始就进行的离散的、单个的活动组成的过程来考虑。

软件需求知识域与软件设计、软件测试、软件维护、软件配置管理、软件工程管理、软件工程过程和软件质量等的知识域紧密相关。

软件需求主题的结构分解

1. 软件需求基础

1.1. 软件需求定义

软件需求的最基本含义是一个为了解决真实世界问题而必须展示的特性。指南将需求与软件联系起来，因为需求涉及的是软件要解决的问题。因此，软件需求是一个为解决特定问题而必须由被开发或被修改的软件展示的特性。这个问题可能是使用软件的某人的任务中的一个自动化部分，或是支持委托开发软件的组织的业务过程，或修正当前软件的缺点，或是控制一个设备等等。用户、业务过程和设备的功能通常很复杂，因此，特定软件的需求在外延上通常是来自一个组织不同层次的不同人员的需求和来自软件将要在其中运行的环境的需求的复杂组合。

所有软件需求的一个基本特性就是：可验证。验证某些软件需求可能很困难或则成本很高，例如，验证呼叫中心的吞吐量需求就需要开发模拟软件。软件需求和软件质量人员都必须保证，可以在现有的资源约束下，需求可以被验证。

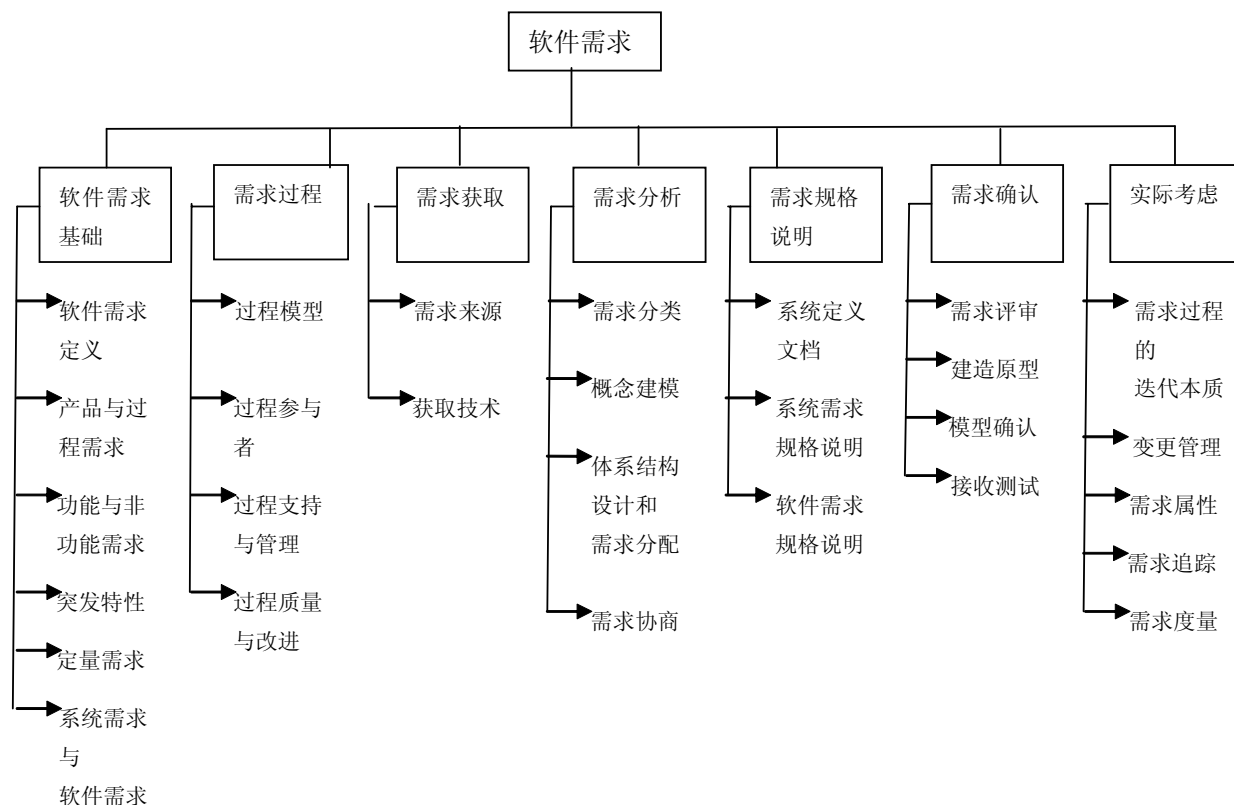


图1 软件需求知识域主题的结构分解

除了其表达的行为特性外，需求还有其它的属性。普遍的例子是一个优先级别，以便在资源有限时进行权衡，或者一个分情况的价值，以便监控项目的进展。通常，软件需求要唯一地标识，才能在整个软件生命周期中，进行软件配置控制和管理[Kot00; Pf101; Som05; Tha97]。

1.2 产品和过程需求

可以区别产品参数与过程参数，产品参数是需要开发的软件上的需求（例如，“软件应该验证一个学生在注册一门课程时，已经满足了所有的前提条件”），过程参数本质上是对软件开发的约束（例如，“软件应该用Ada编写”），过程参数有时也叫过程需求。

一些软件需求可以产生隐含的过程需求，一个例子就是选择验证技术，另外一个例子可能是要求使用特别严格的分析技术（如形式化规格说明方法）来减少可能导致不适当可靠性的故障。开发组织、客户或第三方（如安全管理人员）也可能直接提出过程需求[Kot00; Som97]。

1.3 功能与非功能需求

功能需求描述软件要执行的功能，例如，将某些文本格式化或调制一个信号，功能有时叫做能力。

非功能需求是限制解决方案的需求，非功能需求有时叫做约束或质量需求，可以进一步划分为：性能需求、可维护性需求、安全性需求、可靠性需求，以及其它类型的软件需求，这些主题也在软件质量知识域中讨论[Kot00; Som97]。

1.4 突发特性

一些需求表现了软件的突发 (Emergent) 特性, 即, 这些需求不能由单个组件完成, 根据其满足程度, 依赖于所有软件组件之间的互操作。例如, 呼叫中心的吞吐量需求可能依赖于电话系统、信息系统和接线员在实际运行条件下的相互作用。突发特性特别依赖于系统体系结构[Som05]。

1.5 定量需求

软件需求应尽可能清楚地无二义性地陈述, 可能时, 应该定量地陈述。避免含糊和不可验证的需求非常重要, 因为这些需求依赖于主观判断的解释 (“软件应该可靠”、“软件应该用户友好”)。这对于非功能性需求特别重要。下面是两个定量需求的例子: 呼叫中心软件必须增加20%的吞吐量; 在运行的任一小时内, 系统产生致命错误的概率应该小于 1.0×10^{-8} 。吞吐量需求是一个高层次的需求, 可以导出许多详细需求, 可靠性需求将紧密约束系统体系结构[Dav93; Som05]。

1.6 系统需求与软件需求

在这个主题中, 系统的含义是 “多个元素相互作用的组合, 以实现为其定义的目标, 元素包括: 硬件、软件、固件、人员、信息、技术、设施、服务和其它支持元素”, 这是国际系统工程理事会的定义 (INC0SE00)。

系统需求是对整个系统的需求, 在包含软件组件的系统中, 软件需求是从系统需求中导出的。关于需求的文献有时将系统需求称为 “用户需求”。本指南将 “用户需求” 严格定义为系统客户或终端用户的需求。这样, 系统需求包括: 用户需求, 其它干系人的需求 (如管理层), 以及没有可标识的人类来源的需求。

2 需求过程

本节介绍软件需求过程, 它面向剩余的5个子域, 说明软件需求过程如何与总的软件工程过程吻合 [Dav93; Som05]。

2.1 过程模型

这个主题的目的是提供对需求过程的如下理解: (1) 需求过程不是软件生命周期中的一个离散的从头到尾 (front-end) 的活动, 而是一个过程, 开始于项目的起始阶段, 并在整个生命周期中需要持续精化。(2) 需求过程要将软件需求标识配置项, 并使用与其它软件生命周期过程的产品相同的软件配置管理实践来管理软件需求。(3) 需求过程需要与组织和项目上下文保持适应。

这个主题也包括为需求过程提供输入的活动, 如市场和可行性研究等 [Kot00; Rob99; Som97; Som05]。

2.2 过程参与者

这个主题介绍参与需求过程的人员的角色。需求过程本身是交叉学科的需求专家需要在干系人和软件工程师之间进行协调。除了需求专家外, 有许多人员参与, 每个人都与软件有某种干系。随项目不同, 干系人也不同, 但总是包括: 用户/操作人员和客户 (二者可以不同) [Gog93]。软件干系人的典型例子包括 (但不局限于): (1) 用户: 这个群体由将要操作软件的人员组成, 通常是一个杂合群体, 包含有不同角色和需求的人员。(2) 客户: 这个群体由委托软件开发的人员和代表软件的目标市场的人员组成。(3) 市场分析师: 大众市场产品不止一个委托方, 因此需要市场营销人员来确定市场的要求和作为代理客户。(4)

管制人员：许多应用领域，如银行和公共运输，是被政府管制的，这些领域的软件必须遵从管制当局的需求。（5）软件工程师：这些人员对从软件开发中通过复用其它产品的组件而获利感兴趣，在这种情形下，如果特定产品的用户有复用组件的特殊需求，软件工程师必须权衡自身的利益和客户的利益。

满足所有干系人的需求，通常是不可能的，软件工程师的任务就是协调各种权衡，使之能被主要的干系人接受，并能满足预算、技术、管制规则和其它约束。作到这一点的前提是，已经标识了所有干系人，分析了他们的利益本质，获得了他们的需求 [Dav93;Kot00; Rob99; Som97; You01]。

2.3 过程支持与管理

这个主题介绍需求过程需要和消耗的管理资源，它为软件工程管理知识域的第一个子域（启动和范围定义）建立上下文，它的主要目的是建立2.1中标识的过程活动与成本、人力资源、培训和工具等问题之间的联系[Rob99; Som97; You01]。

2.4 过程质量与改进

这个主题涉及需求过程的质量评定和改进，其目的是强调需求过程在软件产品的成本和准时性方面、客户对需求过程的满意度方面所起的关键作用[Som97]。它有助于需求过程适应软件和质量标准的过程改进模型。过程质量和改进与软件质量知识域和软件工程过程知识域都密切相关，特别是软件质量属性和度量的问题，以及软件过程定义的问题。这个主题覆盖下列问题：（1）过程改进标准和模型覆盖的需求过程范围；（2）需求过程度量与基准；（3）改进的计划与实现[Kot00;Som97; You01]。

3 需求获取[Dav93; Gog93; Lou95; Pf101]

需求获取涉及软件需求来自何方，软件工程师如何收集它们。这是理解需要软件解决的问题的第一阶段，这是本质是一个人类活动，活动中要标识干系人，建立开发小组与客户之间的联系。需求获取又称为“需求捕获”、“需求发现”和“需求获得”。

良好的软件工程的一个基本原则就是，软件工程师与软件用户之间有良好的沟通。在开发活动开始前，需求专家必须为这个沟通建立渠道，他们必须协调软件用户（及其它干系人）的业务领域和软件工程师的技术领域。

3.1 需求来源[Dav93; Gog93; Pf101]

在典型的软件中，需求有许多来源，有必要标识所有潜在的来源，并评价其对软件的影响。设计这个主题是为了促进对软件需求的各种来源的意识和管理这些来源的框架的意识。要点如下：

（1）目标：术语“目标”（有时称为“业务关注”或“关键成功因素”）指的是软件总体的、高层次的目标。目标为软件提供了动机，但通常被含糊地表达。软件工程师需要特别注意评定目标（相对与优先权）的价值和成本，一个低成本的评定方法是可行性研究[Lou95]。

（2）领域知识：软件工程师需要获得或具有关于应用领域的知识，这样，能够推导出干系人没有清楚说明的隐性知识，能够评定相互矛盾的需求之间的必要的权衡，并作为“用户”支持者。

（3）干系人（参见2.2过程参与者）：由于过分强调了一部分干系人的需求而牺牲了其它干系人的需求，许多软件的结果不令人满意。因此，交互的软件难以使用，或违反了客户组织的文化或政策结构。软件工程师需要标识、描绘和管理很多不同类型的干系人的视点

[Kot00]。

(4) 运行环境：需求能够从软件将要运行的环境中导出。例如，实时软件的定时约束、办公环境中的互操作约束。必须主动地寻找这些需求，因为它们可能对软件的可行性和成本有重大影响，并限制设计的选择[Tha97]。

(5) 组织环境：通常要求软件支持某个业务过程，业务过程的选择受组织的结构、文化和内部策略的限制。软件工程师对这些事情应该敏感，因为，一般要求新的软件不能对业务过程施加没有计划的变更。

3.2 获取技术[Dav93; Kot00; Lou95; Pf101]

一旦标识了需求来源后，软件工程师就开始从需求来源获取需求。这个主题主要是让人类干系人清晰地表达他们的需求的技术。这是一个非常困难的领域，软件工程师需要记住这个事实（例如）：用户可能难以描述他们的任务，可能遗漏重要的信息，可能不愿意或不能够合作。特别重要的是，需要理解，获取不是一个被动的任务，即使干系人愿意合作并能清晰表达其任务，软件工程师也必须努力工作，以获取正确的信息。有大量的需求获取技术，下面是主要的几个[Gog93]。

(1) 面谈：这是一个“传统”的获取需求的方法。了解面谈的优点和限制，以及如何引导面谈，非常重要。

(2) 场景：这是一个为获取用户需求提供上下文的有价值的工具，场景让软件工程师为有关用户任务的问题提供一个框架，允许提出“如果…那么…”和“这是如何做的”之类的问题。场景最普通的类型就是用况，这里提供一个到主题4.2（概念建模）的指示器，因为用况和用况图等场景符号在概念建模中很普通。

(3) 原型：这是一个阐明不清楚的需求的有价值的工具，原型与场景的作用类似，向用户提供一个可以更好理解他们需要提供的信息的上下文。原型技术的范围很广，从纸面的屏幕模拟设计到软件产品的β版本，原型用于需求获取和用于需求验证，二者之间有很多重叠（参见主题6.2建立原型）。

(4) 恳谈会：目的是试图达到一个综合效果，因为一群人对其软件需求的洞察比与向单个人了解的要深，他们可以采用头脑风暴，精化其想法，这些想法是使用面谈难以得到的。另一个优点是，相互冲突的需求出现得早，让干系人能认识到存在冲突。如果组织得好，这个技术可以比其它技术得到更丰富更一致的需求。但是，需要仔细控制会议（因此，需要一个推动者），以避免出现小组中批判的能力被对团体的忠诚消蚀，或者出现赞同少数人说出的需求而损害其他人员。

(5) 观察：在组织环境中软件的上下文的重要性，导致修改观察技术来获取需求。软件工程师通过将自己投入到环境中，观察用户如何与其软件交互或相互交互，来学习用户的任务。这些技术成本相对高，但却是有益的，因为它们能阐明许多参与者不能轻易描述清楚的微妙和复杂的用户任务和业务过程。

4 需求分析[Som05]

这个主题涉及分析需求的过程，目的是：（1）检测 and 解决需求之间的冲突；（2）发现软件的边界，以及软件与其环境如何交互；（3）详细描述系统需求，以导出软件需求。需求分析的传统观点是：需求分析被精简为使用多个分析方法中的一个来进行概念建模，这些方法包括：结构化分析与设计（Structured Analysis and Design Technique, SADT）。尽管概念建模很重要，我们将需求分类也包括进来，以帮助需求之间的权衡的告知（需求分类）和建立这些权衡的过程（需求协商）[Dav93]。

描述需求时，必须仔细，应该精确到能确认需求，验证需求的实现，估算需求的成本。

4.1 需求分类[Dav93; Kot00; Som05]

需求可以在多个维度上分类，下面是一些例子：

- (1) 需求是功能性的还是非功能性的（参见主题1.3功能与非功能需求）。
- (2) 需求是从一个或多个高层次需求或突发性特性（参见主题1.4突发特性）导出的，还是由于系人或其它来源直接施加在软件上的。
- (3) 需求是针对产品的还是过程的。对过程的需求可以约束合同商的选择、软件工程过程的采用、要遵循的标准。
- (4) 需求的优先性。一般，优先性越高，需求对于满足软件的总体目标就越基本。通常用固定尺度进行分类，如：强制的、特别需要的、需要的、可选的。优先性通常要与开发和实现的成本进行平衡。
- (5) 需求的范围。范围指的是需求影响软件和软件组件的程度。一些需求，特别是一些非功能性需求，具有全局范围，因为对这些需求的满足不能分配到某一个离散的组件。因此，全局性需求可能强烈地影响软件的体系结构和许多组件的设计，范围小的需求对于满足需求没有多少影响。
- (6) 易变性/稳定性。一些需求在软件生命周期内将发生变化，甚至在开发过程本身中发生变化。估计需求可能发生的变化概率，是有用的。例如，在银行应用中，对客户帐户计算和除息功能的需求可能比支持特定类型的免税帐户的需求更稳定，前者反映了银行领域的一个基本特征（帐户可以有利息），后者可能由于政府的法规而被废止。标记潜在的易变需求，有助于软件工程师建立一个更容许变化的设计。

也可以采用其它适当的分类，这取决于组织的平常实践和应用本身。

需求分类和需求属性之间较大的重叠（参见主题7.3需求属性）。

4.2 概念建模[Dav93; Kot00; Som05]

开发真实世界问题的模型是软件需求分析的关键，模型的目的是帮助理解问题，而不是启动方案的设计。因此，概念模型由来自问题域的实体模型组成，实体模型反映了它们的真实世界联系和依赖。可以开发几类模型，包括：数据和控制流、状态模型、事件追踪、用户交互、对象模型、数据模型，以及其它模型。影响模型选择的因素有：

- (1) 问题的本质：一些类型的软件要求某些方面的分析特别严格，例如，控制流和状态模型对于实时软件来说，就比管理信息软件重要得多，后者通常使用数据模型。
- (2) 软件工程师的技能：采用软件工程师有经验的建模符号或方法，具有更高的生产率。
- (3) 客户的过程需求：客户可能要求使用其喜欢的符号或方法，或者禁止使用其不熟悉的方法。这个因素与前面一个因素是冲突的。
- (4) 方法和工具的可用性：尽管适合于某些特定的问题，培训和工具不常支持的符号或方法可能不被广泛接受。

注意，几乎所有情况下，从构造软件的上下文模型开始，是有用的。软件上下文提供了打算开发的软件与其外部环境之间的联系，这对于理解软件运行环境中的软件上下文，对于标识软件与环境的接口，都很关键。

建模问题与方法紧密联系，对于实用目的，方法是由引导符号应用的过程支持的一个符号（或一组符号）。几乎没有经验迹象表明一个符号对另一个符号的优越性。但是，特定方法和符号的广泛接受可能导致工业界的技能和知识的聚集，这就是目前UML（统一建模语言）情况（UML04）。

可以追溯到逻辑推理的、使用基于离散数学符号的建模，已经对某些特定领域产生影响。

客户或标准可能要求使用这些方法,并可能对某些关键功能或组件的分析提供引人注目的优点。

本主题并不寻求“教授”一种特定的建模风格或符号,而是提供建模目的和意图的指南。两个标准提供了可能在进行概念建模时有用的符号:IEEE1320.1功能建模的IDEF0标准和IEEE1320.2信息建模的IDEF1X97(IDEF对象)标准。

4.3 体系结构设计 with 需求分配[Dav93; Som05]

在某些时候,必须导出方案的体系结构。体系结构设计是需求过程与软件或系统设计重叠时进行的,这说明将二者截然分开是不可能的[Som01]。本主题与软件设计知识域的软件结构与体系结构子域紧密联系。在许多情况下,软件工程师要作为软件体系结构师,因为分析和详细阐明需求的任务,要求标识负责满足需求的组件。这是一个需求分配:将满足需求的责任分配到组件上。

分配对于需求的详细分析很重要,例如,一旦将一组需求分配给一个组件,就可以进一步分解单个需求,以发现有关组件需要如何与其它组件交互以满足分配的需求的更深入的需求。在大型项目中,分配刺激新一轮的对子系统的分析,例如,对轿车的特定刹车性能的需求(刹车距离、恶劣驾驶条件下的安全性、应用的平滑性、需要的踏板压力等),可以分配给刹车硬件(机械和液压组件)和一个反锁定的刹车系统(ABS)。只有在标识了一个反锁定的刹车系统,并将需求分配给它后,才能使用ABS的能力、刹车硬件和突发特性(如轿车重量)等来标识详细的ABS软件需求。

体系结构设计 with 概念建模差不多,从真实世界领域实体到软件组件的映射并不总是显然的,因此将体系结构设计标识为一个独立的主题。对于概念建模和小、体系结构设计,二者对于符号和方法的需求广泛地相同。

IEEE1471-2000标准(软件密集系统的体系结构描述的推荐实践)建议使用多视点的途径来描述系统的体系结构及其软件项(IEEE1471-00)。

4.4 需求协商

本子主题中另一个普遍使用的术语是“解决冲突”。这涉及需求冲突的问题,冲突发生在两个干系人需要不兼容的特征,或者发生在需求与资源之间A,或者在功能与非功能需求之间[Kot00, Som97]。多数情况下,软件工程师作出单边的决定是不明智的,与干系人协商达成适当的权衡,就有必要。从合同的原因上,这样一个决定应该追溯到客户。我们将需求协商分类为一个软件需求分析主题,是因为问题是作为分析的结果出现的。但是,也可以将其作为需求确认的一个主题。

5 需求规格说明

对于许多工程职业,术语“规格说明”指的是将数值或限制分配给产品的设计目标(Vin90)。一般的物理系统的值的数目相对少,而通常软件却有大量的需求,在完成数量化和大量需求的复杂的相互作用之间有相同的重点。在软件工程术语中,“软件需求规格说明”一般指产生一个文档或其电子版本,以便系统地评审、评价和批准。对于复杂的系统,特别是涉及许多非软件组件的系统,最多要产生3类文档:系统定义、系统需求和软件需求。对于简单的软件产品,只需求软件需求文档。下面描述所有3类文档,需要指出,它们应该适当组合。对于系统工程的描述可以参见第12章“软件工程相关学科”。

5.1 系统定义文档

这个文档(有时称为用户需求文档或操作概念)记录系统的需求,它从领域角度,定义

高层系统需求，其读者包括系统用户/客户代表（对于市场驱动的软件，市场人员可以担任这些角色），因此，它的术语必须采用领域术语。文档需要列举系统需求，以及有关系统总体目标的背景信息、系统的目标环境，以及对约束、假设和非工程需求的陈述。它也可以包括设计用于说明系统上下文的概念模型、使用场景和主要的领域实体，还包括数据、信息和工作流。IEEE1362标准“操作文档概念”为准备这种文档的内容提出了建议(IEEE1362-98)。

5.2 系统需求规格说明[Dav93; Kot00; Rob99; Tha97]

开发包含许多软件和非软件组件的系统（例如，现代民航客机）的人员，通常将系统需求描述与软件需求描述分开。按这个观点，首先规格说明系统需求，软件需求从系统需求导出，然后为每个软件组件的需求进行规格说明。严格地讲，系统需求规格说明是一个系统工程活动，不属于本指南的范围。IEEE1233标准是一个开发系统需求的指南(IEEE1233-98)。

5.3 软件需求规格说明[Kot00; Rob99]

软件需求规格说明为客户和合同商或供应商（对于市场驱动的项目，这些角色可以有市场人员和开发部门担任）之间达成的一致建立了基础：软件产品要做什么，软件产品不应该做什么。对于非技术领域读者，除了软件需求规格说明文档，通常还需要软件需求定义文档。

软件需求规格说明允许在设计开始之前，对需求进行严格的评定，以减少以后的重新设计工作，它也为估算产品的成本、风险和进度提供一个现实的基础。组织结构也可使用软件需求规格说明文档来更有效地开发自己的确认和验证计划。

软件需求规格说明为将软件产品传递到新用户或新机器提供了一个信息基础。最后，它为软件的增强也提供一个基础。

软件需求通常用自然语言编写，但是对于软件需求规格说明，需要用形式化或半形式化描述离开补充。选择适当的符号，可以更精确更简明地描述软件体系结构的特定需求和特定方面。一般的规则是，使用的符号应该尽可能简明地描述需求，这对于某些安全性要求很高的软件，以及某些对其依赖性强的软件，特别重要。但是，符号的选择通常受培训、技能和文档的作者与读者的限制。

已经有了大量的质量指标，用于将软件需求规格说明的质量与产品的其它属性联系起来，诸如成本、可接收性、性能、进度、可重现性等。单个软件需求规格说明语句的质量指标包括：祈使语气、引导语句、弱短语、选项和连续性。整个软件需求规格说明文档的指标包括：篇幅、可读性、规范、深度和文本结构[Dav93; Tha97] (Ros98)。

IEEE为软件需求规格说明的制作和内容发布了一个标准，称为IEEE830标准[IEEE830-98]。另外，IEEE1465标准（类似ISO/IEC12119标准）是一个处理软件包中质量需求的标准（IEEE1465-98）。

6 需求确认[Dav93]

需求文档需要经过确认和验证过程。确认需求以保证软件工程师已经理解了需求，验证需求文档遵从公司标准，并且是可理解的、一致的和完备的，也很重要。形式化符号在（严格地说，至少）证明后面两个特性方面具有很强的优点。不同的干系人，包括客户和开发人员代表，应该评审文档。需求文档与软件生命周期过程中其它可交付产品一样，要由软件配置管理来管理(Bry94, Ros98)。

通常需要在需求过程中明确一个或多个点，以进行需求的确认。目标是分配资源给需求之前，发现存在的问题。需求验证涉及检查需求文档，以保证它确实定义了正确的软件（即，这是用户期望的软件）[Kot00]。

6.1 需求评审[Kot00; Som05; Tha97]

最普通的确认手段可能就是检查或评审需求文档,分配一组评审人员简要地检查是否存在:错误、不正确的假设、阐述不清楚、与标准不一致等。引导评审的小组的组成很重要(例如,对于市场驱动的产品,至少应该包括一个来自客户的代表),小组可以提供检查清单中应该出现的内容。

在系统定义文档、软件需求规格说明文档、新发布版本的基线规格说明或过程中任何其它步骤完成时,应该进行评审。IEEE1028标准提供了如何进行评审的指导(IEEE1028-97)。软件质量知识域中也覆盖了评审,参见主题2.3评审与审计。

6.2 建立原型[Dav93; Kot00; Som05; Tha97]

原型是确认软件工程师对软件需求的解释、获取新需求的常用手段。就获取需求而言,有大量的原型技术,在过程中也有大量的检查点可以使用原型确认。原型的一个优点是:它使得解释软件工程师的假设变得容易,并能在需要的时候,给出有用的反馈,说明一些假设为什么不正确。例如,使用动画原型比文本描述或图形模型,能更好地理解用户界面的动态行为。但是,原型也有缺点。由于原型的外观或质量问题,它可能将用户的注意力从基础的核心功能转移到其它方面。因此,一些人推荐不使用软件的原型,例如基于翻动图表的模拟。开发原型的成本可能会高。但是,如果原型能够避免由于试图满足错误的需求而浪费的资源,这些成本也是值得付出的。

6.3 模型确认[Dav93; Kot00; Tha97]

在分析时,通常有必要确认开发的模型的质量。例如,在对象模型中,进行静态分析,验证对象之间存在通信路径(在干系人领域,就是交换数据),就有用处。如果使用了形式化规格说明符号,就有可能使用形式化推理来证明规格说明特性。

6.4 接收测试[Dav93]

软件需求的一个本质特性就是,应该能够确认完成的产品满足需求。不能被确认的需求实际上仅仅是“愿望”。因此,一个重要的任务就是计划如何确认每个需求,在多数情况下,设计接收测试来完成这个任务。标识和设计非功能需求(参见主题1.3功能与非功能需求)的接收测试可能比较困难,为了确认它们,必须分析它们,用定量的形式描述它们。

额外的信息可以在软件测试知识域的子主题2.2.4遵从性测试中找到。

7 实际考虑

本知识域的第一级分解看起来描述的是线性的活动序列,这是一个过程的简化视图[Dav93]。需求过程跨越了整个软件生命周期。在某个状态中的需求的变更管理和维护,是软件工程过程成功的关键,它们精确地反映了待构造的软件或已经构造的软件[Kot00; Lou95]。并非每个组织都有将需求文档化和需求管理的文化,在动态启动的公司中。受强烈的“产品视觉”驱动和资源的限制,常常将需求文档看成是不必要的额外开销。但是,随着这些公司的扩展、客户的增加、公司产品的进化,他们经常发现,自己需要恢复那些促进产品特征的需求,以评定建议的变更的影响范围。这样,需求文档化和变更管理是任何需求过程成功的关键。

7.1 需求过程的迭代本质[Kot00; You01]

软件工业的普遍压力是要求更短的开发周期,特别是竞争非常激烈的市场驱动部门。此外,许多项目多少受环境约束,很多是升级或修订给定了体系结构的现存软件。在实践中,

将需求过程实现为线性的、确定性的过程，是不现实的。在线性的、确定性的过程中，从干系人获得需求，确定基线，分配需求，交付给软件开发小组，顺次完成。大型软件项目的需求被完美理解或完美规格说明，这只是一个神话[Som97]。

相反，需求通常是反复地向一个详细和质量水平前进的，这个水平允许人们作出设计和采购决定。在一些项目中，这样可能导致需求在所有其它特性被充分理解之间，被作为基线。如果问题在以后的软件工程过程中出现，就会有推倒重来的高成本风险。但是，软件工程师应该受项目管理计划的约束，并采取措施，确保需求的“质量”在给定的资源条件下，尽可能高。例如，他们应该清楚说明所有支持需求的假设，以及其它任何已知的问题。

几乎在所有情况下，对需求的理解会随着设计和开发的进行而继续演化，这常常导致在生命周期后期修改需求。也许，理解需求工程中最关键的一点是：相当大一部分需求将回改变。有时，这是由于分析中的错误，但它经常是“环境”改变的一个不可避免的后果：例如，客户的运行或业务环境、软件将要在其中销售的市场环境。无论什么原因，认识到变化的不可避免性和采取措施来缓和它的影响，非常重要。必须管理这些变更，要求提出的变更必须经过一个事先定义的评审和批准过程，并使用仔细的需求追踪、后果分析和软件配置管理（参加软件配置管理知识域）。因此，需求过程不仅仅是软件生命周期中一个简单的任务，而是跨越整个软件生命周期。在一个典型的项目中，软件需求活动从获取到变更管理，都随时间演化。

7.2 变更管理[Kot00]

变更管理是需求管理的中心。这个主题描述了变更管理的角色、需要经过的程序、应该对提出的变更进行的分析。它与软件配置管理知识域有较强的联系。

7.3 需求的属性[Kot00]

需求不仅仅由需要的事物的规格说明组成，还包括帮助管理和解释需求的辅助信息。这应该包括需求的各种不同的分类维度（参见主题4.1需求分类）和验证方法或接收测试计划。它还可能包括每个需求的概要性原理、每个需求的来源、变更历史等附加信息。但是，最重要的需求属性是一个标识符，它允许需求被唯一地和无二义地被标识。

7.4 需求追踪[Kot00]

需求追踪涉及恢复需求的来源和预测需求的效果。追踪是完成需求变更时的效果分析的基础。一个需求应该能够追踪回溯到原始需求和提出需求的干系人（例如，从软件需求回溯到其需要满足的系统需求）。一个需求还应能够向前追踪到满足的需求和设计实体（例如，从系统需求追踪到详细描述它的软件需求，追踪到实现它的代码模块）。典型项目的需求追踪将形成需求的一个复杂的有向无环图(DAG)。

7.5 度量需求

作为一个实用问题，对于一个特定的软件产品，有一个需求的“量”的概念是有用的。这个数在评价需求变更的“规模”、估算开发或维护任务的成本时，很有用，也可简单用于其它度量的分母。功能规模度量（Functional Size Measurement, FSM）是评价功能需求体系的规模的技术，IEEE14143.1标准定义了FSM的概念[IEEE14143.1-00]。ISO/IEC的标准和其它标准描述了特定的FSM方法。

有关规模度量和标准的其它信息可以在软件工程过程知识域中找到。

主题与参考资料矩阵

	[DA V93]	[Go g93]	[IE EE8 30- 98]	[IE EE1 414 3.1 -10 0]	[Ko t00]	[Lo u95]	[Pf 101]	[Ro b99]	[So m97]	[S0 m05]	[T ha 97]	[Yo u01]
1软件需求基础												
1.1软件需求定义					*		*			C5	C1	
1.2产品与过程需求					*				C1			
1.3功能与非功能需求					*				C1			
1.4紧急特性										C2		
1.5定量需求	C3 S4									C6		
1.6系统需求与软件需求												
2需求过程	*									C5		
2.1过程模型					C2 S1			*	C2	C3		
2.2过程参与者	C2	*			C2 S2			C3	C2			C3
2.3过程支持与管理								C3	C2			C2 C7
2.4过程质量与改进					C2 S4				C2			C5
3需求获取	*	*				*	*					
3.1需求来源	C2	*			C3 S1	*	*				C1	
3.2获取技术	C2	*			C3 S2	*	*					
4需求分析	*									C6		
4.1需求分类	*				C8 S1					C6		
4.2概念建模	*				*					C7		
4.3体系结构设计与需求分配	*									C10		
4.4需求协商					C3 S4				*			
5需求规格说明												
5.1系统定义文档												
5.2系统需求规格说明	*				*			C9			C3	

5.3软件需求规格说明	*		*		*			C9			C3	
6需求确认	*				*							
6.1需求评审					C4 S1					C6	C5	
6.2建造原型	C6				C4 S2					C8	C6	
6.3模型确认	*				C4 S3						C5	
6.4接收测试	*											
7实际考虑	*				*	*						
7.1需求过程的迭代本质					C5 S1				C2			C6
7.2变更管理					C5 S3							
7.3需求属性					C5 S2							
7.4需求追踪					C5 S4							
7.5需求度量				*								

推荐的软件需求参考文献

- [Dav93] A. M. Davis, *Software Requirements: Objects, Functions and States*: Prentice-Hall, 1993.
- [Gog93] J. Goguen and C. Linde, "Techniques for Requirements Elicitation," presented at International Symposium on Requirements Engineering, San Diego, California, 1993
- [IEEE830-98] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*: IEEE, 1998.
- (IEEE14143.1-00) IEEE Std 14143.1-2000//ISO/IEC14143-1:1998, *Information Technology-Software Measurement-Functional Size Measurement-Part 1: Definitions of Concepts*: IEEE, 2000.
- [Kot00] G. Kotonya and I. Sommerville, *Requirements Engineering: Processes and Techniques*: John Wiley and Sons, 2000.
- [Lou95] P. Loucopulos and V. Karakostas, *Systems Requirements Engineering*: McGraw-Hill, 1995.
- [Pfl101] S. L. Pfleeger, "Software Engineering: Theory and Practice," Second ed: Prentice-Hall, 2001, Chap. 4.
- [Rob99] S. Robertson and J. Robertson, *Mastering the Requirements Process*: Addison-Wesley, 1999.
- [Som97] I. Sommerville and P. Sawyer, "Requirements engineering: A Good Practice Guide," John Wiley and Sons, 1997, Chap. 1-2.
- [Som05] I. Sommerville, "Software Engineering," Seventh ed: Addison-Wesley, 2005.
- [Tha97] R. H. Thayer and M. Dorfman, Eds., "Software Requirements Engineering." IEEE

Computer Society Press, 1997, 176–205, 389–404.

[You01] R. R. You, *Effective Requirements Practices*: Addison-Wesley, 2001.

附录A 深入阅读的文献

(Ale02) I. Alexander and R. Stevens, *Writing Better Requirements*: Addison-Wesley, 2002.

(Ard97) M. Ardis, "Formal Methods for Telecommunication System Requirements: A survey of Standardized Languages," *Annals of Software Engineering*, vol. 3, 1997

(Ber97) V. Berzins and al, "A Requirements Evolution Model for Computer Aided Prototyping," presented at Ninth IEEE International Conference on Software Engineering and Knowledge Engineering, Skokie, Illinois: Knowledge Systems Institute, 1997

(Bey95) H. Beyer and K. Holtzblatt, "Apprenticing with the Customer," *Communications of the ACM*, vol. 38, iss.5, 45–52, May, 1995

(Bru95) G. Bruno and R. Agarwal, "Validating Software Requirements Using Operational Models," presented at Second Symposium on Software Quality Techniques and Acquisition Criteria, Florence, Italy, 1995

(Bry94) E. Bryne, "IEEE Standard 830: Recommended Practice for Software Requirements Specification," presented at IEEE International Conference on Requirements Engineering, 1994

(Buc94) G. Bucci and al, "An Object-Oriented Dual Language for Specifying Reactive Systems," presented at IEEE International Conference on Requirements Engineering, 1994

(Bus95) D. Bustard and P. Lundy, "Enhancing Soft Systems Analysis with Formal Modeling," presented at Second International Symposium on Requirements Engineering, 1995

(Che94) M. Chechik and J. Gannon, "Automated Verification of Requirements Implementation," presented at Proceedings of the International Symposium on Software Testing and Analysis, Special Issue, 1994

(Chu95) L. Chung and B. Nixon, "Dealing with Non-Functional Requirements: Three Experimental Studies of a Process-Oriented Approach," presented at Seventeenth IEEE International Conference on Software Engineering, 1995

(Cia97) P. Ciancarini and al, "Engineering Formal Requirements: An Analysis and Testing Method for Z Documents," *Annals of Software Engineering*, vol. 3, 1997

(Cre94) R. Crespo, "We Need to Identify the Requirements of the Statements of Non-Functional Requirements," presented at International Workshop on Requirements Engineering: Foundations of Software Quality, 1994

(Cur94) P. Curran and al, "BORIS-R Specification of the Requirements of a Large-Scale Software Intensive System," presented at Requirements Elicitation for Software-Based Systems, 1994

(Dar97) R. Darimont and J. Souquieres, "Reusing Operational Requirements: A Process-Oriented Approach," presented at IEEE International Symposium on Requirements Engineering, 1997

(Dav94) A. Davis and P. Hsia, "Giving Voice to Requirements Engineering: Guest

Editors' Introduction," *Requirements Engineering: Guest Editors' Introduction*," *IEEE Software*, vol. 11, iss. 2, 12-16, March, 1994

(Def94) J. DeFoe, "Requirements Engineering Technology in Industrial Education," presented at IEEE International Conference on Requirements Engineering, 1994

(Dem97) E. Demirors, "A Blackboard Framework for Supporting Teams in Software Development," presented at Ninth IEEE International Conference on Software Engineering and Knowledge Engineering, Skokie, Illinois: Knowledge Systems Institute, 1997

(Die95) M. Diepstraten, "Command and Control System Requirements Analysis and System Requirements Specification for a Tactical System," presented at First IEEE International Conference on Engineering of Complex Computer Systems, 1995

(Dob94) J. Dobson and R. Strens, "Organizational Requirements Definition for Information Technology," presented at IEEE International Conference on Requirements Engineering, 1994

(Duf95) D. Duffy and al, "A Framework for Requirements Analysis Using Automated Reasoning," presented at Seventh International Conference on Advanced Information Systems Engineering, 1995

(Eas95) S. Easterbrook and B. Nuseibeh, "Managing Inconsistencies in an Evolving Specification," presented at Second International Symposium on Requirements Engineering, 1995

(Edw95) M. Edwards and al, "RECAP: A Requirements Elicitation, Capture, and Analysis Process Prototype Tool for Large Complex Systems," presented at First IEEE International Conference on Engineering of Complex Computer Systems, 1995

(ElE95) K. El-Emam and N. Madhavji, "Requirements Engineering Practices in Information Systems Development: A Multiple Case Study," presented at Second International Symposium on Requirements Engineering, 1995

(Fai97) R. Fairley and R. Thayer, "The Concept of Operations: The Bridge From Operational Requirements to Technical Specifications," *Annals of Software Engineering*, vol. 3, 1997

(Fic95) S. Fickas and M. Feather, "Requirements Monitoring in Dynamic Environments," presented at Second International Symposium on Requirements Engineering, 1995

(Fie95) R. Fields and al, "A Task-Centered Approach to Analyzing Human Error Tolerance Requirements," presented at Second International Symposium on Requirements Engineering, 1995

(Gha94) J. Ghajar-Dowlatshahi and A. Varnekar, "Rapid Prototyping in Requirements Specification Phase of Software Systems," presented at Fourth International Symposium on Systems Engineering, Sunnyvale, California: National Council on Systems Engineering, 1994

(Gib95) M. Gibson, "Domain Knowledge Reuse During Requirements Engineering," presented at Seventh International Conference on Advanced Information Systems Engineering (CAiSE '95), 1995

(Gol94) L. Goldin and D. Berry, "AbstFinder: A Prototype Abstraction Finder for Natural Language Text for Use in Requirements Elicitation: Design, Methodology and Evaluation," presented at IEEE International Conference on Requirements Engineering,

1994

(Got97) O. Gotel and A. Finkelstein, "Extending Requirements Traceability: Lessons Learned from an Industrial Case Study," presented at IEEE International Symposium on Requirements Engineering, 1997

(Hei96) M. Heimdahl, "Errors Introduced during the TACS II Requirements Specification Effort: A Retrospective Case Study," presented at Eighteenth IEEE International Conference on Software Engineering, 1996

(Hei96a) C. Heitmeyer and al, "Automated Consistency Checking Requirements Specifications," *ACM Transactions on Software Engineering and Methodology*, vol. 5, iss. 3, 231-261, July, 1996

(Hol95) K. Holtzblatt and H. Beyer, "Requirements Gathering: The Human Factor," *Communications of the ACM*, vol. 38, iss. 5, 31-32, May, 1995

(Hud96) E. Hudlicka, "Requirements Elicitation with Indirect Knowledge Elicitation Techniques: Comparison of Three Methods," presented at Second IEEE International Conference on Requirements Engineering, 1996

(Hug94) K. Hughes and al, "A Taxonomy for Requirements Analysis Techniques," presented at IEEE International Conference on Requirements Engineering, 1994

(Hug95) J. Hughes and al, "Presenting Ethnography in the Requirements Process," presented at Second IEEE International Symposium on Requirements Engineering, 1995

(Hut94) A. T. F. Hutt, Ed., "Object Analysis and Design - Comparison of Methods. Object Analysis and Design - Description of Methods." John Wiley & Sons, 1994.

(INCOSE00) INCOSE, *How To: Guide for all Engineers, Version 2*: International Council on Systems Engineering, 2000.

(Jac95) M. Jackson, *Software Requirements and Specifications*. Reading, Massachusetts: Addison Wesley, 1995.

(Jac97) M. Jackson, "The Meaning of Requirements," *Annals of Software Engineering*, vol. 3, 1997

(Jon96) S. Jones and C. Britton, "Early Elicitation and Definition of Requirements for an Interactive Multimedia Information System," presented at Second IEEE International Conference on Requirements Engineering, 1996

(Kir96) T. Kirner and A. Davis, "Nonfunctional Requirements for Real-Time Systems," *Advances in Computers*, 1996

(Kle97) M. Klein, "Handling Exceptions in Collaborative Requirements Acquisition," presented at IEEE International Symposium on Requirements Engineering, 1997

(Kos97) R. Kosman, "A Two-Step Methodology to Reduce Requirements Defects," *Annals of Software Engineering*, vol. 3, 1997

(Kro95) J. Krogstie and al, "Towards a Deeper Understanding of Quality in Requirements Engineering," presented at Seventh International Conference on Advanced Information Systems Engineering (CAiSE '95), 1995

(Lal95) V. Lalioti and B. Theodoulidis, "Visual Scenarios for Validation of Requirements Specification," presented at Seventh International Conference on Software Engineering and Knowledge Engineering, Skokie, Illinois: Knowledge Systems Institute, 1995

(Lam95) A. v. Lamsweerde and al, "Goal-Directed Elaboration of Requirements for a

Meeting Scheduler: Problems and Lessons Learnt," presented at Second International Symposium on Requirements Engineering, 1995

(Lei97) J. Leite and al, "Enhancing a Requirements Baseline with Scenarios," presented at IEEE International Symposium on Requirements Engineering, 1997

(Ler97) F. Lerch and al., "Using Simulation-Based Experiments for Software Requirements Engineering," *Annals of Software Engineering*, vol. 3, 1997

(Lev94) N. Leveson and al, "Requirements Specification for Process-Control Systems," *IEEE Transactions on Software Engineering*, vol. 20, iss. 9, 684-707, September, 1994

(Lut96a) R. Lutz and R. Woodhouse, "Contributions of SFMEA to Requirements Analysis," presented at Second IEEE International Conference on Requirements Engineering, 1996

(Lut97) R. Lutz and R. Woodhouse, "Requirements Analysis Using Forward and Backward Search," *Annals of Software Engineering*, vol. 3, 1997

(Mac96) L. Macaulay, *Requirements Engineering*. London UK: Springer, 1996.

(Mai95) N. Maiden and al, "Computational Mechanisms for Distributed Requirements Engineering," presented at Seventh International Conference on Software Engineering and Knowledge Engineering, Skokie, Illinois: Knowledge Systems Institute, 1995

(Mar94) B. Mar, "Requirements for Development of Software Requirements," presented at Fourth International Symposium on Systems Engineering, Sunnyvale, California, 1994

(Mas97) P. Massonet and A. v. Lamsweerde, "Analogical Reuse of Requirements Frameworks," presented at IEEE International Symposium on Requirements Engineering, 1997

(McF95) I. McFarland and I. Reilly, "Requirements Traceability in an Integrated Development Environment," presented at Second International Symposium on Requirements Engineering, 1995

(Mea94) N. Mead, "The Role of Software Architecture in Requirements Engineering," presented at IEEE International Conference on Requirements Engineering, 1994

(Mos95) D. Mostert and S. v. Solms, "A Technique to Include Computer Security, Safety, and Resilience Requirements as Part of the Requirements Specification," *Journal of Systems and Software*, vol. 31, iss. 1, 45-53, October, 1995

(Myl95) J. Mylopoulos and al, "Multiple Viewpoints Analysis of Software Specification Process," *IEEE Transactions on Software Engineering*, 1995

(Nis92) K. Nishimura and S. Honiden, "Representing and Using Non-Functional Requirements: A Process-Oriented Approach," *IEEE Transactions on Software Engineering*, December, 1992

(Nis97) H. Nissen and al, "View-Directed Requirements Engineering: A Framework and Metamodel," presented at Ninth IEEE International Conference on Software Engineering and Knowledge Engineering, Skokie, Illinois: Knowledge Systems Institute, 1997

(OBr96) L. O'Brien, "From Use Case to Database: Implementing a Requirements Tracking System," *Software Development*, vol. 4, iss. 2, 43-47, February, 1996

(UML04) Object Management Group, "Unified Modeling Language," 2004, available at

<http://www.uml.org>

(Opd94) A. Opdahl, "Requirements Engineering for Software Performance," presented at International Workshop on Requirements Engineering: Foundations of Software Quality, 1994

(Pin96) F. Pinheiro and J. Goguen, "An Object-Oriented Tool for Tracing Requirements," *IEEE Software*, vol. 13, iss. 2, 52-64, March, 1996

(Pla96) G. Playle and C. Schroeder, "Software Requirements Elicitation: Problems, Tools, and Techniques," *Crosstalk: The Journal of Defense Software Engineering*, vol. 9, iss. 12, 19-24, December, 1996

(Poh94) K. Pohl and al, "Applying AI Techniques to Requirements Engineering: The NATURE Prototype," presented at IEEE Workshop on Research Issues in the Intersection Between Software Engineering and Artificial Intelligence, 1994

(Por95) A. Porter and al, "Comparing Detection Methods for Software Requirements Inspections: A Replicated Experiment," *IEEE Transactions on Software Engineering*, vol. 21, iss. 6, 563-575, June, 1995

(Pot95) C. Potts and al, "An Evaluation of Inquiry-Based Requirements Analysis for an Internet Server," presented at Second International Symposium on Requirements Engineering, 1995

(Pot97) C. Potts and I. Hsi, "Abstraction and Context in Requirements Engineering: Toward a Synthesis," *Annals of Software Engineering*, vol. 3, 1997

(Pot97a) C. Potts and W. Newstetter, "Naturalistic Inquiry and Requirements Engineering: Reconciling Their Theoretical Foundations," presented at IEEE International Symposium on Requirements Engineering, 1997

(Ram95) B. Ramesh and al, "Implementing Requirements Traceability: A Case Study," presented at Second International Symposium on Requirements Engineering, 1995

(Reg95) B. Regnell and al, "Improving the Use Case Driven Approach to Requirements Engineering," presented at Second IEEE International Symposium on Requirements Engineering, 1995

(Reu94) H. Reubenstein, "The Role of Software Architecture in Software Requirements Engineering," presented at IEEE International Conference on Requirements Engineering, 1994

(Rob94) J. Robertson and S. Robertson, "Complete Systems Analysis," vol. 1 and 2. Englewood Cliffs, New Jersey: Prentice Hall, 1994.

(Rob94a) W. Robinson and S. Fickas, "Supporting Multi-Perspective Requirements Engineering," presented at IEEE International Conference on Requirements Engineering, 1994

(Ros98) L. Rosenberg, T. F. Hammer and L. L. Huffman, "Requirements, testing and metrics," presented at 15th Annual Pacific Northwest Software Quality Conference, Utah, 1998

(Sch94) W. Schoening, "The Next Big Step in Systems Engineering Tools: Integrating Automated Requirements Tools with Computer Simulated Synthesis and Test," presented at Fourth International Symposium on Systems Engineering, Sunnyvale, California, 1994

(She94) M. Shekaran, "The Role of Software Architecture in Requirements

- Engineering," presented at IEEE International Conference on Requirements Engineering, 1994
- (Sid97) J. Siddiqi and al, "Towards Quality Requirements Via Animated Formal Specifications," *Annals of Software Engineering*, vol. 3, 1997
- (Span97) G. Spanoudakis and A. Finkelstein, "Reconciling Requirements: A Method for Managing Interference, Inconsistency, and Conflict," *Annals of Software Engineering*, vol. 3, 1997
- (Ste94) R. Stevens, "Structured Requirements," presented at Fourth International Symposium on Systems Engineering, Sunnyvale, California, 1994
- (Vin90) W. G. Vincenti, *What Engineers Know and How They Know It - Analytical Studies form Aeronautical History*. Baltimore and London: John Hopkins University Press, 1990.
- (Wei03) K. Weigers, *Software Requirements*, 2nd ed: Microsoft Press, 2003.
- (Whi95) S. White and M. Edwards, "A Requirements Taxonomy for Specifying Complex Systems," presented at First IEEE International Conference on Engineering of Complex Computer Systems, 1995
- (Wil99) B. Wiley, *Essential System Requirements: A Practical Guide to Event-Driven Methods*: Addison-Wesley, 1999.
- (Wyd96) T. Wyder, "Capturing Requirements With Use Cases," *Software Development*, vol. 4, iss. 2, 36-40, February, 1996
- (Yen97) J. Yen and W. Tiao, "A Systematic Tradeoff Analysis for Conflicting Imprecise Requirements," presented at IEEE International Symposium on Requirements Engineering, 1997
- (Yu97) E. Yu, "Towards Modeling and Reasoning Support for Early-Phase Requirements Engineering," presented at IEEE International Symposium on Requirements Engineering, 1997
- (Zav96) P. Zave and M. Jackson, "Where Do Operations Come From? A Multiparadigm Specification Technique," *IEEE Transactions on Software Engineering*, vol. 22, iss. 7, 508-528, July, 1996

附录B 有关标准

- (IEEE830-98) IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*: IEEE, 1998.
- (IEEE1028-97) IEEE Std 1028-1997 (R2002), *IEEE Standard for Software Reviews*: IEEE, 1997.
- (IEEE1233-98) IEEE Std 1233-1998, "IEEE Guide for Developing System Requirements Specifications," 1998
- (IEEE1320.1-98) IEEE Std 1320.1-1998, *IEEE Standard for Functional Modeling Language-Syntax and Semantics for IDEF0*: IEEE, 1998.
- (IEEE1320.2-98) IEEE Std 1320.2-1998, "IEEE Standard for Conceptual Modeling Language-Syntax and Semantics for IDEFIX97 (IDEF Objectt)," IEEE, 1998.
- (IEEE1362-98) IEEE Std 1362-1998, *IEEE Guide for Information Technology-System Definition-Concept of Operations (ConOps) Document*: IEEE, 1998.
- (IEEE1465-98) IEEE Std 1465-1998//ISO/IEC12119:1994, *IEEE Standard Adoption of*

International Standard ISO/IEC12119:1994(E), Information Technology-Software packages-Quality requirements and testing: IEEE, 1998.
(IEEEP1471-00) IEEE Std 1471-2000, *IEEE Recommended Practice for Architectural Description of Software Intensive Systems: Architecture Working Group of the Software Engineering Standards Committee, 2000.*
(IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes, IEEE, 1996.*
(IEEE14143.1-00) IEEE Std 14143.1-2000//ISO/IEC14143-1:1998, *Information Technology-Software Measurement-Functional Size Measurement-Part 1: Definitions of Concepts: IEEE, 2000.*

////////////////////////////////////

第3章 软件设计

术语和缩写

ADL: Architecture Description Languages, 体系结构描述语言
CRC: Collaboration Responsibilities Card, 协作责任卡
ERD: Entity-Relationship Diagrams, 实体联系图
IDL: Interface Description Languages, 接口描述语言
DFD: Data Flow Diagram, 数据流图
PDL: Pseudo-Code and Program Design Languages, 伪码与程序描述语言
CBD: Component-based design, 基于组件的设计

引言

在文献[IEEE610.12-90]中, 设计被定义为“定义一个系统或组件的体系结构、组件、接口和其它特征的过程”和“这个过程的结果”。作为过程看待时, 软件设计是一种软件生命周期活动, 在这个活动中, 要分析软件需求, 以产生一个将作为软件构造的基础的软件内部结构的描述。更精确地说, 软件设计(结果)必须描述软件体系结构(即, 软件如何分解成组件并组织起来)和这些组件之间的接口, 它必须在详细的层次上描述组件, 以便能构造这些组件。

软件设计在软件开发中起着重要作用: 它让软件工程师产生形成要实现的方案的蓝图的各种不同的模型, 我们可以分析和评价这些模型, 以确定使用它们能否实现各种不同的需求, 我们可以检查和评价各种不同的候选方案, 进行权衡, 最后, 除了作为构造和测试的输入和起始点外, 我们可以使用作为结果的模型, 来规划后续的开发活动。

在《IEEE/EIA 12207软件生命周期过程》[IEEE12207.0-96]等软件生命周期过程的标准列表中, 软件设计由两个处于软件需求和软件构造之间的活动组成: (1) 软件体系结构设计(有时叫做高层设计): 描述软件的搞成结构和组织, 标识各种不同的组件。(2) 软件详细设计: 详细地描述各个组件, 使之能被构造。

对于软件设计知识域的范围, 目前的知识域描述并没有讨论每一个其名字包含“设计”一词的主题。在Tom DeMarco的术语中(DeM99), 本章讨论的知识域主要处理D设计(decomposition design: 分解设计, 将软件映射到组件上)。

但是, 由于日益增长的软件体系结构领域的重要性, 我们也将讨论FP设计(family pattern design: 族模式设计, 目标是在一族软件中建立可利用的公共部分)。但是, 软件

设计知识域没有包含I设计（invention design：创造性设计，通常在软件需求过程中完成，目标是将软件概念化，并说明软件能满足已经发现的要求和需求），因为这个主题是需求分析和规格说明的一部分。

软件设计知识域描述与软件需求、软件构造、软件工程管理、软件指令和软件工程相关学科特别相关。

软件设计主题分解结构

1 软件设计基础

这里介绍的概念、符号和术语形成理解软件设计的作用和范围的基础。

1.1 一般设计概念

软件并不是有设计的唯一领域。一般而言，我们可以将设计看作是一种问题求解[Bud03:c1]，例如，难题（没有确定性答案的问题）对于理解设计的限制就很有趣[Bud04:c1]，一般意义上，其它许多符号和概念对于理解设计也有趣：目标、约束、候选方案、代表、答案[Smi93]。

1.2 软件设计上下文

为理解软件设计的作用，理解其所处的上下文—软件工程生命周期—很重要，这样，理解软件需求分析与软件设计与软件构造与软件测试，就非常重要。[IEEE12207.0-96；Lis01:c11；Mar02；Pfl01:c2；Pre04:c2]。

1.3 软件设计过程

通常认为软件设计是一个两步的过程[Bas03；Dor02:v1c4s2；Fre83:I；IEEE12207.0-96]；Lis01:c13；Mar02:D]：

1.3.1 体系结构设计

体系结构设计描述软件如何分解和组织成组件（软件体系结构）[IEEEP1471-00]。

1.3.2 详细设计

详细设计描述这些组件的特定行为。

软件设计过程的输出是一组模型和人造物品，它们记录了采用的主要决策[Bud04:c2；IEEE1016-98；Lis01:c13；Pre04:c9]。

1.4 使能技术

根据牛津英语词典，“原理”是“一个基本的真理或普遍的法则，用来作为推理的基础或行动的指南”。软件设计原理又称为使能技术[Bus96]，是针对许多不同的软件设计方法和概念的关键观念。使能技术包括[Bas98:c6；Bus96:c6；IEEE1016-98；Jal97:c5,c6；Lis01:c1,c3；Pfl01:c5；Pre04:c9]：

1.4.1 抽象

抽象是“遗忘一些信息，使得不同的事物可以当作同样的事物来处理”[Lis01]。在软件设计上下文中，有两个关键的抽象机制：参数化和规范。规范抽象主要由类：过程抽象、

数据抽象和控制（迭代）抽象[Bas98:c6; Jal97:c5, c6; Lis01:c1, c2, c5, c6; Pre04:c1]。

1.4.2 耦合与聚合

耦合定义为模块之间相互联系的强度，聚合定义为组成一个模块的各个元素如何相互联系[Bas98:c6; Jal97:c5; Pfl01:c5; Pre04:c9]。

1.4.3 分解与模块化

将大型软件分解和模块化为大量小规模独立模块，一般目标是将不同的功能或责任放置到不同的组件中[Bas98:c6; Bus96:c6; Jal97 :c5; Pfl01:c5; Pre04:c9]。

1.4.4 封装与信息隐藏

封装与信息隐藏意味着将元素和抽象的细节分组打包，使得外部不能访问细节[Bas98:c6; Bus96:c6; Jal97:c5; Pfl01:c5; Pre04:c9]。

1.4.5 接口和实现的分离

接口和实现的分离涉及通过规格说明一个公共接口（称为客户）来定义一个组件，并将如何实现细节分离出来[Bas98:c6; Bos00:c10; Lis01:c1, c9]。

1.4.6 充分性、完备性和原始性

要实现充分性、完备性和原始性，就要保证一个软件组件包括了一个抽象的所有重要特征，并且没有多余的特征[Bus96:c6; Lis01:c5]。

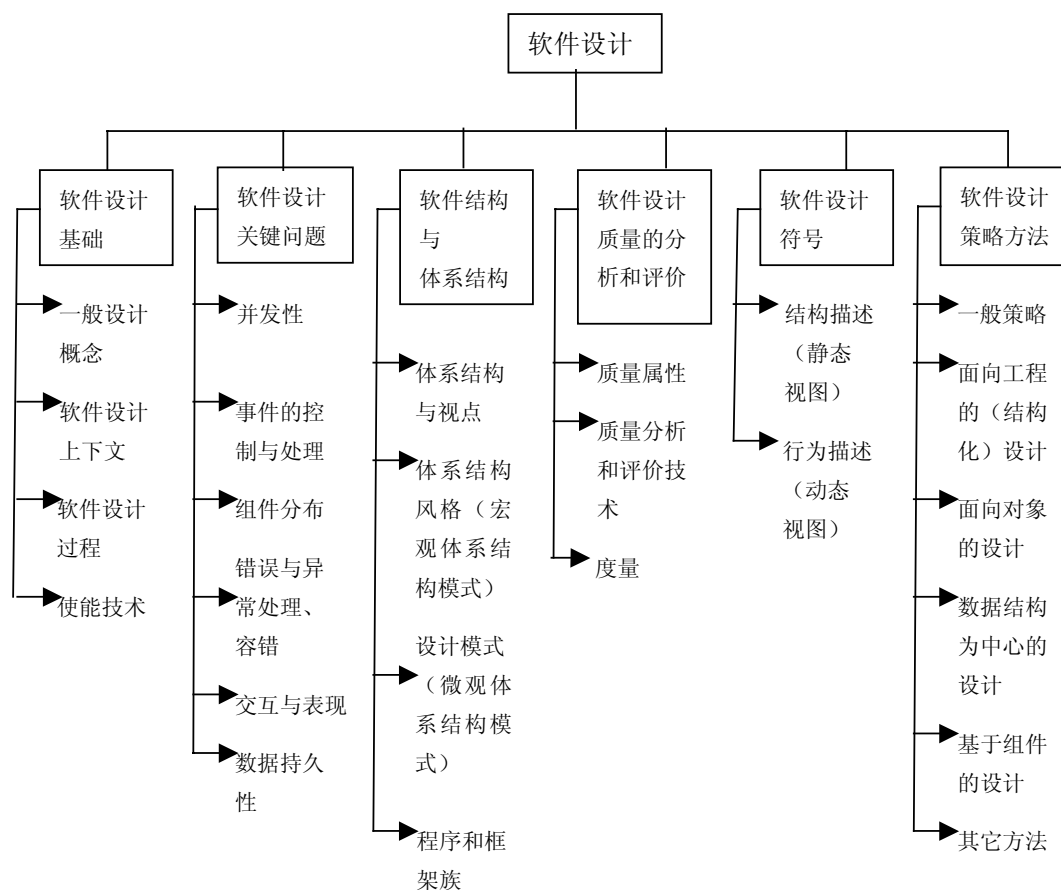


图1 软件设计知识域的主题分解结构

2 软件设计关键问题

设计软件时，必须处理许多关键问题。一些是所有软件都必须处理的质量问题，例如，性能。另一个重要问题是软件组件的分解、组织、打包方法，这个问题很基本，所有设计方法都必须以某种方式处理它（参见主题1.4使能技术和软件设计策略和方法子域）。其它的问题“处理软件行为的某些不在应用领域内的方面，而这些行为涉及支持领域”[Bos00]。这些问题通常与系统的功能性横断相交，被成为剖面（aspect），“剖面一般不是软件功能分解的单元，而是以系统的方式影响组件的性能和语义”（Kic97）。下面是一些关键的、横断问题：

2.1 并发性

如何将软件分解为多个进程、任务和线程，以处理相关的效率、原子性、同步和调度问题 [Bos00:c5; Mar02:CSD; Mey97:c30; Pre04:c9]。

2.2 实践的控制与处理

如何组织数据和控制流，如何通过不同的机制（如隐含调用和回调）处理交互和临时事件 [Bas98:c5; Mey97:c32; Pf101:c5]。

2.3 组件的分布

如何将软件分布到各个硬件上，组件如何通信，如何使用中间件来处理异构软件

[Bas03:c16; Bos00:c5; Bus96:c2 Mar94:DD; Mey97:c30; Pre04:c30]。

2.4 错误和异常处理、容错

如何阻止和容许故障，如何处理异常条件[Lis01:c4; Mey97:c12; Pf101:c5]。

2.5 交互和表现

如何设计结构和组织来实现与用户的交互、信息的表现形式（例如，使用模型-控制器-视图方法，将表现与业务逻辑分离）[Bas98:c6; Bos00:c5; Bus96:c2; Lis01:c13; Mey97:c32]。应该指出，这个主题不是关于如何规格说明用户接口细节的，后者是用户接口设计（软件人类工程学的一部分）的任务，参见软件工程相关学科。

2.6 数据持久性

如何处理需要长时间生存的数据[Bos00:c5; Mey97:c31]。

3 软件结构与体系结构

严格地说，软件体系结构是“一个描述软件系统的子系统和组件，以及它们之间相互关系的学科”（Bus96:c6）。体系结构试图定义结果软件的内部结构（根据牛津英语词典，“结构”是某个事物被构造和组织的方式）。但是，在1990年代中期，软件体系结构开始作为一个更广泛的学科出现了，它涉及以更一般的方式研究软件结构和体系结构[Sha96]，这引发了大量有趣的关于在不同抽象层次上设计软件的思想，其中一些概念在特定软件的体系结构设计中有用（例如，体系结构风格），也在详细设计中有用（例如，低层的设计模式）。但它们也可以用于设计一般的系统，导致了程序族（也称为产品线）的设计。有趣的是，可以认为，多数这些概念是试图描述或复用一般的设计知识。

3.1 体系结构和视图

可以描述软件设计的不同高层剖面，并形成文档，这些剖面通常称为视图：“一个视图表示了软件体系结构的显示软件系统某个特定特性的某个特定方面”[Bus96:c6]。这些不同的图点与关联软件设计的不同问题相关，例如：逻辑视图（满足功能需求）与进程图点（并发问题）、物理视图（分布问题）与开发视图（设计如何分解为实现单元）。其它作者使用了不同的术语，例如，行为、功能、结构和数据模型视图。总之，软件设计是多剖面的产品，它由设计过程产生，并由一些相对独立和正交的视图组成[Bas03:c2; Boo99:c31; Bud04:c5; Bus96:c6; IEEE1016-98; IEEE1471-00]。

体系结构风格（宏观体系结构模式）：体系结构风格是“施加在一个体系结构上的、定义一组或一族满足它们的体系结构的一组约束”[Bas03:c2]。体系结构风格可以被认为是提供软件高层组织（宏观体系结构）的元模型。不同的作者已经标识了大量的主要体系结构风格[Bas03:c5; Boo99:c28; Bos00:c6; Bus96:c1, c6; Pf101:c5]：（1）一般结构（例如，分层、管道线、过滤器、黑板）；（2）分布式系统（例如，客户/服务器、3级结构、代理）；（3）交互式系统（例如，模型-视图-控制器、表现-抽象-控制）；（4）自适应系统（例如，微内核、反射）；（5）其它（例如，批处理、解释器、过程控制、基于规则）。

3.2 设计模式（微观体系结构模式）

简洁地讲，模式是“给定上下文中普遍问题的普遍解决方案”（Jac99）。体系结构风格可以被认为是描述软件高层组织的模式（宏观体系结构），其它设计模式可用于描述较低层次的、更局部的细节（微观体系结构）[Bas98:c13; Boo99:c28; Bus96:c1; Mar02:DP]：（1）

创建型模式（例如，构造者、工厂、原型、单件）；（2）结构型模式（例如，适配器、桥接器、组合、装饰器、剖面、蝇量、代理）；（3）行为型模式（例如，命令、解释器、迭代器、协调器、备忘录、观察者、状态、策略、模板、访问者）。

3.3 程序和框架族

另一个复用软件设计和组件的途径是设计程序族，又成为软件产品线。标识族中成员的公共特性，使用可复用可裁剪组件来解决族内成员之间的可变性问题，就可以实现程序族[Bos00:c7, c10; Bas98:c15; Pre04:c30]。

在面向对象编程中，一个关键概念是框架：可以通过适当的特定插件（又成为热点）实例化的、部分完成的软件子系统[Bos00:c11; Boo99:c28; Bus96:c6]。

4 软件设计质量的分析与评价

本节包含大量的与软件设计特别相关的质量和评价主题，多数主题也包含在软件质量知识域中。

4.1 质量属性

对于得到一个高质量（可维护性、可移植性、可测试性、可追踪性、正确性、健壮性、目的的适应性）的软件设计，多种质量属性都认为是重要的[Bos00:c5; Bud04:c4; Bus96:c6; ISO9126.1-01; ISO15026-98; Mar94:D; Mey97:c3; Pf101:c5]。一个有趣的区分是在运行时间可区别的质量属性（性能、安全性、可用性、功能性、可使用性）、运行时间不可区别的质量属性（可修改性、可移植性、可复用性、可集成性、可测试性）、与体系结构本质质量相关的质量属性（概念完整性、正确性、完备性和可构造性）[Bas03:c4]。

4.2 质量分析与评价技术

有多种工具和技术来帮助人们确保软件设计的质量。

（1）软件设计评审：有正式的和半正式的，通常是以小组方式进行，来验证和保证设计结果的质量（例如，体系结构评审[Bas03:c11]、设计评审和检查[Bud04:c4; Fre83:VIII; IEEE1028-97; Ja197:c5, c7; Lis01:c14; Pf101:c5]、基于场景的技术[Bas98:c9; Bos00:c5]、需求追踪[Dor02:vlc4s2; Pf101:c11]）。

（2）静态分析：正式或半正式的静态（不可执行的）分析技术，可以用于评价一个设计（例如，故障树分析或自动交叉检查）[Ja197:c5; Pf101:c5]。

（3）模拟与原型：这是评价设计的动态的技术（例如，性能模拟或可行性原型[Bas98:c10; Bos00:c5; Bud04:c4; Pf101:c5]）。

4.3 度量

使用度量可以评定或定量估计软件设计的不同方面：规模、结构、质量。已经提出了许多度量，它们多数依赖产生设计的方法。这些度量可以分为两类。

（1）面向功能（结构化）设计的度量：通过功能分解得到的设计结构，通常表示为结构图（有时称为层次图），可以计算其多种度量[Ja197:c5, c7, Pre04:c15]。

（2）面向对象设计度量：设计的总体结构通常表示为类图，可以计算多种度量，也可以计算每个类内部的内容的度量[Ja197:c6, c7; Pre04:c15]。

5 软件设计符号

有许多表达软件设计成果的符号和语言，一些主要用于描述设计的结构组织，另一些用

于描述软件行为。某些符号主要在体系结构设计中使用，另一些则主要用于详细设计，少部分可以用于这两个步骤。另外，一些符号通常用于特定方法的上下文中（参见软件设计策略与方法子域）。这里，我们将符号分类为描述结构（静态）视图和行为（动态）视图两类。

5.1 结构描述（静态视图）

下面的符号，主要是（但不总是）图形，描述和表示软件设计的结构方面，即，它们描述主要的组件和组件间的联系（静态视图）：

（1）体系结构描述语言（Architecture description languages, ADL）：文本（通常是形式化的）语言，以组件和组件间相互联系的方式描述软件体系结构[Bas03:c12]。

（2）类图和对象图：用于表示类或对象的集合，以及它们之间的联系[Boo99:c8, c14; Jal97:c5, c6]。

（3）组件图：用于表示组件（[遵从和提供]一组接口的实现的系统的物理的和替代的部分）集合和组件间联系[Boo99:c12, c31]。

（4）协作责任卡（Collaboration responsibilities cards, CRC）：用于表示组件（类）的名称、责任和协作组件名称[Boo99:c4; Bus96]。

（5）部署图：用于表示一组（物理）节点，及其相互联系，表示了系统的物理外观[Boo99:c30]。

（6）实体联系图：用于表示存储在信息系统中数据的模型[Bud04:c6; Dor02:v1c5; Mar02:DR]。

（7）接口描述语言（Interface description languages, IDL）：类编程语言，用于定义软件组件的接口（输出的操作的名字和类型）[Bas98:c8; Boo99:c11]。

（8）Jackson结构图：以顺序、选择和重复的方式描述数据结构[Bud04:c6; Mar02:DR]。

（9）结构图：用于描述程序的调用结构（一个模块调用的其它模块，调用某个模块的其它模块）[Bud04:c6; Jal97:c5; Mar02:DR; Pre04:c10]。

5.2 行为描述（动态视图）

下面的符号和语言，一些是图形的，一些是文本的，用于描述软件和组件的动态行为。多数符号用于详细设计。

（1）活动图：用于表示从活动（在一个状态机内进行的非原子执行）到活动的控制流[Boo99:c19]。

（2）协作图：用于表示发生在一组对象之间的交互，重点是对象、对象的链接、对象在链接上交换的消息[Boo99:c18]。

（3）数据流图：用来表示数据在一组处理过程之间的流动[Bud04:c6; Mar02:DR; Pre04:c8]。

（4）决策表和决策图：用于表示条件和行动的复杂组合[Pre04:c11]。

（5）流程图和结构化流程图：用于表示控制流和要完成的对应活动[Fre83:VII; Mar02:DR; Pre04:c11]。

（6）序列图：用于表示一组对象之间的交互，重点在按时间顺序的消息交换[Boo99:c18]。

（7）状态变迁与状态图：用于表示状态机中，状态之间的控制流[Boo99:c24; Bud04:c6; Mar02:DR; Jal97:c7]。

（8）形式化描述语言：文本语言，使用来自数学的基本符号（例如，逻辑、集合、顺序）来严格和抽象地定义软件组件接口和行为，通常形式是前置条件和后置条件[Bud04:c18; Dor02:v1c6s5; Mey97:c11]。

(9) 伪码和程序设计语言：结构化的、类编程语言，通常在详细设计阶段，用于描述一个过程或方法的行为[Bud04:c6; Fre83:VII; Jal97:c7; Pre04:c8, c11]。

6 软件设计策略与方法

有各种一般的策略来帮助指导设计过程[Bud04:c9, Mar02:D]。与一般的策略不同，方法则更为专门，方法通常建议和提供了与方法一起使用的一组符号，并描述了遵循方法时要使用的过程，以及使用方法的指南[Bud04:c8]。这些方法作为传递知识的手段和作为软件工程师小组的公共构架，很有用处[Bud03:c8]。可以参见软件工程工具与方法知识域。

6.1 一般策略

常常被引用的设计过程中有用的一般策略是分而治之和逐步求精[Bud04:c12; Fre83:V]、子项向下与自底向上[Jal97:c5; Lis01:c13]、数据抽象与信息隐藏[Fre83:V]、使用启发式规则[Bud04:c8]、使用模式和模式语言[Bud04:c10; Bus96:c5]、使用迭代和增量方法[Pf101:c2]。

6.2 面向功能（结构化）设计[Bud04:c14; Dor02:v1c6s4; Fre83:V; Jal97:c5; Pre04:c9, c10]

这是软件设计的一个经典方法，分解的中心集中在标识主要的软件工程上，然后以自顶向下的方式，不断详细描述和精化这些功能。结构化设计通常在结构化分析后进行，产生数据流图对应的过程描述。研究人员提出了各种策略（例如，变换分析和事务分析）和启发式方法（例如，扇入/扇出、影响范围/控制范围）来将一个数据流图变换为通常用结构图表示的软件体系结构。

6.3 面向对象的设计[Bud04:c16; Dor02:v1:c6s2, s3; Fre83:VI; Jal97:c6; Mar02:D; Pre04:c9]

已经提出了许多基于对象的软件设计方法，这个领域从1980年代中期的基于对象的设计（名词=对象，动词=方法、形容词=属性）发展到面向对象的设计，其中，继承和多态性起着关键的作用，在发展到基于组件的设计，其中，可以定义和访问元信息（例如，通过反射）。虽然面向对象设计源于数据抽象，人们提出了责任驱动的设计作为面向对象设计的另一个选择。

6.4 数据结构为中心的设计[Bud04:c15; Fre83:III, VII; Mar02:D]

数据结构为中心的设计（例如Jackson方法、Warnier-Orr方法）从程序要操纵的数据结构开始，而不是从程序要完成的功能开始。软件工程师首先描述输入输出的数据结构（例如，使用Jackson的结构图），然后基于这些数据结构图来开发程序的控制结构。人们提出了各种启发式规则来处理特殊情况，例如，当输入结构与输出结构不匹配时的情况。

6.5 基于组件的设计

一个软件组件是一个独立的单元，具有良定义的接口和可以独立组合和部署的依赖性。基于组件的设计要解决为了改进复用而提供、开发、集成这些组件有关的问题[Bud04:c11]。

6.6 其它方法

还有一些其它有趣但非主流的方法：形式化和严密的方法[Bud04:c18; Dor02:c5; Fre83; Mey97:c11; Pre04:c29]和变换方法[Pf198:c2]。

主题与参考资矩阵

	[Bas03] {Bas98}	[Boo99]	[Bos00]	[Bud03]	[Bus96]	[Dor02]	[Fre83]	[IEEE1016-98]	[IEEE1028-97]	[IEEE1471-00]
1 软件设计基础										
1.1 一般设计概念				C1						
1.2 软件设计上下文										
1.3 软件设计过程	C2s1, c2s4			C2		V1c4s3	2-22	*		*
1.4 使能技术	{c6s1}		C10s3		C6s3			*		
2 软件设计关键问题										
2.1 并发性			C5s4.1							
2.2 事件的控制和处理	{c5s2}									
2.3 组件分布	C16s2, c16s4		C5s4.1		C2s3					
2.4 错误和异常处理、容错										
2.5 交互和表现	{c6s2}		C5s4.1		C2s4					
2.6 数据持久性			C5s4.1							
3 软件结构与体系结构										
3.1 体系结构与视点	C2s5	C31		C5	C6s1			*		*
3.2 体系结构风格	C5s9	C28	C6s3.1		C1s1-c1s3, c6s2					
3.3 设计模式	{c13s3}	C28			C1s1-c1s3					
3.4 程序和	{c15s	C28	C7s1,		C6s2					

框架族	1, c15s3}		c7s2, c10s2-c10s4, c11s2, c11s4							
4软件设计质量的分析 and 评价										
4.1质量属性	C4s2		C5s2.3	C4	C6s4					
4.2质量分析与评价技术	C11s3, {c9s1, c9s2, c10s2, c10s3}		C5s2.1, c5s2.2, c5s3, c5s4	C4		V1c4s2	542-576		*	
4.3度量										
5软件设计符号										
5.1结构描述（静态视图）	{c8s4}C12S1, C12S2	C4, C8, C11, C12, C14, C30, c31		C6	429					
5.2行为描述（动态视图）		C18, c19, c24		C6, c18		V1c5	485-490, 506-513			
6软件设计策略与方法										
6.1一般策略				C8, c10, c12	C5s1-c5s4		304-320, 533-539			
6.2面向功能（结构化）设计				C14		V1c6s4	328-352			
6.3面向对象设计				C16		V1c6s2, v1c6s3	420-436			
6.4数据结构为中心的设计				C15			201-210, 514-532			
6.5基于组				C11						

[illegible]

6. 6其它方法							C11	C2s2	C29	
----------	--	--	--	--	--	--	-----	------	-----	--

推荐的软件设计参考文献

- [Bas98] L. Bass, P. Clements and R. Kazman, *Software Architecture in Practice*: Addison-Wesley, 1998.
- [Bas03] L. Bass, P. Clements and R. Kazman, *Software Architecture in Practice*, Second ed: Addison-Wesley, 2003.
- [Boo99] G. Booch, J. Rumbaugh and I. Jacobson, *The Unified Modeling Language User Guide*, First ed: Addison-Wesley, 1999.
- [Bos00] J. Bosch, *Design & Use of Software Architectures: Adopting and Evolving a Product-line Approach*, First ed: ACM Press, 2000.
- [Bud04] D. Budgen, *Software Design*, Second ed: Addison-Wesley, 2004.
- [Bus96] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad and M. Stal, *Pattern-oriented Software Architecture: A System of Patterns*, First ed: John Wiley & Sons, 1996.
- [Dor02] M. Dorfman and R. H. Thayer, Eds., "Software Engineering." (Vol. 1 & vol. 2), IEEE Computer Society Press, 2002.
- [Fre83] P. Freeman and A. I. Wasserman, *Tutorial on Software Design Techniques*, Fourth ed: IEEE Computer Society Press, 1983.
- [IEEE610.12-90] IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.
- [IEEE1016-98] IEEE Std 1016-1998, *IEEE Recommended Practice for Software Design Descriptions*: IEEE, 1998.
- [IEEE1028-97] IEEE Std 1028-1997 (R2002), *IEEE Standard for Software Reviews*: IEEE, 1997.
- [IEEE1471-00] IEEE Std 1471-2000, *IEEE Recommended Practice for Architectural Description of Software Intensive Systems*: Architecture Working Group of the Software Engineering Standards Committee, 2000.
- [IEEE12207.0-96] IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- [ISO9126-01] ISO/IEC 9126-1:2001, *Software Engineering-Product Quality-Part 1: Quality Model*: ISO and IEC, 2001.
- [ISO15026-98] ISO/IEC 15026-1998, *Information technology-- System and software integrity levels*: ISO and IEC, 1998.
- [Jal97] P. Jalote, *An Integrated Approach to Software Engineering*, Second ed. New York: Springer-Verlag, 1997.
- [Lis01] B. Liskov and J. Guttag, *Program Development in Java: Abstraction, Specification, and Object-Oriented Design*, First ed: Addison-Wesley, 2001.
- [Mar94] J. J. Marciniak, *Encyclopedia of Software Engineering*: J. Wiley & Sons, 1994.
- The references to the Encyclopedia are as follows:
- CBD = Component-based Design

D = Design

DD = Design of the Distributed System

DR = Design Representation

[Mar02] J. J. Marciniak, *Encyclopedia of Software Engineering*, Second ed: J. Wiley & Sons, 2002.

[Mey97] B. Meyer, *Object-Oriented Software Construction*, Second ed: Prentice-Hall, 1997.

[Pfl101] S. L. Pfleeger, *Software Engineering: Theory and Practice*, Second ed: Prentice-Hall, 2001.

[Pre04] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, Sixth ed: McGraw-Hill, 2004.

[Smi93] G. Smith and G. Browne, "Conceptual foundations of design problem-solving," *IEEE Transactions on Systems, Man and Cybernetics*, vol. 23, iss. 5, 1209-1219, Sep-Oct, 1993.

附录A 深入阅读的文献

(Boo94a) G. Booch, *Object Oriented Analysis and Design with Applications*, Second ed: The Benjamin/Cummings Publishing Company, Inc., 1994.

(Coa91) P. Coad and E. Yourdon, *Object-Oriented Design*: Yourdon Press, 1991.

(Cro84) N. Cross, *Developments in Design Methodology*: John Wiley & Sons, 1984.

(DSO99) D. F. D'Souza and A. C. Wills, *Objects, Components, and Frameworks with UML - The Catalysis Approach*: Addison-Wesley, 1999.

(Dem99) T. DeMarco, "The Paradox of Software Architecture and Design," *Stevens Prize Lecture*, August, 1999

(Fen98) N. E. Fenton and S. L. Pfleeger, *Software Metrics: A Rigorous & Practical Approach*, Second ed: International Thomson Computer Press, 1998.

(Fow99) M. Fowler, *Refactoring: Improving the Design of Existing Code*: Addison-Wesley, 1999.

(Fow03) M. Fowler, *Patterns of Enterprise Application Architecture*, First ed. Boston, MA: Addison-Wesley, 2003.

(Gam95) E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*: Addison-Wesley, 1995.

(Hut94) A. T. F. Hutt, *Object Analysis and Design - Comparison of Methods. Object Analysis and Design - Description of Methods*: John Wiley & Sons, 1994.

(Jac99) I. Jacobson, G. Booch and J. Rumbaugh, *The Unified Software Development Process*: Addison-Wesley, 1999.

(Kic97) G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier and J. Irwin, "Aspect-oriented programming," presented at ECOOP '97 - Object-Oriented Programming, 1997

(Kru95) P. B. Kruchten, "The 4+1 view model of architecture," *IEEE Software*, vol. 12, iss. 6, 42-50, 1995

(Lar98) C. Larman, *Applying UML and Patterns: An introduction to Object-Oriented Analysis and Design*: Prentice-Hall, 1998.

(McC93) S. McConnell, *Code Complete: A Practical Handbook of Software Construction*:

Microsoft Press, 1993.

(Pag00) M. Page-Jones, *Fundamentals of Object-Oriented Design in UML*: Addison-Wesley, 2000.

(Pet92) H. Petroski, *To Engineer is Human: The role of failure in successful design*: Vintage Books, 1992.

(Pre95) W. Pree, *Design Patterns for Object-Oriented Software Development*: Addison-Wesley and ACM Press, 1995.

(Rie96) A. J. Riel, *Object-Oriented Design Heuristics*: Addison-Wesley, 1996.

(Rum91) J. Rumbaugh, M. Blaha, W. Premerlani, F. Eddy and W. Lorensen, *Object-Oriented Modeling and Design*: Prentice-Hall, 1991.

(Sha96) M. Shaw and D. Garlan, *Software architecture: Perspectives on an emerging discipline*: Prentice-Hall, 1996.

(Som05) I. Sommerville, *Software Engineering*, Sixth ed: Addison-Wesley, 2001.

(Wie98) R. Wieringa, "A Survey of Structured and Object: Oriented Software Specification Methods and Techniques," *ACM Computing Surveys*, vol. 30, iss. 4, 459-527, 1998

(Wir90) R. Wirfs-Brock, B. Wilkerson and L. Wiener, *Designing Object-Oriented Software*: Prentice-Hall, 1990

附录B 有关标准

(IEEE610.12-90) IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.

(IEEE1016-98) IEEE Std 1016-1998, *IEEE Recommended Practice for Software Design Descriptions*: IEEE, 1998.

(IEEE1028-97) IEEE Std 1028-1997 (R2002), *IEEE Standard for Software Reviews*: IEEE, 1997.

(IEEE1471-00) IEEE Std 1471-2000, *IEEE Recommended Practice for Architectural Description of Software Intensive Systems*: Architecture Working Group of the Software Engineering Standards Committee, 2000.

(IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.

(ISO9126-01) ISO/IEC 9126-1:2001, *Software Engineering Product Quality-Part 1: Quality Model*: ISO and IEC, 2001.

(ISO15026-98) ISO/IEC 15026-1998, *Information technology-- System and software integrity levels*: ISO and IEC, 1998.

////////////////////////////////////

第4章 软件构造

术语和缩写

OMG: Object Management Group, 对象管理集团

UML: Unified Modeling Language, 统一建模语言

引言

术语“软件构造”指的是通过编码、验证、单元测试、集成测试和调试的组合，详细地创建可工作的、有意义的软件。

软件构造知识域与其它所有知识域都有联系，特别是软件设计和软件测试，这是因为，软件构造过程本身涉及大量的软件设计和测试活动，软件构造也使用了设计的输出，并为测试提供一个输入，这时，设计和测试都是活动，而不是知识域。设计、构造和测试之间的详细边界（如果存在的话）取决于项目使用的生命周期过程。

尽管某些详细设计可以在构造之前进行，但多数设计是在构造活动内部完成的，这样，软件构造知识域与软件设计知识域紧密相关。在整个构造阶段，软件工程师要进行单元测试，并将其工作集成，这样，软件构造知识域也与软件测试紧密相关。

软件构造一般产生需要在软件项目中管理的最数量的配置项（源文件、内容、测试用例等等），这样，软件构造知识域也与软件配置管理知识域相关。由于软件构造强烈依赖工具和方法，并且可能是工具最密集的知识域，它与软件工程工具和方法知识域有联系。尽管软件质量对所有知识域都重要，但是，代码是软件项目的最终可交付产品，因此，软件质量与软件构造密切相关。

在软件工程相关学科中，软件构造知识域与计算机科学最接近，因为软件构造使用了计算机科学的算法知识和详细编码实践经验，通常认为二者属于计算机科学领域。软件构造也与项目管理，软件构造的管理对项目管理提出了相当大的挑战。

软件构造主题的分解结构

下面给出软件构造知识域的主题分解结构，以及与知识域相关的主题的简要描述。对每个主题，我们给出了适当的参考文献。图1是这个知识域的分解结构的顶层图形描述。

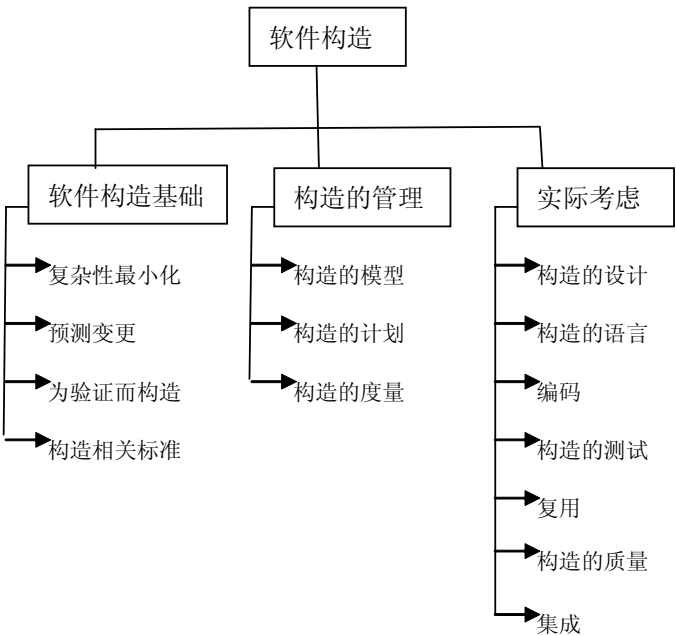


图 1 软件构造知识域主题的分解结构

1 软件构造基础

软件构造基础包括：复杂性最小化、预测变更、为验证而构造、构造中的标准。前3个概念也适用与设计。下面的小节定义这些概念，并描述如何将其应用于构造。

1.1 复杂性最小化[Bec99; Ben00; Hun00; Ker99; Mag93; McC04]

在人们如何将意图传递给计算机中的一个主要因素是，人们在工作记忆中保持复杂结构和信息的能力受到严重限制，特别是长时间保持记忆。这导致了软件构造的一个最强烈的驱动因素：将复杂性最小化。降低复杂性的要求几乎存在于软件构造的每个方面，特别是软件构造的验证和测试过程。在软件构造中，通过创建简单、可读的代码，而不是“聪明”的代码，可以达到降低复杂性的目标。

复杂性最小化可以通过采用标准来实现，这将在主题1.4构造中的标准中讨论，可以采用主题3.3编码中的大量特定技术。也可以用以构造为中心的质量技术来支持复杂性最小化，这将在主题3.5构造的质量中，进行总结。

1.2 预测变更[Ben00; Ker99; McC04]

多数软件要随时间变化，对变更的预测推动着软件构造的许多方面的发展。软件不可避免的是不断变化的外部环境的一部分，外部环境的变化以多种方式影响着软件。可以使用主题3.3编码中的许多特定技术来预测变更。

1.3 为验证而构造[Ben00; Hun00; Ker99; Mag93; McC04]

为验证而构造指的是，用这种方式来构造软件：编写软件的软件工程师、独立的测试活动和运行活动能容易地找出其中的故障。支持为验证而构造的特定技术包括以下：支持代码评审的编码标准、单元测试、代码结构能支持自动测试、限制使用复杂和难以理解的语言结构等。

1.4 构造的相关标准[IEEE12207-95; McC04]

直接影响构造问题的标准包括：编程语言（例如，Java和C++的的语言的语言标准）、通信方法（例如，文档格式和内容标准）、平台（例如，操作系统调用的程序员接口标准）、工具（例如，UML等符号的图示标准）。

应用外部标准：软件构造依赖于应用构造语言的外部标准、构造工具、技术接口和软件构造与其它知识域之间的交互。标准有不同的来源，包括硬件和软件接口规范，诸如对象管理集团OMG和国际标准组织IEEE或ISO。

应用内部标准：可以在一个公司级别或特定项目的组织的基础上，创建标准，这些标准支持小组活动的协调、复杂性最小化、预测变更和为验证而构造。

2 构造的管理

2.1 构造的模型[Bec99; McC04]

人们为开发软件创建了很多模型，其中一些模型强调软件构造。从软件构造的观点，一模型是线性的，如瀑布模型和分阶段交付的生命周期模型。这些模型将构造作为仅仅发生在重要的前提工作（详细需求工作、广泛的设计工作和详细的计划）已经完成后的活动来处理。更线性化的方法倾向于强调构造前面的活动（需求和设计），并倾向与在活动之间创建更多的分隔。在这些模型中，构造的重点是编码。

其他模型是迭代的，如进化式原型、极限编程和结对编程（scrum），这些方法倾向于将构造作为一个活动处理，构造与其它软件开发活动（需求、设计和计划）并行地或重叠地进行。这些方法倾向于将设计、编码和测试活动混合起来，并将这些活动的组合称为构造。这样，构造的含义多少取决于采用的生命周期模型。

2.2 构造的计划[Bec99; McC04]

构造方法的选择是构造计划活动的一个关键方面。构造方法的选择影响着构造前提工作的完成程度、这些前提工作的完成顺序、以及它们在构造工作开始前的完备程度。构造的方法影响着项目降低复杂性、预测变更和为验证而构造的能力。这些目标可以在过程、需求和设计层次上处理，但它们也受选择的构造方法的影响。

构造的计划也要根据选择的构造方法，定义创建和集成组件的顺序、各种软件质量管理过程、将任务分配到特定的软件工程师，以及分配其它任务。

2.3 构造的度量[McC04]

可以度量很多构造活动和产品：开发的代码、修改的代码、复用的代码、销毁的代码、代码复杂性、代码检查的统计数据、故障修正和发现的比率、工作量和进度情况。这些度量对于管理构造、构造中的质量保证、改进构造过程以及其它方面，很有用处。有关度量的更多内容，可以参见软件工程过程知识域。

3 实际考虑

构造是一种活动，其中的软件要遵守任意的和凌乱的真实世界约束，并能精确做到这点。由于与真实世界约束的逼近性，构造比其它知识域更受实际考虑的推动，在构造领域，软件工程也许是最接近工艺的。

3.1 构造的设计[Bec99; Ben00; Hun00; IEEE12207-95; Mag93; McC04]

一些项目为构造分配了更多的设计活动，另一些项目有一个明显的设计阶段。无论实际分配如何，一些详细设计工作将发生在构造的层次上，设计工作倾向于受软件要解决的真实世界问题施加的不可改变的约束。

正如建造物理结构的建筑工人必须进行小规模的修改，以适应建设者计划中的没有预料到的差距，软件构造人员也必须作出或大或小的改动，以在构造中补充软件设计的细节。

设计活动在构造层次上的细节与软件设计知识域中描述的相同，只是应用于更小的规模上。

3.2 构造的语言[Hun00; McC04]

构造的语言包括人类向计算机规格说明一个可执行的问题解决方案的所有形式的通信交流。

构造语言的最简单类型是配置语言，这里，软件工程师从事先定义好的有限选择集中选择需要的东西，创建新的或定制软件装置，在Windows和Unix操作系统中使用的基于文本的配置文件就是一个例子，某些程序生成器的菜单风格的选择列表是另一个例子。

工具箱语言用于从工具（特定应用可复用组件的集成集合）中构造新的应用，比配置语言更复杂。工具箱语言可以显式地定义为应用编程语言（例如，脚本），也可以由工具箱的接口集合隐含定义。

编程语言是最灵活类型的构造语言。它们也包含了特定应用领域和开发过程的最小公共信息，因此，为有效使用它，需要最多的培训。编程语言使用的3类符号包括：语言学的、形式化的、可视化的。

语言学符号特别显著之处是使用类似词的文本串来表示复杂的软件构造，并将类似词的串组合为具有类似句子语法的模式。使用得当时，每个这样的串将具有较强的语义内涵，为其表示的软件构造被执行时要发生的事情的直接、直观的理解。

形式化符号较少依赖于单词和文本串的直观的、日常意义，而是更多地依赖于由简明的、无二义的、形式化（或数学）的定义。形式化构造符号是多数系统编程形式的核心部分，在这里，精确性、时间行为、可测试性比易于到自然语言更重要。形式化构造也使用组合符号

的简明定义方式，以避免许多自然语言构造的二义性。

可视化符号比语言学的和形式化的构造更较少依赖于面向文本的符号，而是依赖于表示软件的可视化实体的直接的视觉解释和放置。可视化构造多少受到仅仅在显示器上移动可视化实体来构造复杂语句的困难的限制，但是，如果主要的编程任务是建造或调整一个详细行为已经定义了的程序的可视化接口时，它就是一个强有力的工具。

3.3 编码[Ben00; IEEE12207-95; McC04]

下面的内容可以应用于软件构造的编码活动：（1）构造可理解的源代码的技术，包括命名和源代码编排；（2）使用类、枚举类型、变量、命名约束和其它类似实体；（3）使用控制结构；（4）处理错误条件：预计的错误和异常（例如，错误的输入数据）；（5）预防代码级的安全泄露（例如，缓冲区超限或数组下标溢出）；（6）通过排斥机制和顺序访问可复用资源规则来使用资源（包括线程和数据库锁）；（7）源代码组织（语句、例程、类、包或其它结构）；（8）代码文档；（9）代码调整。

3.4 构造的测试[Bec99; Hun00; Mag93; McC04]

构造设计通常由编写代码的软件工程师完成的两类测试：单元测试和集成测试。

构造测试的目的是减小故障被带入到代码时的时间与检测到故障的时间之间的差距。有时构造测试在代码完成后进行，有时，测试可以在代码编写前进行。

构造测试一般涉及测试类型的一个子集，测试类型在软件测试知识域中描述。例如，构造测试一般不涉及系统测试， α 测试、 β 测试、强度测试、配置测试、可用性测试，以及其它专门的测试。

本主题有两个已经发布的标准：IEEE829-1998标准“IEEE软件测试文档标准”和IEEE1008-1987标准“IEEE软件单元测试标准”。对于更专门的参考文件，请参见软件测试知识域的相应子主题2.1.1单元测试和子主题2.1.2集成测试。

3.5 复用[IEEE1517-99; Som05].

在IEEE1517-99的引言中指出：实现软件复用比创建是使用程序库需要做更多的工作，它需要通过将复用过程和活动集成到软件生命周期中，来将复用的实践经验形式化。但是，复用软件构造中很重要，因此这里将其作为一个主题处理。

在软件构造的编码和测试中，与复用相关的主题有：（1）选择可复用的单元、数据库、测试过程或测试数据；（2）代码或测试的可复用性评价；（3）在新代码、测试过程或测试数据上的复用信息的报表制作。

3.6 构造的质量[Bec99; Hun00; IEEE12207-95; Mag93; McC04]

有很多保证在构造代码时的质量保证技术，构造使用的主要技术包括：（1）单元测试和集成测试（主题3.4构造的测试）；（2）测试优先的开发（参见软件测试知识域，主题2.2测试目标）；（3）代码逐步求精（stepping）；（4）使用断言；（5）调试；（6）技术评审（参见软件质量知识域，子主题3.3技术评审）；（7）静态分析（IEEE1028）（参见软件质量知识域，主题3.3评审与审计）。

特定技术或选择的技术依赖于被构造的软件的本质，以及完成构造的软件工程师的技能水平。构造质量活动的重点与其它质量活动的重点不同。构造质量活动的重点在代码和与代码紧密相关的产品，与代码相关的产品是小规模的设计，它们与需求、高层设计和计划等其它与代码不密切相关的产品是相对的。

3.7 集成[Bec99; IEEE12207-95; McC04]

构造中的一个关键活动是集成单独构造的例程、类、组件和子系统。另外，一个特定的软件系统可能需要与其它的软件或硬件系统集成。与构造集成相关的东西包括：计划集成组件的顺序、创建支持附加组件来支持软件的过渡版本、确定集成组件前要完成的测试和质量工作的程度、确定项目中测试软件过渡版本的时间点。

主题与参考文献矩阵

	[Bec99]	[Ben00]	[Hun00]	[IEEE1517]	[IEEE12207.0]	[Ker99]	[Mag93]	[McC04]	[Som05]
1 软件构造基础									
1.1 复杂性最小化	C17	C2, c3	C7, c8			C2, c3	C6	C2, c3, c7-c9, c24, c27, c28, c31, c32, c34	
1.2 预测变更		C11, c13, c14				C2, c9		C3-c5, c24, c31, c32, c34	
1.3 为验证而构造		C4	C21, c23, c34, c43			C1, c5, c6	C2, c3, c5, c7	C8, c20-c23, c31, c34	
1.4 构造有关标准					X			C4	
2 构造的管理									
2.1 构造的模型	C10							C2, c3, c27, c29	
2.2 构造的计划	C12, c15, c21							C3, c4, c21, c27-c29	
2.3 构造的度量								C25, c28	
3 实际考虑									
3.1 构造的设计	C17	C8-c10, p	C33		X		C6	C3, c5, c24	

		175-6							
3.2构造的语言			C12, c14-c20					C4	
3.3编码		C6-c10			X			C5-c19, c25-c26	
3.4构造的测试	C18		C34, c43		X		C4	C22, c23	
3.5复用				X					C14
3.6构造的质量	C18		C18		X		C4, c6, c7	C8, c20-c25	
3.7集成	C16				X			C29	

软件构造的推荐参考文献

- [Bec99] K. Beck, "Extreme Programming Explained: Embrace Change," Addison-Wesley, 1999, Chap. 10, 12, 15, 16-18, 21.
- [Ben00a] J. Bentley, "Programming Pearls," Second ed: Addison-Wesley, 2000, Chap. 2-4, 6-11, 13, 14, pp. 175-176.
- [Hun00] A. Hunt and D. Thomas, "The Pragmatic Programmer," Addison-Wesley, 2000, Chap. 7, 8 12, 14-21, 23, 33, 34, 36-40, 42, 43.
- [IEEE1517-99] IEEE Std 1517-1999, *IEEE Standard for Information Technology-Software Life Cycle Processes-Reuse Processes*: IEEE, 1999.
- [IEEE12207.0-96] IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- [Ker99a] B. W. Kernighan and R. Pike, "The Practice of Programming," Addison-Wesley, 1999, Chap. 2, 3, 5, 6, 9.
- [Mag93] S. Maguire, "Writing Solid Code: Microsoft's Techniques for Developing Bug-Free C Software," Microsoft Press, 1993, Chap. 2-7.
- [McC04] S. McConnell, "Code Complete: A Practical Handbook of Software Construction," Microsoft Press, 1993, Chap. 2-32.
- [Som05] I. Sommerville, "Software Engineering," Seventh ed: Addison-Wesley, 2005.

附录A 深入阅读的参考文献

- (Bar98) T. T. Barker, *Writing Software Documentation: A Task-Oriented Approach*: Allyn & Bacon, 1998.
- (Bec02) K. Beck, "Test-Driven Development: By Example," Addison-Wesley, 2002.
- (Fow99) M. Fowler and al, *Refactoring: Improving the Design of Existing Code*: Addison-Wesley, 1999.
- (How02) M. Howard and D. C. Leblanc, *Writing Secure Code*: Microsoft Press, 2002.
- (Hum97b) W. S. Humphrey, *Introduction to the Personal Software Process*:

Addison-Wesley, 1997.

(Mey97) B. Meyer, "Object-Oriented Software Construction," Second ed: Prentice-Hall, 1997, Chap. 6, 10, 11.

(Set96) R. Sethi, "Programming Languages: Concepts & Constructs," Second ed: Addison-Wesley, 1996, Parts II -V.

附录B 有关标准

(IEEE829-98) IEEE Std 829-1998, *IEEE Standard for Software Test Documentation*: IEEE, 1998.

(IEEE1008-87) IEEE Std 1008-1987 (R2003), *IEEE Standard for Software Unit Testing*: IEEE, 1987.

(IEEE1028-97) IEEE Std 1028-1997 (R2002), *IEEE Standard for Software Reviews*: IEEE, 1997.

(IEEE1517-99) IEEE Std 1517-1999, *IEEE Standard for Information Technology-Software Life Cycle Processes-Reuse Processes*: IEEE, 1999.

(IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.

////////////////////////////////////

第5章 软件测试

术语与缩写

SRET: Software Reliability Engineered Testing, 软件可靠性加固测试

引言

测试是为评价、改进产品质量、标识产品的缺陷和问题而进行的活动。

软件测试由一个程序的行为在有限测试用例集合上, 针对期望的行为的动态验证组成, 测试用例是从通常的无限执行域中适当选取的。

在以上定义中, 斜体字对应与标识软件测试知识时的关键问题, 特别是:

(1) 动态: 这意味着, 测试总是隐含在经评价过的输入上执行程序, 更确切地, 输入值本身并不能充分地确定一个测试, 因为一个复杂的、非确定性的系统可能对相同的输入作出不同的反应行为, 这取决于系统的状态。尽管在这个知识域中, “输入” 这个术语在需要的情况下, 仍将继续使用, 并有隐含的约定: 其含义也包括一个特定的输入状态。静态技术与动态不同, 并作为动态技术的补充, 将在软件质量知识域中描述。

(2) 有限: 即使是简单的程序, 理论上也有很多的测试用例, 需要若干年或若干月才能完成穷举测试, 因此, 在实践中, 一般认为整个测试集合是无限的。测试总是隐含了有限的资源和进度与无限的测试需求之间的权衡。

(3) 选取: 人们提出的许多测试技术, 本质上的区别在于如何选择测试集合, 软件工程师必须意识到, 不同的选择准则可能产生差别很大的效果。在给定条件下, 如何标识最适当的选择准则, 是一个非常复杂的问题。在实践中, 需要使用风险分析技术和测试工程技能。

(4) 期望: 确定观察的程序执行的输出是否可以接受, 这尽管不容易但也必须是可能的, 否则测试工作就无用。观察到的结果可以与用户的期望对比 (一般称为确认测试)、与规格说明对比 (验证测试)、或与隐含的需求的期望行为或合理的期望对比, 参见软件需求

知识域的主题6.4接收测试。

软件测试的视图已经演化为更具建设性，测试不再被认为是一种仅在编码阶段完成后才开始的_{活动}，其目标只是检测失效。现在，软件测试被认为是一种应该包括整个开发和维护过程的活动，它本身是实际产品构造的一个重要部分。确实，测试的规划应该在需求过程的早期就开始，随着开发的进展，必须系统地和连续地开发或精化测试计划和过程。测试的规划和设计活动本身为设计者查找潜在的弱点（如设计中的忽略或矛盾之处、文档中的省略或二义之处）提供了有用的输入。

正是目前，人们认为，对质量的正确态度是预防措施之一：避免问题比更正问题好得多。这样，必须认为，测试不仅是检查预防措施是否有效的主要手段，而且是识别由于某种原因预防措施无效时的错误的主要手段。一个可能明显但有值得认识到的事是，在广泛的测试活动成功完成后，软件可能仍包含错误，交付后出现的软件失效的补救措施是由更正性维护，软件维护主题在软件维护知识域中讨论。

在软件质量知识域（参见主题3.3质量管理技术），软件质量管理技术明显分为：静态技术（没有代码执行）和动态技术（代码执行）。这两类都有用。软件测试知识域集中于动态技术。

软件测试也与软件构造有关（参见主题3.4构造测试），即使不是其中一部分，单元和集成测试与软件构造密切相关。

主题的分解结构

软件测试知识域的分解结构如图1所示。

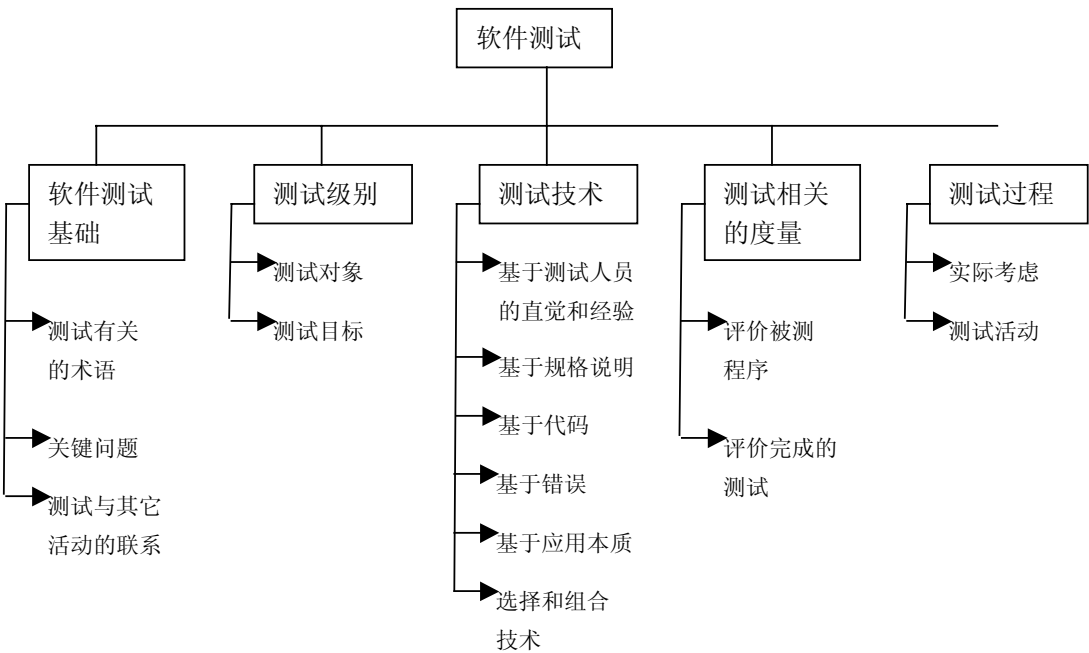


图1 软件测试知识域主题的分解结构

第一个子域描述软件测试基础，它覆盖软件测试领域的基本定义、基本术语和关键问题、与其它活动的联系。

第二个子域是测试级别，有两个（正交的）主题组成：2.1是大型软件测试的传统划分级别，2.2考虑特定条件或特性的测试，称为测试目标。并非所有测试类型都适用于每个软

件产品，也没有列出每个可能的类型。

测试目的和测试目标一起确定如何标识测试集合，二者都与测试集合的一致性（达到规定的目标需要多少测试？）和组成（为达到规定的目标，需要选择哪些测试用例？）有关（尽管“达到规定的目标”通常是隐含的，且一般只提出前面两个问题中的第一个）。有关第一个问题的准则称为测试适合性准则，有关第二个问题的准则称为测试选择准则。

过去几十年来，人们开发了多个测试技术，并仍在提出新的测试技术。子域3覆盖了普遍接受的技术。

子域4讨论与测试有关的度量。

子域5讨论与测试过程有关的问题。

1 软件测试基础

1.1 与测试有关的术语

1.1.1 测试及相关术语的定义[Bei90:c1; Jor02:c2; Lyu96:c2s2.2] (IEEE610.12-90)

软件测试知识域的综合性介绍可以参见推荐的参考文献。

1.1.2 故障 (Fault) 与失效 (Failure) [Jor02:c2; Lyu96:c2s2.2; Per95:c1; Pf101:c8] (IEEE610.12-90; IEEE982.1-88)

软件工程文献中，有许多术语用于描述功能失调，如毛病 (fault)、失效 (failure)、错误 (error)，以及其它。这个术语在IEEE610.12-1990标准“软件工程术语标准词汇 (IEEE610-90)”中有精确定义，也在软件质量知识域中讨论。有必要明确地区分功能失调的原因（这里应该使用错误 fault 或缺陷 defect）和观察到的系统提供的服务中的不希望的效果（称为失效 failure）。测试可以揭露失效，但错误才能够并必须被消除。

但是，应该认识到，失效的原因并非总能毫不含糊的识别。没有理论准则来确定什么错误引发观察到的失效，必须指出，正是要修正错误才能消除问题，但其它方面的修改也能达到这一点。为避免二义性，一些作者使用引发失效的输入 (failure-causing input) (Fra98) 而不是错误 (fault)，即这些输入集合引发一个失效出现。

1.2 一些关键问题

1.2.1 测试选择准则/测试适当性准则（或停止规则）[Pf101:c8s7.3; Zhu97:s1.1] (Wey83; Wey91; Zhu97)

测试选择准则是一种手段，用于确定适当测试用例应该是什么样。选择准则用于：选择测试用例、检查选择的测试组 (suite) 是否合适，即确定测试是否能停止。参见主题5.1 管理问题中的子主题终止。

1.2.2 测试有效性 (effectiveness) /测试目标[Bei90:cls1.4; Per95:c21] (Fra98)

测试是对程序执行的一个样本的观察，可以用不同的目标来指导样本的选择：只有以追求的目标为样板，才能评价测试的有效性。

1.2.3 为标识缺陷而测试[Bei90:c1; Kan99:c1]

在为标识缺陷而进行的测试中，成功的测试是引起系统失效的测试，这与为展示软件满足规格说明或其它需要的特性而进行的测试完全不同，在这类情况中，成功的测试是没有观察到（重大）失效的测试。

1.2.4 神喻 (oracle) 问题[Bei90:c1] (Ber96, Wey83)

神喻是任何决定一个程序是否在给定的测试中具有正确行为、并相应给出一个“通过”

或“失败”的裁定的（人或机械的）智能体。有多种不同类型的神喻，神喻的自动化非常困难和昂贵。

1.2.5 测试的理论和实践局限[Kan99:c2] (How76)

测试理论对将一个没有确定的置信度赋给一系列通过的测试提出了告诫。不幸的是，测试理论建立的多数结论是否定性的，因为它们指出什么是测试永远不能达到的，而不是实际能达到的。关于这点的最著名问题是Dijkstra的格言：程序测试可以用于说明存在错误，但却不能用来说明没有错误。明显的原因是真实软件的完全测试是不可行的。因此，必须基于风险来驱动测试，并将测试作为风险管理策略看待。

1.2.6 不可行路径问题[Bei90:c3]

不可行路径是任何输入都不能到达的控制流路径，这是面向路径的测试中的一个重要问题，特别是在为基于代码的测试技术自动导出测试输入中。

1.2.7 可测试性[Bei90:c3,c13] (Bac90; Ber96a; Voa95)

术语“软件可测试性”有两个相关的但又不同的含义：一方面，它指一个软件完成给定测试覆盖准则的容易程度，见(Bac90)；另一方面，它被定义为如果一个软件是有缺陷的，这个软件在测试中显露失效的概率，可能在统计上度量，见(Voa95, Ber96a)。这两个含义都很重要。

1.3 测试与其它活动的联系

软件测试与静态软件质量管理技术、正确性证明、排错和编程有关，但又不同于它们。但是从软件质量分析员和认证员的角度，来考虑测试，会得到更多的信息。

测试与静态软件质量管理技术的关系可以参见软件质量管理知识域的子域2软件质量管理过程 [Bei90:c1; Per95:c17] (IEEE1008-87)。

测试与正确性证明和形式化验证的关系见[Bei90:c1s5; Pf101:c8]。

测试与排错的关系见软件构造知识域主题3.4构造的测试[Bei90:c1s2.1] (IEEE1008-87)。

测试与编程的关系见软件构造知识域主题3.4构造的测试[Bei90:c1s2.3]。

测试与认证的关系见(Wak99)。

2 测试级别

2.1 测试对象(target)

软件测试随开发和维护过程，通常在不同的级别上进行，即，测试的对象是变化的：单个模块、一组（通过目的、使用、行为或结构而联系起来的）模块、或整个系统[Bei90:c1; Jor02:c13; Pf101:c8]。可以在概念上区分3个大的测试阶段：单元、集成、系统。这里没有隐含任何过程模型，也没有隐含其中一个阶段比其它两个阶段更重要。

2.1.1 单元测试[Bei90:c1; Per95:c17; Pf101:c8s7.3] (IEEE1008-87)

单元测试验证孤立的、可以分别测试的软件片的功能，在不同的上下文中，软件片可以是单个的子程序、或者是由紧密联系的单元组成的较大的组件。在IEEE软件单元测试标准(IEEE1008-87)中对单元测试进行了更精确的定义，这个标准也描述了系统化和文档化的单元测试的集成方法。一般，单元测试与访问被测试的代码一起进行，并需要排错工具的支持，可能涉及编写代码的程序员。

2.1.2 集成测试[Jor02:c13, 14; Pf101:c8s7.4]

集成测试是验证软件组件之间的交互的过程，经典的集成测试策略，如自顶向下、自底向上，与传统的层次结构软件一起使用。现代的系统化集成策略是由体系结构驱动的，它隐含了基于已标识的功能线索来集成软件组件或子系统。集成测试是一个连续的活动，在其每个阶段，软件工程师必须抽象较低层次的视图，并集中到正在集成的层次的视图上。除了小型、简单的软件，通常使用系统化的增量集成策略，而不是一次性将所有组件聚集在一起，后者被成为“大爆炸式（big bang）”测试。

2.1.3 系统测试[Jor02:c15; Pf101:c9]

系统测试关注整个系统的行为。在单元测试和集成测试中，已经标识了多数的功能性失效，通常认为，系统测试适合于评价系统的非功能性需求，如安全性、速度、精确性和可靠性。这个阶段也要评价系统与其它应用、实用程序、硬件设备、运行环境等的接口。更多的功能与非功能性需求的信息可以参见软件需求知识域。

2.2 测试的目标[Per95:c8; Pf101:c9s8.3]

测试是按多少有些明确陈述、精确程度变化的特定目标的观点进行的。用定量的术语精确地陈述目标，可以在测试过程上建立控制。测试可以针对不同的特性，可以设计测试用例来检查是否正确实现了功能规格说明，这在文献中被称为遵从性测试、正确性测试、功能性测试。但是，还应该测试其它几个非功能性特性，这包括：性能、可靠性、可使用性等。

其它重要的测试目标还包括（但不局限于）：可靠性度量、可用性评价和接收，接收可以采用不同的途径。注意，测试目标随测试对象而变化，一般，在测试的不同级别针对不同的目的。

前面为本主题推荐的参考文献描述了潜在的测试目标集合，下面列出的是在文献中最常引用的子主题。注意，某些类型的测试更适合于定制的软件包，安装测试就是一个例子，其它的测试则适合于一般的产品， β 测试就是一个例子。

2.2.1 接收/认定测试[Per95:c10; Pf101:c9s8.5] (IEEE12207.0-96:s5.3.9)

接收测试按照客户的需求来检查系统的行为，需求可以任意用任意方式表达。客户承担或说明典型的任务，以检查需求是否满足，或检查开发机构已经标识了软件的目标市场的需求。这个测试活动可能涉及系统的开发人员，也可能不涉及他们。

2.2.2 安装测试[Per95:c9; Pf101:c9s8.6]

通常，在软件开发和接收测试完成后，可以在目标环境中通过安装来验证软件。可以将安装测试看作是根据硬件配置而再次进行的系统测试，并可以验证安装过程。

2.2.3 α 与 β 测试[Kan99:c13]

在软件发布前，有时会让小规模、有代表性的潜在用户试用，可以在开发机构中进行（ α 测试），也可在用户处进行（ β 测试）。通常，不对 α 与 β 使用进行控制，测试也不总是针对一个测试计划。

2.2.4 遵从性测试/功能测试/正确性测试[Kan99:c7; Per95:c8] (Wak99)

遵从性测试目的是确认观察到的被测试软件的行为是否遵从其规格说明。

2.2.5 可靠性达到和评价[Lyu96:c7; Pf101:c9s.8.4] (Pos96)

为帮助标识错误，测试是改进可靠性的一种手段。与之相反，根据运行大纲，随机地产生测试用例，可以导出可靠性的统计学度量。使用可靠性增长模型，这两个目标都能达到（参见子主题4.1.4寿命测试、可靠性评价）。

2.2.6 回归测试[Kan99:c7; Per95:c11, c12; Pf101:c9s8.1] (Rot96)

根据IEEE610.12-90，回归测试是“系统或组件的选择性测试，以验证修改没有引起不需要的后果……”。在实际中，这个思想用于表明以前通过测试的软件，目前仍能通过。Beizer (Bei90)将其定义为重复的测试，目的在于表明，除了需要的外，软件的行为没有改变。

显然，必须在每当一次变更后进行的回归测试作出的断言和回归测试需要资源之间作出平衡。

在主题2.1测试对象中的每个测试级别上都可以进行回归测试，并可应用与功能和非功能测试。

2.2.7 性能测试[Per95:c17; Pf101:c9s8.3] (Wak99)

性能测试特别针对验证软件满足规格说明的性能需求，例如，容量和响应时间。一种特殊的性能测试是容限测试(Per95:p185, p487; Pf101:p401)，它要测试程序或系统的内部极限。

2.2.8 压力测试[Per95:c17; Pf101:c9s8.3]

压力测试以设计的最大负载运行软件，并以超过最大负载运行软件。

2.2.9 背对背测试

在一个软件产品的两个实现版本上运行同一个测试集合，并比较结果。

2.2.10 恢复测试[Per95:c17; Pf101:c9s8.3]

恢复测试的目的是验证软件在“灾难”后的重新启动能力。

2.2.11 配置测试[Kan99:c8; Pf101:c9s8.3]

开发的软件将服务于不同的用户时，配置测试用于分析软件在规格说明的不同的配置下的行为。

2.2.12 可用性测试[Per95:c8; Pf101:c9s8.3]

这个过程评价终端用户学习和使用软件（包括用户文档）的难易程度、软件功能在支持用户任务的有效程度、从用户的错误中恢复的能力。

2.2.13 测试驱动的开发[Bec02]

测试驱动的开发本质上不是一个测试技术，它将测试作为一个需求规格说明文档的替代手段，而不是将其作为一个独立的检查软件是否正确实现需求的手段。

3 测试技术

测试的一个目标就是尽可能多地揭露潜在的失效，人们为此开发了很多的技术，这些技术试图通过运行一个或多个从被认为是等价的执行的已标识的类中导出的测试，来“中断”程序这些技术的主要原理是尽可能系统化的标识程序行为的代表性集合，例如，考虑输入域、

场景、状态和数据流等的子类。

找出分类所有技术的相同的基础比较困难，这里使用的分类基础是一个折中。分类基于测试用例是如何从软件工程师的直觉和经验、规格说明、代码结构、需要发现的（真实或人造）错误、使用领域或应用的本质中生成的。有时，如果测试依赖于有关软件如何设计和编码的信息，则这些技术被分为白盒或玻璃盒，如果测试仅依赖于输入/输出行为，则这些技术被称为黑盒，还有一类组合使用了两种或两种以上的技术。

显然，这些技术通常不会被测试人员平等地使用，下面是软件工程师应该知道的一些技术。

3.1 基于软件工程师直觉和经验

3.1.1 特定测试[Kan99:c1]

实践中最广泛采用的技术也许是特定测试：测试是根据软件工程师的技能、直觉和对类似程序的经验导出的。特定测试对于标识不容易被形式化技术捕获的特殊的测试有用。

3.1.2 探索性测试

探索性测试定义为边学习、边设计测试和边执行测试，即，没有在制定的测试计划中事先定义测试，而是动态的设计、执行和修改测试。探索性测试的效果依赖于软件工程师的知识，这些知识可以从不同的来源导出：测试中观察到的产品行为、对应用、平台和失效过程的了解、可能的错误和失效类型、与特定产品相关的风险等[Kan01:c3]。

3.2 基于规格说明的技术

3.2.1 等价类划分[Jor02:c7; Kan99:c7]

将输入域划分为一些子集合或等价类，根据一个指定的联系，这些类是等价的，从每个类中提取的测试的代表性测试集合（通常只有一个测试）。

3.2.2 边值分析[Jor02:c6; Kan99:c7]

在变量的输入域的边界上和边界附近选择测试用例，其基本原理是许多错误倾向于发生在输入的极端值附近。这个技术的一个扩展是强健性测试，在变量的输入域外部选择测试用例，以测试程序对于不期望或错误输入的强健性。

3.2.3 决策表[Bei90:c10s3] (Jor02)

决策表表示了条件（粗略地说是输入）和行动（粗略地说是输出）之间的关系。考虑条件和行动的每个可能的组合，系统性地选择测试用例。一个相关的技术成为原因-结果图（*cause-effect graphing*）[Pf101:c9]。

3.2.4 基于有限自动机[Bei90:c11; Jor02:c8]

将一个程序看作一个有限状态自动机，可能根据对其状态和状态迁移的覆盖，来选择测试用例。

3.2.5 根据形式化规格说明的测试[Zhu97:s2.2] (Ber91; Dic93; Hor95)

用形式化语言形式给出规格说明，可以自动导出功能性测试用例，同时也为检查测试结果提供了一个参考输出或神喻。有许多方法可以从模型(Dic93, Hor95)和代数规格说明(Ber91)中导出测试用例。

3.2.6 随机测试[Bei90:c13; Kan99:c7]

测试完全是随机产生的，不要与子主题3.5.1操作大纲中描述的来自操作大纲的统计测试混淆。之所以将这类测试划分到基于规格说明的测试，是因为必须知道输入域，以便在其中选择随机点。

3.3 基于代码的技术

3.3.1 基于控制流准则[Bei90:c3; Jor02:c10] (Zhu97)

基于控制流覆盖准则旨在覆盖程序中所有的语句或语句块，或它们的任意组合。已经提出了几个覆盖准则，如决策条件覆盖。最强的基于控制流的准则是路径测试，其目的是执行流程图中所有从入口到出口的路径。由于循环的存在，路径测试一般是不可行的，实践中采用其它较弱的准则，如语句测试、分支测试、条件/决策测试。这些测试的适当度按百分比度量，例如，当测试将所有分支至少执行一遍时，可以认为达到了100%的分支覆盖。

3.3.2 机遇数据流的准则[Bei90:c5] (Jor02; Zhu97)

在基于数据流的测试中，用有关程序的变量如何被定义、使用和丢弃（取消定义）的信息来注释控制流图。最强的准则是使用所有定义的路径，它要求对于每个变量，每一个从变量的定义到使用定义的控制流路径段被执行。为减少需要的路径数目，可以使用较弱的策略，如所有定义或所有使用。

3.3.3 基于代码测试的参考模型（流程图、调用图）[Bei90:c3; Jor02:c5].

尽管这不是一个技术，在基于代码的测试技术中，程序的控制结构可以用一个流程图表示。流程图是一个有向图，节点和弧分别对应程序元素，例如，节点可以代表语句或不被中断的语句序列，弧代表节点之间的控制转移。

3.4 基于错误的技术 (Mor90)

随形式化的程度不同，基于错误的测试技术特别针对揭露可能的或事先定义的错误类别，来设计测试用例。

3.4.1 错误猜测[Kan99:c7]

在错误猜测中，软件工程师设计特殊的测试用例，试图给定程序中最可能的错误。一个比较好的信息源是以前项目中发现的错误的历史，以及软件工程师的技能。

3.4.2 变异测试[Per95:c17; Zhu97:s3.2-s3.3]

变异是被测试程序的一个微小修改的版本，对程序进行一个小的、语法上的变更。在原始版本和所有变异版本上运行每个测试用例，如果一个测试用例能够成功分辨原始程序和变异版本，就认为后者被丢弃了。最初，变异测试用于评价测试集合（参见4.2），它本身也是一个测试准则：随机地选择测试用例，直到足够多的变异被丢弃，或者特别地设计测试，来丢弃幸存的变异版本。在后一种情况中，可以将变异测试分类为基于代码的测试。变异测试的基本假设是耦合效应，即，通过寻找简单的语法错误，可以发现更复杂的、真实的错误。为了让变异测试有效，必须系统化地自动生成大量的变异。

3.5 基于使用的技术

3.5.1 操作大纲[Jor02:c15; Lyu96:c5; Pf101:c9]

在为进行可靠性评价而进行的测试中，测试环境必须尽可能相近地重现软件的运行/操作环境。其思想是从观察到的测试结果中，推测软件在未来实际使用时的可靠性。为此，根据其在实际操作中出现的情况，为输入分配一个概率分布或大纲。

3.5.2 软件可靠性加固测试[Lyu96:c6]

软件可靠性加固测试（Software Reliability Engineered Testing, SRET）是一个包括整个开发过程的测试方法，这里，测试是“根据可靠性目标、期望的相对使用、领域中不同功能的关键性来设计和进行的”。

3.6 基于应用本质的测试

上面的技术适用与所有类型的软件。但是，对某些应用，为导出测试，需要一些额外的知识，基于被测试应用的本质，下面提供一些专门的测试领域：（1）面向对象的测试[Jor02:c17; Pf101:c8s7.5] (Bin00)；（2）基于组件的测试；（3）基于Web的测试；（4）图形用户接口GUI测试[Jor20]；（5）并发程序的测试(Car91)；（6）协议遵从性测试(Pos96; Boc94)；（7）实时系统测试(Sch94)；（8）极端要求安全性的系统的测试(IEEE1228-94)。

3.7 技术的选择和组合

3.7.1 功能性与结构性[Bei90:cls. 2.2; Jor02:c2, c9, c12; Per95:c17] (Pos96)

通常分别将基于规格说明的技术和基于代码的技术成为功能性的和结构性的。这两类选择测试的方法并不是相互可替代的，而是互补的，事实上，它们使用不同的信息来源，突出了不同种类的问题。根据预算的考虑，可以组合使用它们。

3.7.2 确定性的与随机的(Ham92; Lyu96:p541-547)

根据前面列出的各种技术，可以按确定性方式选择测试用例，也可以从输入分布中随机选择，就象可靠性测试那样。已经进行了一些分析性和经验性的比较，分析了一种方法比另一种方法更有效的条件。

4 测试相关的度量

有时，测试技术与测试目标混淆在一起，测试技术是手段，用来帮助确保测试目标的达到。例如，分支覆盖是常见的测试技术，要达到特定的分支覆盖度量在本质上不是测试的目标，它仅是一个通过系统地从一决策点执行每个程序分支，以增加发现失效的机会的手段。为避免这种误解，应该在提供对被测试程序的评价的测试相关的度量和评价测试集合完备性的测试相关的度量之间，进行明确的区分。附加的程序度量信息在软件工程管理知识域的子域6软件工程度量中。有关度量的附加信息请参见软件工程过程知识域的子域4产品与过程度量。

通常认为，度量是质量分析的仪器手段，度量也可用于优化测试的计划和执行。测试管理可以使用几个过程度量来监控进展。与管理目的测试过程相关的度量见主题5.1实际考虑。

4.1 评价被测试的程序(IEEE982.1-98)

4.1.1 辅助测试的计划和设计的程序度量[Bei90:c7s4.2; Jor02:c9] (Ber96; IEEE982.1-88)

可以使用基于程序规模（例如，源代码行或功能点）或基于程序结构（如复杂性）的度量来指导测试。结构度量也可以包括程序模块之间的度量（如一个模块调用其它模块的频率）。

4.1.2 错误类型、分类和统计[Bei90:c2; Jor02:c2; Pf101:c8] (Bei90; IEEE1044-93; Kan99; Lyu96)

有关错误的分类学和分类的文献有很多，为了让测试更有效，知道可以在被测试的软件中发现哪类错误，以及这些错误在过去发生的相对频率，很重要。这个信息在预测质量和过程改进中，很有用处。更多的信息请参见软件质量知识域的主题3.2缺陷特征。有一个分类软件异常的IEEE标准(IEEE1044-93)。

4.1.3 错误密度[Per95:c20] (IEEE982.1-88; Lyu96:c9)

通过按发现的错误类型来计数和分类，可以评定被测试的程序。对每个错误类型，错误密度定义为发现的错误数目与程序规模的比率。

4.1.4 寿命测试、可靠性评价[Pf101:c9] (Pos96:p146-154)

可以从可靠性达到和评价（参见子主题2.2.5）得到软件可靠性的统计学估计，它被用于评价一个产品和决定是否可以终止测试。

4.1.5 可靠性增长模型[Lyu96:c7; Pf101:c9] (Lyu96:c3, c4)

可靠性增长模型提供了基于在可靠性达到和评价（子主题2.2.5）下观察到的失效而对可靠性的预测，一般，这些模型假设引起观察到的失效的错误已经被改正（尽管一些模型也接受不完全的改正），这样，在平均上，产品的可靠性呈现增长的趋势。已经发表了十多个模型，许多模型基于一些公共假设，其它则不然。特别地，这些模型划分为错误计数模型和失效间隔时间模型。

4.2 评价完成的测试

4.2.1 覆盖/彻底性度量[Jor02:c9; Pf101:c8] (IEEE982.1-88)

有几个测试适当度准则要求，测试用例系统地程序中或规格说明中（见子主题3.3）标识的元素集合。为评价已经执行的测试的彻底性，测试人员必须控制被覆盖的元素，这样才能动态地度量已覆盖的元素和所有元素之间的比率。例如，有可能度量程序流程图中已覆盖的分支的百分比，或已执行的列在规格说明文档中的功能需求的百分比。基于代码的适当性准则要求适当的“仪器”来测量被测试程序。

4.2.2 错误种植[Pf101:c8] (Zhu97:s3.1)

在测试前，可以在程序中人工引入一些，执行测试时，可能揭露一些种植的错误，以及一些程序中本来的错误。在理论上，根据发现的哪些人工错误及其数目，可以评价测试的有效性，并估计剩余的程序原来的错误数目。实际中，统计学家要考虑种植的错误的分布，以及其相对与程序原本错误的代表性，和外推基于的小样本规模。一些人指出，使用这个技术必须非常小心，因为将错误插入软件涉及到将其留在程序中的风险。

4.2.3 变异评分[Zhu97:s3.2-s3.3]

在变异测试（子主题3.4.2）中，被丢弃的变异对生成的总变异数目的比率可以用来度量执行的测试的有效性。

4.2.4 不同技术的比较和相对有效性techniques

[Jor02:c9,c12; Per95:c17; Zhu97:s5] (Fra93; Fra98;Pos96: p64-72)

已经进行了一些研究，来比较不同测试技术的相对有效性。必须精确定义技术评定的特性，这一点很重要，例如，什么是术语“有效性effectiveness”的精确含义？可能的解释是：发现第一个失效需要的测试的数目、通过测试发现的错误数目与测试中和测试后发现的错误的总数目的比率、或者可靠性增加了多少。根据前面列出的有效性的各个概念，对不同技术进行了分析和经验的比较。

5 测试过程

需要将测试概念、策略、技术和度量集成到一个定义好的、受控制的由人来执行的过程中。测试过程支持测试活动，为测试小组提供从测试计划到评价测试输入的指导，这样，就提供了可证明的保证：测试目标可以有效地满足成本。

5.1 实际考虑

5.1.1 态度/无自我的编程[Bei90:c13s3.2; Pf101:c8]

成功测试的一个重要成分是对测试和质量保证活动的合作态度。在培育对开发和维护中发现失效的良好接受态度方面，管理者起着关键作用。例如，阻止程序员之间对代码所有权的固有思想，程序员们就不会感到对其代码揭露的失效负有责任。

5.1.2 测试指导[Kan01]

测试阶段可以由不同的目的来指导，例如：在基于风险的测试中，使用产品风险来对测试策略进行优先排序和聚焦，或者在基于场景的测试中，用基于规格说明的软件场景来定义测试用例。

5.1.3 测试过程管理[Bec02: III; Per95:c1-c4; Pf101:c9] (IEEE1074:97; IEEE12207.0-96:s5.3.9, s5.4.2, s6.4, s6.5)

必须将在不同的级别（见子主题2测试级别）进行测试活动与人、工具、策略和度量组织到一个良好定义的过程中，这个过程是生命周期的一个完整部分。在IEEE/EIA标准12207.0中，测试不是一个孤立的过程，但是测试活动原则包含进了5个主要的生命周期过程和支持过程中。在IEEE1074标准中，测试与其它评价活动一起，是整个生命周期活动的一个完整部分。

5.1.4 测试文档与工作产品[Bei90:c13s5; Kan99:c12; Per95:c19; Pf101:c9s8.8] (IEEE829-98)

文档是测试过程形式化的一个完整部分，IEEE软件测试文档标准(IEEE829-98)为测试文档、文档间相互关系、文档与测试过程关系，提供了很好的描述。测试文档可能包括：测试计划、测试设计规格说明、测试过程规格说明、测试用例规格说明、测试日志、测试事件或问题报告。被测试的软件作为测试项文档。测试文档应该以与软件工程其它类型文档一样的质量水平来生成和连续更新。

5.1.5 内部与独立测试小组 [Bei90:c13s2.2-c13s2.3; Kan99:c15; Per95:c4; Pf101:c9]

测试过程的形式化可能涉及测试小组组织的形式化。测试小组可以由内部成员（即项目组，可能或不涉及软件构造）组成，也可以由外部成员组成，以带来无偏见的、独立的观点，或者同时由内部和外部成员组成。对成本、进度、涉及到的组织的成熟度级别和应用的关键性的考虑，会影响决策。

5.1.6 成本/工作量估计，其它过程度量 [Per95:c4, c21] (Per95: Appendix B; Pos96:p139-145; IEEE982.1-88)

管理人员常使用几个与测试耗费的资源、不同测试阶段的发现错误的相对有效性有关的度量，来控制和改进测试过程。这些测试度量可能覆盖这些方面：指定的测试用例数目、执行的测试用例数目、通过的测试用例数目、没通过的测试用例数目等。

测试阶段评价报告可以与原因分析组合在一起，以尽可能评价测试过程对于发现错误的有效性。这种评价可能与风险分析相关。还有，值得耗费在测试上的资源应该与应用的使用/关键性相称：不同的技术有不同的成本，因而产生产品可靠性的不同置信度。

5.1.7 终止 [Bei90:c2s2.4; Per95:c2]

必须作出多少测试就足够了和什么时候终止测试的决策。彻底性度量（如达到的代码覆盖率或功能完备性）以及错误密度估计、运行可靠性估计，可以提供有用的支持，但这些本身并不充分。决策也涉及到对残余失效带来的成本和风险的考虑，以及继续测试的成本。参见子主题1.2.1测试选择准则/测试适当性准则。

5.1.8 测试复用与测试模式 [Bei90:c13s5]

为了以有组织的、成本/效益比好的方式完成测试或维护，用于测试软件每个部分的手段必须系统地复用。这个测试材料的仓库必须处于软件配置管理的控制下，这样，软件需求或设计的变更可以反映到进行的测试的变更上。

用于在特定环境下的应用类型的测试解决方案，以及决策时的动机，形成了一个测试模式，它本身可以文档化，并在以后类似的项目中复用。

5.2 测试活动

在这个主题下，给出测试活动的一个简要总结，正如以下描述隐含的那样，测试活动的成功管理强烈依赖于软件配置管理过程。

5.2.1 计划 [Kan99:c12; Per95:c19; Pf101:c8s7.6] (IEEE829-98:s4; IEEE1008-87:s1-s3)

与项目管理的任何其它方面一样，必须对测试活动进行计划。测试计划的关键方面包括人员协调、可用的测试设施和设备（可能包括磁介质、测试计划和过程）的管理、对可能的不期望的结果的计划。如果维持了多于一个的软件基线，则主要的计划考虑是保证测试环境设置为适当配置需要的时间和工作量。

5.2.2 产生测试用例 [Kan99:c7] (Pos96:c2; IEEE1008-87:s4, s5)

测试用例的产生基于要完成的测试级别和特定的测试技术。测试用例应该在软件配置管理的控制之下，并包括每个测试的期望结果。

5.2.3 开发测试环境 [Kan99:c11]

用于测试的环境应该与软件工程工具兼容，它应该加快测试用例的开发和控制，以及期

望结果、脚本和其他测试材料的日志和恢复。

5.2.4 执行[Bei90:c13; Kan99:c11] (IEEE1008-87:s6, s7)

测试的执行应该科学实验的基本原理：测试中作的每件事都应该清楚地进行和记录，使其它人员可以重复结果。因此，测试应该根据用被测试软件的清晰定义的版本进行文档化的过程来完成。

5.2.5 测试结果评价[Per95:c20, c21] (Pos96:p18-20, p131-138)

必须评价测试结果，以确定测试是否成功。多数情况下，“成功”表示软件按期望运行，并且没有重大的非期望结果。并非所有的非期望结果必须是错误，而是被认为是简单的噪声。在消除一个失效前，需要进行分析 and 排错，以隔离、标识和描述失效。当测试结果特别重要时，需要召集一个正式的评审委员会来评价这些结果。

5.2.6 问题报告/测试日志[Kan99:c5; Per95:c20] (IEEE829-98:s9-s10)

测试活动应该记录到测试日志，以标识测试何时进行、由谁完成、测试的软件配置基础是什么，以及其它有关的标识信息。非期望的和不正确的测试结果应该记录到一个问题报告系统中，报告中的数据是以后进行排错和修改测试中观察到的失效的问题的基础。还有，异常没有分类为错误时，也应该记录到文档中，可能它们以后会比最初认为的更严重。测试报告也是变更管理请求过程（见软件配置管理知识域的子域3软件配置控制）的输入。

5.2.7 缺陷追踪[Kan99:c6]

测试中观察到的失效多数是由于软件中的错误或缺陷引起的。可以分析这些缺陷，以确认它们是什么时候引入软件的，它们是有余什么类型的错误造成的（定义不清的需求、不正确的变量声明、内存泄漏、程序语法错误等），它们是什么时候在软件中被首次发现的。缺陷追踪信息用于确定需要改进的软件工程的一些方面，以及前面的分析和测试的有效性。

主题与参考文献矩阵

	[Bec 02]	[Bei 90]	[Jor 02]	[Kan 99]	[Kan 01]	[Lyu 96]	[Per 95]	[Pfl 01]	[Zhu 97]
1软件测试基础									
1.1测试相关的术语									
测试和相关术语的定义		C1	C2			C2s2 .2			
故障与失效			C2			C2s2 .2	C1	C8	
1.2关键问题									
测试选择准则/测试适当性准则(测试停止标准)								C8s7 .3	S1.1
测试有效性/测试客观性		C1s1 .4					C21		
为标识缺陷而测试		C1		C1					

神喻问题		C1							
测试的理论和实践局限				C2					
不可达路径问题		C3							
可测试性		C3, c13							
1.3测试与其它活动的联系									
测试与静态分析技术		C1					C17		
测试与正确性证明、形式化验证		C1s5						C8	
测试与调试		C1s2.1							
测试与编程		C1s2.3							
测试与认证									
2测试级别									
2.1测试对象		C1	C13					C8	
单元测试		C1					C17	C8s7.3	
集成测试			C13, c14					C8s7.4	
系统测试			C15					C9	
2.2测试目标							C8	C9s8.3	
接收/认定测试							C10	C9s8.5	
安装测试							C9	C9s8.6	
α 和 β 测试				C13					
遵从性测试/功能测试/正确性测试				C7			C8		
通过测试达到可靠性和评价						C7		C9s8.4	
回归测试				C7			C11, c12	C9s8.1	
性能测试							C17	C9s8.3	
压力测试							C17	C9s8.3	
背对背测试									
恢复测试							C17	C9s8	

								. 3	
配置测试				C8				C9s8 . 3	
可用性测试							C8	C9s8 . 3	
测试驱动的开发	III								
3测试技术									
3.1基于测试人员的直觉和经验									
特定的测试				C1					
探索性测试					C3				
3.2基于规格说明									
等价类划分			C7	C7					
边值分析			C6	C7					
决策表		C10s 3						C9	
基于有限状态机		C11	C8						
基于形式化规格说明									S2. 2
随机测试		C13		C7					
3.3基于代码的测试									
基于控制流的准则		C3	C10					C8	
基于数据流的准则		C5							
基于代码测试的参考模型		C3	C5						
3.4基于错误的测试									
错误猜测				C7					
变异测试							C17		S3. 2 , s3. 3
3.4基于使用的测试									
运行大纲			C15			C5		C9	
软件可靠性加固测试						C6			
3.6基于应用本质的测试									
面向对象的测试			C17					C8s7 . 5	

基于组件的测试									
基于Web的测试									
GUI测试			C20						
并发程序的测试									
协议遵从性测试									
分布式系统的测试									
实时系统的测试									
3.7测试技术的选择和组合									
功能性与结构性		C1s2 .2	C1, c 11s1 1.3				C17		
确定性与随机性									
4测试相关的度量									
4.1被测试程序的评价									
辅助测试计划和设计的程序度量		C7s4 .2	C9						
故障的类型、分类和统计		C2	C1					C8	
错误密度							C20		
寿命测试,可靠性评价								C9	
可靠性增长模型						C7		C9	
4.2评价已完成的测试									
覆盖/彻底性度量			C9					C8	
错误种植								C8	
变异分值									S3.2 , s3. 3
不同技术的比较与相对效果			C8, c 11				C17		S5
5测试过程									
5.1实际考虑									
态度/无私编程		C13s 3.2						C8	
测试指导	III				C5				
测试过程管理							C1-c 4	C9	
测试文档和工作产品		C13s 5		C12			C19	C9s8 .8	

内部与独立测试小组		C13s 2.2, c1s2 .3		C15			C4	C9	
成本/效益估算和其它过程度量							C4, c 21		
终止		C2s2 .4					C2		
测试复用与测试模式		C13s 5							
5.2测试活动									
计划				C12			C19	C87s 7.6	
测试用例产生				C7					
测试环境开发				C11					
执行		C13		C11					
测试结果评价							C20, c21		
问题报表/测试日志				C5			C20		
缺陷追踪				C6					

软件测试的推荐参考文献

- [Bec02] K. Beck, "Test-Driven Development by Example," Addison-Wesley, 2002.
- [Bei90] B. Beizer, "Software Testing Techniques," International Thomson Press, 1990, Chap. 1-3, 5, 7s4, 10s3, 11, 13.
- [Jor02] P. C. Jorgensen, "Software Testing: A Craftsman's Approach," Second Edition, CRC Press, 2004, Chap. 2, 5-10, 12-15, 17, 20.
- [Kan99] C. Kaner, J. Falk and H. Q. Nguyen, "Testing Computer Software," Second ed: Wiley, 1999, Chap. 1, 2, 5-8, 11-13, 15.
- [Kan01] C. Kaner, J. Bach and B. Pettichord, *Lessons Learned in Software Testing*: Wiley Computer Publishing, 2001.
- [Lyu96] M. R. Lyu, "Handbook of Software Reliability Engineering," Mc-Graw-Hill/IEEE, 1996, Chap. 2s2.2, 5-7.
- [Per95] W. Perry, "Effective Methods for Software Testing," Wiley, 1995, Chap. 1-4, 9, 10-12, 17, 19-21.
- [Pfl01] S. L. Pfleeger, "Software Engineering: Theory and Practice," Second ed: Prentice-Hall, 2001, Chap. 8, 9.
- [Zhu97] H. Zhu, P. A. V. Hall and J. H. R. May, "Software Unit Test Coverage and Adequacy," *ACM Computing Surveys*, vol. 29, iss. 4, 366-427 (Sections 1, 2.2, 3.2, 3.3), Dec., 1997

附录A 深入阅读的参考文献

- (Bac90) R. Bache and M. Müllerburg, "Measures of Testability as a Basis for Quality

- Assurance," *Software Engineering Journal*, vol. 5, 86-92, March, 1990
- (Bei90) B. Beizer, "Software Testing Techniques," Second ed: International Thomson Press, 1990.
- (Ber91) G. Bernot, M. C. Gaudel and B. Marre, "Software Testing Based On Formal Specifications: a Theory and a Tool," *Software Engineering Journal*, 387-405, Nov., 1991
- (Ber96) A. Bertolino and M. Marrè, "How many paths are needed for branch testing?," *The Journal of Systems and Software*, vol. 35, iss. 2, 95-106, 1996
- (Ber96a) A. Bertolino and L. Strigini, "On the Use of Testability Measures for Dependability Assessment," *IEEE Transactions on Software Engineering*, vol. 22, iss. 2, 97-108, Feb., 1996
- (Bin00) R. V. Binder, *Testing Object-Oriented Systems Models, Patterns, and Tools*: Addison-Wesley, 2000.
- (Boc94) G. V. Bochmann and A. Petrenko, "Protocol Testing: Review of Methods and Relevance for Software Testing," presented at ACM Proc. Int. Symposium on Software Testing and Analysis (ISSTA' 94), Seattle, Washington, USA, 1994
- (Car91) R. H. Carver and K. C. Tai, "Replay and testing for concurrent programs," *IEEE Software*, 66-74, March, 1991
- (Dic93) J. Dick and A. Faivre, "Automating The Generation and Sequencing of Test Cases From Model-Based Specifications," presented at FME'93: Industrial-Strength Formal Method, LNCS 670, 1993
- (Fran93) P. Frankl and E. Weyuker, "A formal analysis of the fault detecting ability of testing methods," *IEEE Transactions on Software Engineering*, vol. 19, iss. 3, 202-, March, 1993
- (Fran98) P. Frankl, D. Hamlet, B. Littlewood and L. Strigini, "Evaluating testing methods by delivered reliability," *IEEE Transactions on Software Engineering*, vol. 24, iss. 8, 586-601, August, 1998
- (Ham92) D. Hamlet, "Are we testing for true reliability?," *IEEE Software*, 21-27, July, 1992
- (Hor95) H. Horcher and J. Peleska, "Using Formal Specifications to Support Software Testing," *Software Quality Journal*, vol. 4, 309-327, 1995
- (How76) W. E. Howden, "Reliability of the Path Analysis Testing Strategy," *IEEE Transactions on Software Engineering*, vol. 2, iss. 3, 208-215, Sept., 1976
- (Jor02) P. C. Jorgensen, "Software Testing: A Craftsman's Approach," Second Edition, CRC Press, 2004.
- (Kan99) C. Kaner, J. Falk and H. Q. Nguyen, "Testing Computer Software," Second ed: Wiley, 1999.
- (Lyu96) M. R. Lyu, *Handbook of Software Reliability Engineering*: Mc-Graw-Hill/IEEE, 1996.
- (Mor90) L. J. Morell, "A Theory of Fault-Based Testing," *IEEE Transactions on Software Engineering*, vol. 16, iss. 8, 844-857, August, 1990
- (Ost88) T. J. Ostrand and M. J. Balcer, "The Category-Partition Method for Specifying and Generating Functional Tests," *Communications of ACM*, vol. 31, iss. 3, 676-686, June, 1988

- (Ost98) T. Ostrand, A. Anodide, H. Foster and T. Goradia, "A Visual Test Development Environment for GUI Systems," presented at ACM Proc. Int. Symposium on Software Testing and Analysis (ISSTA' 98), Clearwater Beach, Florida, USA, 1998
- (Per95) W. Perry, *Effective Methods for Software Testing*: Wiley, 1995.
- (Pfl101) S. L. Pfleeger, "Software Engineering: Theory and Practice," Second ed: Prentice-Hall, 2001, Chap. 8, 9.
- (Pos96) R. M. Poston, *Automating Specification-based Software Testing*: IEEE, 1996.
- (Rot96) G. Rothermel and M. J. Harrold, "Analyzing Regression Test Selection Techniques," *IEEE Transactions on Software Engineering*, vol. 22, iss. 8, 529-, Aug., 1996
- (Sch94) W. Schütz, "Fundamental Issues in Testing Distributed Real-Time Systems," *Real-Time Systems Journal*, vol. 7, iss. 2, 129-157, Sept., 1994
- (Voa95) J. M. Voas and K. W. Miller, "Software Testability: The New Verification," *IEEE Software*, 17-28, May, 1995
- (Wak99) S. Wakid, D. R. Kuhn and D. R. Wallace, "Toward Credible IT Testing and Certification," *IEEE Software*, 39-47, July-August, 1999
- (Wey82) E. J. Weyuker, "On Testing Non-testable Programs," *The Computer Journal*, vol. 25, iss. 4, 465-470, 1982
- (Wey83) E. J. Weyuker, "Assessing Test Data Adequacy through Program Inference," *ACM Trans. On Programming Languages and Systems*, vol. 5, iss. 4, 641-655, October, 1983
- (Wey91) E. J. Weyuker, S. N. Weiss and D. Hamlet, "Comparison of Program Test Strategies," presented at Proc. Symposium on Testing, Analysis and Verification TAV 4, Victoria, British Columbia, 1991
- (Zhu97) H. Zhu, P. A. V. Hall and J. H. R. May, "Software Unit Test Coverage and Adequacy," *ACM Computing Surveys*, vol. 29, iss. 4, 366-427, Dec., 1997

附录B 有关标准

- (IEEE610.12-90) IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.
- (IEEE829-98) IEEE Std 829-1998, *Standard for Software Test Documentation*: IEEE, 1998.
- (IEEE982.1-88) IEEE Std 982.1-1988, *IEEE Standard Dictionary of Measures to Produce Reliable Software*: IEEE, 1988.
- (IEEE1008-87) IEEE Std 1008-1987 (R2003), *IEEE Standard for Software Unit Testing*: IEEE, 1987.
- (IEEE1044-93) IEEE Std 1044-1993 (R2002), *IEEE Standard for the Classification of Software Anomalies*: IEEE, 1993.
- (IEEE1228-94) IEEE Std 1228-1994, *Standard for Software Safety Plans*: IEEE, 1994.
- (IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes, vol. IEEE, 1996.

////////////////////////////////////

第6章 软件维护

术语和缩写

CMMi: Capability Maturity Model Integration能力成熟度模型集成

ICSM: International Conference on Software Maintenance软件维护国际会议

SCM: Software Configuration Management软件配置管理

SQA: Software Quality Assurance软件质量保证

V&V: Verification and Validation验证和确认

Y2K: Year 2000 2000年

引言

软件开发工作的结果是交付满足用户需求的软件产品。相应地，软件产品必须变更和演化。一旦投入运行，就会发现缺陷，运行环境可能会变化，用户会提出新的需求。生命周期的维护阶段从随后的保修时期或实现后的支持交付开始，但维护活动发生的更早。软件维护是生命周期的一个完整部分，但是，在历史上，它却没有得到与其它阶段一样的重视。在历史上，多数组织中，软件开发的地位比软件维护高得多。但目前发生了变化，各个组织正在通过让软件尽可能久地运行，以减少软件开发的投资。由于2000年问题，引起了对软件维护阶段的重大关注，而且，开放源码范例进一步加强了对于维护由其它人员或组织开发的软件的关注。

在本指南中，软件维护定义为需要提供好的成本/效益的软件支持的活动的全体。这些活动包括在交付前完成的活动，以及交付后阶段完成的活动。交付前完成的活动包括交付后运行的计划、可维护性计划、迁移活动的后勤决策。交付后活动包括软件修改、培训、帮助材料的操作和接口。

软件维护知识域与软件工程所有其它方面都相关，因此，这个知识域描述与本指南所有其它章节都有链接。

软件维护主题的主题分解结构

软件维护知识域的主题分解结构如图1所示。

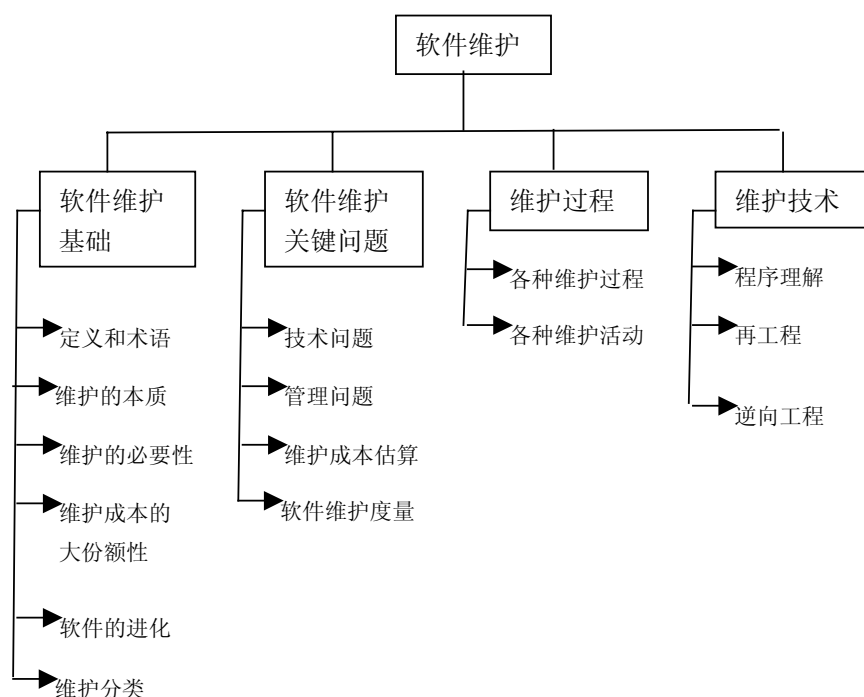


图1 软件维护知识域主题的分解结构

1 软件维护基础

第一节介绍形成理解软件维护的作用和范围的基础的概念和术语，这个主题提供了各个定义，强调为什么需要维护。软件维护的类别对于理解其含义非常重要。

1.1 定义和术语[IEEE1219-98:s3.1.12; IEEE12207.0-96:s3.1, s5.5; ISO14764-99:s6.1]

在IEEE软件维护标准IEEE1219中，将软件维护定义为交付后，修改软件产品以更正错误、改进性能或其他属性、或适应修改后的环境。这个标准也论述了软件产品交付前的维护活动，但是在附录中给出的。

IEEE/EIA 12207软件生命周期过程标准将维护描述为生命周期过程的一个主要过程之一，并将维护描述为软件产品经历“由于一个问题或改进的需要而修改代码和相关文档，目标是修正现有的软件产品并保留其完整性”的过程。ISO/IEC 14764软件维护国际标准对软件维护的定义与IEEE/EIA 12207相同，并强调了交付前的维护方面，例如，计划。

1.2 维护的本质[Pf101:ch1s11.2]

软件维护在软件产品的整个运行生命周期维持软件产品，对于修改要记录到日志中，并能追踪，要确定建议的修改的影响范围，然后修改代码和相关的文档，进行测试，最后发布新的软版本。还要向用户提供培训和日常支持。Pfleeger在[Pf101]指出，“维护（比开发）具有更广的范围，需要更多的追踪和控制”。

在IEEE/EIA 12207中，将维护者定义为一个完成维护活动的组织[IEEE12207.0-96]。在这个知识域中，有时，这个术语指的是完成这些活动的个人，这是相对于开发人员而言。IEEE/EIA 12207标识了软件维护的主要活动：过程实现、问题与修改分析、修改的实现、维护评审/接收、迁移、退出运行。这些活动在主题3.2维护活动中讨论。

维护者可以向开发人员学习软件的知识，与开发人员接触并尽早参与，可以减少维护工作量。有时，无法与软件工程师接触，或者他们参与了其它项目，这对维护者是一个巨大的挑战。维护者必须获得开发的产品、代码、文档，以立即支持它们，并在软件生命周期中进化和维护它们。

1.3 维护的必要[Pf101:c11.s11.2; Pig97:c2s2.3; Tak97:c1]

需要维护来保证软件持续地满足用户需求。维护对于使用任何软件生命周期模型（如螺旋式）开发的软件都是必要的。系统由于更正性或非更正性软件行动而变更，必须进行维护，以便：（1）更正错误；（2）改进设计；（3）实现（性能的）增强；（4）与其它系统接口；（5）改编程序，以便能使用不同的硬件、软件、系统特征、电信设施；（6）迁移遗留软件；（7）让软件退出运行。

根据Pfleeger[Pf101]，维护者的活动有4个关键特征：（1）维持对软件的日常功能的控制；（2）维持对软件修改的控制；（3）优化现有功能；（4）阻止软件性能下降到不可接受的水平。

1.4 维护成本的大额性[Abr93:63-90; Pf101:c11s11.3; Pig97:c3;

Pre01:c30s2.1, c30s2.2]

维护消耗了软件生命周期财力资源的大部分。对于软件维护的一个普遍看法是它仅仅是更正错误，但是，多年来的研究和调查表明，软件维护工作量的大部分，超过80%，用于在非更正性活动上[Abr93, Pig97, Pre01]。Jones在(Jon91)中描述了软件维护管理人员在其管理报告中经常将增强和更正聚集在一起的方式。将增强请求包括在问题报告中，是由于更正成本高的误解，理解了软件维护的分类，有助于理解软件维护成本的结构。还有，理解影响一个系统中可维护性的因素，也有助于容忍成本。Pfleeger在[Pf101]中给出了一些影响软件维护成本的技术性和非技术性因素：（1）应用类型；（2）软件新奇性；（3）软件维护人员的可获得性；（4）软件生命长度；（5）硬件特征；（6）软件设计、构造、文档和测试的质量。

1.5 软件的进化[Art88:cls1.0, s1.1, s1.2, c11, s1.1, s1.2; Leh97:108-124], (Bel72)

Lehman在1969年首次指出软件维护和系统进化，在20年中，他的研究导致了6个“进化律的形成”[Leh97]。关键的发现包括：维护是进化性的开发，维护决策由对于系统（和软件）随时间发生的情况的理解来辅助。其他人指出，维护是持续的开发，只是有一个额外的输入（或约束）：现存的大型软件永远不会完备，需要连续进化。随软件的进化，如果不采取行动来降低其复杂性，软件将越来越复杂。

一些软件展示了有规律的行为和趋势，并且可以度量，已经有人试图开发预测性模型来估计维护工作量，这样，就开发出了有用的管理工具 [Art88], (Bel72)。

1.6 维护的类别[Art88:cls1.2; Lie78; Dor02:vlc9s1.5;

IEEE1219-98:s3.1.1, s3.1.2, s3.1.7, A.1.7; ISO14764-99:s4.1, s4.3, s4.10, s4.11, s6.2; Pig97:c2s2.3]

Lientz和Swanson最初定义了3类维护：更正性、适应性、完善性[Lie78; IEEE 1219-98]。后来在ISO/IEC 14764软件工程标准—软件维护中进行了修改，以包括4类维护：（1）更正性维护：软件产品交付后进行的反应性修改，以更正发现的问题；（2）适应性维护：软件产品交付后进行的修改，以保持软件产品能在变化后或变化中的环境中可以继续使用；（3）完善性维护：软件产品交付后进行的修改，以改进性能和可维护性；（4）预防性维护：软件产品交付后进行的修改，以在软件产品中的潜在错误成有实际错误前，检测和更正它们。ISO/IEC 14764将适应性和完善性维护分类为增强，并将更正性和预防性维护分类为更正，如表1所示。如果安全性特别重要，软件产品中最常进行的维护是最新的预防性维护类别。

表1 软件维护分类

	更正	增强
前瞻性	预防性	完善性
反应性	更正性	适应性

2 软件维护的关键问题

为确保软件的有效维护，必须处理大量关键的问题。软件维护向软件工程师提供了独特的技术性和管理性挑战，这一点很重要。一个例子是，在不是软件工程师开发的有500K行源代码的软件中，试图发现一个错误，类似地，与软件开发人员竞争资源也是经常发生的事。计划未来的发布版本的同时，要为下一个发布版本编码，并为当前版本发送出紧急补丁，这些也是挑战。下面给出与软件维护有关的技术性和管理性问题，它们按主题标题聚集在一起：技术性问题、管理性问题、成本估计、度量。

2.1 技术性问题

2.1.1 有限的理解[Dor02:vlc9s1.11.4; Pf101:c11s11.3; Tak97:c3]

有限的理解指的是，软件工程师能以多快的速度理解在软件中何处能作出变更或更正，且这个软件不是其本人开发的。研究指出，大约40%—60%的维护工作量用于理解要维护的软件，这样，软件理解的主题对于软件工程师有极大的兴趣。在面向文本的表示和源代码中，理解更加困难，例如，如果变更没有记录在文档中，而且没有开发人员来解释时，通过软件的发布/版本来追踪其进化，就很困难，事情也通常如此。这样，软件工程师最初可能对软件只有一个有限的理解，必须作很多工作来弥补这一点。

2.1.2 测试[Art88:c9; Pf101:c11s11.3]

在软件的一个主要部分重复全部测试的成本在时间上和金钱上，都很大。对一个软件或组件进行回归测试或选择性重测试，以验证修改没有引起不需要的后果，这对于维护很重要。还有，找出测试的时间也很困难。当不同的维护小组同时处理不同的问题时，如何协调也是一个挑战[P1f01]。当软件运行关键功能时，可能无法在脱机状态下进行测试。软件测试知识域在子主题2.2.6回归测试中，对这个问题提供了附加的信息和参考文献。

2.1.3 影响（范围）分析[Art88:c3; Dor02:vlc9s1.10; Pf101: c11s11.5]

影响分析描述如何以较好的成本/效益进行现有软件的变更的完备分析。维护者必须具有关于软件结构和内容的详细知识[Pf101]，他们使用这些知识来完成影响分析，以标识软件变更请求将影响的所有系统和软件产品，并作出完成这个变更需要的资源的估计[Art88]。另外，要确定作出变更的风险。变更请求有时叫修改请求（modification request, MR），必须分析并将其翻译为软件术语[Dor02]。在变更请求进入软件配置管理过程后，就可进行变更。Arthur在[Art88]中指出，影响分析的目标是：（1）确定变更的范围，以便计划和实现这个变更；（2）对实现变更需要的资源作出精确的估计；（3）分析请求的变更的成本/效益；（4）与其他人员交流给定变更的复杂性。

通常使用问题的严重性来决定问题如何、何时解决。软件工程师然后标识受影响的组件，提出几个潜在的解决方案，建议一个最佳的行动路线。

在软件设计时就考虑可维护性，可以加速影响分析进程。可以在软件配置知识域中发现更多的信息。

2.1.4 可维护性[IS014764-99:s6.8s6.8.1; Pf101:c9s9.4; Pig97:c16]

如何在开发中就促进和遵循可维护性问题？IEEE在[IEEE610.12-90]中将可维护性定义为软件为满足规定需求而被维护、增强、适应修改、更正的难易程度。ISO/IEC将可维护性定义为质量的一个特征(ISO9126-01)。为降低维护成本，在软件开发活动中，必须说明、评审和控制可维护性的子特征。如果成功作到这点，就能改进软件的可维护性。由于在软件开发过程中，可维护性子特征不是重要的关注点，因此，很难达到可维护性。开发人员要关注其它事情，往往忽略维护者的需求，这通常导致，事实上也如此，缺乏系统性文档，这是程序理解和影响分析中困难的主要原因。人们已经注意到，系统性和成熟的过程、技术和工具，有助于增强系统的可维护性。

2.2 管理问题

2.2.1 与组织目标的一致[Ben00:c6sa; Dor02:v1c9s1.6]

组织的目标描述了如何表现软件维护活动的投资回报。Bennett在[Ben00]中指出，“最初的软件开发通常是基于项目的，具有确定的时间尺度和预算，主要重点是及时交付并在预算内，以满足用户需要。与之相反，软件维护的目标通常是尽可能地延长软件的寿命，另外，它也可以被满足用户对软件更新和增强的要求来驱动。在两种情况下，投资回报不很清楚，这样，高级管理层的观点是：软件维护是消耗大量资源、对组织没有清晰的可量化利润的主要活动”。

2.2.2 人员组织[Dek92:10-17; Dor02:v1c9s1.6; Par86:c4s8-c4s11](Lie81)

人员组织指的是如何吸引和保持软件维护人员，维护通常不被认为是一种有魅力的工作。基于调查数据，Deklava提出了与人员组织有关的问题的列表[Dek92]，结果是，软件维护人员通常被认为是“二等公民”(Lie81)，因而士气低落[Dor02]。

2.2.3 过程[Pau93; Ben00:c6sb; Dor02:v1c9s1.3]

软件过程是一组人们用来开发和维护软件及相关产品的活动、方法、实践和变换[Pau93]。在过程层次上，软件维护活动与软件开发有许多共同之处（例如，两种活动中，软件配置管理都很关键）[Ben00]。维护也需要软件开发中没有出现的几种活动（参加3.2节独特活动），这些活动对管理提出了挑战[Dor02]。

2.2.4 维护组织结果方面[Pf101:c12s12.1-c12s12.3; Par86:c4s7;Pig97:c2s2.5; Tak97:c8]

组织结果方面描述了如何标识负责软件维护的组织和/或功能。一旦软件投入运行，就没有必要指定开发软件的小组来维护它。

在确定将软件维护功能放置在何处时，软件工程组织可以保持与原始开发者在一起，或转移到单独的小组（或维护者）。通常，选择维护者方式应保证软件正常运行和进化，以满足不断变化的用户需求。由于每种方式都有优缺点[Par86,Pig97]，决策时，应该具体问题具体分析，重要的是，不管组织的结构如何，应将软件维护责任委派和分配给一个小组或个人[Pig97]。

2.2.5 外包[Dor02:v1c9s1.7; Pig97:c9s9.1,s9.2], (Car94;McC02)

将维护外包已经成为一个潮流，大型公司将整个软件系统的工作进行外包，包括软件维护在内。通常的是，选择的外包部分是任务不太关键的软件，因为公司不愿意失去其核心业

务中使用的软件的控制。Carey在(Car94)中指出,一些公司只有在找到维持战略性控制的方法后,才会外包。但是,如何度量控制却很难。外包的一个主要挑战是确定需要的维护服务的范围和合同细节。McCracken在(McC02)中指出,50%的外包商在没有任何清晰的服务层协议就提供服务,外包公司花费几个月来评定软件,再签订合同[Dor02]。另一个已标识的挑战是如何将软件传递给外包商[Pig97]。

2.3 维护成本估算

软件工程师必须了解前面讨论的不同的软件维护类别,以能处理软件维护成本估算问题。对于计划而言,成本估算是软件维护的一个重要方面。

2.3.1 成本估算[Art88:c3; Boe81:c30; Jon98:c27; Pfl01:c11s11.3;Pig97:c8]

在子主题2.1.3影响范围中已经提到,影响分析要标识所有受软件变更请求影响的系统和软件产品,并估算完成变更需要的资源[Art88]。

维护成本的估算受到很多技术和非技术因素的影响。ISO/IEC14764 指出,“估算软件维护资源的两个最普遍的方法是:使用参数模型和使用经验”[ISO14764-99:s7.4.1]。通常,要组合使用几种方法。

2.3.2 参数模型[Ben00:s7; Boe81:c30; Jon98:c27; Pfl01:c11s11.3]

人们在应用参数成本模型来估算软件维护方面,做了一些工作[Boe81, Ben00]。需要来自过去项目的数据,才能使用模型,这一点很重要。Jones在[Jon98]中讨论了估算成本的各个方面,包括功能点(IEEE14143.1-00),并为维护估算提供了详细的一章。

2.3.3 经验[ISO14764-00:s7, s7.2, s7.2.1, s7.2.4; Pig97:c8;Sta94]

专家的判断(例如,使用Delphi技术)形式的经验、类比和任务的结构分解,是用于增强来自参数模型数据的集中方法,很明显,维护估算的最好方法是组合经验 数据和经验,这些数据应该作为程序度量的结构来提供。

2.4 软件维护度量[IEEE1061-98:A.2; Pig97:c14s14.6; Gra87 ; Tak97:c6s6.1-c6s6.3]

Grady和Caswell在[Gra87]中讨论了建立公司范围内的软件度量程序,并描述了软件维护度量的表格和数据收集方法。实用软件和系统度量(Practical Software and Systems Measurement,PSM)项目描述了一个由许多组合子使用的问题驱动度量过程,很实用[McG01]。有许多对于所有感兴趣人员都普遍的软件度量,软件工程研究所(SEI)标识了以下度量:规模、工作量、进度和质量[Pig97]。这些度量为维护者提供了一个良好的起点。对于过程和产品度量的讨论在软件工程过程知识域中,软件度量程序在软件工程管理知识域中描述。

2.4.1 特定的度量[Car90:s2-s3; IEEE1219-98:Table3; Sta94:239-249]

Abran在[Abr93]中给出了比较不同内部维护组织的内部基准技术,维护者必须确定什么度量适合于其组织机构。[IEEE1219-98; ISO9126-01; Sta94]提出了更特定于软件维护度量程序的一些度量,这个列表包括可维护性的4个子特征的许多度量:(1)可分析性:度量维护者的工作量或耗费在试图诊断失效的缺陷或原因上的资源,或耗费在标识要修改部分的资源;(2)可变更性:度量与实现特定修改相关的维护者的工作量;(3)稳定性:度量软件的非期望行为,包括测试中遇到的非期望行为;(4)可测试性:度量维护者或用户在试图测试修改后软件上的工作量。

使用已有的商业工具,可以得到软件可维护性的某些度量(Lag96; Apr00)。

3 维护过程

维护过程子主题提供了勇于实现软件维护过程的参考文献和标准。维护活动主题将维护与开发分开，指出其与其它软件工程活动的关系。

软件工程过程的需要已经很好地文档化，CMMi模型可应用于软件维护过程，并与开发者的过程类似[SEI01]。文献(Apr03, Nie02, Kaj01)描述了处理软件维护的特有过程的软件维护能力成熟度模型。

3.1 维护过程[IEEE1219-98:s4; ISO14764-99:s8; IEEE12207.0-96:s5.5; Par86:c7s1; Pig97:c5; Tak97:c2]

维护过程为这些活动提供必须的活动和详细的输入输出，并由IEEE 1219和ISO/IEC 14764软件维护标准描述。软件维护标准(IEEE 1219)中描述的维护过程模型从交付后阶段的软件维护要求开始，讨论了维护计划等问题，这个过程见图2。

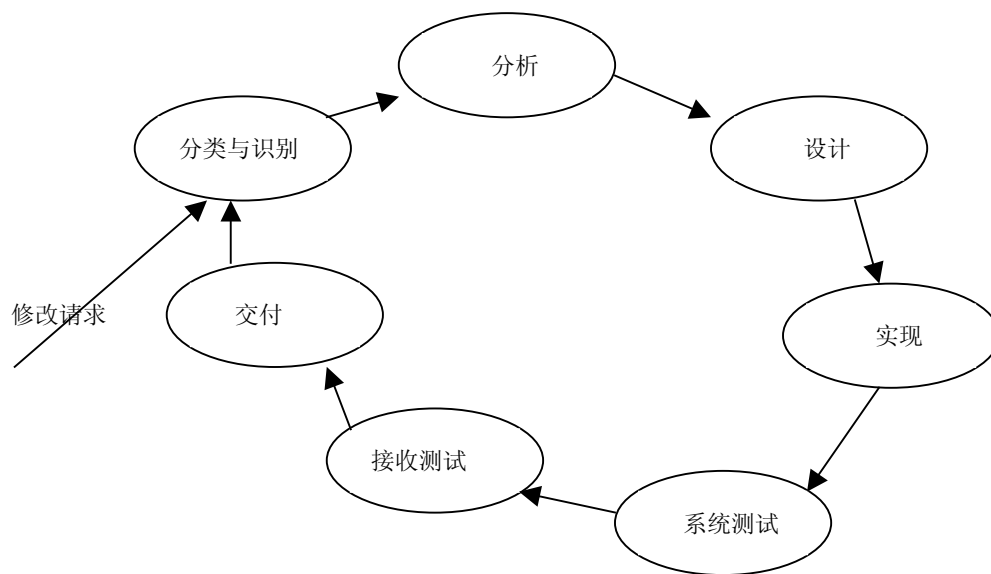


图2 IEEE1219-98 维护过程的各种活动

ISO/IEC 14764 [ISO14764-99]是IEEE/EIA 12207.0-96维护过程的精心实现，ISO/IEC维护过程的活动类似于IEEE，只是它们聚集方式有点不同。ISO/IEC开发的维护活动如图3所示。

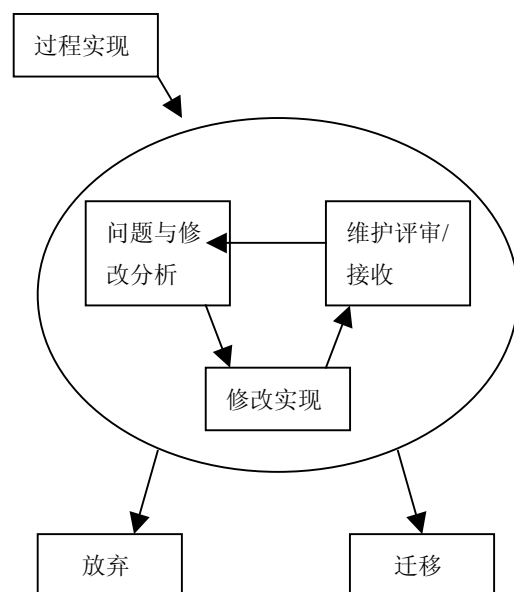


图 3 ISO/IEC14764-00 软件维护过程

ISO/IEC 14764 的每个主要软件维护活动进一步分解为任务，如下：（1）过程实现；（2）问题和修改分析；（3）修改实现；（4）维护评审/接收；（5）迁移；（6）软件退出运行。

Takang 和 Grubb 在 [Tak97] 中提供了维护过程模型进化到 IEEE 和 ISO/IEC 过程模型的历史。Parikh 在 [Par86] 中也给出了一般维护过程的概述。最近，出现了敏捷方法，它提倡轻量级过程，这个需求来自不断增长的快速转变的维护服务要求，在 (Poo01) 中，给出了极限维护的一些实验结果。

3.2 维护活动

前面已经指出，许多维护活动与软件开发中的活动类似，维护者要完成分析、设计、编码、测试和文档。必须象在开发中一样，在活动中追踪需求，在基线改变时，要更新文档。ISO/IEC 14764 推荐，当维护者引用一个类似的开发过程时，必须修改这个过程，以满足特定的需要 [ISO14764-99:s8.3.2.1, 2]，但是对于软件维护，一些活动涉及软件维护特有的过程。

3.2.1 特有活动 [Art88:c3; Dor02:vlc9s1.9.1; IEEE1219-98:s4.1, s4.2; ISO14764-99:s8.2.2.1, s8.3.2.1; Pf101:c11s11.2]

软件维护有许多特有的过活动和实践，例如：（1）迁移：是一个受控的和协调的活动序列，迁移中，软件从开发者逐步传递到维护者 [Dek92, Pig97]；（2）修改请求的接收/拒绝：在一定规模/工作量/复杂性上的修改请求可能被维护者拒绝，并重新转移给开发者 [Dor02], (Apr01)；（3）修改请求和问题报告的帮助桌面：这是一个终端用户支持功能，可以触发修改请求的评定、优先次序和成本分配 [Ben00]；（4）影响分析（详细内容参见 2.1.3）；（5）软件支持：帮助和向关心某个请求的用户得到信息（例如，业务规则、确认、数据含义和特定的请求/报告）(Apr03)；（6）服务级别协议（Service Level Agreements, SLA）和专门（特定领域的）维护合同：这是维护者的责任 (Apr01)。

3.2.2 支持活动 [IEEE1219-98:A.7, A.11; IEEE12207.0-96:c6, c7; ITI01;

Pig97:c10s10.2, c18] ; (Kaj01)

维护者可能也要完成一些支持活动，如软件维护计划、软件配置管理、验证和确认、软件质量保证、评审、审计和用户培训。还需要另一种支持活动：维护培训 [Pig97; IEEE12207.0-96] (Kaj01)。

3.2.3 维护计划活动 [IEEE1219-98:A.3; ISO14764-99:s7; ITI01; Pig97:c7, c8]

软件维护的一个重要活动是计划，维护者必须处理与大量计划观点相关的问题：（1）业务规划（组织层次上的）；（2）维护计划（迁移层次上的）；（3）发布/版本计划（软件层次上的）；（4）单个软件变更请求计划（请求层次上的）。

在单个请求层次上，计划是在影响分析中完成的（参见子主题2.1.3影响分析）。发布/版本计划活动要求维护者完成：（1）收集单个请求的可获得日期；（2）与用户达成对随后的发布/版本的一致；（3）标识潜在的冲突，提出替代方案；（4）评定给定发布的风险，并作出出现问题时的回收计划；（5）通知所有干系人。

软件开发项目一般持续几个月到几年，而维护阶段通常持续多年。估算资源是维护计划的关键因素，这些资源应该包括到开发人员的项目计划预算中。软件维护计划应该从决定开发一个新系统开始，并考虑质量目标 (IEEE1061-98)。应该开发一个概念文档，然后作出维护计划。维护的概念文档应该包括 [ISO14764-99:s7.2]：（1）软件维护范围；（2）软件维护过程的修改；（3）软件维护组织的标识；（4）软件维护成本估算。

下一步是开发一个相应的软件维护计划，这个计划应该在软件开发中准备完毕，并应该指明用户如何请求软件修改或报告问题。IEEE 1219 [IEEE1219-98]和ISO/IEC 14764论述了软件维护计划 [Pig97]。

最后，在最高层次上，维护组织必须进行业务规划活动（预算、财务和人力资源），如同组织中所有其它部门一样。完成这些工作需要的管理知识可以参见软件工程相关学科一章。

3.2.4 软件配置管理 [Art88:c2, c10; IEEE1219-98:A.11; IEEE12207.0-96:s6.2; Pfl01:c11s11.5; Tak97:c7]

IEEE软件维护标准IEEE 1219 [IEEE1219-98]中，将软件配置管理描述为维护过程的关键因素。软件配置管理过程为软件产品的标识、授权、实现和发布需要的每一步提供验证、确认和审计。

简单地追踪修改请求或问题报告是不够的，必须控制软件产品并对其作的任何变更。通过实现和强制一个批准的软件配置管理过程，可以建立这种控制。软件配置管理知识域提供了软件配置管理的细节，讨论了提交、评价和批准软件变更请求的过程。软件维护的软件配置管理与软件开发的软件配置管理不同之处是：在运行的软件上必须控制的小变更的数量。软件配置管理过程的实现是：开发和遵循配置管理计划和操作过程。维护者要参与配置控制委员会，以确定下一个发布/版本的内容。

3.2.5 软件质量 [Art98:c7s4; IEEE12207.0-96:s6.3; IEEE1219-98:A.7; ISO14764-99:s5.5.3.2]

同样，简单地希望软件的维护可以提高软件质量也是不够的，必须进行计划并实现规定的过程，以支持维护过程。必须选择软件质量保证、验证和确认、评审和审计的活动和技术，于其它过程协同，以达到希望的质量等级。建议维护者适应性地修改软件开发过程、技术和可交付产品（如测试文档和测试结果） [ISO14764-99]。

详细内容参见软件质量知识域。

4 维护技术

这个子域介绍一些软件维护中普遍接受的技术。

4.1 程序理解[Arn92:c14; Dor02:vlc9s1.11.4; Tak97:c3]

程序员在阅读和理解程序上，花费大量时间，以便实现变更。代码浏览器是程序理解的关键工具。清晰和简明的文档有助于程序理解。

4.2 再工程[Arn92:c1, c3-c6; Dor02:vlc9s1.11.4; IEEE1219-98:B.2], (Fow99)

再工程定义为检查和修改软件，用新的形式重构软件。Dorfman和Thayer在[Dor02]中指出是修改的最激进（和昂贵）形式，其他人认为，再工程可用于少量的变更，通常不是用再工程来提高可维护性，而是用来替代老化的遗留软件。Arnold在[Arn92]中提供了一个综合性的主题概要，例如：与再工程有关的概念、工具和技术、实例研究、风险与效益。

4.3 逆向工程[Arn92:c12; Dor02:vlc9s1.11.3; IEEE1219-98:B.3; Tak97:c4, Hen01]

逆向工程是分析软件，标识软件的组件及其相互关系，以另一种形式或在更高的抽象层次上创建软件的表示的过程。逆向工程是被动的，它不改变软件，也不产生新软件。逆向工程的结果是来自源代码的调用图和控制流图。一种类型的逆向工程是重写文档，另一类型是恢复设计[Dor02]。重构（Refactoring）是程序变换，它重新组织程序而不改变其行为，也是一种逆向工程，目标是改进程序结构(Fow99)。

最后，在过去几年，数据逆向工程得到重视，它是从物理数据库中恢复逻辑模式。

主题参考文献矩阵

	[Abr93]	[Arn92]	[Art88]	[Ben00]	[Boe81]	[Car90]	[Dek92]	[Dor97]	[IEEE610.12-90]	[IEEE1061.98]
1 软件维护基础										
1.1 定义与术语	63-90			S4						
1.2 维护的本质										
1.3 维护的必要										
1.4 维护成本的大份额性	63-90									
1.5 软件的进化					C1s1.0-c1s1.2, c11s1.1, c11					

					s1.2					
1.6 维护 的类别	63-90				C1s1. 2	S5				V1c9s1. 5
2 软件维 护 的 关 键问题										
2.1 技术 性问题						S6c				
有 限 的 理解						S11.4				V1c9s1. 11.4
测试					C9					
影 响 分 析					C3	S10.1 -s10. 3				V1c9s1. 10.1-v1 c9s1.11 .3
可 维 护 性		S3				S5, s9 .3, s1 1.4				
2.2 管理 性问题										
与 组 织 目 标 的 协 调 一 致						S6a				V1c9s1. 6
人 员 雇 佣									10-17	V1c9s1. 6
过程						S6b				V1c9s1. 3
维 护 的 组 织 方 面						S6a, s 7				
外包						S7				V1c9s1. 7
2.3 维护 成 本 估 算										
成 本 估 算					C3	S7	C30			
参 数 模 型						S7	C30			
经验										
2.4 度量	63-90		A2			S9.1, s9.2, s10.1				

特 定 度 量						S9. 2		S2, s3		
3 维护过 程										
3.1 各种 维 护 过 程								S8		
3.2 维护 活动								S9. 1		
特 有 活 动	63-90				C3	S6b				V1c9s1. 9. 1
支 持 活 动										
维 护 计 划活动										
软 件 配 置管理					C2, c1 0	S2, s6 b, s9. 2, s10 . 1, s1 1. 4				
软 件 质 量	63-90				C7s4	S7, s8 , s9. 2 , s9. 3 , s11. 4, s11 . 5				
4 维护技 术										
4.1 程序 理解				C14		S11. 4				V1c9s1. 11. 4
4.2 再工 程				C1, c3 -c6		S9. 2, s11. 4 , s11. 5				V1c9s1. 11. 4
4.3 逆向 工程				C12		S9. 2, s10. 3 , s11. 2, s11 . 3, s1 1. 5				V1c9s1. 11. 3

	[IEEE 1219-	[IEEE 12207	[ISO1 4764-	[Jon9 8]	[Leh9 7]	[Par8 6]	[Pf10 1]	[Pi g97	[Pr e04	[St a94	[Tak9 7]
--	----------------	----------------	----------------	-------------	-------------	-------------	-------------	------------	------------	------------	-------------

	98]	. 0-96]	00]					]	]	]	
1 软件维 护基础											
1.1 定义 与术语	S3. 1. 12		S6. 1								
1.2 维护 的本质		S3, s5 . 5					C11s1 1. 2				
1.3 维护 的必要							C11s1 1. 2	C2s 2. 3			C1
1.4 维护 成本 的 大 份 额 性							C11s1 1. 3	C3	C31		
1.5 软件 的进化					108-1 24						
1.6 维护 的类别	S3. 1. 1, s3. 1. 2, s 3. 1. 7 , A1. 7		S4. 1, s4. 3, s4. 10 , s4. 1 1, s6. 2					C2s 2. 3			
2 软件维 护 的 关 键问题											
2.1 技术 性问题											
有 限 的 理解							C11s1 1. 3				C3
测试							C11s1 1. 3				
影 响 分 析							C11s1 1. 5				
可 维 护 性			S6. 8, s6. 8. 1				C9s9. 4	C16			
2.2 管理 性问题											
与 组 织 目 标 的 协 调 一 致											
人 员 雇 佣						C4s8- c4s11					

过程											
维 护 的 组 织 方 面						C4s7	C12s1 2.1-c 12s12 .3	C2s 2.5			C8
外包								C9s 9.1 -c9 s9. 2			
2.3 维护 成 本 估 算											
成 本 估 算				C27			C11s1 1.3	C8			
参 数 模 型				C27			C11s1 1.3				
经验			S7, s7 .2, s7 .2.1, s7.2. 4					C8			
2.4 度量								C14 s14 .6			C6s6. 1-c6s 6.3
特 定 度 量	Table 3									239 -24 9	
3 维护过 程											
3.1 各种 维 护 过 程	S4		S8			C7s1		C5			C2
3.2 维护 活动		S5.5									
特 有 活 动	S4.1- s4.2		S8.2. 2.1, s 8.3.2 .1				C11s1 1.2				
支 持 活 动	A7, A1 1	C6, c7						C10 s10 .2, c18			
维 护 计	A3		S7					C7,			

划活动								c8			
软件配置管理	A11	S6.2					C11s1 1.5				C7
软件质量	A7	S6.3	S5.5. 3.2								
4 维护技术											
4.1 程序理解											C3
4.2 再工程	B2										
4.3 逆向工程	B3										C4

软件维护的推荐参考文献

- [Abr93] A. Abran and H. Nguyenkim, "Measurement of the Maintenance Process from a Demand-Based Perspective," *Journal of Software Maintenance: Research and Practice*, vol. 5, iss. 2, 63-90, 1993.
- [Arn93] R. S. Arnold, *Software Reengineering*: IEEE Computer Society, 1993.
- [Art98] L. J. Arthur, *Software Evolution: The Software Maintenance Challenge*: John Wiley & Sons, 1988.
- [Ben00] K. H. Bennett, "Software Maintenance: A Tutorial," in *Software Engineering*, M. Dorfman and R. Thayer, Eds.: IEEE Computer Society Press, 2000.
- [Boe81] B. W. Boehm, *Software Engineering Economics*: Prentice-Hall, 1981.
- [Car90] D. N. Card and R. L. Glass, *Measuring Software Design Quality*: Prentice Hall, 1990.
- [Dek92] S. Dekleva, "Delphi Study of Software Maintenance Problems," presented at Proceedings of the International Conference on Software Maintenance, 1992.
- [Dor02] M. Dorfman and R. H. Thayer, Eds., "Software Engineering." (Vol. 1 & vol. 2), IEEE Computer Society Press, 2002.
- [Gra87] R. B. Grady and D. L. Caswell, *Software Metrics: Establishing A Company-Wide Program*. Englewood Cliffs NJ, USA: Prentice-Hall, 1987.
- [IEEE610.12-90] IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.
- [IEEE1061-98] IEEE Std 1061-1998, *IEEE Standard for a Software Quality Metrics Methodology*: IEEE, 1998.
- [IEEE1219-98] IEEE Std 1219-1998, *IEEE Standard for Software Maintenance*: IEEE, 1998.
- [IEEE12207.0-96] IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- [ISO9126-01] ISO/IEC 9126-1:2001, *Software Engineering-Product Quality-Part 1: Quality Model*: ISO and IEC, 2001.

- [IS014764-99] ISO/IEC 14764-1999, *Software Engineering-Software Maintenance*: ISO and IEC, 1999.
- [ITI01] IT Infrastructure Library, "Service Delivery and Service Support," Stationary Office, Office of Government of Commerce, 2001
- [Jon98] T. C. Jones, *Estimating Software Costs*: McGraw-Hill, 1998.
- [Leh97] M. M. Lehman, "Laws of Software Evolution Revisited," presented at EWSPT96, 1997
- [Lie78] B. Lienz, E. B. Swanson and G. E. Tompkins, "Characteristics of Applications Software Maintenance," *Communications of the ACM*, vol. 21, 1978
- [Par86] G. Parikh, *Handbook of Software Maintenance*: John Wiley & Sons, 1986.
- [Pfl01] S. L. Pfleeger, *Software Engineering: Theory and Practice*, Second ed: Prentice-Hall, 2001.
- [Pig97] T. M. Pigoski, *Practical Software Maintenance: Best Practices for Managing your Software Investment*, First ed: John Wiley & Sons, 1997.
- [Pre04] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, Sixth ed: McGraw-Hill, 2004.
- [SEI01] Software Engineering Institute, "Capability Maturity Model Integration, v1.1," CMU/SEI-2002-TR-002, ESC-TR-2002-002, December, 2001
- [Sta94] G. E. Stark, L. C. Kern and C. V. Vowell, "A Software Metric Set for Program Maintenance Management," *Journal of Systems and Software*, vol. 24, iss. 3, March, 1994
- [Tak97] A. Takang and P. Grubb, *Software Maintenance Concepts and Practice*: International Thomson Computer Press, 1997.

附录A 深入阅读的参考文献

- (Abr93) A. Abran, "Maintenance Productivity & Quality Studies: Industry Feedback on Benchmarking," presented at Proceedings of the Software Maintenance Conference, ICSM93, Montréal, 1993
- (Apr00) A. April and D. Al-Shurougi, "Software Product Measurement for Supplier Evaluation," presented at FESMA2000, Madrid, 2000
- (Apr01) A. April, J. Bouman, A. Abran and D. Al-Shurougi, "Software Maintenance in a Service Level Agreement: Controlling the Customer's Expectations," presented at European Software Measurement Conference, Heidelberg, Germany, 2001
- (Apr03) A. April, A. Abran and R. Dumke, "Software Maintenance Capability Maturity Model (SM-CMM): Process Performance Measurement," presented at 13th International Workshop on Software Measurement _IWSM2003, Montreal, 2003
- (Bas85) V. R. Basili, "Quantitative Evaluation of Software Methodology," presented at Proceedings First Pan-Pacific Computer Conference, 1985
- (Bel72) L. Belady and M. M. Lehman, "An introduction to growth dynamics," in *Statistical Computer Performance Evaluation*, W. Freiberger, Ed. New York: Academic Press, 1972.
- (Ben00) K. H. Bennett and V. T. Rajlich, "Software Maintenance and Evolution: A Roadmap," in *The Future of Software Engineering*, A. Finklestein, Ed. : ACM Press, 2000.

- (Bol95) C. Boldyreff, E. Burd, R. Hather, R. Mortimer, M. Munro and E. Younger, "The AMES Approach to Application Understanding: A Case Study," presented at Proceedings of the International Conference on Software Maintenance-1995, Los Alamitos, CA, 1995
- (Boo94) G. Booch and D. Bryan, *Software Engineering with Ada*, Third ed: Benjamin/Cummings, 1994.
- (Cap94) M. A. Capretz and M. Munro, "Software Configuration Management Issues in the Maintenance of Existing Systems," *Journal of Software Maintenance: Research and Practice*, vol. 6, iss. 2, 1994
- (Car92) J. Cardow, "You Can't Teach Software Maintenance!," presented at Proceedings of the Sixth Annual Meeting and Conference of the Software Management Association, 1992
- (Car94) D. Carey, "Executive round-table on business issues in outsourcing- Making the decision," *CIO Canada*, iss. June/July, 1994
- (Dor97) M. Dorfman and R. H. Thayer, Eds., "Software Engineering." IEEE Computer Society Press, 1997.
- (Dor02) M. Dorfman and R. H. Thayer, Eds., "Software Engineering." (Vol. 1 & vol. 2), IEEE Computer Society Press, 2002.
- (Fow99) M. Fowler and al, *Refactoring: Improving the Design of Existing Code*: Addison-Wesley, 1999.
- (Gra87) R. B. Grady and D. L. Caswell, *Software Metrics: Establishing A Company-Wide Program*. Englewood Cliffs NJ, USA: Prentice-Hall, 1987.
- (Gra92) R. B. Grady, *Practical Software Metrics for project Management and Process Management*: Prentice Hall, Englewood Cliffs, NJ 07632, 1992.
- (Jon91) C. Jones, *Applied Software Measurement*: McGraw-Hill, 1991.
- (Kaj01) M. Kajko-Mattson, "Motivating the Corrective Maintenance Maturity Model (Cm3)," presented at 7th International Conference on Engineering of Complex Systems, 2001
- (Kaj01a) M. Kajko-Mattson, S. Forssander and U. Olsson, "Corrective Maintenance Maturity Model: Maintainer's Education and Training," presented at International Conference on Software Engineering, 2001
- (Kho95) T. M. Khoshgoftaar, R. M. Szabo and J. M. Voas, "Detecting Program Module with Low Testability," presented at Proceedings of the International Conference on Software Maintenance-1995, Los Alamitos, CA, 1995
- (Lag96) B. Laguë and A. April, "Mapping for the ISO9126 Maintainability Internal Metrics to an industrial research tool," presented at SESS, Montréal, 1996
- (Leh85) M. M. Lehman and L. A. Belady, *Program Evolution: Processes of Software Change*. (London) Ltd.: Academic Press Inc., 1985.
- (Leh97) M. M. Lehman, "Laws of Software Evolution Revisited," presented at EWSPT96, 1997
- (Lie81) B. P. Lientz and E. B. Swanson, "Problems in application software maintenance," *Communications of ACM*, vol. 24, iss. 11, 763-769, 1981
- (McC02) B. McCracken, "Taking Control of IT Performance," presented at InfoServer LLC, Dallas, Texas, 2002
- (Nie02) F. Niessink, V. Clerk and H. v. Vliet, "The IT Capability Maturity Model,"

- release L2+3-0.3 draft, 2002, available at
<http://www.itservicecmm.org/doc/itscmm-123-0.3.pdf>
- (Oma91) P. W. Oman, J. Hagemeister and D. Ash, "A Definition and Taxonomy for Software Maintainability," University of Idaho, Software Engineering Test Lab, Technical Report, 91-08 TR November, 1991
- (Oma92) P. Oman and J. Hagemeister, "Metrics for Assessing Software System Maintainability," presented at Proceedings of the International Conference on Software Maintenance-1992, Los Alamitos, CA, 1992
- (Pig93) T. M. Pigoski, "Maintainable Software: Why You Want It and How to Get It," presented at Proceedings of the Third Software Engineering Research Forum-November 1993, University of West Florida Press, 1993
- (Pig94) T. M. Pigoski, "Software Maintenance," in *Encyclopedia of Software Engineering*. New York, NY: John Wiley & Sons, 1994.
- (Pol03) M. Polo, M. Piattini and F. R. (editors), Eds., "Advances in Software Maintenance Management: Technologies and Solutions.." Hershey, PA, Idea Group Publishing, 2003.
- (Poo00) C. Poole and W. Huisman, "Using Extreme Programming in a Maintenance Environment," *IEEE Software*, iss. November/December, 42-50, 2001
- (Put97) L. H. Putman and W. Myers, "Industrial Strength Software - Effective Management Using Measurement," Los Alamitos, CA, 1997
- (Sch99) S. R. Schach, *Classical and Object-Oriented Software Engineering With UML and C++*: McGraw-Hill, 1999.
- (Sch97) S. L. Schneberger, *Client/Server Software Maintenance*: McGraw-Hill, 1997.
- (Sch87) N. F. Schneidewind, "The State of Software Maintenance," presented at Proceedings of the IEEE, 1987
- (Som96) I. Sommerville, *Software Engineering*, Fifth ed: Addison-Wesley, 1996.
- (Val94) J. D. Vallett, S. E. Condon, L. Briand, Y. M. Kim and V. R. Basili, "Building on Experience Factory for Maintenance," presented at Proceedings of the Software Engineering Workshop, Software Engineering Laboratory, 1994

附录B 有关标准

- (IEEE610.12-90) IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.
- (IEEE1061-98) IEEE Std 1061-1998, *IEEE Standard for a Software Quality Metrics Methodology*: IEEE, 1998.
- (IEEE1219-98) IEEE Std 1219-1998, *IEEE Standard for Software Maintenance*: IEEE, 1998.
- (IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- (IEEE14143.1-00) IEEE Std 14143.1-2000//ISO/IEC14143-1:1998, *Information Technology-Software Measurement-Functional Size Measurement-Part 1: Definitions of Concepts*: IEEE, 2000.
- (ISO9126-01) ISO/IEC 9126-1:2001, *Software Engineering-Product Quality-Part 1:*

Quality Model: ISO and IEC, 2001.

(ISO14764-99) ISO/IEC 14764-1999, *Software Engineering-Software Maintenance*: ISO and IEC, 1999.

(ISO15271-98) ISO/IEC TR 15271:1998, *Information Technology - Guide for ISO/IEC 12207, (Software Life Cycle Process)*: ISO and IEC, 1998. [Abr93]

////////////////////////////////////

第7章 软件配置管理

术语和缩写

CCB: Configuration Control Board, 配置控制委员会

CM: Configuration Management, 配置管理

FCA: Functional Configuration Audit, 功能配置审计

MTBF: Mean Time Between Failures, 平均失效间隔时间

PCA: Physical Configuration Audit, 物理配置审计

SCCB: Software Configuration Control Board, 软件配置控制委员会

SCI: Software Configuration Item, 软件配置项

SCM: Software Configuration Management, 软件配置管理

SCMP: Software Configuration Management Plan, 软件配置管理计划

SCR: Software Change Request, 软件变更请求

SCSA: Software Configuration Status Accounting, 软件配置状态簿记

SEI/CMMi: Software Engineering Institute's Capability Maturity Model Integration, 软件工程研究所的能力成熟度模型集成

SQA: Software Quality Assurance, 软件质量保证

SRS: Software Requirement Specification, 软件需求规格说明

USNRC: U.S. Nuclear Regulatory Commission, 美国原子能管制委员会

引言

系统可以定义为一组被组织起来以完成特定功能或特定功能集合的组件 (IEEE 610.12-90), 系统的配置是在技术文档中规定或在产品达到的硬件、固件、软件、或它们的组合的功能和/或物理特征 (Buc96), 也可将其视作一组根据特定的建造流程组合起来以服务于特定目的的硬件、固件、软件项的特定版本。这样, 配置管理就是为了系统地控制配置变更, 在系统的整个生命周期中维持配置的完整性和可追踪性, 而标识系统在不同时间点上的配置的学科 (Ber97)。在 (IEEE610.12-90) 中, 将“配置管理”正式定义为“应用技术的和管理的指导和监督来: 标识和用文档记录配置项的功能和物理特征、控制对这些特征的变更、记录和报告变更处理过程和实现状态、验证与规定的需求的一致性。”

软件配置管理是一个支持性的软件生命周期过程 (IEEE12207.0-96), 它有益于项目管理、开发和维护活动、各种保证活动、最终产品的客户和用户。尽管硬件配置管理和软件配置管理的实现有所不同, 配置管理的概念可以应用于所有要控制的项。

软件配置管理与软件质量保证活动密切相关, 正如软件质量知识域中定义, 软件质量保证过程, 通过计划、实施和完成一系列活动, 为“质量已经建造到软件中”提供适当的置信度, 为项目生命周期中软件的各种产品和过程符合其规格说明的需求, 提供了保证。软件配置管理活动可以帮助完成这些软件质量保证目标。在某些项目环境中 (例如, IEEE730-02), 特定的软件质量保证的需求要求某些软件配置管理活动。

软件配置管理活动有：软件配置管理过程的管理和计划、软件配置标识、软件配置控制、软件配置状态簿记、软件配置审计、软件发布管理与交付。图1给出了这些活动的程式化表示。

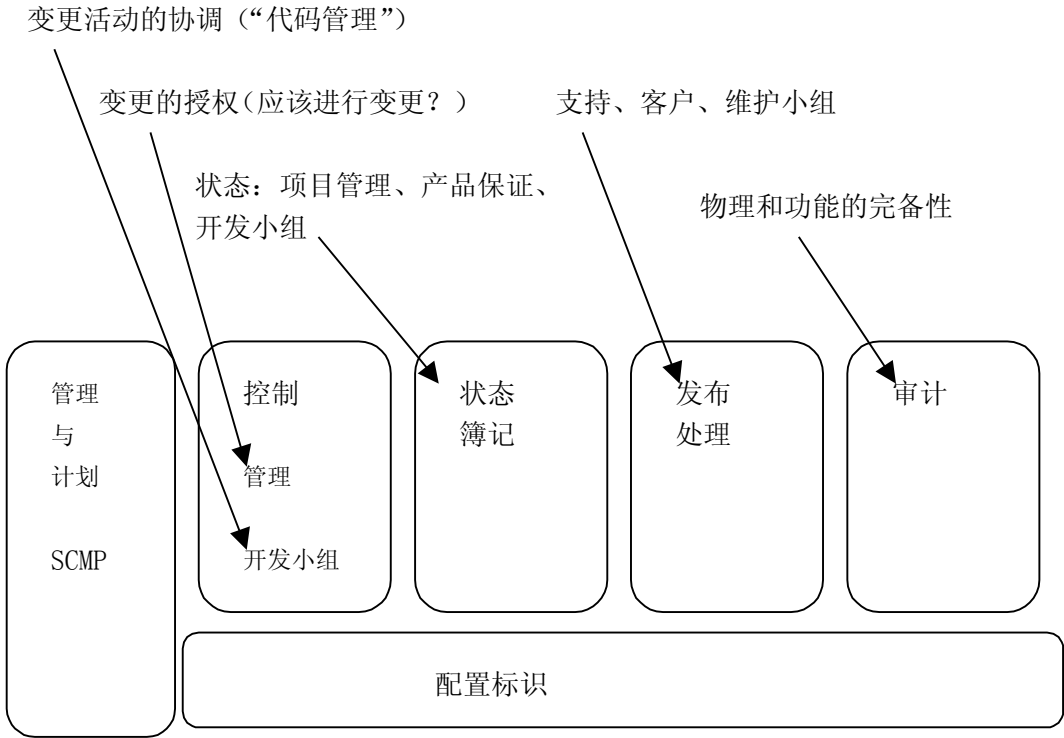


图 1 软件配置管理活动

由于配置管理的目标是整个软件工程过程中产生和使用的人工制品，软件配置管理知识域与所有其它知识域有联系。

软件配置管理的主题分解结构

1 软件配置管理过程的管理

软件配置管理通过标识产品的元素、管理和控制变更、验证、记录和报告配置信息，来控制产品的进化和完整性。从软件工程师的角度，软件配置管理加速了开发和变更实现活动。成功的软件配置管理实现需要仔细的计划和管理，这反过来要求理解软件配置管理过程的设计与实现的组织结构上下文和有关约束。

1.1 软件配置管理的组织结构上下文[Ber92 :c4; Dar90:c2; IEEE828-98:c4s2.1]

为了给项目的软件配置管理制订计划，有必要理解组织结构上下文和组织的元素之间的联系。软件配置管理要与几个其它活动或组织元素进行交互。可以用不同的方式来组织负责软件工程支持过程的组织元素，尽管完成某些可以以不同的方式任务的责任可以分配给组织的其它部分，诸如开发组织，软件配置管理的总体责任通常赋予单独的组织元素或指定的人。

软件通常作为包含硬件和固件元素的更大系统的一部分来进行开发的。这种情况下，软件配置管理活动与硬件和固件的配置管理活动平行进行。Buckley在[Buc96:c2]中描述了这个上下文中的软件配置管理。注意，固件包含硬件和软件，因此，硬件和软件的配置管理都适用于它。

软件配置管理在记录管理和非遵从项（non-conforming）等问题上，可能与组织的质量保证活动交互。对于前者，一些在软件配置管理控制下的项可能也是受到组织的质量保证程序规定的项目记录。管理非遵从项通常是质量保证活动的责任，但是，软件配置管理可以协助追踪和报告属于此类的软件配置项。

软件配置管理也许与软件开发和维护组织的联系最紧密。正是在这个上下文中，进行了许多软件配置控制任务。通常，同样的工具可以支持开发、维护和软件配置管理。

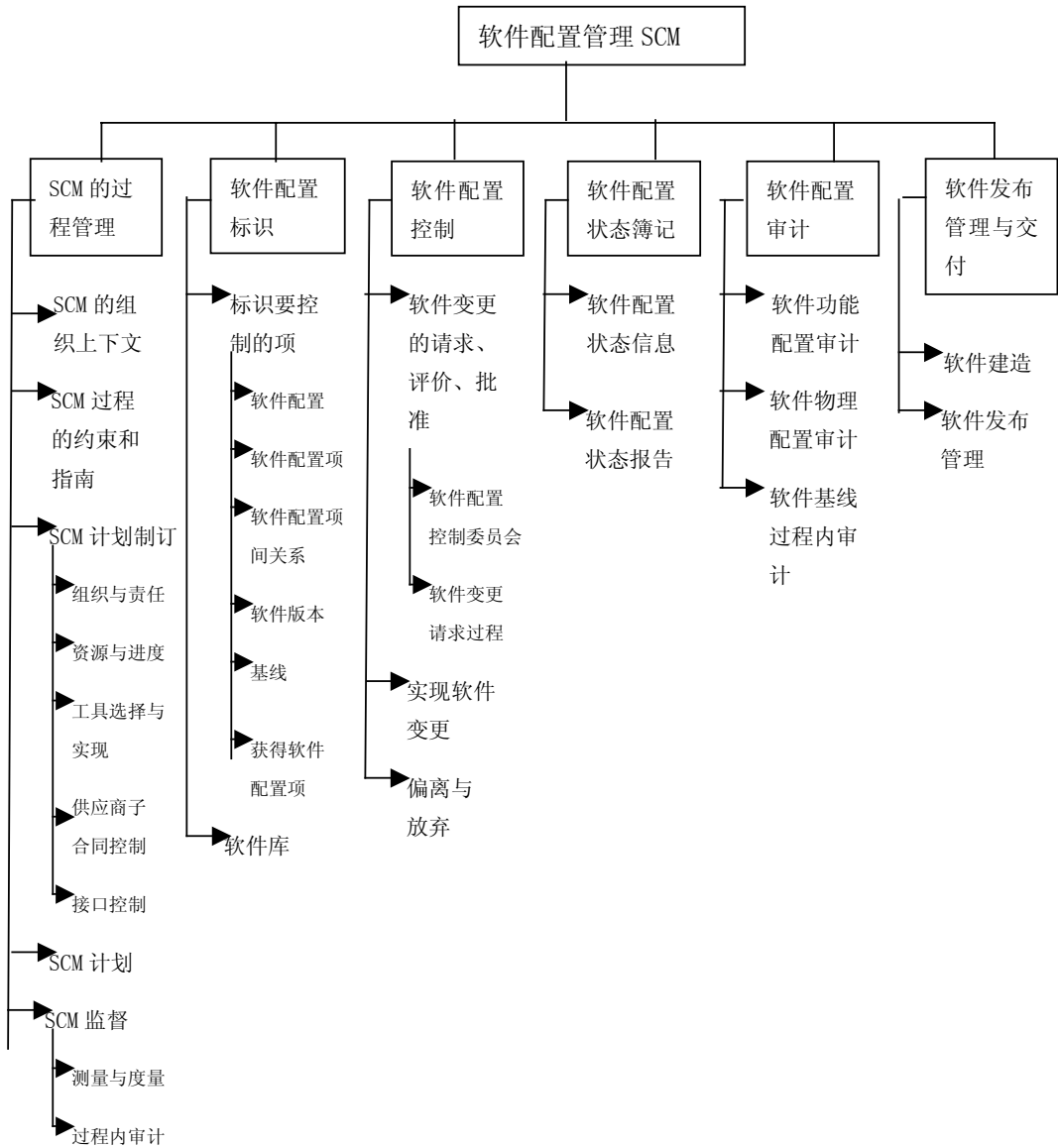


图 2 软件配置管理知识域的主题分解

1.2 软件配置管理过程的约束与指南[Ber92 :c5; IEEE828-98:c4s1, c4s2.3; Moo98]

软件配置管理过程的约束和指南有许多来源。公司或其它组织级别上设置的策略和流程可能影响或规定给定项目的软件配置管理过程的设计和实现。另外，获取者与供应者之间的合同可能包含一些影响软件配置管理过程的规定，例如，可能需要某些配置审计，或者明确

指出某些项应该在配置管理的控制之下。当要开发的软件产品对公共安全有潜在影响时，外部的管制机构也可能施加约束（例子参见USNRC1.169-97）。最后，为软件项目选择的特定软件生命周期过程，以及为实现软件选择的工具，也影响软件配置管理过程的设计和实现[Ber92]。

设计和实现软件配置管理过程的指南也可以从“最佳实践”中得到，如不同标准组织颁布的软件工程标准。Moore在[Moo98]中提供了这些组织的路线图及其标准。最佳实践也可以在过程改进和过程评价模型中体现，如软件工程研究所的能力成熟度模型集成(SEI/CMMi)(SEI01)和ISO/IEC15504软件工程—过程评定(ISO/IEC15504-98)。

1.3 软件配置管理的计划制订[Dar90 :c2; IEEE12207.0-96 :c6.s2.1;Som01 :c29]

为给定项目制订软件配置管理过程计划时，应该与组织的上下文、可应用的约束、普遍接受的指南、项目的本质（例如，规模和关键性）保持一致。覆盖的主要活动包括：软件配置标识、软件配置控制、软件配置状态簿记、软件配置审计、软件发布管理与交付。另外，一般还要考虑一些问题，诸如，组织与责任、资源与进度、工具选择与实现、销售商与子合同控制、接口控制等。制订计划活动的结果记录在软件配置管理计划中，它要接受软件质量保证的评审和审计。

1.3.1 软件配置管理的组织与责任[Ber92:c7; Buc96:c3; IEEE828-98:c4s2]

为防止完成给定的软件配置管理活动或任务的人员感到混淆不清，需要清晰地标识参与软件配置管理过程的组织。也需要将给定软件配置管理活动或任务的特定责任分配给组织实体或组织的元素。还应标识软件配置管理的总体授权和报告渠道，尽管这可以在项目管理或质量保证计划制订阶段完成。

1.3.2 软件配置管理资源与进度安排[Ber92:c7; Buc96:c3; IEEE828-98:c4s4; c4s5]

制订软件配置管理计划时，要标识承担软件配置管理活动和任务的人员和工具，通过建立必要的软件配置管理任务顺序，标识这些任务与项目管理计划阶段建立的项目进度和里程碑之间的关系，解决进度安排问题。还要指明实现计划必须的培训需求和培训新职员计划。

1.3.3 工具选择和实现[Ber92:c15; Con98:c6; Pre01:c31]

工具能力的不同类型、工具使用的流程，都支持软件配置管理活动。根据不同情况，组合一些手工工具、提供单个软件配置管理能力的自动工具、集成一系列软件配置管理能力（可能还包括其它能力）的自动工具、为软件工程过程中多个参与者服务的集成化工具环境（例如，软件配置管理、开发、验证与确认），可以得到这些工具能力。随着项目规模的增长和项目环境变得更加复杂，自动化工具的支持变得越来越重要，并越来越难以建立。这些工具能力提供下面的支持：（1）软件配置管理库；（2）软件变更请求和批准流程；（3）代码（和相关工作制品）和变更管理任务；（4）报告软件配置状态和收集软件配置管理度量；（5）软件配置审计；（6）管理和追踪软件文档；（7）完成软件建造；（8）管理和追踪软件发布及其交付。

在这些领域使用工具，增加了获得用于项目管理和过程改进目的的产品和过程度量的可能。

Royce在[Roy98]中描述了管理软件工程过程中的价值的7个核心度量。从不同软件配置管理工具可以得到的信息与Royce的工作与进展管理指示器联系，并与他的变更流量和稳定性、分解与模块性、重复工作与可适应性、平均失效间隔时间与成熟度等质量指示器联系。可以用不同方式组织这些指示器的报表，诸如，按软件配置项，或按请求的变更类型。图4

给出了工具能力和流程到软件配置管理活动的代表性映射。

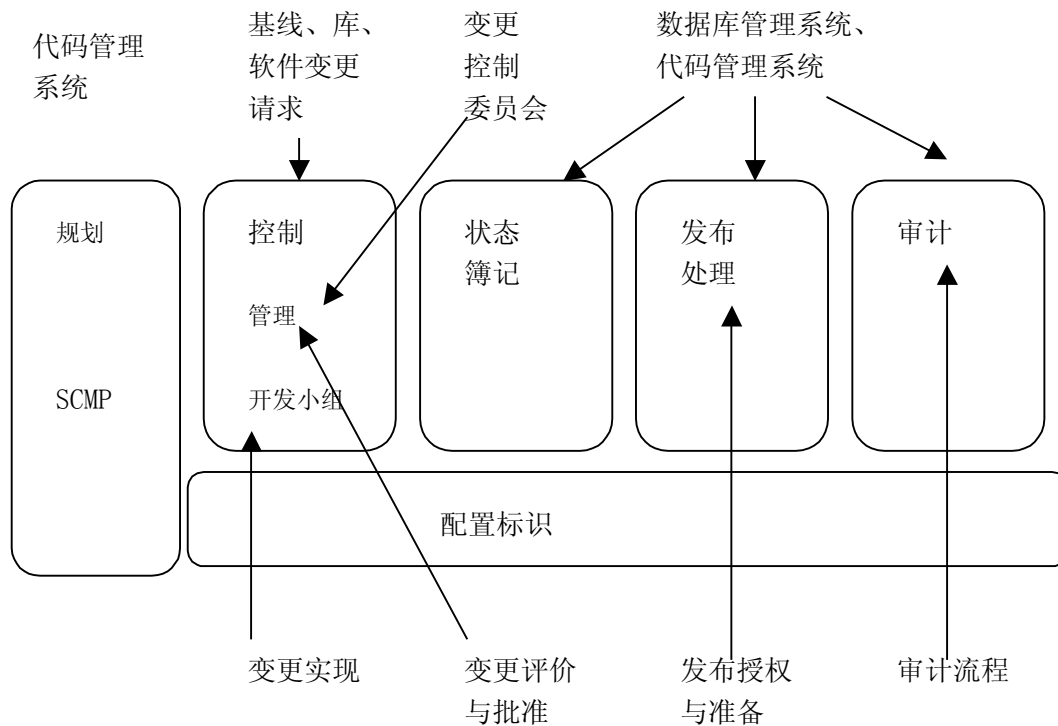


图3 软件配置管理工具与相关流程的特征

在这个例子中，代码管理系统通过控制对库元素的访问，协调多个用户的活动，帮助实行操作流程，来支持软件库的操作。其它工具支持从包含在库中的软件元素来建造软件和发布文档，管理软件变更请求的工具支持应用到可控软件项的变更控制流程。其它工具可以为管理、开发和质量保障活动提供数据库管理和报告制作能力。如前所述，可以将几个工具类型的能力集成到软件配置管理系统中，这反过来与各种其它软件活动紧密耦合。计划制订活动在要使用的其它软件工程工具的上下文中，为给定项目评估软件配置管理工具的需求，并选择软件配置管理将要使用的工具。计划制订要考虑实现这些工具时可能出现的问题，特别是需要进行某些文化变更时。[Dar90:c3, AppA]给出了软件配置管理系统和选择考虑的综述，[Mid97]给出了选择软件配置管理系统的一个实例。有关软件配置管理工具的补充信息可以参见软件工程工具和方法知识域。

1.3.4 销售商/子合同控制[Ber92:c13; Buc96:c11; IEEE828-98:c4s3.6]

软件项目可能获取或利用采购的软件产品，如编译器或其它工具。软件配置管理计划制订时要考虑这些项是否并如何置于配置控制之下（如，集成进项目库中），如何评价和管理变更和更新。

类似需要考虑还包括子合同软件，这种情况下，需要建立施加到子合同商的软件配置管理过程的软件配置管理需求，它作为子合同的一部分，并作为监控符合性的手段。后者包括为进行有效的符合性监控，必须包括哪些软件配置管理信息。

1.3.5 接口控制[IEEE828-98:c4s3.5]

当一个软件项要与其它软件或硬件项接口时，对其中任一个项的变更都可以影响洽谈

项。软件配置管理过程计划制订时，要考虑如何标识相互接口的项，如何管理和通信交流这些项。软件配置管理角色可能是更大的、系统级别的接口规格说明和控制过程的一部分，可能涉及接口规格说明、接口控制计划、接口控制文档。在这种情况下，软件配置管理的接口控制计划制订应在系统级别过程的上下文中进行。有关完成接口控制活动的讨论见[Ber92:c12]。

1.4 软件配置管理计划[Ber92:c7; Buc96:c3; Pau93:L2-81]

为给定项目制订软件配置管理计划的结果记录在软件配置管理计划中，这是一个作为软件配置管理过程的参考的“活动文档”，有必要在软件生命周期中维护（更新或批准）它。在实现软件配置管理计划时，通常有必要开发大量定义规定的需求如何在日常活动中执行的更详细的、次级流程。

基于制订计划的活动产生的信息，关于软件配置管理计划的创建和维护的指南，可以在许多地方找到，如[IEEE828-98:c4]，这个参考文献提供了软件配置管理计划中要包含的信息的需求，它也定义和描述了要包含在软件配置管理计划中的6类软件配置管理信息：（1）引言（目的、范围、使用的术语）；（2）软件配置管理的管理（组织、责任、职权、可应用的政策、指示、和流程）；（3）软件配置管理活动（配置标识、配置控制等）；（4）软件配置管理进度安排（与项目其它活动的协调）；（5）软件配置管理资源（工具、物理资源、人力资源）；（6）软件配置管理计划的维护。

1.5 软件配置管理的监督[Pau93 :L2-87]

在软件配置管理过程实现后，为保证软件配置管理计划的规定被正常执行，某种程度的监督是必要的（参见[Buc96]）。可能有一些为保证符合规定的软件配置管理过程和流程的特定软件质量保证需求，这可能涉及一个软件配置管理职权，以保证被分配了责任的组织或人员正确地完成定义的软件配置管理任务。软件质量保证职权作为符合审计活动的一部分，也可以完成这个监督。

使用具有过程控制能力的集成软件配置管理工具，可以使监督任务更容易些。一些工具促进了过程符合性，同时为软件工程师适应流程提供了灵活性。其它工具强制实现过程，留给软件工程师的余地小一些。监督的需求和提供给软件工程师的灵活性是工具选择中的一个重要考虑事项。

1.5.1 软件配置管理测量与度量[Buc96:c3; Roy98]

可以设计软件配置管理测量（measures），以提供关于产品进化的特定的信息，或提供对软件配置管理过程的运行情况的洞察。监控软件配置管理过程的一个相关目标是为过程改进发现机会。软件配置管理过程的度量为在现行基础上，监控软件配置管理活动的有效性提供了一个好的手段。这些度量对于刻画过程的当前状态，以及提供不同时间的比较的基础，都有用处。分析度量可能提供一些洞察，导致过程变更和对软件配置管理进行相应的更新。

软件库和各种软件配置管理工具能力为提取有关软件配置管理过程的特征的信息提供了源泉（以及提供项目和管理信息）。例如，有关不同类型变更需要的处理时间的信息，对于评价确定什么样的职权级别对授权某些变更的类型的准则就有用。

必须小心，才能保持监督集中在可以从度量得到的洞察上，而不是度量本身。对于产品和过程度量的讨论可参见软件工程过程知识域，软件度量程序在软件工程管理知识域中描述。

1.5.2 软件配置管理的过程内审计[Buc96:c15]

可以在软件工程过程中进行审计，以调查配置的特定元素的当前状态，或评定软件配置管理过程的实现。软件配置管理的过程内审计为监控选择的过程的方面提供了更正式的机制，并可以和软件质量保证功能协调。可以参见子域5软件配置审计。

2 软件配置标识[IEEE12207.0-96 :c6s2.2]

软件配置标识活动标识要控制的项，为这些项及其版本建立标识模式，安装获取和管理受控项时使用的工具。这些活动为其它软件配置管理活动提供了基础。

2.1 标识要控制的项[Ber92 :c8; IEEE828-98:c4s3.1; Pau93 :L2-83;Som05:c29]

控制变更的第一步就是标识要控制的软件项，这涉及理解在系统配置上下文中的软件配置、选择软件配置项、开发为软件项加标签并描述它们之间关系的策略、标识要使用的基线以及获取基线的项的流程。

2.1.1 软件配置[Buc96:c4; c6, Pre04:c27]

软件配置是技术文档中建立的或产品达到的软件的功能和物理特征的集合(IEEE610.12-90)，可以将其视为整个系统配置的一部分。

2.1.2 软件配置项[Buc96:c4;c6; Con98:c2; Pre04:c27]

软件配置项是指派给配置管理的软件的聚合体，并在软件配置管理过程中作为一个单独的实体处理(IEEE610.12-90)。除了代码本身外，项的各种变体也通常由软件配置管理控制。可能成为软件配置项的软件项包括：各种计划、规格说明和设计文档、测试材料、软件工具、源代码和可执行代码、代码库、数据和数据字典，以及安装、维护、操作和软件使用的文档。

选择软件配置项是一个重要的过程，在过程中，必须在为项目控制目的提供可见性和提供可管理数目的可控制项之间，进行平衡软件配置项选择的准则列表参见[Ber92]。

2.1.3 软件配置项之间的联系[Con98:c2; Pre04:c27]

选择的软件配置项之间的结构联系及其组成成分，影响着其它软件配置管理活动或任务，诸如软件构造或分析建议的变更的影响。对于支持可追踪性，正常地追踪这些联系也很重要。为软件配置项设计标识方案时，应该考虑将标识项映射到软件结构的需要，以及支持软件项进化及其联系的需要。

2.1.4 软件版本[Bab86:c2]

软件项随项目的进行而进化，软件项的一个版本是一个特别标识和说明的项，可以被认为是进化中的项的一个状态[Con98:c3-c5]。修订是项的一个新版本，它试图替代项的旧版本。变体则是一个项的新版本，它将被加入到配置中但不替代旧版本。

2.1.5 基线[Bab86:c5; Buc96:c4; Pre04:c27]

软件基线是软件配置项的集合，它在软件生命周期的特定时间被正式指定和固定。这个术语也指软件配置项的一个特定被批准的版本。两种情况下，基线都只能在正式的变更控制流程下，才能变更。基线及其被批准的所有变更，代表当前被批准的配置。

通常使用的基线是功能的、已分配的、开发的和产品基线（例子见[Ber92]）。功能基线对应评审后的系统需求。已分配的基线对应评审后的软件需求规格说明和软件接口需求规格说明。开发基线表示软件生命周期中选定的时间的进化中软件配置，这个基线的变更权力职权通常主要由开发组织掌握，但可以与其它组织分享（例如，软件配置管理或测试）。产

品基线对应交付给系统集成的完备软件产品。给定项目使用的基线及其变更批准需要的权力职权的级别，通常在软件配置管理计划中规定。

2.1.6 获取软件配置项[Buc96:c4]

软件配置项在不同时间都处于软件配置管理的控制下，即，在软件生命周期的特定时间点，它们被集成到一个特定的基线中。触发事件是某种形式的正式接收任务的完成，如正式评审。图4描述了作为基线的项随着生命周期进展的增长特征。由于仅是一个说明性例子，图4基于瀑布模型，图中下标指明进化中项的版本。软件变更请求在主题3.1软件变更的请求、评价和批准中描述。

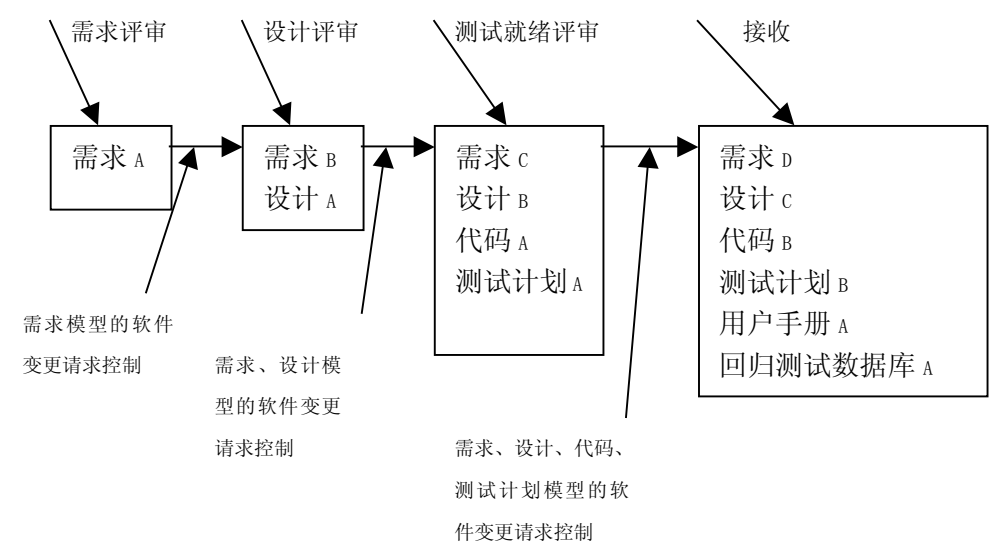


图 4 配置项的获得

软件配置项获取后，对项的变更必须被正式批准为适合于涉及的软件配置项和基线，象在软件配置管理计划中定义的一样。批准后，根据适当的流程，项被集成到软件基线中。

2.2 软件库[Bab86 :c2 ;c5; Buc96 :c4; IEEE828-98:c4s3.1;Pau93:L2-82; Som01:c29]

软件库是一组受控制的、辅助软件开发、使用和维护的软件及相关的文档(IEEE610.12-90)，它在软件发布管理和交付活动中，起着器械性的作用。可能会使用几类软件库，每类对应与软件项的特定成熟度，例如，工作库可以支持编码，项目支持库可以支持测试，主库可以用于完成后的产品。（与基线和变更职权相关的）软件配置管理控制的适当层次，是与每个库相关的。在访问控制和备份设施方面的安全性，是库管理的关键问题。

[Ber92:c14]描述了软件库的模型，每个库使用的工具必须在控制软件配置项和控制库的访问两个方面，支持这个库的软件配置管理控制需求。在工作库层次上，这是一个为开发人员、维护人员和软件配置管理服务的代码管理能力，它集中在管理软件项的版本上，并支持多个开发人员的活动。在更高的控制层次上，访问更严格，软件配置管理是主要的用户。这些库是度量工作和进展的信息的一个重要来源。

3 软件配置控制[IEEE12207.0-96 :c6s2.3; Pau93 :L2-84]

软件配置控制关注管理软件生命周期中的变更，它覆盖确定要作什么样的变更的过程、

批准某些变更的权力职权、支持这些变更的实现，以及与项目需求偏离和放弃这些偏离的概念。从这些活动导出的信息对度量变更流量和间歇、重复工作的方面等有用。

3.1 软件变更的请求、评价和批准[IEEE828-98:c4s3.2; Pre04:c27; Som05:c29]

管理对受控项的变更的第一步是确定要作什么变更，软件变更请求过程（图5）为提交和记录变更请求、评价建议的变更的潜在的成本和影响范围、接受/修改/拒绝建议的变更，提供了正式的流程。对软件配置项的变更请求可以在软件生命周期的任何时间点，由任何人发起，并可包含建议的解决方案和请求的优先权。变更请求的一个来源是响应问题报告的更正活动的启动。

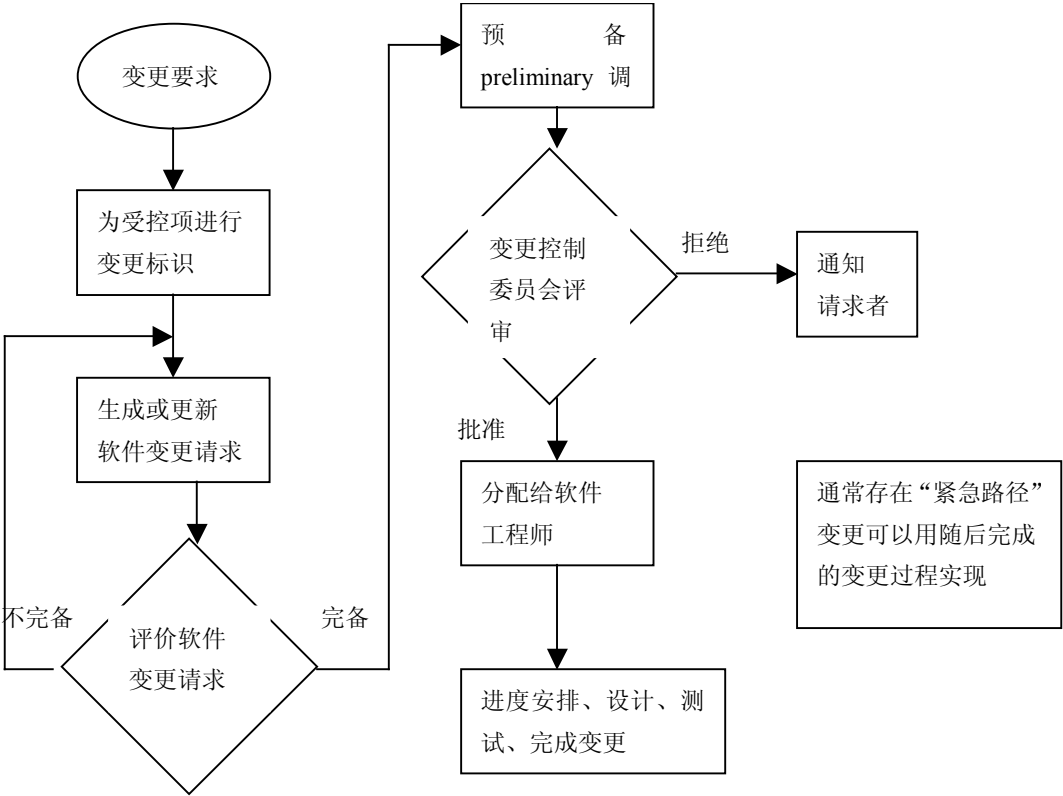


图 5 变更控制流程

无论来源如何，变更的类型（例如，缺陷或增强）通常记录在软件变更请求中。这为根据变更类型追踪缺陷和收集变更活动度量提供了机会。一旦接收到一个软件变更请求，就要进行技术评价（又叫影响分析），以确定如果接受变更请求而必须的修改程度。理解软件（可能还包括硬件）项之间的管理，对于这个任务很重要。

最后，已经建立的职权机构，以及相当的受影响的基线、涉及的软件配置项和变更的本质，将评价变更请求的技术和管理方面，可能会接受/修改/拒绝/推迟建议的变更。

3.1.1 软件配置控制委员会[Ber92:c9; Buc96:c9, c11; Pre04:c27]

接受或拒绝建议的变更的职权通常是由一个称为配置控制委员会的实体行使的。在较小的项目中，这个职权可能实际上由领导或单个人掌握，而不是由多人组成的委员会掌握。根据不同的准则，如涉及的项的关键程度、变更的本质（例如，对预算和进度的影响）、当前所在的生命周期时间点，可能有多个层次的变更职权。给定系统的配置控制委员会的组成

随这些准则而变化（但总有一个软件配置管理的代表），所有的干系人在适当的层次上也有相应的代表。当配置控制委员会的职权范围是严格的软件时，就称为软件变更控制委员会。变更控制委员会的活动通常要接受软件质量审计或评审。

3.1.2 软件变更请求过程[Buc96:c9, c11; Pre04:c27]

有效的软件变更请求过程要求使用支持工具和流程，包括从纸质表格和文档化流程到用于发起变更请求、强制执行变更过程的流程、获得配置控制委员会决定、报告变更过程信息的电子工具。这些工具能力与问题报告系统之间的链接可以加速追踪报告的问题的解决方案。变更过程描述和支持表格（信息）可在多种参考文献中找到，例如[Ber92:c9]。

3.2 实现软件变更[Bab86 :c6; Ber92 :c9; Buc96 :c9, c11; IEEE828-98:c4s3.2.4; Pre04:c27; Som05:c29]

根据可以应用的进度需求，使用定义好的软件流程来实现已经批准的软件变更请求。由于可能同时实现大量的被批准的软件变更请求，有必要提供一种工具来追踪哪个软件变更请求被集成到特定的软件版本和基线中。作为变更过程关闭的一部分，可以对已经完成的变更进行配置审计和软件质量验证。这包括保证，只进行了已经批准的变更。上面描述的变更请求过程一般要用文档记录软件配置管理（和其它）的批准变更的信息。变更的实际实现是由库工具能力支持的，它提供版本管理和代码存储支持。这些工具至少需要提供出入登记和版本控制能力。更强大的工具可能支持平行开发和地理上分布的环境。这些工具可能被声明为独立的软件配置控制小组控制下的分离的专门应用。它们也可以作为软件工程环境的一个集成部分。最后，它们可能象与一个操作系统提供的根本的变更控制系统一样基本。

3.3 偏离和放弃 (*Deviations and Waivers*) [Ber92 :c9; Buc96 :c12]

施加在开发活动中产生的软件工程成果或规格说明上的约束可能包含一些在生命周期指定时间点上无法满足的规定。偏离是一个对先于项的开发之前偏离规定的认可或授权。放弃是一个使用一个在其开发中以某种方式偏离其规定的项的授权。在这些情况下，要使用正式的过程来获得对规定的偏离或放弃。

4 软件配置状态簿记[IEEE12207.0-96 :c6s2.4; Pau93 :L2-85; Pre04:c27; Som05:c29]

软件配置状态簿记要记录和报告进行有效的软件配置管理需要的信息。

4.1 软件配置状态信息[Buc96 :c13; IEEE828-98:c4s3.3]

软件配置状态簿记活动为当生命周期进展时，捕获和报告必要的信息设计和运行一个系统，同任何信息系统一样，必须标识、收集和维持为进化中的配置要管理的配置状态信息。需要各种信息和度量来支持软件配置管理过程，和满足管理、软件工程和其它相关活动的配置状态报告的需要。可用的信息类型包括批准的配置标识，以及变更、偏离和放弃的标识和当前实现状态。[Ber92:c10]给出了重要的数据元素的列表。

为完成软件配置状态簿记数据收集和报告任务，需要某种形式的自动工具支持，这可能是数据库能力，或者是单独的工具或更大的、集成工具环境的能力。

4.2 软件配置状态报告制作[Ber92 :c10; Buc96 :c13]

报告的信息可以被各种组织和项目元素使用，包括：开发小组、维护小组、项目管理、软件质量活动。报告制作可以采用回答特定问题的特定查询的形式，或周期性生成事先定义

好的报表。在生命周期中状态簿记活动产生的一些信息可以成为质量保证记录。

除了报告当前配置状态外，软件配置状态簿记获得的信息可以作为管理、开发和软件配置管理感兴趣的各種度量的基础。例如，每个软件配置项的变更请求数目、实现一个变更请求需要的平均时间等。

5 软件配置审计[IEEE828-98:c4s3.4; IEEE12207.0-96 :c6s2.5;Pau93 :L2-86; Pre04:c276c27]

软件审计是独立评价软件产品和过程对可以应用的规则、标准、指南、计划和流程的遵从性而完成的活动(IEEE1028-97)。审计是根据由不同的审计者规则和责任而良定义的过程进行的，因此，必须仔细计划每个审计。一次审计可能需要大量人员在短时间内完成各种任务。支持审计的计划和进行的工具可以大大加快这个过程。在各种参考文献中可以找到进行审计的指南，如[Ber92:c11; Buc96:c15]和(IEEE1028-97)。

软件配置审计活动确定一个项满足需求的功能和物理特征的程度。非正式的这类审计可以在生命周期的任何点上进行。合同管理可能要求进行两类正式审计（例如，覆盖关键软件的合同中）：功能配置审计和物理配置审计。成功地完成这些审计是建立产品基线的前提。Buckley 在[Buc96:c15]中对比了在硬件和软件上下文中的这两类审计，并推荐进行审计前，仔细评价是否需要进行软件的功能配置审计和物理配置审计。

5.1 软件功能配置审计

软件功能配置审计的目的是保证被审计的软件现与其支配性规格说明一致。软件验证和确认活动的输出是这个审计的一个关键输入。

5.2 软件物理配置审计

软件物理配置审计的目的是保证设计和参考文档与建造软件产品的要求一致。

5.3 软件基线的过程内审计

如前所述，可以在开发过程中进行审计，以调查配置的特定元素的当前状态。在这种情况下，可以对抽样的基线项进行审计，以保证性能与规格说明一致，或保证进化中的文档继续与开发中的基线项保持一致。

6 软件发布管理和交付[IEEE12207.0-96:c6s2.6]

在这个上下文中，术语“发布”用来指将软件配置项分布到开发活动外，这包括内部发布和分布到客户。当软件项的不同版本可以用于交付时，如不同平台的版本或具有不同能力的版本，通常有必要重新创建特定的版本，并包装交付版本的正确材料。完成发布和交付任务时，软件库是关键的元素。

6.1 软件建造[Bab6 :c6; Som05:c29]

软件建造是使用适当的配置数据，将软件配置项的正确版本组合进用于交付给客户或其它接收者（如测试活动）的可执行程序。对于有硬件或固件的系统，可执行程序交付到系统建造活动。建造指令要保证，采取了正确的建造步骤和正确的顺序。除了为新发布建造软件外，通常也有必要让软件配置管理具有为恢复、测试或附加发布目的，而重新产生以前发布的能力。

软件使用支持工具（如编译器）的特定版本来建造。可能有必要为以前建造的软件配置重新建造一个精确的副本。这时，支持工具和相关的建造指令需要处于软件配置管理的控制

下，以保证工具的正确版本的可获得性。

工具能力对于为给定目标环境、让从选择的版本和适当的配置数据中建造软件的过程自动化，选择软件项的正确版本是有用的。对于具有平行开发或分布式开发环境的大型项目，这个工具能力很有必要。多数软件工程环境提供了这个能力。这些工具的复杂性变化大，从需要软件工程师学习专门的脚本语言，到隐藏了“智能”建造设施的复杂性的面向图形的方法。

建造过程和产品通常要进行软件质量验证。建造过程的输出可能在未来的引用中需要，并可能作为质量保证记录。

6.2 软件发布管理[Som05:c29]

软件发布管理包括：标识、包装和交付产品的元素，例如，可执行程序、文档、发布注意事项和配置数据。假定产品变更可以连续发生，发布管理的一个方面是确定何时发行一个发布。发布解决的问题的严重性和上一次发布的错误密度的度量，影响着这个决定(Som01)。给定产品的目的应用，包装任务必须标识要交付哪个产品项，然后选择这些项的正确变体。记录发布的物理内容的信息称为版本描述文档。发布注意事项一般描述新的能力、已知的问题和正常的产品运行必需的平台需求。要发布的包也包括安装或升级指令，后者可能由于一些当前用户可能拥有老几个发布的版本而复杂化。最后，在某些情况下，发布管理活动可能需要追踪产品对各种用户或目标系统的分布。一个例子是，需要供应商向用户提醒新报告的问题。

需要一个工具能力来支持这些发布管理功能。与支持变更请求过程的工具能力保持联系，将发布内容映射到接收到的软件变更请求，这很有用处。这个工具能力可能也要维护不同目标平台上的信息，以及各种客户环境的信息。

主题与参考文献矩阵

	[Bab 86]	[Ber 92]	[Buc 96]	[Con 98]	[Dar 90]	[IEEE8 28-98]	[IEEE1 2207.0 -96]
1 软件配置管理过程的管理							
1.1 软件配置管理组织上下文		C4	C2		C2	C4s2.1	
1.2 软件配置管理过程约束与指南		C5				C4s1, c 4s2.3	
1.3 软件配置管理的计划制订					C2		6.2.1
软件配置管理组织与责任		C7	C3			C4s2	
软件配置管理资源与进度		C7	C3			C4s4, c 4s5	
工具选择与实现		C15		C6	C3, A ppA		
销售商/子合同控制		C13	C11			C4s3.6	
接口控制		C12				C4s3.5	
1.4 软件配置管理计划		C7	C3			C4	
1.5 软件配置管理监督			*			C4	
软件配置管理测量与度量			C3				
软件配置管理过程内统计			C15				

2 软件配置标识							C6s2. 2
2.1 标识要控制的项		C8				C4s3. 1	
软件配置			C4, c 6				
软件配置项		*	C4, c 6	C2			
软件配置项之间联系				C2			
软件版本	C2						
基线	C5	*	C4				
获取软件配置项			C4				
2.2 软件库	C2, c 5	C14	C4			C4s3. 1	
3 软件配置控制							C6s2. 3
3.1 软件变更的请求、评价和批准						C4s3. 2	
软件配置控制委员会		C9	C9, c 11				
软件变更请求过程		C9	C9, c 11				
3.2 实现软件变更	C6	C9	C9, c 11			C4s3. 2 . 4	
3.3 偏离与放弃		C9	C12				
4 软件配置状态簿记							C6s2. 4
4.1 软件配置状态信息		C10	C13			C4s3. 3	
4.2 软件配置状态报告		C10	C13				
5 软件配置审计		C11	C15			C4s3. 4	C6s2. 5
5.1 软件功能配置审计							
5.2 软件物理配置审计							
6 软件发布管理与交付							C6s2. 6
6.1 软件建造	C6						
6.2 软件发布管理							

	[Mid97]	[Moo98]	[Pau93]	[Pre04]	[Roy 98]	[Som 05]
1 软件配置管理过程的管理						
1.1 软件配置管理组织上下文						
1.2 软件配置管理过程约束与指南		*				
1.3 软件配置管理的计划制订						C29
软件配置管理组织与责任						
软件配置管理资源与进度						
工具选择与实现	*				*	

销售商/子合同控制						
接口控制						
1.4 软件配置管理计划			L2-81			
1.5 软件配置管理监督			L2-87			
软件配置管理测量与度量					188-202, 283-298	
软件配置管理过程内统计						
2 软件配置标识						
2.1 标识要控制的项			L2-83			C29
软件配置				C27		
软件配置项				C27		
软件配置项之间联系				C27		
软件版本				C27		
基线				C27		
获取软件配置项						
2.2 软件库			L2-82			C29
3 软件配置控制			L2-84			
3.1 软件变更的请求、评价和批准				C27		C29
软件配置控制委员会				C27		
软件变更请求过程				C27		
3.2 实现软件变更				C27		C29
3.3 偏离与放弃						
4 软件配置状态簿记			L2-85	C27		C29
4.1 软件配置状态信息						
4.2 软件配置状态报告						
5 软件配置审计			L2-85	C26, c27		
5.1 软件功能配置审计						
5.2 软件物理配置审计						
6 软件发布管理与交付						
6.1 软件建造						C27
6.2 软件发布管理						C27

软件配置管理的推荐参考文献

- [Bab86] W. A. Babich, *Software Configuration Management, Coordination for Team Productivity*: Addison-Wesley, 1986.
- [Ber92] H. R. Berlack, *Software Configuration Management*. New York: John Wiley & Sons, 1992.
- [Buc96] F. J. Buckley, *Implementing Configuration Management: Hardware, Software,*

- and Firmware*, Second ed. Los Alamitos, CA: IEEE Computer Society Press, 1996.
- [Con98] R. Conradi and B. Westfechtel, "Version Models for Software Configuration Management," *ACM Computing Surveys*, vol. 30, iss. 2, June, 1998
- [Dar90] S. A. Dart, *Spectrum of Functionality in Configuration Management Systems*. Carnegie Mellon University: Software Engineering Institute, 1990.
- [IEEE828-98] IEEE Std 828-1998, *IEEE Standard for Software Configuration Management Plans*: IEEE, 1998.
- [IEEE12207.0-96] IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- [Mid97] A. K. Midha, "Software Configuration Management for the 21st Century," *Bell Labs Technical Journal*, vol. 2, iss. 1, 154-165, Winter, 1997
- [Moo98] J. W. Moore, *Software Engineering Standards, A User's Roadmap*. Los Alamitos, CA: IEEE Computer Society, 1998.
- [Pau93] M. C. Paulk and al, "Key Practices of the Capability Maturity Model, Version 1.1," Software Engineering Institute, Carnegie Mellon University, Technical Report CMU/SEI-93-TR-025, 1993
- [Pre04] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, Sixth ed: McGraw-Hill, 2004.
- [Roy98] W. Royce, *Software Project Management, A United Framework*: Addison-Wesley, 1998.
- [Som05] I. Sommerville, *Software Engineering*, Seventh ed: Addison-Wesley, 2005..©

附录A 深入阅读的参考文献

- (Bab86) W. A. Babich, *Software Configuration Management, Coordination for Team Productivity*: Addison-Wesley, 1986.
- (Ber92) H. R. Berlack, *Software Configuration Management*. New York: John Wiley & Sons, 1992.
- (Ber97) E. H. Bersoff, "Elements of Software Configuration Management," in *Software Engineering*, M. Dorfman and R. H. Thayer, Eds. Los Alamitos, CA: IEEE Computer Society Press, 1997.
- (Buc96) F. J. Buckley, *Implementing Configuration Management: Hardware, Software, and Firmware*, Second ed. Los Alamitos, CA: IEEE Computer Society Press, 1996.
- (ElE98) K. El-Emam and al, "SPICE, The Theory and Practice of Software Process Improvement and Capability Determination," presented at IEEE Computer Society, Los Alamitos, CA, 1998
- (Est95) J. Estublier, "Software Configuration Management," presented at ICSE SCM-4 and SCM-5 Workshops Selected Papers, Berlin, 1995
- (Gra92) R. B. Grady, *Practical Software Metrics for project Management and Process Management*: Prentice Hall, Englewood Cliffs, NJ 07632, 1992.
- (Hoe02) A. v. d. Hoek, "Configuration Management Yellow Pages," 2002, available at http://www.cmtoday.com/yp/configuration_management.html
- (Hum89) W. Humphrey, *Managing the Software Process*: Massachusetts: Addison Wesley, 1989.

- (Pau95) M. C. Paulk and al, *The Capability Maturity Model, Guidelines for Improving the Software Process*. Reading, Massachusetts: Addison-Wesley, 1995.
- (Som01a) I. Sommerville, "Software Configuration Management," presented at ICSE SCM-6 Workshop, Berlin, 2001
- (USNRC1.169-97) USNRC Regulatory Guide 1.169, "Configuration Management Plans for Digital Computer Software Used in Safety Systems of Nuclear Power Plants," presented at U.S. Nuclear Regulatory Commission, Washington DC, 1997
- (Vin88) J. Vincent, A. Waters and J. Sinclair, *Software Quality Assurance: Practice and Implementation*. Englewood Cliffs, NJ: Prentice-Hall, 1988.
- (Whi91) D. Whitgift, *Methods and Tools for Software Configuration Management*. Chichester, England: John Wiley & Sons, 1991

附录B 有关标准

- (IEEE730-02) IEEE Std 730-2002, *IEEE Standard for Software Quality Assurance Plans*: IEEE, 2002.
- (IEEE828-98) IEEE Std 828-1998, *IEEE Standard for Software Configuration Management Plans*: IEEE, 1998.
- (IEEE1028-97) IEEE Std 1028-1997 (R2002), *IEEE Standard for Software Reviews*: IEEE, 1997.
- (IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- (IEEE12207.1-96) IEEE/EIA 12207.1-1996, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes - Life cycle data*: IEEE, 1996.
- (IEEE12207.2-97) IEEE/EIA 12207.2-1997, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes - Implementation. Considerations*: IEEE, 1997.
- (ISO15846-98) ISO/IEC TR 15846:1998, *Information Technology - Software Life Cycle Processes -Configuration Management*. Geneva, Switzerland,: ISO and IEC, 1998..

////////////////////////////////////

第8章 软件工程管理

术语和缩写

PMBOK: Guide to the Project Management Body of Knowledge, 项目管理知识体系指南
SQA: Software Quality Assurance, 软件质量保证

引言

软件工程管理定义为应用管理活动—计划、协调、度量、监控、控制和报表—以保证软件的开发和维护是系统的、有纪律的和量化的(IEEE610.12-90)。

这样,软件工程管理知识域处理软件工程的管理和度量问题,尽管度量是所有知识域的一个重要方面,但正是在这里提出了度量程序的主题。

尽管在某种意义上可以认为,有可能象任何其它(复杂)过程一样来管理软件工程,软

件产品和软件生命周期过程有其特定的方面，使得有效的管理变得复杂，这些方面包括：（1）客户的感受是，通常缺乏对软件工程中固有的复杂性的正确评价，特别是在与不断变化的需求的影响的关系中。（2）软件工程过程本身将不可避免地产生新的或变更的客户的要求。（3）其后果是，软件通常是以迭代的过程开发的，而不是以一个相继关闭的任务序列开发的。（4）软件工程有必要集成创造性和纪律两个方面，在二者之间维持平衡常常很困难。（5）软件的创新性和复杂性通常很高。（6）基础技术的变化速度很快。

对于软件工程，管理活动发生在3个层次上：组织和内部基础结构管理、项目管理、度量程序的计划制订和控制。本知识域描述中，详细描述了最后两个层次，但是，这并不能减少组织管理问题的重要性。

由于与相关学科（很明显是管理）的链接很重要，将比其它知识域更详细地描述。组织管理方面在其对软件工程、策略管理的影响上很重要，例如，组织策略和标准提供了开展软件工程的框架。这些策略可能受有效的软件开发和维护策略的需求的影响，可能需要为组织层次上软件工程的有效管理建立大量的软件工程特定策略，例如，为建立特定的组织范围内的软件工程任务（如设计、实现、估算、追踪和报表）的过程或流程，策略通常是必要的。这些策略对于有效的长期软件工程管理是必要的，例如，建立一个在其上分析过去的性能和实现改进的一致的基础。

管理的另一个重要的方面是人员管理：雇佣、培训和激励人员的策略和流程，职业生涯发展的指导不仅在项目层次上重要，而且组织的长期成功也重要。软件工程人员管理可能提出特有的培训或人事管理挑战（例如，在技术基础经受连续和迅速变化的上下文中维持流动）。通信管理通常被认为是被忽略的，但却是在需要精确理解用户要求、复杂的需求和设计是必要的领域的个人性能的主要方面。最后，部门级管理不仅具有要开发的软件的总体观点，而且具有组织中已经使用的软件的总体视图，这个管理也是必要的。

还有，软件复用是维持和改进生产率和竞争力的关键因素，有效的复用需要一个反映这个技术的独特力量和需求的战略性眼光。

除了理解受软件独特影响的管理方面外，软件工程师必须具有更一般方面的知识，即使在本指南目标规定的毕业后头4年内。

采购、供应链管理、市场、销售和分布方式的组织文化和行为、功能性企业管理都对组织的软件工程过程有影响，尽管可能只是间接的。

与这个知识域相关的是项目管理概念，因为“构造有用的软件制品”通常以单个项目的形式（或程序）进行管理的。对于这一点，我们在项目管理知识体系指南中可以找到广泛的支持（PMI00），它本身包括了下列管理知识域：项目集成管理、项目范围管理、项目时间管理、项目成本管理、项目质量管理、项目人力资源管理和项目通信管理。显然，所有这些主题都与软件工程管理知识域直接有关。试图在这里重复PMBOK指南的内容是不可能和不适当的，因此，我们建议对软件工程项目管理以外领域感兴趣的读者去阅读PMBOK本身。项目管理也可以在软件工程相关学科一章中找到。

软件工程管理知识域由软件项目管理过程（前5个子域）和软件工程度量（最后一个子域）这两个主题组成。通常认为这两个主题是分离的，它们也确实具有独特的方面，但由于它们的紧密联系，这个知识域将它们组合在一起处理。不幸的是，软件工业的一个普遍感觉是产品交付迟、超过预算、低质量、不确定的功能。有度量信息的管理—任何真正工程学科的假定的基本原则—可以帮助人们改变这种感觉。实际上，没有度量的管理，无论是定性还是定量，都说明缺乏严格性，没有管理的度量则说明了缺乏目标或上下文。同样，没有专家知识的管理和度量都是无效的，我们必须小心，以避免过分强调软件工程管理的定量方面。有效的管理需要数字和经验的组合。

这里采用了下列工作定义：（1）管理过程：指为保证软件工程过程以与组织的策略、

目标和标准一致的方式进行，而开展的活动。（2）度量：指将值或标签分配给软件工程的各个方面（如[Fen98]定义的产品、过程、资源），或从值或标签中导出的模型，无论这些模型是使用统计学、专家知识开发的，还是使用其它技术开发的。

软件工程项目管理子域广泛地利用了软件工程度量子域。

并非出乎意料，这个知识域与SWEBOK指南中其它知识域有紧密联系，将下面知识域的材料与本知识域一起阅读，将很有用处。（1）软件需求：描述了项目的启动和范围定义阶段要完成的一些活动。（2）软件配置管理：它处理软件配置的标识、控制、状态簿记和审计，以及软件发布管理和交付。（3）软件工程过程：过程和项目是紧密联系的（这个知识域也描述了过程与产品度量）。（4）软件质量：质量是管理的一个永恒目标，是许多必须管理的活动的目标。

软件工程管理主题分解结构

由于在这里，软件工程管理知识域被认为是一个组织过程，它要集成过程和项目的管理的概念，我们创建了一个分解结构，它既是基于主题的，也是基于生命周期的。但是，顶层分解结构的主要基础是管理软件工程项目的过程。有6个子域，前5个子域主要遵循

IEEE/EIA12207管理过程。6个子域是：（1）启动和范围定义：处理启动软件工程项目的决定。（2）软件项目计划：从管理的角度，处理为成功的软件工程作准备而要采取的活动。

（3）软件项目实施：处理软件工程中发生的普遍接受的软件工程管理活动。（4）评审和评价：处理软件要满足的保证。（5）关闭：处理软件工程项目完成后的活动。（6）软件工程度量：处理在软件工程组织中有效地开发和实现度量程序(IEEE12207.0-96)。

软件工程管理知识域的主题分解结构见图1。

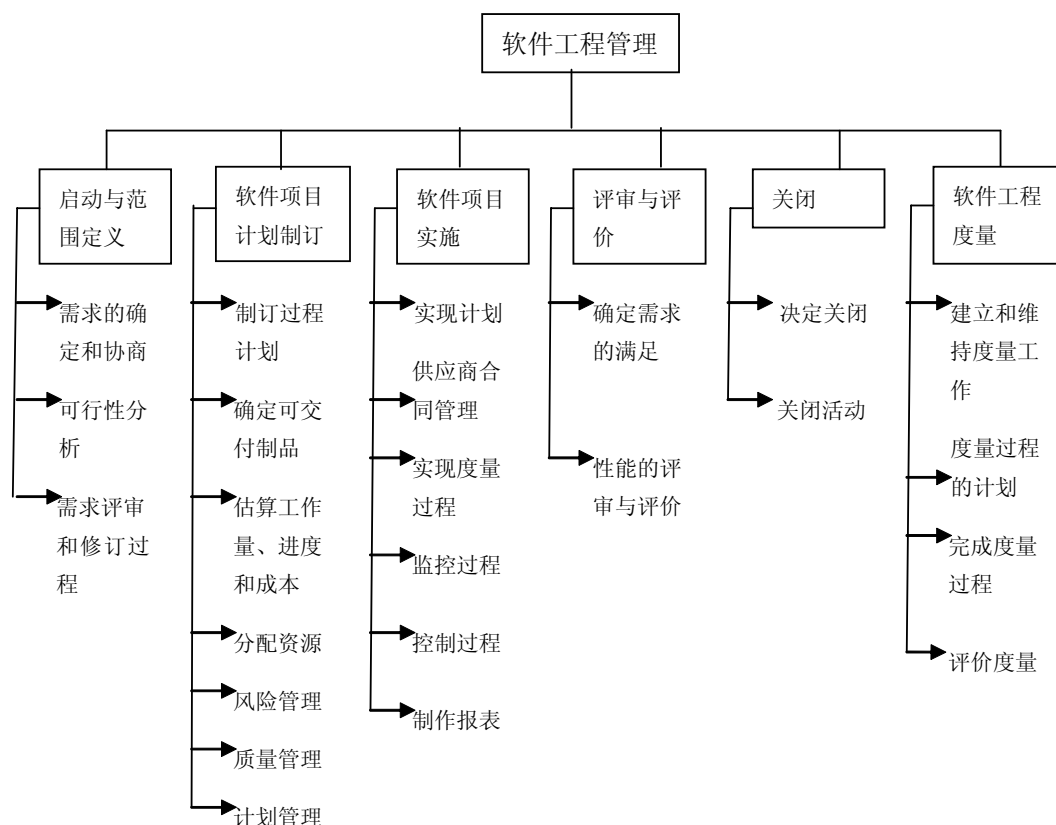


图1 软件工程管理知识域的主题分解结构

1 启动和范围定义

这组活动的焦点是通过各种获取方法来有效地确定软件需求,并从不同的视点来评估项目的可行性。一旦可行性建立后,这个过程中余下的任务就是需求验证和变更流程的规格说明(参校软件需求知识域)。

1.1 需求的确定和协商[Dor02: v2c4; Pf101: c4; Pre04: c7; Som05: c5]

必须选择和应用用于需求获取(例如,观察)、分析(例如,数据建模、用例建模)、规格说明和验证(例如,原型)的软件需求方法,并从不同干系人的角度考虑。这导致了确定项目的范围、目标和约束。这总是一个重要的活动,因为它要为将要开展的任务集合建立可见的边界,特别是当任务的创新性比较高时。附加的信息可以参见软件需求知识域。

1.2 可行性分析(技术的、运行的、财务的、社会的/政治的)[Pre04: c6; Som05: c6]

必须向软件工程师保证:在人员、技能、设备、基础设施和支持(内部和外部)方面,有足够的能力和资源可以使用,以保证项目可以按时完成,并具有很好的成本/效益比(例如,可以使用需求能力矩阵)。这通常需要一些基于适当方法(如给专家信息的模拟技术)来进行工作量和成本的估算。

1.3 需求的评审和修订过程

变更是不可避免的,干系人之间在早期时间点评审和修改范围和需求的手段方面达成一致非常重要(例如,一致同意的变更管理流程)。这清晰地隐含,范围和需求不是一成不变的,而是应该在过程展开时,在预定的点进行评定(例如,设计评审、管理评审)。如

果接受了变更，必须进行某种形式的可追踪性分析和风险分析（参见主题2.5风险分析）来确认这些变更的影响范围。受管理的变更方法在评审项目的输出时，很有用处，因为范围和需求形成了评价是否成功的基础[Som05: c6]。可以参见软件配置管理知识域的软件配置控制子域。

2 软件项目计划制订

迭代式的计划制订过程的输入信息是范围和需求，以及建立的可行性。在这一点，要评价软件生命周期过程，并选择最适当的（给定项目的本质、创新程度、功能与技术复杂性、质量需求等）。在相关的地方，项目本身要用任务的层次分解的方式进行计划，以质量和其它属性的方式并与需求列在一起，规定每个任务的可交付制品及特征，对每个任务进行详细的工作量、进度和成本估算。然后，将资源分配到各个任务上，以优化个人生产率（在个人、小组和组织层次上）、设备和物资利用、与进度的一致。要进行详细的风险管理，在所有干系人间讨论项目的风险概要，并被其接受。将综合性的软件质量管理过程作为计划制订过程的一部分来确定，给出软件质量保证、验证和确认、评审和审计（参加软件质量知识域）的流程和责任。作为一个迭代过程，明确陈述将要进行的计划管理、评审和修订的过程和责任，并达成一致，非常重要。

2.1 过程计划制订

以项目的特定范围和需求为样板，要采取适当的软件生命周期模型（例如，螺旋、进化原型），并修改和部署适当的软件生命周期过程，同时要选择相关的方法和工具[Dor02: v1c6, v2c8; Pf101: c2; Pre04: c2; Rei02: c1, c3, c5; Som05: c3; Tha97: c3]。在项目层次上，使用适当的方法和工具来将项目分解为任务，任务有相关的输入、输出和完成条件（例如，工作分解结构）[Dor02: v2c7; Pf101: c3; Pre04: c21; Rei02: c4, c5; Som05: c4; Tha97: c4, c6]。这又反过来影响项目的高层次进度和组织结构。

2.2 确定可交付制品

要规定每个任务的产品（例如，体系结构设计、检查报告等），说明产品的特性[Pf101: c3; Pre04: c24; Tha97: c4]。要评估复用以前的开发中的组件或利用其它商品软件的可能性，对使用第三方软件或采购的软件要作出相应计划，并选择供应商。

2.3 工作量、进度和成本估算

基于任务的分解、输入和输出，使用校正后的估算模型，基于可用的或相关的历史规模/工作量数据，要确定每个任务的期望工作量范围，也可以使用专家判断等模型。要建立任务之间的依赖关系，使用适当的方法（例如，关键路径分析）标识潜在瓶颈，在可能的地方解决瓶颈，生成期望的任务进度安排，为每个任务确定起始时间、持续时间和结束时间（例如，PERT图）。将资源（人力、工具）需求转换为成本估算[Dor02: v2c7; Fen98: c12; Pf101: c3; Pre04: c23, c24; Rei02: c5, c6; Som05: c4, c23; Tha97: c5]。这是一个高度迭代性的活动，必须进行协商和修订，直到干系人（主要是工程人员和管理人员）之间达成一致。

2.4 资源分配[Pf101: c3; Pre04: c24; Rei02: c8, c9; Som05: c4; Tha97: c6, c7]

将设备、工具和人员分配到安排的任务上，包括分配完成任务的责任（例如，可以使用Gantt图）。这个活动输入信息和约束条件是资源的可用性和在这些环境下资源的最优使用，以及与个人相关的问题（例如，个人/小组的生产率、小组变动、组织和小组结构）。

2.5 风险管理

要进行这些工作：风险的标识和分析（什么会出事、如何出事、为什么、可能的后果是什么）、关键风险评估（按显露程度，哪些是最重要的风险，我们能做什么来影响调节风险）、风险缓解和意外事件计划（形成一个策略来处理风险和管理风险大纲）。应使用风险评估方法（例如，决策树和过程模拟）来突出和评价风险。与所有其它干系人讨论后，应该在这点上确定项目放弃策略[Dor02: v2c7; Pf101: c3; Pre04: c25; Rei02: c11; Som05: c4; Tha97: c4]。软件特有的风险方面，诸如软件工程师可能增加不需要的特征或软件不可见本质带来的风险，也会影响项目的风险管理。

2.6 质量管理[Dor02: v1c8, v2c3-c5; Pre04: c26; Rei02: c10; Som05: c24, c25; Tha97: c9, c10]

质量定义为特定项目和任何相关产品的有关属性，可以定量，也可以定性。在详细的软件需求规格说明中，必须确定这些质量特征。可以参见软件需求知识域。为每个指标设置保持质量的限定值，以适合干系人对软件的期望。这个阶段也要规定与整个过程和产品（可交付制品）验证和确认中要进行的软件质量保证相关的流程（例如，技术评审和检查）（参见软件质量知识域）。

2.7 计划/管理[Som05: c4; Tha97: c4]

如何管理项目和计划如何管理，也必须对此作出计划。项目的报表制作、监控和控制必须适合选择的软件工程过程和项目的现实情况，必须反映在用于管理项目的各种制品中。但是，在变更是期望的而不是突如其来的环境中，计划本身也要受管理，这点很重要。这要求，对计划的遵照必须系统地指导、监控、报告，必要时，还要修改。与其它面向管理的支持过程相关的计划（例如，文档、软件配置管理和问题解决方案）也必须用同样的方式来管理。

3 软件项目实施

这里，计划被付诸实现，体现在计划中的过程被实施，自始至终，焦点是计划的遵循，最重要的期望是遵循计划可以导致成功地满足干系人的需求，并达到项目的目标。实施的基础是要进行的度量、监控、控制和报告制作的管理活动。

3.1 计划的实现[Pf101: c3; Som05: c4]

项目启动后，根据进度开展项目活动。在过程中，资源被利用（例如，人力、资金），产生可交付制品（例如，体系结构设计文档、测试用例）。

3.2 供应商合同管理[Som05: c4]

准备和执行与供应商的协议，监督供应商的完成情况，接收供应商的产品，适当时候，将它们集成。

3.3 实现度量过程[Fen98: c13, c14; Pre04: c22; Rei02: c10, c12; Tha97: c3, c10]

度量过程随软件项目而实施，保证收集到相关和有用的数据（参见主题6.2度量过程计划和主题6.3完成度量过程）。

3.4 监控过程[Dor02: v1c8, v2c2-c5, c7; Rei02: c10; Som05: c25; Tha97: c3; c9]

要连续地在预定义的间隔，评估对计划的遵循程度，分析每个任务的输出和完成情况，按照要求的特征来评价可交付制品（例如，采用评审和审计）。调查迄今为止花费的工作量、

进度情况和成本情况，检查资源使用情况。修改项目的风险大纲，评价对质量需求的遵循程度。

建立度量数据的模型并进行分析，进行基于实际与期望结果和数值的偏离的偏差分析，可能以成本超出、进度推迟等形式给出。还进行质量和其它度量数据的异常标识和分析（例如，缺陷密度分析）。重新计算风险显露和调节，按照新数据，重新运行决策树和模拟等。这些活动可以完成基于超过阈值的问题检测和异常标识。当可接受的阈值被超过后，或必要的时候，将输出结果制作成报表。

3.5 控制过程[Dor02: v2c7; Rei02: c10; Tha97: c3, c9]

过程监控活动的输出为下一步采取行动提供了基础，适当时，或建立变更的影响范围和相关的风险的模型，并进行分析后，可以对项目作出变更，可以采用更正性行动的方式（例如，重新测试某些组件），这可能涉及意外事件的合并，这样可以避免类似的情况发生（例如，使用原型来辅助软件需求确认的决定），也可能要求修订各种计划和其它项目文档（例如，需求规格说明）以容纳非预期的结果及其隐含的结果。在一些情况下，这可能导致放弃项目。在所有情况下，要遵循变更控制和软件配置管理流程（参见软件配置管理知识域），将决定记录到文档中，与相关各方交流，必要时，重新审查和修改计划，相关的数据要记录到中心数据库中（参见主题6.3完成度量过程）。

3.6 制作报表[Rei02: c10; Tha97: c3, c10]

在规定和同意的期间内，要向组织内部（例如，向项目级别监督委员会）和外部干系人（例如，客户、用户）报告计划的遵循情况。这类情况的报告应集中于总体情况，这与项目组内频繁要求的详细报告不同。

4 评审和评价

在项目的关键点，要评价向着达到规定的目标和满足干系人需求的总体进展。同样，在特定的里程碑，要评估迄今为止的总体过程的有效性、涉及的人员、使用的工具和方法。

4.1 确定需求的满足情况[Rei02: c10; Tha97: c3, c10]

由于达到干系人（用户和客户）的满意是主要的目标，必须正式地和周期性地评估向这个目标的进展，这点很重要。在达到项目的主要里程碑（例如，软件设计体系结构的确认、软件集成技术评审）时，就要进行评估，要标识对期望的偏离，并采取适当的行动。与前面的控制过程活动中一样（参见主题3.5控制过程），在所有情况下，要遵循变更控制和软件配置管理流程（参见软件配置管理知识域），将决定记录到文档中，与相关各方交流，必要时，重新审查和修改计划，相关的数据要记录到中心数据库中（参见主题6.3完成度量过程）。更多的信息可以参见软件测试知识域主题2.2测试目标和软件质量知识域主题2.3评审和审计。

4.2 性能的评审和评价[Dor02: v1c8, v2c3, c5; Pf101: c8, c9; Rei02: c10; Tha97: c3, c10]

对项目人员进行周期性性能评审，为、可以了解遵循计划的可能性和可能会有困难的地方（例如，小组成员的冲突）。要评价使用的各种方法、工具和技术的有效性和适当性，过程本身在项目上下文中的相关性、效用和功效，也要系统地和周期性地评价。适当的时候，要进行变更并对之进行管理。

5 关闭

当所有的计划和包含的过程都实施和完成后，项目达到关闭阶段，在这个阶段，重新审查项目成功的准则。一旦关闭成立，就进行归档、事后分析和过程改进活动。

5.1 确定关闭[Dor02: v1c8, v2c3, c5; Rei02: c10; Tha97: c3, c10]

计划中规定的任务完成后，要确认完备准则的满意达到。所有计划的产品都要交付并具有可接受的特征，核对需求，确认已被满足，项目的目标已经达到。这些过程一般涉及所有干系人，形成客户接收记录文档和任何余留已知问题报告。

5.2 关闭活动[Pf101: c12; Som05: c4]

确认关闭后，要将项目材料按符合干系人同意的的方法、位置和保存期限进行归档，用最终的项目数据更新组织的度量数据库，进行项目后的分析。要进行项目的事后分析，这样可以分析过程（特别是通过评审和评价，见子域4）中遇到的结果、问题和机会，从过程中获得教训，加入到在组织的学习和改进活动中（参见软件工程过程知识域）。

6 软件工程量度[IS015939-02]

度的重要性及其在更好的管理实践中的作用已被广泛认同，因此，在以后，它的重要性只会增加。有效的度量已经成为组织成熟度的一个基础。基于ISO国际度量衡词汇表[IS093]，[IS015939-02]定义了软件度量和度量方法的关键术语。不过，读者仍可能在文献中遇到不同的术语，例如，有时，用“度量学(metrics)”代替“度量(measure)”。这个主题遵循国际标准ISO/IEC 15939，它描述了定义实现软件度量过程必要的活动和任务的过程，并包括一个度量信息模型。

6.1 建立和维持度量工作

接受度量的需求：每个度量工作应该由组织目标指导，并由组织和项目建立的一组度量需求驱动，例如，组织的目标可能是“新产品最先进入市场”[Fen98: c3, c13; Pre04: c22]，这反过来可能产生一个需求，要求度量对这个目标有贡献的因素，以便管理项目来满足这个目标。（1）定义度量的范围：必须建立要应用每个度量需求的组织单元，这可能由一个功能区域、单个项目、单个地点、甚至整个企业组成，以后，与这个需求有关的所有度量任务应该在定义的范围内开展。另外，应该标识干系人。（2）委派管理和职员进行度量。工作必须正式建立，进行交流和有资源支持（见下）。

为度量分配资源：组织的度量工作是成功的基本因素，要为实现度量过程分配资源。分配资源包括为各种度量过程的任务分配责任（如用户、分析员和资料管理员），提供适当的资金、培训、工具和支持，以便能持续地进行这个过程。

6.2 度量过程的计划

确定组织单元特征：组织单元提供了度量的上下文，将上下文明确提出，清晰地指出它包含的内容和它施加的约束，很重要。特征可以是组织的各种过程、应用领域、技术和组织接口。组织过程模型通常是组织单元特征的元素[IS015939-02: 5.2.1]。

标识信息要求：信息要求基于组织单元的目标、约束、风险和问题，它们可以从业务、组织结构、规则和/或产品目标中导出，必须标识它们并排列优先顺序。然后，选择要解决的一个子集，将结果记录到文档中，进行交流，由干系人评审[IS015939-02: 5.2.2]。

选择度量：必须选择候选的度量，与信息要求有清晰的联系，然后基于信息要求的优先性和其它准则（收集成本、收集中过程的破坏程度、分析的难易、获得精确一致数据的难易等），选择度量 [IS015939-02: 5.2.3 和Appendix C]。

定义数据的收集、分析和报表制作流程：这包括数据的收集流程和进度、存储、验证、分析、报表制作和配置管理[IS015939-02:5.2.4]。

定义评价信息产品的准则：评价的准则受组织单元的技术和业务目标的影响，信息产品包括与正在生产的产品相关的，以及与用来管理和度量项目的过程相关的[IS015939-02: 5.2.5 and Appendices D, E]。

评审、批准和为度量任务提供资源：（1）必须评审度量计划，并由适当的干系人批准，这包括所有的数据收集流程、存储、分析和报表制作流程、评价准则、进度和责任。评审这些制品的准则应该在组织单元或更高的层次上建立，并用作评审的基础。这些准则应该考虑以前的经验、资源的可用性、当建议对目前的实践进行变更时对项目的破坏程度。批准表示了对度量过程的承诺[IS015939-02: 5.2.6.1 and Appendix F]。（2）应该让资源对于实现列入计划并被批准的度量任务是可用的，资源可用性可能在变更被控制在广泛部署之前就准备好。应该考虑为成功部署新流程或度量所必须的资源[IS015939-02: 5.2.6.2]。

获取和部署支持技术：这包括评价可用的支持技术、选择最适当的技术、获取这些技术和部署这些技术[IS015939-02:5.2.7]。

6.3 完成度量过程

将度量流程与相关的过程集成：诸如数据收集等度量流程，必须集成到它们度量的过程中，这可能涉及改变目前的过程以包容数据收集或产生活动，也可能涉及分析目前的过程，将对雇员的影响的附加的工作量和评价最小化，以保证度量流程被接受。需要考虑士气问题和其它人类因素问题。此外，必须与提供数据的人员交流度量流程，可能需要提供培训，还要提供支持。通常，数据分析和报表制作流程必须用类似的方式集成到组织和/或项目过程[IS015939-02: 5.3.1]。

收集数据：必须收集、验证和存储数据[IS015939-02 :5.3.2]。

分析数据和开发信息产品：作为分析过程的一部分，使用适合于数据本质和信息要求的某种严密程度，数据被聚集、变换或重新编码。分析的结果通常是一些指标，如图形、数值或其它必须解释的指标，得到要提交给干系人的初步结论。必须使用组织定义的过程（可能是正式或非正式的），来评审结果和结论。数据提供者和度量用户应参与数据评审，以保证数据有意义和精确，并能导致合理的行动[IS015939-02: 5.3.3 and Appendix G]。

交流结果：信息产品必须记录到文档中，并与用户和干系人交流[IS015939-02: 5.3.4]。

6.4 评价度量

评价信息产品：根据规定的评价准则来评价信息产品，确定信息产品的优缺点。这可以由内部过程或外部审计来完成，并要包括来自度量用户的反馈。将经验和教训记录到适当的数据库中[IS015939-02: 5.4.1 and Appendix D]。

评价度量过程：根据规定的评价准则来评价度量过程，确定过程的优缺点。这可以由内部过程或外部审计来完成，并要包括来自度量用户的反馈。将经验和教训记录到适当的数据库中 [IS015939-02: 5.4.1和Appendix D]。

标识潜在的改进：这类改进可能是指标格式的改变、度量单位的改变或重新分类。确定潜在改进的成本和效益，选择适当的改进行动。与度量过程所有者和干系人进行交流，以便评审和批准。如果分析没有标识出改进，也要指出缺乏潜在的改进[IS015939-02: 5.4.2]。

主题与参考文献矩阵

	[Dor 02]	[ISO1593 9-02]	[Fen 98]	[Pf1 01]	[Pre 04]	[Rei 02]	[Som 05]	[Tha9 7]
1 启动和范围定义								
1.1 需求的确定和协商	V2c4			C4	C7		C5	
1.2 可行性研究					C6		C6	
1.3 需求的评审和修订过程							C6	
2 软件项目计划制订								
2.1 过程计划制订	V1c6 , v2c 7, v2 c8			C2, c 3	C2, c 21	C1, c 3, c5	C3, c 4	C3, c4 , c6
2.2 确定可交互制品				C3	C24			C4
2.3 工作量、进度和成本估算	V2c7		C12	C3	C23, c24	C5, c 6	C2, c 23	C5
2.4 资源分配				C3	C24	C8, c 9	C4	C6, c7
2.5 风险管理	V2c7			C3	C25	C11	C4	C4
2.6 质量管理	V1c8 , v2c 3-c5				C26	C10	C24, c25	C9, c1 0
2.7 计划管理							C4	C4
3 软件项目实施								
3.1 计划的实现				C3			C4	
3.2 供应商合同管理							C4	
3.3 度量过程的实现			C13, c14		C22	C10, c12		C3, c1 0
3.4 监控过程	V1c8 , v2c 2-c5 , c7					C10	C25	C3, c9
3.5 控制过程	V2c7					C10		C3, c9
3.6 制作报表						C10		C3, c1 0
4 评审与评价								
4.1 确定需求的满足						C10		C3, c1 0
4.2 评审和评价性能	V1c8 , v2c 3, c5			C8, c 9		C10		C3, c1 0
5 关闭								
5.1 确定关闭	V1c8					C10		C3, c1

	, v2c 3, c5							0
5.2 关闭活动				C12			C4	
6 软件工程度量		*						
6.1 建立和维持度量 工作			C3, c 13		C22			
6.2 度量过程计划		C5, C, D, E , F						
6.3 完成度量过程		C5, G						
6.4 评价度量		C5, D						

软件工程管理的推荐参考文献

- [Dor02] M. Dorfman and R. H. Thayer, Eds., "Software Engineering." (Vol. 1 & vol. 2), IEEE Computer Society Press, 2002, Vol. 1, Chap. 6, 8, Vol. 2, Chap. 3, 4, 5, 7, 8.
- [Fen98] N. E. Fenton and S. L. Pfleeger, "Software Metrics: A Rigorous & Practical Approach," Second ed: International Thomson Computer Press, 1998, Chap. 1-14.
- [ISO15939-02] ISO/IEC 15939:2002, *Software Engineering-Software Measurement Process*: ISO and IEC, 2002.
- [Pf101] S. L. Pfleeger, "Software Engineering: Theory and Practice," Second ed: Prentice-Hall, 2001, Chap 2-4, 8, 9, 12, 13.
- [Pre04] R. S. Pressman, "Software Engineering: A Practitioner's Approach," Sixth ed: McGraw-Hill, 2004, Chap. 2, 6, 7, 22-26.
- [Rei02] D. J. Reifer, Ed., "Software Management." IEEE Computer Society, 2002, Chap. 1-6, 7-12, 13.
- [Som05] I. Sommerville, "Software Engineering," Seventh ed: Addison-Wesley, 2005, Chap. 3-6, 23-25.
- [Tha97] R. H. Thayer, Ed., "Software Engineering Project Management." IEEE Computer Society, 1997, Chap. 1-10

附录A 深入阅读的参考文献

- (Ad199) T. R. Adler, J. G. Leonard and R. K. Nordgren, "Improving Risk Management: Moving from Risk Elimination to Risk Avoidance," *Information and Software Technology*, vol. 41, 29-34, 1999
- (Bai98) R. Baines, "Across Disciplines: Risk, Design, Method, Process, and Tools," *IEEE Software*, 61-64, July/August, 1998
- (Bin97) R. V. Binder, "Can a Manufacturing Quality Model Work for Software?," *IEEE Software*, 101-102, 105, September/October, 1997
- (Boe97) B. W. Boehm and T. DeMarco, "Software Risk Management," *IEEE Software*, 17-19, May/June, 1997
- (Bri96) L. C. Briand, S. Morasca and V. R. Basili, "Property-Based Software Engineering Measurement," *IEEE Transactions on Software Engineering*, vol. 22, iss. 1, 68-86, 1996

- (Bri96a) L. Briand, K. E. Emam and S. Morasca, "On the Application of Measurement Theory in Software Engineering," *Empirical Software Engineering*, vol. 1, 61-88, 1996
- (Bri97) L. C. Briand, S. Morasca and V. R. Basili, "Response to: Comments on 'Property-based Software Engineering Measurement: Refining the Additivity Properties'," *IEEE Transactions on Software Engineering*, vol. 23, iss. 3, 196-197, 1997
- (Bro87) F. P. J. Brooks, "No Silver Bullet: Essence and Accidents of Software Engineering," *Computer*, 10-19, Apr., 1987
- (Cap96) J. Capers, *Applied Software Measurement: Assuring Productivity and Quality*, Second ed: McGraw-Hill, Inc., 1996.
- (Car97) M. J. Carr, "Risk Management May Not Be For Everyone," *IEEE Software*, 21-24, May/June, 1997
- (Cha96) R. N. Charette, "Large-Scale Project Management is Risk Management," *IEEE Software*, 110-117, July, 1996
- (Cha97) R. N. Charette, K. M. Adams and M. B. White, "Managing Risk in Software Maintenance," *IEEE Software*, 43-50, May/June, 1997
- (Col96) B. Collier, T. DeMarco and P. Fearey, "A Defined Process for Project Postmortem Review," *IEEE Software*, 65-72, July, 1996
- (Con97) E. H. Conrow and P. S. Shishido, "Implementing Risk Management on Software Intensive Projects," *IEEE Software*, 83-89, May/June, 1997
- (Dav98) A. M. Davis, "Predictions and Farewells," *IEEE Software*, 6-9, July/August, 1998
- (Dem87) T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*: Dorset House Publishing, 1987.
- (Dem96) T. DeMarco and A. Miller, "Managing Large Software Projects," *IEEE Software*, 24-27, July, 1996
- (Fav98) J. Favaro and S. L. Pfleeger, "Making Software Development Investment Decisions," *ACM SIGSoft Software Engineering Notes*, vol. 23, iss. 5, 69-74, 1998
- (Fay96) M. E. Fayad and M. Cline, "Managing Object-Oriented Software Development," *Computer*, 26-31, September, 1996
- (Fen98) N. E. Fenton and S. L. Pfleeger, *Software Metrics: A Rigorous & Practical Approach*, Second ed: International Thomson Computer Press, 1998.
- (Fle99) R. Fleming, "A Fresh Perspective on Old Problems," *IEEE Software*, 106-113, January/February, 1999
- (Fug98) A. Fuggetta, L. Lavazza, S. Morasca, S. Cinti, G. Oldano and E. Orazi, "Applying GQM in an Industrial Software Factory," *ACM Transactions on Software Engineering and Methodology*, vol. 7, iss. 4, 411-448, 1998
- (Gar97) P. R. Garvey, D. J. Phair and J. A. Wilson, "An Information Architecture for Risk Assessment and Management," *IEEE Software*, 25-34, May/June, 1997
- (Gem97) A. Gemmer, "Risk Management: Moving beyond Process," *Computer*, 33-43, May, 1997
- (Gla97) R. L. Glass, "The Ups and Downs of Programmer Stress," *Communications of the ACM*, vol. 40, iss. 4, 17-19, 1997
- (Gla98) R. L. Glass, "Short-Term and Long-Term Remedies for Runaway Projects,"

- Communications of the ACM*, vol. 41, iss. 7, 13-15, 1998
- (Gla98a) R. L. Glass, "How Not to Prepare for a Consulting Assignment, and Other Ugly Consultancy Truths," *Communications of the ACM*, vol. 41, iss. 12, 11-13, 1998
- (Gla99) R. L. Glass, "The Realities of Software Technology Payoffs," *Communications of the ACM*, vol. 42, iss. 2, 74-79, 1999
- (Gra99) R. Grable, J. Jernigan, C. Pogue and D. Divis, "Metrics for Small Projects: Experiences at the SED," *IEEE Software*, 21-29, March/April, 1999
- (Gra87) R. B. Grady and D. L. Caswell, *Software Metrics: Establishing A Company-Wide Program*. Englewood Cliffs NJ, USA: Prentice-Hall, 1987.
- (Hal97) T. Hall and N. Fenton, "Implementing Effective Software Metrics Programs," *IEEE Software*, 55-64, Mar/Apr, 1997
- (Hen99) S. M. Henry and K. T. Stevens, "Using Belbin's Leadership Role to Improve Team Effectiveness: An Empirical Investigation," *Journal of Systems and Software*, vol. 44, 241-250, 1999
- (Hoh99) L. Hohmann, "Coaching the Rookie Manager," *IEEE Software*, 16-19, January/February, 1999
- (Hsi96) P. Hsia, "Making Software Development Visible," *IEEE Software*, 23-26, March, 1996
- (Hum97) W. S. Humphrey, *Managing Technical People: Innovation, Teamwork, and the Software Process*: Addison-Wesley, 1997.
- (IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- (Jac98) M. Jackman, "Homeopathic Remedies for Team Toxicity," *IEEE Software*, 43-45, July/August, 1998
- (Kan97) K. Kansala, "Integrating Risk Assessment with Cost Estimation," *IEEE Software*, 61-67, May/June, 1997
- (Kar97) J. Karlsson and K. Ryan, "A Cost-Value Approach for Prioritizing Requirements," *IEEE Software*, 87-74, September/October, 1997
- (Kar96) D. W. Karolak, "Software Engineering Risk Management," *IEEE Computer Society*, 1996
- (Kau99) K. Kautz, "Making Sense of Measurement for Small Organizations," *IEEE Software*, 14-20, March/April, 1999
- (Kei98) M. Keil, P. E. Cule, K. Lyytinen and R. C. Schmict, "A Framework for Identifying Software Project Risks," *Communications of the ACM*, vol. 41, iss. 11, 76-83, 1998
- (Ker99) B. Kernighan and R. Pike, "Finding Performance Improvements," *IEEE Software*, 61-65, March/April, 1999
- (Kit97) B. Kitchenham and S. Linkman, "Estimates, Uncertainty, and Risk," *IEEE Software*, 69-74, May/June, 1997
- (Lat98) F. v. Latum, R. v. Solingen, M. Oivo, B. Hoisl, D. Rombach and G. Ruhe, "Adopting GQM-Based Measurement in an Industrial Environment," *IEEE Software*, 78-86, January-February, 1998
- (Leu96) H. K. N. Leung, "A Risk Index for Software Producers," *Software Maintenance:*

- Research and Practice*, vol. 8, 281-294, 1996
- (Lis97) T. Lister, "Risk Management is Project Management for Adults," *IEEE Software*, 20-22, May/June, 1997
- (Mac96) K. Mackey, "Why Bad Things Happen to Good Projects," *IEEE Software*, 27-32, May, 1996
- (Mac98) K. Mackey, "Beyond Dilbert: Creating Cultures that Work," *IEEE Software*, 48-49, January/February, 1998
- (Mad97) R. J. Madachy, "Heuristic Risk Assessment Using Cost Factors," *IEEE Software*, 51-59, May/June, 1997
- (McC96) S. C. McConnell, *Rapid Development: Taming Wild Software Schedules*: Microsoft Press, 1996.
- (McC97) S. C. McConnell, *Software Project Survival Guide*: Microsoft Press, 1997.
- (McC99) S. C. McConnell, "Software Engineering Principles," *IEEE Software*, 6-8, March/April, 1999
- (Moy97) T. Moynihan, "How Experienced Project Managers Assess Risk," *IEEE Software*, 35-41, May/June, 1997
- (Ncs98) P. Ncsi, "Managing OO Projects Better," *IEEE Software*, 50-60, July/August, 1998
- (Nol99) A. J. Nolan, "Learning From Success," *IEEE Software*, 97-105, January/February, 1999
- (Off97) R. J. Offen and R. Jeffery, "Establishing Software Measurement Programs," *IEEE Software*, 45-53, Mar/Apr, 1997
- (Par96) K. V. C. Parris, "Implementing Accountability," *IEEE Software*, 83-93, July, 1996
- (Pfl97) S. L. Pfleeger, "Assessing Measurement (Guest Editor's Introduction)," *IEEE Software*, 25-26, March/April, 1997
- (Pfl97a) S. L. Pfleeger, R. Jeffery, B. Curtis and B. Kitchenham, "Status Report on Software Measurement," *IEEE Software*, 33-43, March/April, 1997
- (Put97) L. H. Putman and W. Myers, "Industrial Strength Software - Effective Management Using Measurement," Los Alamitos, CA, 1997
- (Rob99) P. N. Robillard, "The Role of Knowledge in Software Development," *Communications of the ACM*, vol. 42, iss. 1, 87-92, 1999
- (Rod97) A. G. Rodrigues and T. M. Williams, "System Dynamics in Software Project Management: Towards the Development of a Formal Integrated Framework," *European Journal of Information Systems*, vol. 6, 51-66, 1997
- (Rop97) J. Ropponen and K. Lyytinen, "Can Software Risk Management Improve System Development: An Exploratory Study," *European Journal of Information Systems*, vol. 6, 41-50, 1997
- (Sch99) C. Schmidt, P. Dart, L. Johnston, L. Sterling and P. Thorne, "Disincentives for Communicating Risk: A Risk Paradox," *Information and Software Technology*, vol. 41, 403-411, 1999
- (Sco92) R. L. v. Scoy, "Software Development Risk: Opportunity, Not Problem," Software Engineering Institute, Carnegie Mellon University CMU/SEI-92-TR-30, 1992
- (Sla98) S. A. Slaughter, D. E. Harter and M. S. Krishnan, "Evaluating the Cost of

Software Quality," *Communications of the ACM*, vol. 41, iss. 8, 67-73, 1998

(Sol98) R. v. Solingen, R. Berghout and F. v. Latum, "Interrupts: just a minute never is," *IEEE Software*, 97-103, September/October, 1998

(Whi95) N. Whitten, *Managing Software Development Projects: Formulas for Success*: Wiley, 1995.

(Wil99) B. Wiley, *Essential System Requirements: A Practical Guide to Event-Driven Methods*: Addison-Wesley, 1999.

(Zel98) M. V. Zelkowitz and D. R. Wallace, "Experimental Models for Validating Technology," *Computer*, vol. 31, iss. 5, 23-31, 1998

附录B 有关的标准

(IEEE610.12-90) IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.

(IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.

(ISO15939-02) ISO/IEC 15939:2002, *Software Engineering-Software Measurement Process*: ISO and IEC, 2002.

(PMI00) Project Management Institute Standards Committee, *A guide to the project management body of knowledge (PMBOK)*: Project Management Institute, 2000

////////////////////////////////////

第9章 软件工程过程

术语和缩写

CMMi: Capability Maturity Model Integration, 能力成熟度模型集成

EF: Experience Factory, 经验工厂

FP: Function Point, 功能点

HRM: Human Resources Management, 人力资源管理

IDEAL: Initiating-Diagnosing-Establishing-Acting-Leaning (model), 启动—诊断—建立—行动—学习 (模型)

OMG: Object Management Group, 对象管理集团

QIP: Quality Improvement Paradigm, 质量改进范例

SCAMPI: CMM Based Appraisal for Process Improvement using the CMMi, 基于CMM的对使用CMMi的过程改进的鉴定

SCE: Software Capability Evaluation, 软件能力评价

SEPG: Software Engineering Process Group, 软件工程过程小组

引言

可以在两个层次上分析软件工程过程知识域,第一个层次包括软件生命周期过程中的技术的和管理的活动,它们是在软件获取、开发、维护和退出运行中完成的。第二个层次是元层次,涉及软件生命周期过程本身的定义、实现、评估、管理、变更和改进。第一个层次由指南中其它知识域覆盖,本知识域关注第二个。

术语“软件工程过程”可以用不同方式解释,并可能引起混淆。(1)在一种使用了冠

词the的含义中，隐含只有一种正确的方式来完成软件工程任务。本指南将避免这种含义，因为这种过程不存在。IEEE12207等标准谈到各种软件工程过程，表示涉及了许多过程，如开发过程或配置管理过程。（2）第二种含义指对与软件工程相关的过程的一般性讨论，这是这个知识域的标题打算使用的含义，也是知识域描述中经常打算使用的含义。（3）最后，第三个含义可能突出组织内完成的一组活动，它们可以被认为是一个过程，特别是从组织内部来看。本知识域中在少数地方 使用了这个含义。

这个知识域可应用于要为过程或过程改进引入流程性或技术性的变更的软件生命周期过程的管理的任何部分。

软件工程过程不仅仅与大型组织有关，相反，与过程有关的活动可以并已经成功地由小型组织、小组和个人完成。

管理软件生命周期过程的目标是在实践中实现新的或更好的过程，无论是个人、项目，还是组织。

这个知识域并不明显地处理人力资源管理，例如，象人员能力成熟度模型（People CMM）（Cur02）和系统工程过程中体现的那样[ISO1528-028； IEEE 1220-98]。

应该认识到，许多软件工程过程问题与其它学科紧密相关，如管理，虽然有时使用了不同的术语。

软件工程过程主题分解结构ENGINEERING PROCESS

图1出了这个知识域的主题分解结构。

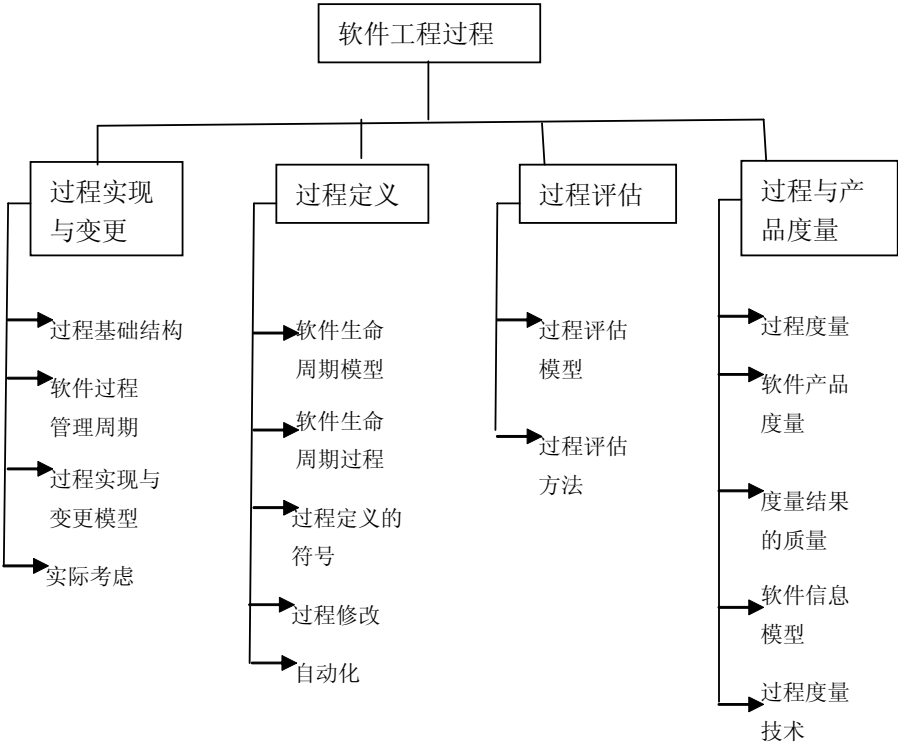


图 1 软件工程过程知识域主题分解结构

1 过程实现与变更

这个子域集中在组织变更上，它描述了过程实现和变更的基础结构、活动、模型和实际考虑。这里描述的是第一次开展过程时的情况（例如，引入一个检查过程到项目中或引入一个覆盖整个生命周期的方法），和变更当前过程的情况（例如，引入一个工具或优化一个流

程)，后者可以称为过程进化。在两种情况下，都必须修改现存的实践，如果修改是广泛的，就有必要改变组织的文化。

1.1 过程基础结构[IEEE12207.0-96; ISO15504-98; SEL96]

这个主题包括与软件工程过程基础结构相关的知识。

为建立软件生命周期过程，在适当的地方有必要有一个适当的基础结构，即必须有可用的资源（有能力的职员、工具和资金）并分配了责任。当完成这些任务后，它就是管理对软件工程过程工作的义务和所有权的指示。必须成立不同的委员会，诸如监督软件工程过程工作的指导委员会。

[McF96]提供了一般过程改进的基础结构的描述，实践中使用了两类主要的基础结构：软件工程过程小组和经验工厂。

1.1.1 软件工程过程小组(SEPG)

软件工程过程小组应该成为软件过程改进的中心点，在启动和维持过程改进方面具有大量的责任，[Fow90]描述了这些责任。

1.1.2 经验工厂(EF)

经验工厂的概念将项目组织（例如，软件开发组织）与改进组织分开。项目组织集中于软件的开发和维护，而经验工厂关注软件工程过程改进。

经验工厂的目的是通过开发、更新和向项目组织交付经验包（例如，指南、模型和培训课程，又叫过程财产），将组织的集体学习制度化。项目组织向经验工厂提供其产品、开发中使用的计划、开发和运行中收集的数据。经验包的例子见[Bas92]。

1.2 软件过程管理周期[Bas92; Fow90; IEEE12207.0-96; ISO15504-98; McF96; SEL96]

软件过程管理由迭代周期中顺序排列的4个活动组成，以允许软件过程的连续反馈和改进：（1）建立过程基础结构活动由向过程实现和变更建立承诺（包括获得管理大宗买进）、将适当的基础结构（资源和责任）放在适当位置，让其起作用。（2）计划活动的目标是理解单个项目或组织的业务目标和过程需求，以标识其缺点，作出过程实现和变更的计划。（3）过程实现与变更的目标是执行计划，开展新的过程（可能涉及工具部署和职员培训），和/或变更现有过程。（4）过程评价关注发现实现和变更进行的如何，期望的收益是否已经物化。结果将作为随后周期的输入。

1.3 过程实现与变更的模型

已经有两个导出实现和变更的一般模型：质量改进范例QIP[SEL96]和启动—诊断—建立—行动—学习模型IDEAL[McF96]，[SEL96]中比较了这两个模型，过程实现与变更的评价的结果可以是定性的或定量的。

1.4 实际考虑

过程实现与变更组成了组织变更的一个实例，许多成功的组织变更工作将变更作为一个项目处理，并具有适当的计划、监控和评审。[Moi98; San98; Sti99]中给出了软件工程组织内的过程实现和变更指南，包括行动计划、管理发起和委派、选择覆盖过程和工具的示范项目等，(ElE99a)报告了过程变更的成功因素的经验研究。

(Hut94)讨论了在这个活动中变更机构的作用。过程实现与变更也可以看成（内部或外部）咨询的例子。也可以从技术转移的角度看待组织变更(Rog83)，(Pf199; Rag89)讨论了

技术转移和新技术（可能包括与过程有关的技术）接收者的特征。

有两种方式来评价过程实现与变更：过程本身的变更或过程输出的变更（例如，度量进行变更的投资回报）。（Her98）给出了从这类评价研究可以得到什么的实用观点。如何评价过程实现与变更的综述，以及如何进行的例子，可以参见[Go199]，（Kit98；Kra99；McG94）。

2 过程定义

过程的定义可以是一个流程、一个策略或一个标准。根据大量的原因，定义了软件生命周期过程，包括提高产品质量、加速人类理解和通信、支持过程改进、支持过程管理、提供自动化的过程指导、提供自动化的执行支持。需要的过程定义的类型在很大程度上依赖于定义的原因。

应该指出，项目和组织的上下文将确定最有用的过程定义的类型，要考虑的重要变量包括工作的本质（例如，维护或开发）、应用领域、生命周期模型、组织的成熟度。

2.1 生命周期模型[Pf101:c2；IEEE12207.0-96]

软件生命周期模型作为开发中发生的阶段的高层定义，它们目标不是提供详细的定义，而是突出关键活动及其相互依赖。软件生命周期模型的例子是瀑布模型、丢弃式原型模型、进化开发、增量/迭代交付、螺旋模型、可复用软件模型和自动软件综合。这些模型的比较可参见[Com97]，（Dav88），如何选择这些模型可以参见（Ale91）。

2.2 软件生命周期过程

软件生命周期过程的定义比软件生命周期模型更详细，但是软件生命周期过程并不试图按时间顺序排列这些过程，这意味着，在原则上，可以安排软件生命周期过程，以适合任何软件生命周期模型。这个领域的主要参考文献是IEEE/EIA12207.0：信息技术—软件生命周期过程[IEEE12207.0-96]。

IEEE 1074:1997关于开发生命周期过程的标准也为软件开发和维护提供了一个过程和活动列表[IEEE1074-97]，以及一个可以映射过程并按任何软件生命周期模型组织的生命周期活动列表。此外，它标识了其它IEEE软件标准，并将这些标准链接到这些活动。原理上，IEEE 1074标准可用于建造遵从任何生命周期模型的过程。集中于维护过程的标准有IEEE 1219-1998和ISO 14764: 1998 [IEEE1219-98]。

其它提供过程定义的重要标准包括：（1）IEEE 1540标准：软件风险管理（IEEE1540-01）。（2）IEEE 1517标准：软件复用过程（IEEE1517-99）。（3）ISO/IEC 15939标准：软件度量过程[ISO15939-02]。这个过程的详细描述可以参见软件工程管理知识域。

在某些情况下，必须定义考虑组织的质量管理过程的软件工程过程，ISO 9001 [ISO9001-00]为质量管理过程提供了需求，ISO/IEC 90003为组织开发软件解释了这些需求（ISO9003-04）。

一些软件生命周过程强调快速的交付和较强的用户参与，例如极限编程等敏捷方法[Bec99]，一个选择问题的表格关注沿敏捷的计划驱动方法轴的选择，（Boe03a）描述了作出这个决定的基于风险的方法。

2.3 过程定义符号

可以在不同的抽象层次上定义过程（例如，一般的定义与适应性定义、描述性与说明性与强制性）[Pf101]。可以定义过程的各种元素，例如，活动、产品（人工制品）和资源。（Mad94）描述了将定义过程需要的信息类型组织起来的详细的框架。

有许多符号用于定义过程（SPC92），它们之间的关键区别是前面提到的框架定义、捕获

和使用的信息类型，软件工程师应该知道以下方法：表示过程目的和结果的数据流图 [ISO15504-98]和分解为用自然语言定义的组成活动和任务的过程列表[IEEE12207.0-96]、状态图(Har98)、ETVX (Rad85)、行动者依赖模型(Yu94)、结构化分析设计技术SADT符号 (Mcg93)、Petri网(Ban95)、IDEF0 (IEEE1320.1-98)和基于规则的符号(Bar95)。最近，对象管理集团发布了试图协调建模符号的过程建模标准，称为SPEM (Software Process Engineering Meta-Model，软件过程工程元模型)规范[OMG02]。

2.4 过程修改[IEEE12207.0-96; ISO15504-98; Joh99]

预先定义的过程——甚至是标准过程——也必须修改以适应局部需要，例如，组织上下文、项目规模、管制需求、业界实践和公司文化，这一点很重要。一些标准如IEEE/EIA 12207，包含完成这些修改的机制和推荐方法。

2.5 自动化[Pf101:c2]

自动化工具或者支持过程定义的执行，或为完成定义的过程的人提供指导。当要进行过程分析时，一些工具可以用来进行不同类型的模拟（例如，离散事件模拟）。

此外，有一些工具支持上面的过程定义的符号，而且，这些工具可以执行过程定义，以为实际过程提供自动支持，或在一些情况下将过程定义完全自动化。过程建模工具的综述可以参见[Fin94]，以过程为中心的环境的综述可以参见(Gar96)，(Kel98)描述了将Internet应用到准备实时过程指导的工作。

3 过程评估

过程评估可以用评估模型和评估方法来开展，有时，术语“批准”用来替代评估，批准用于授权合同目的时，使用术语“能力评价”。

3.1 过程评估模型

评估模型捕获被认为是良好的实践，这些实践可以仅适合于技术上的软件工程活动，也可以指管理、系统工程、人力资源管理活动等。

ISO/IEC 15504标准[ISO15504-98]定义了一个样板评估模型和对其它评估模型的遵从性需求。可以使用并已经使用的特定评估模型包括Sw-CMM (SEI95)、CMMi[SEI01]和Bootstrap [Sti99]。已经定义了许多其它的能力和成熟度模型，例如，设计、文档和形式化方法等。ISO 9001是另一个普遍的评估模型，已经被软件组织采用(ISO9001-00)。

已经开发了一个系统工程的成熟度模型，当项目或组织在开发或维护系统，包括软件时，很有用处(EIA/IS731-99)。评估模型对于小型组织的可应用性参见[Joh99; San98]。

评估模型有两种一般的体系结构，区别在于对评估过程的顺序假设：连续的、分阶段的(Pau94)，它们差别大，应该由组织根据它们是否符合需要和目标来评价。

3.2 过程评估方法[Go199]

为完成评估，需要遵循特定的评估方法，以产生刻画过程能力（或组织成熟度）的定量的分值。

例如，CBA-IPI评估方法集中于过程改进(Dun96)，SCE方法集中于评价供应商能力(Bar95)，这两种方法都是为SW-CMM开发的，反映了被认为是良好评估实践的两类方法的需求由[ISO15504-98]提供(Mas95)。SCAMPI方法朝CMMi评估[SEI01]进行了调整。评估中完成的活动、这些活动上的工作量分布和评估中的气氛，在用于改进和用于合同授权时，是不同的。

人们对过程评估方法和模型提出了一些批评(Fay97; Gra98)，多数批评关注支持应用评估模型和方法的经验证据。但是，自这些文章发表后，已经有一些系统性的证据来支持过程评估的有效性(Cla97; Ele00; Ele00a; Kri99)。

4 过程和产品度量

虽然软件工程中应用度量很复杂，特别是建模和分析方法，但是在几个方面，软件工程度量是基础的，并是许多更高级的度量和分析过程的基础。此外，只有在基线度量建立后，才能评估过程和产品改进成就的达到。

可以进行度量来支持过程实现和变更的开始，或在产品上完度量。

在ISO国际度量衡词汇的基础上，ISO/IEC15939定义了软件度量和度量方法的关键术语，ISO/IEC 15359也为度量过程和产品特征提供了一个标准过程[VIM93]。

不过，读者可能回在文献中遇到术语上的差异，例如，有时用术语“度量学metric”代替“度量measure”。

4.1 过程度量[ISO15539-02]

这里的术语“过程度量”表示收集、分析和解释关于过程的定量信息。度量用于标识过程的优缺点，评价实现和/或变更后的过程。

过程度量有几个用途，例如，过程度量对于管理软件工程项目有用。这里，焦点是用过程实现和变更的过程度量。

图2中的路径显示了多数软件工程项目中作出的重要假设，通常，过程对项目结果有重要影响，上下文影响着过程和过程结果之间的联系，这意味着，过程到过程结果的联系依赖于上下文。

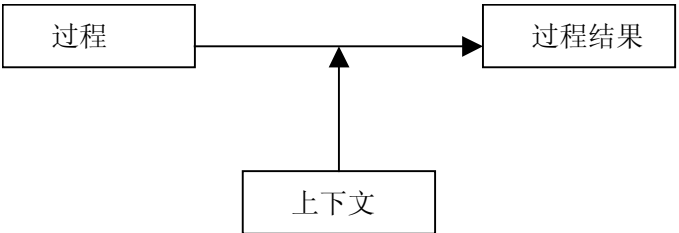


图 2 说明过程和结果之间关系的路径图

并非每个过程都对所有结果有正面影响，例如，引入软件检查可以减少测试工作量和成本，但如果由于安排了大型检查会议，每个检查持续较长，这也可能增加时间(Vot93)。因此，最好使用多个对于组织的业务重要的多个过程结果度量。

虽然一些工作量花费在评估工具和硬件的利用上，软件工程中需要管理的主要资源是人员，其结果是，感兴趣的主要度量是与小组或过程生产率有关的度量（例如，使用每个人-工作量单元产生的功能点度量）以及小组或过程在软件工程中（特定技术上）的经验水平[Fen98: c3, c11; Som05: c25]。

过程结果可以是产品质量（每千行代码/每功能点的错误）、可维护性（进行某种类型变更的工作量）、生产率（代码行或功能点/人月）、上市时间或客户满足程度（通过客户调查得到）。这个关系依赖与特定的上下文（例如，组织的规模或项目规模）。

一般，我们最关心过程结果，但是，为得到希望的过程结果（例如，更好的质量、更好的可维护性、更好的客户满足度），我们必须实施适当的过程。

当然，不仅仅是过程对结果有影响，其它因素，如职员的能力和使用的工具的能力，都起着

重要作用。评价过程变更的影响时，将这些其它的影响因素分解出来，就很重要。此外，过程制度化的程度（过程执行的保真度）也很重要，因为它可以解释为什么“好的”过程在给定情况下并不总是给出期望的结果。

4.2 软件产品度量[IS09126-01]

软件产品度量包括：产品规模、产品结构、产品质量。

4.2.1 规模度量

软件产品规模最常用长度（例如，模块中代码行数、软件规格说明文档页数）或功能（例如规格说明中的功能点数目）来度量。功能规模度量的原则在IEEE 14143.1标准中，功能规模度量方法的国际标准包括ISO/IEC 19761、20926 和20968 [IEEE14143.1-00; ISO19761-03; ISO20926-03; ISO20968-02]。

4.2.2 结构度量

多种软件产品结构度量可以应用到设计和代码人工制品的高层和低层，以反映控制流（例如，回路数目、代码节点）、数据流（例如，片段的数量）、嵌套（例如，嵌套多项式度量、BAND度量）、控制结构（例如，向量度量、NPATH度量）和模块结构与交互（例如，信息流、基于树的度量、耦合与聚合）[Fen98:c8; Pre04: c15]。

4.2.3 质量度量

作为多维属性，质量度量的定义不如前面的度量直接，此外，质量的一些维度可能需要定性度量而不是定量。软件质量度量的更详细讨论在软件质量知识域中主题4.4。ISO软件产品质量模型和相关的度量参见IS09126的1—4部分[IS09126-01]。[Fen98: c9, c10; Pre04: c15; Som05: c24]。

4.3 度量结果的质量

度量结果的质量（精确性、可重复性、可转换性、随机度量误差）对于度量程序提供有效的和有界的结果很关键。度量结果的关键特征和度量仪器的相关质量在ISO国际度量衡词汇中定义[VIM93]。

度量理论建立了可以进行有意义度量的基础，度量理论和尺度类型参见[Kan02]。在理论中，度量定义为“以系统的方式将数字赋给对象，以表示对象的特性”。

软件度量尺度的正确评估和每个尺度类型与随后如何选择数据分析方法的联系的含义特别重要[Abr96; Fen98: c2; Pf101: c11]。有意义的尺度与尺度分类有关，度量理论为此提供了一系列越来越严格的分配度量的方式。如果分配的数字仅仅是提供分类对象的标签，则它们可以是名词性的，如果要使用它们来评定对象等级（例如，好、较好、最好），则它们是序数性的，如果它们表示与定义的度量单元相关的特性的幅度，则就是间隔性的（除非特别指出，间隔在数字间是均匀的，因此可以相加）。如果度量有绝对0点，就是在比率层次，到绝对点距离的比率就是有意义的。

4.4 软件信息模型

随着数据的收集和度量数据库中被填充数据，我们可以使用数据和知识来建立模型。这些模型用于分析、分类和预测。需要评价这些模型，以保证它们有足够的精度，并充分了解了它们的局限。模型的精化可以在项目中进行，也可以在项目完成后进行，这是一个重要的活动。

4.4.1 模型建造

模型建造包括模型的校正和评价。度量的目标驱动方法告知模型建造过程，模型已经构造完毕，可以回答相关问题，并达到软件改进目标，即模型达到的程度。这个过程也受特定度量尺度与选择的分析方法关系隐含的局限的影响。模型要校正（通过特定相关的观察，例如，最近的项目、使用类似技术的项目），并评价模型的有效性（例如，在支持样本上测试模型的性能）[Fen98: c4, c6, c13; Pf101: c3, c11, c12; Som05: c25]。

4.4.2 模型实现

模型实现包括模型的解释和精化，将校正后的模型应用到过程中，解释结果，在过程/项目的上下文中评价结果，适当时，精化模型[Fen98: c6; Pf101: c3, c11, c12; Pre04: c22; Som05: c25]

4.5 过程度量技术

度量技术可以用来分析软件工程过程和标识其优缺点，可以以次启动过程实现和变更，或评价过程实现和变更的后果。

度量结果的质量，如精度、可重复性，是软件工程过程度量的关键问题，因为有基于仪器的度量和基于判断的度量，例如，当评估者将分值分配给特定的过程时。达到高质量度量的方法和讨论参见[Go199]。

过程度量技术分为两大基本类型：分析性的和基准性的。两类技术可以一起使用，因为它们基于不同类型的信息(Car91)。

4.5.1 分析性技术

分析性技术特征是依赖“量化证据以确定什么地方需要改进和改进工作是否成功”。分析性类型的样板是质量改进范例，它由理解、评估、打包循环组成[SEL96]。下面给出的技术将作为其它分析技术的例子，反映了实际中作了什么[Fen98; Mus99], (Lyu96; Wei93, Ze198)，特定的组织是否使用全部这些技术，这（至少在部分上）依赖于组织的成熟度。

(1) 实验研究：实验涉及在组织中建立受控的或准受控的实验来评价过程(McG94)。通常，要将新过程与当前过程比较，确定前者是否具有更好的过程结果。另一类实验研究是过程模拟，这类研究用于分析过程行为，探索过程改进的潜在性，预测如果目前过程以某种方式变更时的过程结果，控制过程的执行。但是，需要收集当前过程性能的初始数据，以作为模拟的基础。

(2) 过程定义评审：是一种评审（描述性或说明性）过程定义、标识缺陷和潜在的过程改进的手段。典型的例子见(Ban95; Kel98)。分析过程的一种容易操作的方式是将其与现存标准（国家、国际或行业标准）比较，如IEEE/EIA 12207.0[IEEE12207.0-96]。使用这个方法，在过程中不收集定量数据，即使有这些数据，也是支持性角色。完成过程定义分析的人员使用他们的知识和能力，来决定什么过程变更可能潜在地导致期望的过程结果。观察性研究也可以为标识过程改进提供有用的改进(Agr99)。

(3) 正交缺陷分类：是一个用于将错误与发现的潜在原因联系的技术，它依赖于错误类型和错误触发器之间的映射(Chi92; Chi96)，IEEE错误（或异常）分类标准在这个上下文中可能有用(IEEE1044-93)。

(4) 根本原因分析：是实践中的另一个普遍的分析技术，它涉及从检测到的问题（例如，错误）开始追踪，以标识过程的原因，目标是改变过程以在将来避免这些问题。不同过程类型的例子见(Co193; Ele97; Nak91)。前面描述的正交缺陷分类技术可用于找出许多问

题存在的类别，可以在这点上分析这些问题。这样，正交缺陷分类可以为在何处进行根本原因分析作出定量选择。

(5) 统计过程控制：是一个使用控制图及其解释，来标识过程中稳定性或缺乏稳定性的有效方法。软件工程上下文中，对统计过程控制的良好介绍见 (Flo99)。

(6) 个体软件过程：用规定的顺序，定义了一系列对个人开发实践的改进 [Hum95]，它规定了个体数据收集和基于数据解释的改进，在这个意义上，它是自底向上的。

4.5.2 基准技术

第二类技术成为基准技术，“依赖于标识一个领域中的‘优秀’组织，将其实践和工具记录到文档中”。基准技术假设，如果不太熟练的组织采用优秀组织的实践，它也可成为优秀的。基准技术涉及评估组织的成熟度或组织过程的能力，它由软件过程评估工作来例证，过程评估及其应用的介绍性综述见 (Zah98)。

主题与参考文献矩阵

	[Abr96]	[Bas92]	[Beck99]	[Boehm03]	[Corm97]	[Fen98]	[Fin94]	[Fow90]	[Goll99]	[Johnson99]
1 过程实现与变更										
1.1 过程基础结构		*						*		
1.2 软件过程管理周期		*						*		
1.3 过程实现与变更模型										
1.4 实际考虑									*	
2 过程定义										
2.1 软件生命周期模型					*					
2.2 软件生命周期过程			*	*						
2.3 过程定义符号										
2.4 过程修改										*
2.5 自动化							*			
3 过程评估										
3.1 过程评估模型										*
3.2 过程评估方法									*	
4 度量										
4.1 过程度量						C3, c11				
4.2 软件产品度量						C8-c10				
4.3 度量结果质量	*					C2				
4.4 软件信息模型										
模型建造						C4, c6, c13				
模型实现						C6				
4.5 过程度量技术						*			*	

3 过程评估										
3.1 过程评估模型	*				*					
3.2 过程评估方法					*					
4 度量										
4.1 过程度量							*			
4.2 软件产品度量			*				*	*	*	
4.3 度量结果质量										
4.4 软件信息模型										
模型建造										
模型实现										
4.5 过程度量技术										

	ISO VIM	IEE E10 44	IEE E10 61	IEE E10 74	IEE E12 19	IEE E15 17	IEE E15 40	IEE E12 207	IEEE1 4143. 1
1 过程实现与变更									
1.1 过程基础结构								*	
1.2 软件过程管理周期								*	
1.3 过程实现与变更模型									
1.4 实际考虑									
2 过程定义									
2.1 软件生命周期模型									
2.2 软件生命周期过程				*	*	*	*	*	
2.3 过程定义符号									
2.4 过程修改								*	
2.5 自动化									
3 过程评估									
3.1 过程评估模型									
3.2 过程评估方法									
4 度量									
4.1 过程度量	*								
4.2 软件产品度量									*
4.3 度量结果质量	*								
4.4 软件信息模型									
模型建造									
模型实现									
4.5 过程度量技术		*						*	

软件工程过程的推荐参考文献

[Abr96] A. Abran and P. N. Robillard, "Function Points Analysis: An Empirical Study

- of its Measurement Processes," *IEEE Transactions on Software Engineering*, vol. 22, 895-909, 1996
- [Bas92] V. Basili, G. Caldiera, F. McGarry, R. Pajerski, G. Page and S. Waligora, "The Software Engineering Laboratory – An Operational Software Experience Factory," presented at Proceedings of the International Conference on Software Engineering, 1992
- [Bec99] K. Beck, *Extreme Programming Explained: Embrace Change*: Addison-Wesley, 1999.
- [Boe03] B. Boehm and R. Turner, "Using Risk to Balance Agile and Plan-Driven Methods," *Computer*, 57-66, June, 2003
- [Com97] E. Comer, "Alternative Software Life Cycle Models," presented at Software Engineering, 1997
- [ElE99] K. El-Emam and N. Madhavji, *Elements of Software Process Assessment and Improvement*: IEEE CS Press, 1999.
- [Fen98] N. E. Fenton and S. L. Pfleeger, *Software Metrics: A Rigorous & Practical Approach*, Second ed: International Thomson Computer Press, 1998.
- [Fin94] A. Finkelstein, J. Kramer and B. Nuseibeh, "Software Process Modeling and Technology," *Research Studies Press Ltd.*, 1994
- [Fow90] P. Fowler and S. Rifkin, "Software Engineering Process Group Guide," Software Engineering Institute, Technical Report CMU/SEI-90-TR-24, 1990, available at <http://www.sei.cmu.edu/pub/documents/90.reports/pdf/tr24.90.pdf>
- [Gol99] D. Goldenson, K. El-Emam, J. Herbsleb and C. Deephouse, "Empirical Studies of Software Process Assessment Methods," presented at Elements of Software Process Assessment and Improvement, 1999
- [IEEE1074-97] IEEE Std 1074-1997, *IEEE Standard for Developing Software Life Cycle Processes*: IEEE, 1997.
- [IEEE12207.0-96] IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.
- [VIM93] ISO VIM, *International Vocabulary of Basic and General Terms in Metrology*. Geneva, Switzerland: ISO, 1993.
- [ISO9126-01] ISO/IEC 9126-1:2001, *Software Engineering-Product Quality-Part 1: Quality Model*: ISO and IEC, 2001.
- [ISO15504-98] ISO/IEC TR 15504:1998, *Information Technology – Software Process Assessment (parts 1-9)*: ISO and IEC, 1998.
- [ISO15939-02] ISO/IEC 15939:2002, *Software Engineering-Software Measurement Process*: ISO and IEC, 2002.
- [Joh99] D. Johnson and J. Brodman, "Tailoring the CMM for Small Businesses, Small Organizations, and Small Projects," presented at Elements of Software Process Assessment and Improvement,, 1999
- [McF96] B. McFeeley, "IDEAL: A User's Guide for Software Process Improvement," Software Engineering Institute CMU/SEI-96-HB-001, 1996, available at <http://www.sei.cmu.edu/pub/documents/96.reports/pdf/hb001.96.pdf>
- [Moi98] D. Moitra, "Managing Change for Software Process Improvement Initiatives:

- A Practical Experience-Based Approach," *Software Process – Improvement and Practice*, vol. 4, iss. 4, 199–207, 1998
- [Mus99] J. Musa, *Software Reliability Engineering: More Reliable Software, Faster Development and Testing*: McGraw Hill, 1999.
- [OMG02] Object Management Group, "Software Process Engineering Meta-model Specification," 2002, available at <http://www.omg.org/docs/formal/02-11-14.pdf>
- [Pfl101] S. L. Pfleeger, *Software Engineering: Theory and Practice*, Second ed: Prentice-Hall, 2001.
- [Pre04] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, Sixth ed: McGraw-Hill, 2004.
- [San98] M. Sanders, "The SPIRE Handbook: Better, Faster, Cheaper Software Development in Small Organisations," *European Commission*, 1998
- [SEI01] Software Engineering Institute, "Capability Maturity Model Integration, v1.1," 2001, available at <http://www.sei.cmu.edu/cmmi/cmmi.html>
- [SEL96] Software Engineering Laboratory, "Software Process Improvement Guidebook," NASA/GSFC, Technical Report SEL-95-102, April, 1996, available at <http://sel.gsfc.nasa.gov/website/documents/online-doc/95-102.pdf>
- [Som05] I. Sommerville, *Software Engineering*, Seventh ed: Addison-Wesley, 2005.
- [Sti99] H. Stienen, "Software Process Assessment and Improvement: 5 Years of Experiences with Bootstrap," in *Elements of Software Process Assessment and Improvement*, K. El-Emam and N. Madhavji, Eds.: IEEE CS Press, 1999

附录A 深入阅读的参考文献

- (Agr99) W. Agresti, "The Role of Design and Analysis in Process Improvement," presented at Elements of Software Process Assessment and Improvement, 1999
- (Ale91) L. Alexander and A. Davis, "Criteria for Selecting Software Process Models," presented at Proceedings of COMPSAC' 91, 1991
- (Ban95) S. Bandinelli, A. Fuggetta, L. Lavazza, M. Loi and G. Picco, "Modeling and Improving an Industrial Software Process," *IEEE Transactions on Software Engineering*, vol. 21, iss. 5, 440–454, 1995
- (Bar95) N. Barghouti, D. Rosenblum, D. Belanger and C. Alliegro, "Two Case Studies in Modeling Real, Corporate Processes," *Software Process – Improvement and Practice*, vol. Pilot Issue, 17–32, 1995
- (Boe03a) B. Boehm and R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*: Addison-Wesley, 2003.
- (Bur99) I. Burnstein, A. Homyen, T. Suwanassart, G. Saxena and R. Grom, "A Testing Maturity Model for Software Test Process Assessment and Improvement," *Software Quality Professional*, vol. 1, iss. 4, 8–21, 1999
- (Chi92) R. Chillarege, I. Bhandhari, J. Chaar, M. Halliday, D. Moebus, B. Ray and M. Wong, "Orthogonal Defect Classification – A Concept for In-Process Measurement," *IEEE Transactions on Software Engineering*, vol. 18, iss. 11, 943–956, 1992
- (Chi96) R. Chillarege, "Orthogonal Defect Classification," in *Handbook of Software Reliability Engineering*, M. Lyu, Ed.: IEEE CS Press, 1996.
- (Col93) J. Collofello and B. Gosalia, "An Application of Causal Analysis to the

Software Production Process," *Software Practice and Experience*, vol. 23, iss. 10, 1095-1105, 1993

(Cur02) B. Curtis, W. Hefley and S. Miller, "The People Capability Maturity Model: Guidelines for Improving the Workforce," Addison-Wesley, 2002.

(Dav88) A. Davis, E. Bersoff and E. Comer, "A Strategy for Comparing Alternative Software Development Life Cycle Models," *IEEE Transactions on Software Engineering*, vol. 14, iss. 10, 1453-1461, 1988

(Dun96) D. Dunnaway and S. Masters, "CMM-Based Appraisal for Internal Process Improvement (CBA IPI): Method Description," Software Engineering Institute CMU/SEI-96-TR-007, 1996, available at <http://www.sei.cmu.edu/pub/documents/96.reports/pdf/tr007.96.pdf>

(EIA/IS731-99) EIA, "EIA/IS 731 Systems Engineering Capability Model," 1999, available at <http://www.geia.org/eoc/G47/index.html>

(ElE-97) K. El-Emam, D. Holtje and N. Madhavji, "Causal Analysis of the Requirements Change Process for a Large System," presented at Proceedings of the International Conference on Software Maintenance, 1997

(ElE-99a) K. El-Emam, B. Smith and P. Fusaro, "Success Factors and Barriers in Software Process Improvement: An Empirical Study," in *Better Software Practice for Business Benefit: Principles and Experiences*, R. Messnarz and C. Tully, Eds.: IEEE CS Press, 1999.

(ElE-00a) K. El-Emam and A. Birk, "Validating the ISO/IEC 15504 Measures of Software Development Process Capability," *Journal of Systems and Software*, vol. 51, iss. 2, 119-149, 2000

(ElE-00b) K. El-Emam and A. Birk, "Validating the ISO/IEC 15504 Measures of Software Requirements Analysis Process Capability," *IEEE Transactions on Software Engineering*, vol. 26, iss. 6, 541-566, June, 2000

(Fay97) M. Fayad and M. Laitinen, "Process Assessment: Considered Wasteful," *Communications of the ACM*, vol.40, iss. 11, November, 1997

(Flo99) W. Florac and A. Carleton, *Measuring the Software Process: Statistical Process Control for Software Process Improvement*: Addison Wesley, 1999.

(Gar96) P. Garg and M. Jazayeri, "Process-Centered Software Engineering Environments: A Grand Tour," in *Software Process*, A. Fuggetta and A. Wolf, Eds.: John Wiley & Sons, 1996.

(Gra97) R. Grady, *Successful Software Process Improvement*: Prentice Hall, 1997.

(Gra88) E. Gray and W. Smith, "On the Limitations of Software Process Assessment and the Recognition of a Required Re-Oriented for Global Process Improvement," *Software Quality Journal*, vol. 7, 21-34, 1998

(Har98) D. Harel and M. Politi, *Modeling Reactive Systems with Statecharts: The StateMate Approach*: McGraw-Hill, 1998.

(Her98) J. Herbsleb, "Hard Problems and Hard Science: On the Practical Limits of Experimentation," *IEEE TCSE Software Process Newsletter*, vol. 11, 18-21, 1998, available at <http://www.seg.iit.nrc.ca/SPN>

(Hum95) W. Humphrey, *A Discipline for Software Engineering*: Addison Wesley, 1995.

(Hum99) W. Humphrey, *An Introduction to the Team Software Process*: Addison-Wesley,

1999.

(Hut94) D. Hutton, *The Change Agent's Handbook: A Survival Guide for Quality Improvement Champions*: Irwin, 1994.

(Kan02) S. H. Kan, *Metrics and Models in Software Quality Engineering*, Second ed: Addison-Wesley, 2002.

(Kel98) M. Kellner, U. Becker-Kornstaedt, W. Riddle, J. Tomal and M. Verlage, "Process Guides: Effective Guidance for Process Participants," presented at Proceedings of the 5th International Conference on the Software Process, 1998

(Kit98) B. Kitchenham, "Selecting Projects for Technology Evaluation," *IEEE TCSE Software Process Newsletter*, iss.11, 3-6, 1998, available at <http://www.seg.iit.nrc.ca/SPN>

(Kra99) H. Krasner, "The Payoff for Software Process Improvement: What it is and How to Get it," presented at Elements of Software Process Assessment and Improvement, 1999

(Kri99) M. S. Krishnan and M. Kellner, "Measuring Process Consistency: Implications for Reducing Software Defects," *IEEE Transactions on Software Engineering*, vol. 25, iss. 6, 800-815, November/December, 1999

(Lyu96) M. R. Lyu, *Handbook of Software Reliability Engineering*: Mc-Graw-Hill/IEEE, 1996.

(Mad94) N. Madhavji, D. Hoeltje, W. Hong and T. Bruckhaus, "Elicit: A Method for Eliciting Process Models," presented at Proceedings of the Third International Conference on the Software Process, 1994

(Mas95) S. Masters and C. Bothwell, "CMM Appraisal Framework - Version 1.0," Software Engineering Institute CMU/SEI-TR-95-001, 1995, available at <http://www.sei.cmu.edu/pub/documents/95.reports/pdf/tr001.95.pdf>

(McG94) F. McGarry, R. Pajerski, G. Page, S. Waligora, V. Basili and M. Zelkowitz, "Software Process Improvement in the NASA Software Engineering Laboratory," Software Engineering Institute CMU/SEI-94-TR-22, 1994, available at <http://www.sei.cmu.edu/pub/documents/94.reports/pdf/tr22.94.pdf>

(McG01) J. McGarry, D. Card, C. Jones, B. Layman, E. Clark, J. Dean and F. Hall, *Practical Software Measurement: Objective Information for Decision Makers*: Addison-Wesley, 2001.

(McG93) C. McGowan and S. Bohner, "Model Based Process Assessments," presented at Proceedings of the International Conference on Software Engineering, 1993

(Nak91) T. Nakajo and H. Kume, "A Case History Analysis of Software Error Cause-Effect Relationship," *IEEE Transactions on Software Engineering*, vol. 17, iss. 8, 1991

(Pau94) M. Paulk and M. Konrad, "Measuring Process Capability Versus Organizational Process Maturity," presented at Proceedings of the 4th International Conference on Software Quality, 1994

(Pfl99) S. L. Pfleeger, "Understanding and Improving Technology Transfer in Software Engineering," *Journal of Systems and Software*, vol. 47, 111-124, 1999

(Pfl01) S. L. Pfleeger, *Software Engineering: Theory and Practice*, Second ed: Prentice-Hall, Inc., 2001.

- (Rad85) R. Radice, N. Roth, A. O. H. Jr. and W. Ciarfella, "A Programming Process Architecture," *IBM Systems Journal*, vol. 24, iss. 2, 79-90, 1985
- (Rag89) S. Raghavan and D. Chand, "Diffusing Software-Engineering Methods," *IEEE Software*, 81-90, July, 1989
- (Rog83) E. Rogers, *Diffusion of Innovations*: Free Press, 1983.
- (Sch99) E. Schein, *Process Consultation Revisited: Building the Helping Relationship*: Addison-Wesley, 1999.
- (SEI95) Software Engineering Institute, *The Capability Maturity Model: Guidelines for Improving the Software Process*: Addison-Wesley, 1995.
- (SEL96) Software Engineering Laboratory, "Software Process Improvement Guidebook," Software Engineering Laboratory: NASA/GSFC, Technical Report SEL-95-102, April, 1996, available at <http://sel.gsfc.nasa.gov/website/documents/online-doc/95-102.pdf>
- (SPC92) Software Productivity Consortium, "Process Definition and Modeling Guidebook," Software Productivity Consortium, SPC-92041-CMC, 1992
- (Som97) I. Sommerville and P. Sawyer, *Requirements engineering: A Good Practice Guide*: John Wiley and Sons, 1997.
- (Vot93) L. Votta, "Does Every Inspection Need a Meeting ?," *ACM Software Engineering Notes*, vol. 18, iss. 5, 107-114, 1993
- (Wei93) G. M. Weinberg, "Quality Software Management," *First-Order Measurement (Ch. 8, Measuring Cost and Value)*, vol. 2, 1993
- (Yu94) E. Yu and J. Mylopoulos, "Understanding 'Why' in Software Process Modeling, Analysis, and Design," presented at Proceedings of the 16th International Conference on Software Engineering, 1994
- (Zah98) S. Zahran, *Software Process Improvement: Practical Guidelines for Business Success*: Addison Wesley, 1998.
- (Zel98) M. V. Zelkowitz and D. R. Wallace, "Experimental Models for Validating Technology," *Computer*, vol. 31, iss. 5, 23-31, 1998

附录B 有关标准

- (IEEE1044-93) IEEE Std 1044-1993 (R2002), *IEEE Standard for the Classification of Software Anomalies*: IEEE, 1993.
- (IEEE1061-98) IEEE Std 1061-1998, *IEEE Standard for a Software Quality Metrics Methodology*: IEEE Computer, 1998.
- (IEEE1074-97) IEEE Std 1074-1997, *IEEE Standard for Developing Software Life Cycle Processes*: IEEE, 1997.
- (IEEE1219-98) IEEE Std 1219-1998, *IEEE Standard for Software Maintenance*: IEEE, 1998.
- (IEEE1220-98) IEEE Std 1220-1998, *IEEE Standard for the Application and Management of the Systems Engineering Process*: IEEE, 1998.
- (IEEE1517-99) IEEE Std 1517-1999, *IEEE Standard for Information Technology-Software Life Cycle Processes-Reuse Processes*: IEEE, 1999.
- (IEEE1540-01) IEEE Std 1540-2001//ISO/IEC16085:2003, *IEEE Standard for Software Life Cycle Processes-Risk Management*: IEEE, 2001.

(IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.

(IEEE12207.1-96) IEEE/EIA 12207.1-1996, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes -Life cycle data*: IEEE, 1996.

(IEEE12207.2-97) IEEE/EIA 12207.2-1997, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes -Implementation. Considerations*: IEEE, 1997.

(IEEE14143.1-00) IEEE Std 14143.1-2000//ISO/IEC14143-1:1998, *Information Technology-Software Measurement-Functional Size Measurement-Part 1: Definitions of Concepts*: IEEE, 2000.

(ISO9001-00) ISO 9001:2000, *Quality Management Systems-Requirements*: ISO, 1994.

(ISO9126-01) ISO/IEC 9126-1:2001, *Software Engineering-Product Quality-Part 1: Quality Model*: ISO and IEC, 2001.

(ISO14674-99) ISO/IEC 14674:1999, *Information Technology- Software Maintenance*: ISO and IEC, 1999.

(ISO15288-02) ISO/IEC 15288:2002, *Systems Engineering-System Life cycle process*: ISO and IEC, 2002.

(ISO15504-98) ISO/IEC TR 15504:1998, *Information Technology - Software Process Assessment (parts 1-9)*: ISO and IEC, 1998.

(ISO15939-02) ISO/IEC 15939:2002, *Software Engineering-Software Measurement Process*: ISO and IEC, 2002.

(ISO19761-03) ISO/IEC 19761:2003, *Software Engineering-Cosmic FPP-A functional Size Measurement Method*: ISO and IEC, 2003.

(ISO20926-03) ISO/IEC 20926:2003, *Software Engineering-IFPUG 4.1 Unadjusted functional Size measurement method-Counting practices manual*: ISO and IEC, 2003.

(ISO20968-02) ISO/IEC 20968:2002, *Software Engineering-MK II Function Point Analysis- Counting Practices Manual*: ISO and IEC, 2002.

(ISO90003-04) ISO/IEC 90003:2004, *Software and Systems Engineering-Guidelines for the Application of ISO9001:2000 to Computer Software*: ISO and IEC, 2004.

(VIM93) ISO VIM, *International Vocabulary of Basic and General Terms in Metrology*. Geneva, Switzerland: ISO, 1993

////////////////////////////////////

第10章 软件工程工具和方法

术语和缩写

CASE: Computer Assisted Software Engineering, 计算机辅助软件工程

引言

软件开发工具是用于辅助软件生命周期过程的基于计算机的工具, 工具将复性的、良定义的动作自动化, 减少了软件工程师的认知负担, 使他们可以集中在过程的创造性方面。通常设计工具来支持特定的软件工程方法, 减少手工应用方法相关的管理性负担。与软件工程

方法一样，它们试图让软件工程更加系统化，工具的范围覆盖支持单个任务到囊括整个生命周期。

软件工程方法将结构施加到软件工程活动上，目标是使活动系统化并最终更可能成功。方法通常提供一种符号和词汇表，为完成可标识的任务提供流程，为检查过程和产品提供指南。方法的覆盖范围从单个生命周期阶段到整个生命周期。这个知识域的重点是包括多个生命周期阶段的软件工程方法，因为特定阶段的方法由其它知识域覆盖。

尽管对于特定的工具有详尽的手册，对于新型工具有大量的论文，但是，关于软件工程工具的一般性技术读物却很少。一个困难是软件工具的高变化率，特定的细节有规则地变化着，使得难以提供实质性的、最新的例子。

软件工程工具和方法知识域覆盖整个生命周期过程，因此，与指南中每个知识域都有联系。

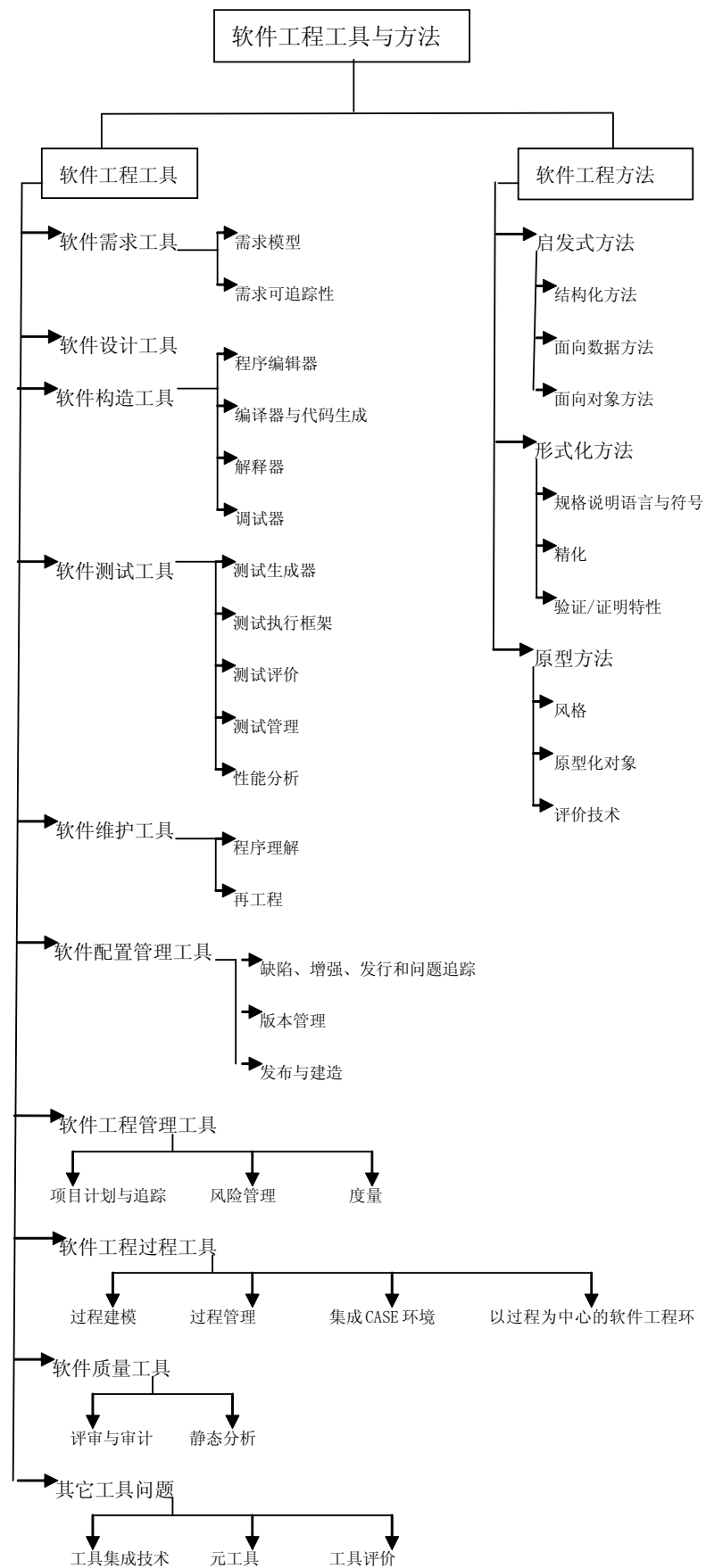


图 1 软件工程工具与方法知识域的主题结构分解

软件工程工具与方法主题的分解结构

1 软件工程工具

软件工程工具子域的前5个主题对应指南的前5个知识域（软件需求、软件设计、软件构造、软件测试、软件维护），随后4个主题对应余下的知识域（软件配置管理、软件工程管理、软件工程过程、软件质量），一个附加的主题是“其它”，涉及工具集成技术等可能应用与所有类型工具的领域。

1.1 软件需求工具[Dor97, Dor02]

处理软件需求的工具划分为两类：建模和可追踪性工具。（1）需求建模工具：这些工具用于获取、分析、规格说明和验证软件需求。（2）需求可追踪性工具[Dor02]：这些工具随着软件复杂性的增加而变得越来越重要，由于它们与其它生命周期过程也有关，因此将它们从需求建模工具中独立出来。

1.2 软件设计工具[Dor02]

这个主题覆盖创建和检查软件设计，这类工具的种类很多，原因在于软件设计符号和方法的多样性。尽管多样，但这个主题却没有引人注目的划分。

1.3 软件构造工具[Dor02, Rei96]

这个主题覆盖构造工具，这些工具用于产生和翻译对于机器执行来说是足够详细和明显的程序表示（例如，源代码）。（1）程序编辑器：这些工具用于创建和修改程序，可能也创建和修改相关的文档。它们可以是一般的文本或文档编辑器，也可以专用于特定的目标语言。（2）编译器和代码生成器：传统上，编译器是源代码的非交互式翻译器，但有趋势将编译器和程序编辑集成，以提供集成化的编程环境，这个主题也覆盖预处理器、连接器/装载器和代码生成器。（3）解释器：这些工具通过模拟来提供软件的执行，它们通过为程序执行提供更可控制的和更可观察的环境，来支持软件构造活动。（4）调试器：由于支持软件构造过程，这些工具作为单独的类别考虑，但它们与程序编辑器和编译器是不同的。

1.4 软件测试工具[Dor02, Pf101, Rei96]

（1）测试生成器：这些工具辅助测试用例的开发。（2）测试执行框架：这些工具使测试用例在一个受控的环境中执行，可以观察被测试对象的行为。（3）测试评价工具：这些工具支持测试执行结果的评估，帮助确定观察到的行为是否符合期望的行为。（4）测试管理工具：这些工具为软件测试过程的所有方面提供支持。（5）性能分析工具[Rei96]：这些工具用于度量和分析软件性能，这是一种特殊的测试，目标是评估性能行为而不是功能行为（正确性）。

1.5 软件维护工具[Dor02, Pf101]

这个主题包括对于修改现存软件的软件维护特别重要的工具，有两类：理解工具和再工程工具。（1）理解工具[Rei96]：这些工具辅助人类理解程序，例子包括可视化工具，诸如动画和程序分片器。（2）再工程工具：在软件维护知识域，再工程定义为检查和修改目标软件，以新的形式重构它。再工程工具支持这种活动。

逆向工程工具辅助通过从现有产品向前工作，创建诸如规格说明和设计描述等制品，支持维护过程，创建的制品和变换产生新的制品。

1.6 软件配置管理工具[Dor02, Rei96, Som05]

配置管理工具划分为3类：追踪、版本管理和发布工具。（1）缺陷、增强、发行和问题追踪工具：这些工具与特定软件产品中的问题追踪问题一起使用。（2）版本管理工具：这些工具用于管理产品的多个版本。（3）发布和建造工具：这些工具用于管理软件发布和建造任务，包括目前已经广泛用于配置软件产品安装的安装工具。

软件配置管理知识域主题2.3软件配置管理计划制订中，有附加的信息。

1.7 软件工程管理工具[Dor02]

软件工程管理工具分为3类：项目计划与追踪、风险管理和度量。（1）项目计划和追踪工具：这些工具用于软件项目工作量度量和成本估算，以及项目进度安排。（2）风险管理工具：这些工具用于标识、估计和监控风险。（3）度量工具：度量工具辅助完成与软件度量程序相关活动。

1.8 软件工程过程工具[Dor02, Som05]

软件工程过程工具分为：建模工具、管理工具、软件开发环境。（1）过程建模工具[Pf101]：这些工具用于软件工程过程的建模和研究。（2）过程管理工具：这些工具为软件工程管理提供支持。（3）集成的CASE环境[Rei96, Som05] (ECMA55-93, ECMA69-94, IEEE1209-92, IEEE1348-95, Mu196)：覆盖软件工程生命周期的多个阶段的集成的计算机辅助软件工程工具或环境属于这个子域，这类工具完成多个功能，并潜在地与正在执行的软件生命周期过程交互。（4）以过程为中心的软件工程环境[Rei96] (Gar96)：这些环境显式地包含软件生命周期过程信息，根据定义好的过程，引导和监督用户。

1.9 软件质量工具[Dor02]

质量工具分为2类：检查工具和分析工具。（1）评审和审计工具：这些工具用于支持评审和审计。（2）静态分析工具[Cla96, Pf101, Rei96]：这些工具用于分析软件制品，如语法和语义分析器、数据流/控制流/依赖性分析器。这些工具用于检查软件制品对需要的特性的遵从性或验证需要的特性。

1.10 其它工具问题[Dor02]

这个主题覆盖可以应用到所有类型工具的问题，有3类：工具集成技术、元工具和工具评价。（1）工具集成问题[Pf101, Rei96, Som01] (Bro94)：工具集成对于让单个工具协作很重要，这与应用了集成技术的集成化CASE环境潜在地重叠，但是，有必要将其独立为一类。工具集成的典型类别是平台、表现、过程、数据和控制。（2）元工具：元工具生成其它工具，编译程序的编译程序是经典的例子。（3）工具评价：[Pf101] (IEEE1209-92, IEEE1348-95, Mos92, Val97)：由于软件工程工具的连续进化，工具评价是一个有实质的主题。

2 软件工程方法

软件工程方法子域分3个主题：处理非形式化方法的启发式方法、处理基于数学的形式化方法、处理基于各种原型形式的软件工程方法的原型方法。前3个主题不是分离的，它们代表的是不同的思想，例如，面向对象的方法可以集成形式化技术，并依赖于原型来验证和确认。与软件工程工具一样，方法学也连续地进化，这样，对知识域的描述尽可能避免命名

特定的方法学。

2.1 启发式方法[Was96]

这个主题包含4类：结构化、面向数据、面向对象和领域特定。领域特定的类包括开发实时、人身安全、系统安全系统的专门方法。（1）结构化方法[Dor02, Pf101, Pre04, Som05]：从功能的观点构造系统，从高层的视图开始，逐步将其精化为更详细的设计。（2）面向数据的方法[Dor02, Pre04]：这里，出发点是程序操作的数据结构，而不是其完成的功能。（3）面向对象的方法[Dor02, Pf101, Pre04, Som05]：将系统看作是一组对象而不是一组功能。

2.2 形式化方法[Dor02, Pre04, Som05]

这个主题处理基于数学的软件工程方法，根据形式化方法的各个方面进行分类。（1）规格说明语言和符号[Cla96, Pf101, Pre01]：这个主题关注规格说明符号或使用的语言。规格说明语言可以分为面向模型的、面向特性的或面向行为的。（2）精化[Pre04]：这个主题处理如何将规格说明精化（或变换）到更接近期望的可执行程序的最终形式。（3）验证/证明特性[Cla96, Pf101, Som05]：这个主题覆盖形式化方法特定的验证特性，包括理论证明和模型检查。

2.3 原型方法[Pre04, Som05, Was96]

这个主题覆盖涉及软件原型的方法，分为：原型风格、对象和评价技术。（1）原型风格[Dor02, Pf101, Pre04] (Pom96)：原型风格主题标识各类方法：抛弃、进化和可执行规格说明。（2）原型对象[Dor97] (Pom96)：原型方法的对象的例子可以是需求、体系结构设计或用户界面。（3）原型评价技术：这个主题覆盖使用原型工作的结果的方式。

主题与参考文献矩阵

	[Cla96]	[Do2021]{Dor97}	[Pf101]{Pf198}	[Pre04]	[Rei96]	[Som05]	[Was96]
1 软件工具							
1.1 软件需求工具		{c4s1}, v2c8s4					
需求建模工具							
需求追踪工具		V1c4s2					
1.2 软件设计工具		V2c8s4					
1.3 软件构造工具		V2c8s4			C112s2		
程序编辑器							
编译器和代码生成器							
解释器							
调试器							
1.4 软件测试工具		V2c8s4	C8s7, c9s7		C112s2		

测试生成器							
测试执行框架							
测试评价工具							
测试管理工具							
性能分析工具							
1.5 软件维护工具		V2c8s4	C11s5				
理解工具					C112s5		
再工程工具							
1.6 软件配置管理工具		V2c8s4	C11s5		C112s3	C29	
缺陷、增强、发行和问题追踪							
版本管理工具							
发布与建造工具							
1.7 软件工程管理工具		V2c8s4					
项目计划和追踪工具							
风险管理工具							
度量工具							
1.8 软件工程过程工具		V2c8s4					
过程建模工具			C2s3, c2s4				
过程管理工具							
集成的 CASE 环境					C112s3, c112s4	C3	
以过程为中心的软件工程环境					C112s5		
1.9 软件质量工具		V2c8s4					
评审与审计工具							
静态分析工具	*		C8s7		C112s5		
1.10 其它工具问题		V2c8s5					
工具集成技术			C1s8		C112s4		*
元工具							
工具评价			C9s10				
2 开发方法							
2.1 启发式方法							*
结构化方法		V1c5s1, v1c6s3	C4s5	C7-c9		C15	
面向数据的方法		V1c5s1		C7-c			

		mv1c6s 3		9			
面向对象方法		V1c6s2 , v1c6s 3	C4s4, c 6, c8s5	C7-c 9		C12	
2.2 形式化方法		V1c6s5		C28		C9	
规格说明语言与符号	*		C4s5				
精化							
验证/证明特性	*		C5s7, c 8s3				
2.3 原型方法						C8	*
风格		V1c4s4	C4s6, c 5s6				
原型化对象		V1c4s4					
评价技术							

软件工程工具和方法的推荐参考文献

- [Cla96] E. M. Clarke, J. M. Wing and al, "Formal Methods: State of the Art and Future Directions," *ACM Computer Surveys*, vol. 28, iss. 4, 626-643, 1996
- [Dor97] M. Dorfman and R. H. Thayer, Eds., "Software Engineering." IEEE Computer Society Press, 1997.
- [Dor02] M. Dorfman and R. H. Thayer, Eds., "Software Engineering, Vol. 1 & vol.2." IEEE Computer Society Press, 2002.
- [Pf101] S. L. Pfleeger, *Software Engineering: Theory and Practice*, Second ed: Prentice-Hall, Inc., 2001.
- [Pre04] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, Sixth ed: McGraw-Hill, 2004.
- [Rei96] S. P. Reiss, *Software Tools and Environments in The Computer Science and Engineering Handbook*: CRC Press, 1996.
- [Som05] I. Sommerville, *Software Engineering*, Seventh ed: Addison-Wesley, 2005.
- [Was96] A. I. Wasserman, "Toward a Discipline of Software Engineering," *IEEE Software*, vol. 13, iss. 6, 23-31, November, 1996

附录A 深入阅读的参考文献

- (Ber93) E. V. Berard, *Essays on Object-Oriented Software Engineering*: Prentice-Hall, 1993.
- (Bis92) W. Bischofberger and G. Pomberger, *Prototyping-Oriented Software Development: Concepts and Tools*: Springer-Verlag, 1992.
- (Bro94) A. W. Brown and al, *Principles of CASE Tool Integration*: Oxford University Press, 1994.
- (Car95) D. J. Carney and A. W. Brown, "On the Necessary Conditions for the Composition of Integrated Software Engineering Environments," presented at Advances in Computers, 1995

- (Col94) D. Coleman, P. Arnold, S. Godoff, C. Dollin, H. Gilchrist, F. Hayes and P. Jeremaes, *Object-Oriented Development: The Fusion Method*: Prentice Hall, 1994.
- (Cra95) D. Craigen, S. Gerhart and T. Ralston, "Formal Methods Reality Check: Industrial Usage," *IEEE Transactions on Software Engineering*, vol. 21, iss. 2, 90-98, February, 1995
- (Fin00) A. Finkelstein, Ed., "The Future of Software Engineering." ACM, 2000.
- (Gar96) P. K. Garg and M. Jazayeri, *Process-Centered Software Engineering Environments*: IEEE Computer Society, 1996.
- (Har00) W. Harrison, H. Ossher and P. Tarr, "Software Engineering Tools and Environments: A Roadmap," 2000
- (Jar98) S. Jarzabek and R. Huang, "The Case for User-Centered CASE Tools," *Communications of the ACM*, vol. 41, iss. 8, 93-99, August, 1998
- (Kit95) B. Kitchenham, L. Pickard and S. L. Pfleeger, "Case Studies for Method and Tool Evaluation," *IEEE Software*, vol. 12, iss. 4, 52-62, July, 1995
- (Lam00) A. v. Lamsweerde, "Formal Specification: A Roadmap," in *The Future of Software Engineering*, A. Finkelstein, Ed.: ACM, 2000, 149-159.
- (Mey97) B. Meyer, *Object-Oriented Software Construction*, Second ed: Prentice-Hall, 1997.
- (Moo98) J. W. Moore, *Software Engineering Standards, A User's Roadmap*. Los Alamitos, CA: IEEE Computer Society, 1998.
- (Mos92) V. Mosley, "How to Assess Tools Efficiently and Quantitatively," *IEEE Software*, vol. 9, iss. 3, 29-32, May, 1992
- (Mül96) H. A. Muller, R. J. Norman and J. Slonim, Eds., "Computer Aided Software Engineering." A special issue of *Automated Software Engineering*, 3(3/4), Kluwer, 1996.
- (Mül00) H. Müller and al, "Reverse Engineering: A Roadmap," in *The Future of Software Engineering*, A. Finkelstein, Ed.: ACM, 2000, 49-60.
- (Pom96) G. Pomberger and G. Blaschek, *Object-Oriented and Prototyping in Software Engineering*: Prentice Hall, 1996.
- (Pos96) R. M. Poston, *Automating Specification-based Software Testing*: IEEE, 1996.
- (Ric92) C. Rich and R. C. Waters, "Knowledge Intensive Software Engineering Tools," *IEEE Transactions on Knowledge and Data Engineering*, vol. 4, iss. 5, 424-430, October, 1992
- (Son92) X. Song and L. J. Osterweil, "Towards Objective, Systematic Design-Method Comparisons," *IEEE Software*, vol. 9, iss. 3, 43-53, May, 1992
- (Tuc96) A. B. Tucker, *The Computer Science and Engineering Handbook*: CRC Press, 1996.
- (Val97) L. A. Valaer and R. C. B. II, "Choosing a User Interface Development Tool," *IEEE Software*, vol. 14, iss. 4, 29-39, 1997
- (Vin90) W. G. Vincenti, *What Engineers Know and How They Know It - Analytical Studies form Aeronautical History*. Baltimore and London: John Hopkins University Press, 1990.
- (Wie98) R. Wieringa, "A Survey of Structured and Object-Oriented Software Specification Methods and Techniques," *ACM Computing Surveys*, vol. 30, iss. 4, 459-527, 1998

附录B 有关的标准

- (ECMA55-93) ECMA, *TR/55 Reference Model for Frameworks of Software Engineering Environments*, Third ed, 1993.
- (ECMA69-94) ECMA, *TR/69 Reference Model for Project Support Environments*, 1994.
- (IEEE1175.1-02) IEEE Std 1175.1-2002, *IEEE Guide for CASE Tool Interconnections-Classification and Description*: IEEE, 2002.
- (IEEE1209-92) IEEE Std 1209-1992, *Recommended Practice for the Evaluation and Selection of CASE Tools, (ISO/IEC 14102, 1995)*, 1992.
- (IEEE1348-95) IEEE Std 1348-1995, *Recommended Practice for the Adoption of CASE Tools, (ISO/IEC 14471)*, 1995.
- (IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996

////////////////////////////////////

第11章 软件质量

术语和缩写

- CMMi: Capability Maturity Model Integrated, 能力成熟度模型集成
- COTS: Commercial Off-the-shelf Software, 商用商品化软件
- PDCA: Plan, Do, Check, Act, 计划、实施、检查、改进
- SQA: Software Quality Assurance, 软件质量保证
- SQM: Software Quality Management, 软件质量管理
- TQM: Total Quality Management, 全面质量管理
- V&V: Verification and Validation, 验证与确认

引言

什么是软件质量？为什么它如此重要而在SWEBOK指南中到处出现？多年以来，许多作者和组织对术语“质量”有着不同的定义。对于Phil Crosby (Cro79)，质量就是“遵从用户需求”，Watts Humphrey (Hum89)认为质量就是“达到适合使用的卓越层次”。IBM发明了术语“市场驱动的质量”，它基于达到全面的客户满意，关于组织质量的Baldrige准则(NIST03)使用了一个类似的短语“客户驱动的质量”，将客户满意作为主要的考虑。最近，在 (ISO9001-00)中，指令被定义为“一组内在特征满足需求的程度”。

本章讨论跨越生命周期过程的软件质量考虑，软件质量是软件工程中无所不在的问题，许多知识域也考虑了质量问题。总之，SWEBOK指南描述了许多达到软件质量的方式。特别是，本知识域将覆盖静态技术，这些技术不需要执行被评价的软件，动态技术在软件测试知识域中。

软件质量主题的分解结构

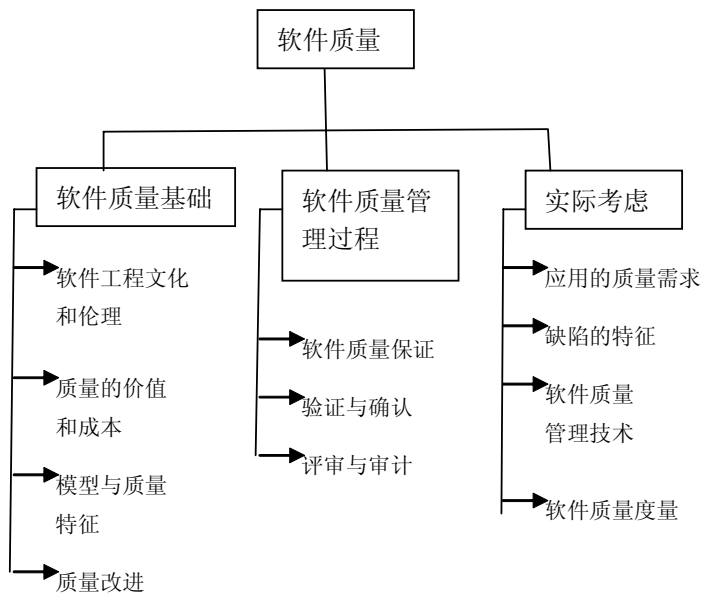


图1 软件质量知识域主题的分解结构

1 软件质量基础

要在质量需求上达成一致，并与软件工程师交流质量的构成，就需要正式地定义和讨论质量的许多方面。软件工程师应该了解质量概念和特征的基础意义，以及它们对要开发或维护软件的价值。重要的概念是软件需求定义了需要的软件质量特征，并影响评价这些特征的度量方法和接收准则。

1.1 软件工程文化与伦理

通常期望软件工程师共同认识到对软件质量的承诺作为他们文化的一部分，[Wie96]中描述了健康的软件工程文化。伦理在软件质量、文化、和软件工程师的态度中起着重要作用。IEEE计算机学会和ACM[IEEE99]发布了基于8项原则的伦理和职业实践准则，以帮助软件工程师增强有关质量和其工作独立性的看法。

1.2 质量的价值和成本[Boe78; NIST; Pre01; Wei93]

“质量”的概念并非看起来那样简单，对于任何工程化产品，在确定产品需求时，有许多与产品特定方面有关的希望的质量需要讨论和确定。各种质量特性可能需要或不需要，或可能需要加强或削弱，需要在它们之间作出权衡[Pf101]。质量的成本可以分为预防成本、批准成本、内部失效成本和外部失效成本[Hou99]。

软件产品后面的动机是期望创建一个有价值的产品，这个价值可能也不可能被量化为成本。客户牢记着最大成本，反过来期望软件的基本目的能够实现。客户也可能有对软件质量的期望，有时，客户可能完全没有考虑质量问题以及它们相关的成本。特性仅仅是一个装饰？还是对软件是必须的？如果答案位于二者之间，通常也是这种情况，就要求客户作于决策过程，并完全意识到质量的成本和效益。理想情况下，多数这些决策是在软件需求过程中作出的（参见软件需求知识域），但这些问题可能在整个软件生命周期中出现。还没有如何能作出这些决策的确定性规则，但软件工程师应该能提出质量解决方案及其成本。涉及质量需求的成本和价值的讨论可以参见[Jon96:c5; Wei96:c11]。

1.3 模型与质量特征[Dac01; Kia95; Lap91; Lew92; Mus99; NIST; Pre01; Rak97; Sei02;

Wal96]

在各种分类学（或软件质量模型）中，软件质量特征这一术语是不相同的，每个模型可能具有不同数目的层次级别，并有不同数目的特征。不同的作者产生了各种软件质量特征或属性，可用于软件产品质量的讨论、计划制订和评分[Boe78; McC77]。ISO/IEC定义了3个产品质量的相关模型（内部质量、外部质量、使用质量）(ISO9126-01)和一组相关的指南(ISO14598-98)。

1.3.1 软件工程过程质量

软件质量管理和软件工程过程质量对于软件产品质量有直接的关系。

评价软件组织能力的模型和准则主要是项目组织和管理的考虑，参见软件工程管理和软件工程过程知识域。

当然，不可能将过程质量与产品质量完全分开，本指南的软件工程过程知识域讨论了过程质量，过程质量影响软件产品的质量特征，这反过来影响客户观察到的使用质量。

两个重要的质量标准是TicKIT [Llo03]和ISO9001[ISO9001-00]，后者对软件质量有重要影响，它应用到软件的指南是[ISO9003-04]。

另一个软件质量的工业标准是CMMi [Sei02]，软件工程过程知识域对其进行了讨论。CMMi是一个提出了最佳时间的过程基准和改进模型。与质量管理相关的特定过程是：（1）过程和产品质量保证；（2）过程验证；（3）过程确认。CMMi将评审和审计分类为验证方法，而不是象(IEEE12207.0-96)一样作为特定的过程。

最初，对于软件工程师是否应该用ISO9001或CMMi来保证质量，有过争论，发表了许多文章，最后结果是，二者是互补的，通过ISO9001认证可以极大地帮助达到更高的CMMi成熟度层次 [Dac01]。

1.3.2 软件产品质量

首先，软件工程师需要确定软件的真实目的，在这方面，必须牢记：首先有客户的需求，这些需求不仅包括功能，而且包括了质量等约束，这一点非常重要，这样软件工程师有责任去获取最先可能不是明显的质量需求，并讨论各种质量需求的重要性，以及获得它们的困难程度。以后在设计所有与软件质量有关的过程（例如，建造、检查、改进质量）时，都必须考虑质量需求，这些过程会有附加的成本。

(ISO9126-01)标准为其3个质量模型的每一个都定义相关的质量特征和子特征，以及对于评估软件产品质量有用的度量(Sur03)。

术语“产品”的含义被扩充到包括任何作为用于建造最终软件产品的过程的输出的人工制品，产品的例子包括，但不局限于：整个系统的需求规格说明、系统的一个软件组件的软件需求规格说明、设计模块、代码、测试文档或作为质量分析任务结果的报表。尽管多数质量的处理都以最后的软件和系统性能来描述，但合理的工程实践要求，必须在整个软件工程过程中检查与质量有关的中间产品。

1.4 质量改进[NIST03; Pre01; Wei96]

软件产品的质量可以通过连续改进的迭代过程来改进，这需要许多并发过程的管理控制、协调和反馈：（1）软件生命周期过程。（2）错误/缺陷的检测、消除和预防过程。（3）质量改进过程(Kin92)。

质量改进的理论和概念，诸如通过对错误的预防和早期检测、持久改进、用户焦点等来提高质量，都与软件工程有关。这些概念基于质量领域的专家的工作，他们指出产品的质量与创建产品的过程的质量直接联系(Cro79, Dem86, Jur89)。

诸如全面质量管理TQM、计划-实施-检查-改进PDCA过程等方法，都是可以满足质量目标的工具。管理层首先发起支持过程和产品的评价和发现的结果，然后开发改进程序来标识详细的行动和在合理的时间框架内可以改进的项目。管理层的支持意味着每个改进项目有足够的资源来达到为其定义的目标。必须经常通过实现前瞻性的交流活动，来获得管理层的支持。工作小组的参与、中层管理的支持和项目级的资源分配，在软件工程过程知识域中讨论。

2 软件质量管理过程

软件质量管理要应用于软件过程、产品和资源的各个方面，它定义了过程、过程所有者、对过程的需求、过程及其输出的度量、反馈渠道(Art93)。软件质量管理过程由许多活动组成，一些活动可直接发现缺陷，其它指出深入的检查是否有价值，前者也称为直接缺点发现活动，许多活动可以达到这两个目的。

软件质量计划制订涉及：（1）从质量特征方面，定义需要的产品（软件工程管理知识域有详细的描述）。（2）为过程制订计划，以达到需要的产品质量（软件设计和软件构造知识域中有详细描述）。

这些方面与软件质量管理计划制订过程本身有所不同，前者评估列入计划的质量特征，后者 实际实现这个计划：软件产品在满足客户和干系人的需求，为客户和其它干系人提供价值、提供满足软件需求而需要的软件质量等方面，做的如何？使用软件质量管理来评价中间产品和最终产品。

(IEEE12207.0-96)定义了一些特定的软件质量管理过程：（1）质量保证过程。（2）验证过程。（3）确认过程。（4）评审过程。（5）审计过程。

这些过程提高了质量，也能发现可能的问题，但它们与其重点多少有些不同。软件质量管理过程保证确保给定项目中更高的质量，它们也提供作为副产品的一般管理用信息，包括整个软件工程过程的质量指标。软件工程过程和软件工程管理知识域为开发软件的组织讨论了质量程序，软件质量管理可以为这些领域提供相关的反馈。

软件质量管理过程由一些任务和技术组成，这些任务和技术指出软件计划（例如，管理、开发、配置管理）正在如何执行，以及中间和最终产品满足规定需求的程度。这些任务的结果在采取更正性行动之前，被汇编到管理用的报表中，软件质量管理过程的管理也是一个任务，用于保证这些报表的结果是正确的。

正如本知识域描述的那样，各个软件质量管理过程密切相关，它们之间是重叠的，有时要组合在一起，它们看起来在本质上是反应性的，因为它们将过程作为实践后的过程，并将产品作为生产后的产品处理，但是它们在计划制订阶段的主要作用是前瞻性的，将过程和流程作为获得质量特征的必要，将质量程度作为软件中干系人的需要。

风险管理在交付高质量软件方面也有重要作用，将严格的风险分析和风险管理技术集成到软件生命周期过程，可以提高生产高质量软件的潜力(Cha89)，风险管理的有关材料可以参见软件工程管理知识域。

2.1 软件质量保证

软件质量保证过程提供保证：通过计划制订、实施和完成一组活动，这些活动提供适当的质量已经建造进软件的置信度，这样项目生命周期中的软件产品和过程符合其规定的要求。这意味着，确保问题被清晰地和适当地陈述，已经恰当地定义和表达了解决方案的需求。软件质量保证通过在不同阶段执行各种活动，这些活动可以及早标识问题，这是任何复杂活动几乎不可避免的特征，来寻求在产品的整个开发和维护中维持质量。软件质量保证对于过程的角色是确保计划的过程是适当的，并在随后回根据计划实施，而且为适当的组织提供了相关的度量过程。

软件质量保证计划定义了用于保证为特定产品开发的软件满足用户需求并在项目的约束内具有最高的质量的手段。为此，它必须首先保证已经清晰地定义和理解了质量目标，它必须为软件考虑管理、开发和维护计划。详细请参见有关标准(IEEE730-98)。列出了特定的质量活动和任务，包括成本和资源需求、总体管理目标、进度等与软件工程管理、开发或维护计划中的目标的关系。软件质量保证计划应该与软件配置管理计划(参见软件培植管理知识域)一致。软件质量保证计划标识了支配项目及如何检查和监控才能保证适当性和符合性的文档、标准、实践和会议。软件质量保证计划也标识了度量、统计技术、报告问题和更正行动的流程、工具/技术/方法学等资源、物理介质安全性、培训、软件质量保证的报表制作和文档。此外，软件质量保证计划要说明软件计划中描述的任何类型活动的软件质量保证活动，如为项目供应商软件的采购或商用商品化软件的采购、软件交付后的服务。它也包含接收标准，以及对于软件质量很关键的报表制作和管理活动。

2.2 验证与确认[SOM04 :c19]

为简明起见，验证与确认在本指南中作为一个主题处理，而不是象(IEEE12207.0-96)标准中作为两个主题对待。“软件验证和确认是一个严格方法，用于评估整个产品申明周期的软件产品，验证和确认工作试图保证质量已经建造到软件中，并且软件满足用户需求”(IEEE1059-93)。

验证和确认直接处理软件产品质量，使用能够定位缺陷以便以后改正的测试技术，但是它也评估中间产品，在这里是软件证明周期过程的中间步骤。

验证和确认过程决定给定的开发和维护活动的产品是否符合活动的需求，最终的软件产品是否达到其意图并满足用户需求。验证试图确保活动的输出产品已经被正确制造，即活动的输出产品满足前面活动施加的规格说明，确认则试图确保建造了正确的产品，即产品满足其特定的目的。验证过程和确认过程都在开发或维护阶段的早期开始，它们提供关键产品特征与产品的前道工序产品和产品必须满足的规格说明的关系的检查。

制订验证和确认计划的目的是保证已经明确地分配了每个资源、角色和责任，得到的验证与确认计划记录和描述了各种资源及角色和责任，以及要使用的技术和工具。充分理解每个验证与确认活动的不同目的，有助于对完成其目的而需要的技术和资源作出仔细的计划。一些标准(IEEE1012-98:s7, IEEE1059-93: Appendix A)规定了通常的验证和确认计划的内容。这个计划也处理验证和确认活动的管理、交流、策略和交互流程，以及缺陷报告和文档需求。

2.3 评审与审计

为简洁起见，评审和审计在本指南中作为一个主题处理，不象(IEEE12207.0-96)中作为两个分离的主题。(IEEE12207.0-96)定义了评审和审计，(IEEE1028-97)定义更为详细，并给出了5类评审或审计：管理评审、技术评审、检查、走查、审计。

2.3.1 管理评审

“管理评审的目的是监控进展，决定计划和进度的状态，确认需求及其系统分配，或评价用于达到目标适应性的管理方法的有效性”[IEEE1028-97]，它们支持有关软件项目期间需要的变更和更正性行动的决策。管理评审决定计划、进度安排和需求的适合性，监控它们的进展和不一致性。这些评审可能针对产品进行，诸如审计报告、进展报告、验证和确认报告、各类计划，包括风险管理、项目管理、软件配置管理、软件安全性和风险评估。参见软件工程管理和软件配置管理知识域。

2.3.2 技术评审[Fre98; Hor03; Lew92; McC93; Pf101; Pre01; Som01; Voa99; Wal89; Wal96]

技术评审的目的是评价软件产品，以确定其对使用意图的适合性，目标是标识与批准的规格说明和标准的差异，结果应该向管理提供证据，以表明或否定产品满足规格说明并遵从标准，而且变更处于控制之下。技术评审中必须成立特定的角色：决策者、评审领导者、记录员和支持评审活动的技术职员。技术评审要求必须具有强制性的输入后才能进行：目标的陈述、特定的软件产品、特定的项目管理计划、与产品有关的问题列表、技术评审流程。小组遵照流程，技术上过关的人员说明产品的概观，在一次或多次会议上对产品进行检查，当检查中列出的所有活动都完成后，技术评审才完成。

2.3.3 检查[Ack97; Ebe94; Fre98; Gra92; Hor03; McC93; Pf101; Pre01; Rak97; Sch98; Som01; Voa99; Wal89; Wal96]

“检查的目的是检测和标识软件产品异常”（IEEE1028-97），检查的两个重要的与众不同特征是：（1）管理职位高于检查小组的任何人员不得参与检查；（2）检查应该由受过检查训练的公正的促进者来领导。

软件总是涉及中间或最终产品作者，其它评审则不然。检查也包括一位检查领导、一位记录员和一些检查人员。检查组的成员可以具有不同的技能，如领域技能、设计方法技能或语言技能。一次检查通常针对产品的一个相对小的部分。每个检查组成员必须在评审会前，检查软件产品和其它评审输入，可能是将分析技术（参见3.3.3节）应用到在产品的一个小部分，或应用到整个产品但只集中在某个方面。例如，接口。

发现的任何异常都要记录到文档中，并提交给检查组领导。在检查过程中，检查组领导主持会议，并确认每个成员都为检查作好准备。检查中常用的工具是检查列表，列出了与感兴趣的问题有密切关系的异常和问题，检查结果的表通常将异常分类（详细参见IEEE1044-93），小组要评审其完备性和精确性。退出检查的决定必须对应于下面三个准则之一：（1）接受或有少量的重作；（2）接受但需要验证重作的工作。（3）重新检查。

检查会议通常持续几个小时，如果技术评审和审计的范围比较大时，时间可以长一些。

2.3.4 走查[Fre98; Hor03; McC93; Pf101; Pre01; Som01; Wal89; Wal96]

走查的目的是评价软件产品，走查也可以用于培训软件产品的听众，主要目标是[IEEE1028-97]：发现异常、改进软件产品、考虑另外的实现、评价对标准和规格说明的遵从性。

走查类似与检查，但通常不那么正式，走查通常主要由软件工程师组织，让其同事评审其工作，以作为一种保障技术。

2.3.5 审计[Fre98; Hor03; McC93; Pf101; Pre01; Som01; Voa99; Wal89; Wal96]

“软件审计的目的是提供软件产品和过程对于可应用的规则、标准、指南、计划和流程的遵从性的独立评价”[IEEE1028-97]，审计是正式组织的活动，参与者有特定的角色，诸如主审计员、审计员、记录员、发起人，包括被审计组织的代表。审计将标识非遵从性的实例，并产生一个报告，要求小组采取更正性行动。

评审和审计虽然有很多正式的名称，但最重要的一点是，它们几乎在任何产品的开发或维护过程的任何阶段都可以进行。

3 实际考虑

3.1 软件质量需求[Hor03; Lew92; Rak97; Sch98; Wal89; Wal96]

3.1.1 影响因素

各种因素影响着软件质量管理活动的计划制订、管理，以及技术的选择，这些因素包括：

(1) 软件将存在与其中的系统的领域（安全最关键的、任务最关键的、业务最关键的）。
(2) 系统和软件需求。(3) 系统中将要使用的商用（外部）和标准（内部）组件。(4) 可以应用的特定软件工程标准。(5) 开发和维护、质量评价与改进要使用的方法和软件工具。(6) 所有过程的预算、人员安排、项目组织、计划和进度安排。(7) 系统的目标用户和使用方式。(8) 系统的完整性水平。

这些信息影响着如何组织软件质量管理过程和文档生成，如何选择特定的软件质量管理活动，需要什么资源，以及限定工作量的因素。

3.1.2 可依赖性

如果系统失效将造成严重后果时，总体的可依赖性（硬件、软件和人员）是处于基本功能之上的主要的质量需求，软件可依赖性包括这些特征：容错、保险性（safety）、安全性（security）和可使用性。可靠性也是一个可以从可依赖性方面来定义的准则（ISO9126）。

系统的文献主体必须是高度可依赖的（“高度可信”或“高度完整性系统”）。已经引入可传统可能不包含软件的机械和电子系统的术语来讨论威胁、危险、风险、系统完整性和相关的概念，可以在本节引用的参考文献中找到。

3.1.3 软件的完整性级别

完整性级别的确定是基于软件的失效的可能后果和失效的概率的。对于保险性和安全性非常重要的软件，可以使用保险性的危险分析或安全性的威胁分析等技术，来开展一个计划制订活动，这个活动标识潜在的问题点的所在。类似软件的失效历史也可以帮助标识检测错误和评估质量的最有用的技术。完整性级别（例如完整性等级）是在（IEEE1012-98）中提出的。

3.2 缺陷特征[Fri95; Hor03; Lew92; Rub94; Wak99; Wal89]

软件质量管理过程发现缺陷，刻画这些缺陷的特征可以导致对产品的理解，推动过程或产品的更正，向项目管理层和客户通报过程或产品的状态。有许多缺陷（错误）的分类方法，人们试图在错误和失效分类方法上达成一致，但有关文献指出，实际使用的很少[Bei90, Chi96, Gra92], (IEEE1044-93)。缺陷（异常）的特征化也在审计和评审中用到，评审领导通常提出由小组成员提供的异常列表，以在评审会上考虑。

随着新的设计方法和语言的进化，以及总体软件技术的进展，出现了新类型的缺陷，需要大量的工作来解释以前定义的类别。追踪缺陷时，软件工程师不仅对缺陷数目感兴趣，而且对缺陷类型也感兴趣。没有某种分类的信息本身对于标识缺陷的原因几乎没有用处，因为特定类型的问题需要聚集在一起，以便作出有关决定。要点是建立对于组织和软件工程师有意义的缺陷分类。

软件质量管理在软件开发和维护的所有阶段都能发现信息，通常，使用“缺陷(defect)”一词时，指的是下面定义的“错误(fault)”，但是，不同的文献和标准可能使用与这些术语多少不同的意义。从(IEEE610.12-90)中摘录的部分定义如下：(1) 误差(Error)：计算得到的结果与正确结果之间的差异。(2) 错误/故障(Fault)：计算机程序中的不正确的步骤、过程或数据定义。(3) 失效(Failure)：错误(Fault)的[不正确]结果。(4) 错误(Mistake)：产生不正确结果的人类动作。

本节的讨论中，将测试中发现的作为软件错误的失效包括在缺陷中。可靠性模型是从软

件测试或软件运行中收集的数据中建立的，因而可以预测未来的失效，以及辅助何时停止测试的决策[Mus89]。

从软件质量管理发现的结果中，可能采取的行动是从被检查的产品中消除缺陷，其它行动可能实现软件质量管理活动发现的结果的全部价值。这些行动包括：分析和总结发现结果，使用度量技术来改进产品和过程，以及追踪缺陷及缺陷的消除。过程改进主要在软件工程过程知识域中讨论，那里，软件质量管理是一个信息源。

如果不记录，就可能丢失在软件质量管理技术的实现中发现的不适当性和缺陷，对于一些技术（例如，技术评审、审计、检查），记录员要出席，以记录这些信息以及问题和决策。如果使用了自动化工具，工具的输出可以提供缺陷信息。可以收集关于缺陷的数据并记录在软件变更请求的表格中，随后手工或自动记录到某种类型的数据库中。关于缺陷的报告应提交给组织的管理层。

3.3 软件质量管理技术[Bas94; Bei90; Con86; Chi96; Fen97; Fri95; Lev95; Mus89; Pen93; Sch98; Wak99; Wei93; Zel98]

软件质量管理技术可以按多种方式分类：静态的、人力密集的、分析性的、动态的。

3.3.1 静态技术

静态技术涉及检查项目文档和软件，以及其它有关软件产品的信息，但不执行软件。这些技术可能包括人力密集的活动（如3.3.2的定义）或由个人组织的分析活动（如3.3.3的定义），可能有，也可能没有使用自动化工具。

3.3.2 人力密集的技术

人力密集的技术包括评审和审计，规模从正式会议到非正式的聚会或桌面检查会议不等，但（通常至少）有两人或两人以上，必须提前作好准备。除了要检查的项目外，还需要其它资源：检查列表、分析技术和测试的结果。这些活动在(IEEE1028-97)的评审和审计部分中讨论[Fre98, Hor03] and [Jon96, Rak97]。

3.3.3 分析型技术

软件工程师一般要应用分析型技术，有时，几个软件工程师使用同一技术，但却是将其应用到产品的不同方面。一些技术是工具驱动的，其它则是手工的；一些技术直接找出缺陷，但通常用来支持其它技术。一些技术也包括各种评估，作为总体质量分析的一部分，这类技术的例子包括：复杂性分析、控制流分析和算法分析。每一类分析都有其特定目的，并非所有类型的分析都要应用到每个项目上。支持技术的一个例子是复杂性分析，它对于确定设计或实现是否太复杂而不能正确开发、测试或维护，很有用。复杂性分析的结果可以用于测试用例的开发。诸如控制流分析等缺陷发现技术也可能用于支持其它活动。对于有许多算法的软件，算法分析很重要，特别是不正确的算法将引起灾难性的后果时。有许多分析技术，这里不能全部列举出来，这里提供的列表和参考文献可能为选择技术提供意见和建议深入的阅读。

其它更形式化的分析技术类型，称为形式化方法，它们用来验证软件需求和设计，可以对软件的关键部分应用正确性证明技术，这些技术主要用于关键系统的重要部分的验证，诸如特定的安全性和保险性需求(Nas97)

3.3.4 动态技术

在软件的整个开发和维护阶段，使用了不同类型的动态技术，一般，这些技术是测试技

术，但诸如模拟、模型检查和符号执行也可认为是动态的。代码阅读被认为是静态技术，但有经验的软件工程师在与阅读代码时，可能或执行代码。在这个意义上，代码阅读可以认为是动态技术。这种分类的差异表明组织中不同角色的人员可能会以不同的方式来考虑和应用这些技术。这样，一些测试可能在开发过程中、软件质量保证过程中或验证与确认过程中进行，这也依赖于项目的组织。由于软件质量管理计划涉及测试，本节将包括一些对测试的评论。软件测试知识域为测试的理论、技术和自动化提供了讨论和参考文献。

3.3.5 测试

软件质量保证和验证与确认中描述的保证过程根据软件需求规格说明检查每个输出，以确保输出的可追踪性、一致性、完备性、正确性和性能。这种确认也包括开发和维护过程的输出，以及结果的收集、分析和度量。软件质量保证要确保：适当的测试类型已经列入计划，并已经进行了开发和实现，验证与确认则要开发测试的计划、策略、用例和流程。

软件测试知识域中详细讨论了测试，有两类测试可能划入软件质量保证和验证与测试内，因为它们负责项目使用的材料的质量：（1）项目中要使用的工具的评价与测试（IEEE1462）。（2）项目中要使用的组件和商用商品化软件的遵从性测试（或遵从性测试的评审），现在已经有一个软件包的标准（IEEE1465）。

有时，会让一个独立的验证与确认组织来监控测试过程，有时还要见证实际的执行，以保证测试是按规定的过程进行的。验证与确认可以用来评价测试本身：计划与流程的适当性、结果的适当性和精确性。

另一类可以划入验证与确认组织的是第三方测试，第三方既不是一个开发者，又与产品的开发没有任何联系，第三方是一个独立的机构，通常由某个权威机构认证，其目的是测试产品是否遵从一组特定的需求。

3.4 软件质量度量[Gra92]

软件产品质量模型通常包括确定每个由产品获得的特征的程度的度量。如果适当选择，度量可以从多种方式支持（在软件生命周期过程的其它方面中）软件质量，它们可以帮助管理决策过程，可以发现软件过程中的有问题的地方和瓶颈，可以帮助软件工程师为软奖质量保证目的或为长期的过程质量改进而评估其工作。

随着软件复杂性的增加，质量问题远远超过了软件是否工作，达到可度量的质量目标的程度如何。

还有更多度量直接支持软件质量管理的主题，这些包括：辅助决定何时停止测试，对此，使用了错误和失效数据的可靠性模型和基准都有用。

软件质量管理过程的成本是一个问题，它几乎总是在决定应该如何组织一个项目时被提出来。通常，使用一般的成本模型，它基于何时发现一个缺陷，和相对于在开发周期早期阶段发现这个缺陷，修复这个缺陷需要的工作量。项目数据可能会给出成本的更好图象，对这个主题的讨论参见[Rak97: pp. 39-50]。相关的信息可以在软件工程过程和软件工程管理知识域中找到。

最后，软件质量管理报告本身不仅提供关于这些过程的有价值的信息，而且提供了有关如何改进所有软件生命周期过程的信息。有关这些主题的讨论参见[McC04]和（IEEE1012-98）。

尽管质量特征和产品特性的度量本身是有用的（例如，有缺陷需求的数目或比例），还可以应用数学和图形技术来辅助这些度量的解释，它们适合与下列类别，有关讨论参见[Fen97, Jon96, Kan02, Lyu96, Mus99]：（1）基于统计学的（例如，Pareto分析，运行图、散布图、正态分布）。（2）统计测试（例如，二项分布测试、 χ^2 分布测试）。（3）趋势

分析。（4）预测（例如，可靠性模型）。

基于统计的技术和测试通常为被检测的软件产品的容易出错的地方提供了一个快照，结果可视化表示为图表和图形，决策者可以用来集中资源到最需要的地方。趋势分析的结果可以用来指示诸如测试中进度没有完成，除非在开发中采取一些更正性行动，某类错误将变得更加密集。一般度量的更多讨论参见软件工程过程和软件工程管理知识域，测试度量的更特定的信息参见软件测试知识域。

参考文献[Fen97, Kan02, Pf101, Jon96]提供了缺陷分析的讨论，缺陷分析包括度量缺陷发生，应用统计方法来理解最常发生的缺陷的类型，即回答问题以评估其密度。它们也辅助理解检测技术的趋势和工作情况、开发和维护过程的进展如何。测试覆盖的度量帮助人们估算还需要多少测试工作量，预测可能的残留缺陷。从这些度量方法，可以得到特定应用领域的缺陷概貌。然后，对于这个组织的下一个软件系统，可以使用这个概貌来指导软件质量管理过程，即，为问题最可能发生的地方分配更多的工作量。同样，这个领域的基准或典型缺陷计数，可作为确定产品是否可以交付的辅助手段。

有关使用来自软件质量管理的数据来改进开发和维护过程的讨论，可以参见软件工程管理和软件工程过程知识域。

主题与参考文献矩阵

	[B o e 7 8]	[D a c 0 1]	[H o u 9 9]	[J o n 9 6]	[K i a p 9 5]	[L e w 9 2]	[L l o 0 3]	[M c C 7]	[M u s 9]	[N I S T]	[P r e 0 4]	[R a k 9 7]	[S e i 0 2]	[W a l 9 6]	[W e i 9 3]	[W e i 9 6]
1 软件质量基础										*						
1.1 软件工程文化与伦理																*
1.2 质量的价值和成本	*		*	*						*	*				*	*
1.3 模型和质量特征		*			*	*	*	*	*	*	*	*	*	*		*
1.4 软件质量改进										*	*					*

	[A c k 9 7]	[E b e 9 4]	[F r e 9 8]	[G r a 9 2]	[H o r 0 3]	[L e w 9 2]	[M c C 9 3]	[P f l 0 1]	[P r e 0 4]	[R a k 9 7]	[S c h 9 8]	[S o m 0 5]	[V o a 9 9]	[W a l 8 9]	[W a l 9 6]
2 软件质量管理过程			*		*	*	*	*	*			*	*	*	*
2.1 软件质量保证	*	*	*	*	*		*	*	*	*	*	*	*	*	*
2.2 验证与确认			*		*		*	*	*			*		*	*

2.3 评审与审计			*		*		*	*	*			*	*	*	*
-----------	--	--	---	--	---	--	---	---	---	--	--	---	---	---	---

	[B a s 8 4]	[B e i 9 0]	[C o n 8 6]	[C h i 9 6]	[F e n 9 7]	[F e r 9 8]	[F e r 9 5]	[G r a 9 2]	[H e a 8 4]	[H o r 0 3]	[J o n 9 6]	[K a n 9 4]	[K a n 0 2]	[L e v 9 5]	[L e w 9 2]	[L y u 9 6]
3 软件质量实际考虑																
3.1 软件质量需求										*					*	
3.2 缺陷特征		*		*			*	*	*	*					*	
3.3 软件质量管理技术	*	*	*	*	*	*	*			*	*	*		*		
3.4 软件质量度量					*			*			*		*			*

	[M c C 9 3]	[M u s 8 9]	[M u s 9 9]	[P e n 9 3]	[P e n 0 1]	[R a u 9 7]	[R a u 9 4]	[S c h 9 8]	[W a k 9 9]	[W a l 8 9]	[W a l 9 6]	[W e i 9 3]	[Z e l 9 8]
3 软件质量实际考虑													
3.1 软件质量需求						*		*		*	*		
3.2 缺陷特征		*					*		*	*			
3.3 软件质量管理技术		*		*		*		*	*			*	*
3.4 软件质量度量	*		*		*	*							

软件质量的推荐参考文献

- [Ack97] F. A. Ackerman, "Software Inspections and the Cost Effective Production of Reliable Software," 1997
- [Bas84] V. R. Basili and D.M. Weiss, "A Methodology for Collecting Valid Software Engineering Data," *IEEE Transactions on Software Engineering*, vol. SE-10, iss. 6, 728-738, November, 1984
- [Bei90] B. Beizer, *Software Testing Techniques*: International Thomson Press, 1990.
- [Boe78] B. W. Boehm and al, "Characteristics of Software Quality," *TRW series on Software Technologies*, vol. 1, 1978
- [Chi96] R. Chillarege, "Orthogonal Defect Classification," in *Handbook of Software Reliability Engineering*, M. Lyu, Ed.: IEEE CS Press, 1996.
- [Con86] S. D. Conte, H. E. Dunsmore and V. Y. Shen, *Software Engineering Metrics and Models*: The Benjamin /Cummings Publishing Company, Inc., 1986.

- [Dac01] G. Dache, "IT Companies will gain competitive advantage by integrating CMM with ISO9001," *Quality System Update*, vol. 11, iss. 11, November, 2001
- [Ebe94] R. G. Ebenau and S. Strauss, *Software Inspection Process*: McGraw-Hill, 1994.
- [Fen98] N. E. Fenton and S. L. Pfleeger, *Software Metrics: A Rigorous & Practical Approach*, Second ed: International Thomson Computer Press, 1998.
- [Fre98] D. P. Freedman and G. M. Weinberg, *Handbook of Walkthroughs, Inspections, and Technical Reviews*: Little, Brown and Company, 1998.
- [Fri95] M. A. Friedman and J. M. Voas, *Software Assessment: Reliability, Safety Testability*: John Wiley & Sons, Inc., 1995.
- [Gra92] R. B. Grady, *Practical Software Metrics for project Management and Process Management*: Prentice Hall, Englewood Cliffs, NJ 07632, 1992.
- [Hor03] J. W. Horch, *Practical Guide to Software Quality Management*: Artech-House Publishers, 2003.
- [Hou99] D. Houston, "Software Quality Professional," *ASQC*, vol. 1, iss. 2, 1999
- [IEEE-CS-99] IEEE-CS-1999, "Software Engineering Code of Ethics and Professional Practice," IEEE-CS/ACM, 1999, available at <http://www.computer.org/certification/ethics.htm>
- [Jon96] C. Jones and J. Capers, *Applied Software Measurement: Assuring Productivity and Quality*, Second ed: McGraw-Hill, 1996.
- [Kan02] S. H. Kan, *Metrics and Models in Software Quality Engineering*, Second ed: Addison-Wesley, 2002.
- [Kia95] D. Kiang, "Harmonization of International Software Standards on Integrity and Dependability," presented at Proc. IEEE International Software Engineering Standards Symposium, Los Alamitos, CA, 1995
- [Lap91] J. C. Laprie, *Dependability: Basic Concepts and Terminology in English, French, German, Italian and Japanese, IFIP WG 10.4*. New York: Springer-Verlag, 1991.
- [Lev95] N. G. Leveson, *Safeware: System Safety and Computers*: Massachusetts: Addison-Wesley, 1995.
- [Lew92] R. O. Lewis, *Independent Verification and Validation: A Life Cycle Engineering Process for Quality Software*: John Wiley & Sons, Inc., 1992.
- [Llo03] Lloyd's Register, "TickIT Guide," iss. 5, 2003, available at <http://www.tickit.org>
- [Lyu96] M. R. Lyu, *Handbook of Software Reliability Engineering*: Mc-Graw-Hill/IEEE, 1996.
- [McC77] J. A. McCall, "Factors in Software Quality -General Electric," n77C1502, June, 1977
- [McC93] S. McConnell, *Code Complete: A Practical Handbook of Software Construction*: Microsoft Press, 1993.
- [Mus89] J. D. Musa and A. F. Ackerman, "Quantifying Software Validation: When to stop testing?," *IEEE Software*, vol. 6, iss. 3, 19-27, May, 1989
- [Mus99] J. Musa, *Software Reliability Engineering: More Reliable Software, Faster Development and Testing*: McGraw Hill, 1999.
- [NIST] National Institute of Standards and Technology, "Balridge National Quality Program," available at <http://www.quality.nist.gov>

- [Pen93] W. W. Peng and D. R. Wallace, "Software Error Analysis," National Institute of Standards and Technology, Gaithersburg, NIST SP 500-209 MD 20899, December, 1993, available at <http://hissa.nist.gov/SWERROR/>
- [Pf101] S. L. Pfleeger, *Software Engineering: Theory and Practice*, Second ed: Prentice-Hall, Inc., 2001.
- [Pre04] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, Sixth ed: McGraw-Hill, 2004.
- [Rak97] S. R. Rakitin, *Software Verification and Validation: A Practitioner's Guide*: Artech House, Inc., 1997.
- [Rub94] J. Rubin, *Handbook of Usability Testing: How to Plan, Design, and Conduct Effective Tests*: John Wiley & Sons, 1994.
- [Sch99] G. C. Schulmeyer and J. I. McManus, *Handbook of Software Quality Assurance*, Third ed: Prentice Hall, 1999.
- [SEI02] Software Engineering Institute, "Capability Maturity Model Integration for Software Engineering. (CMMI)," Software Engineering Institute, Carnegie Mellon University CMU/SEI-2002-TR-028, ESC-TR-2002-028, 2002
- [Som05] I. Sommerville, *Software Engineering*, Seventh ed: Addison-Wesley, 2005.
- [Voa99] J. Voas, "Certifying Software For High Assurance Environments," *IEEE Software*, vol. 16, iss. 4, 48-54, July-August, 1999
- [Wak99] S. Wakid, D. R. Kuhn and D. R. Wallace, "Toward Credible IT Testing and Certification," *IEEE Software*, 39-47, July-August, 1999
- [Wal89] D. R. Wallace and R. U. Fujii, "Software Verification and Validation: An Overview," *IEEE Software*, vol. 6, iss. 3, 10-17, May, 1989
- [Wal96] D. R. Wallace, L. Ippolito and B. Cuthill, "Reference Information for the Software Verification and Validation Process," @NIST SP 500-234, Gaithersburg, NIST MD 20899, April, 1996, available at <http://hissa.nist.gov/VV234/>
- [Wei93] G. M. Weinberg, "Quality Software Management: First-Order Measurement," vol. 2: Dorset House, 1993, Ch. 8, Measuring Cost and Value.
- [Wie96] K. Wiegers, *Creating a Software Engineering Culture*: Dorset House, 1996.
- [Zel98] M. V. Zelkowitz and D. R. Wallace, "Experimental Models for Validating Technology," *Computer*, vol. 31, iss.5, 23-31, 1998

附录A 深入阅读的参考文献

- (Abr96) A. Abran and P. N. Robillard, "Function Points Analysis: An Empirical Study of its Measurement Processes," presented at IEEE Transactions on Software Engineering, 1996
- (Art93) L. J. Arthur, *Improving Software Quality: An Insider's Guide to TQM*: John Wiley & Sons, 1993.
- (Bev97) N. Bevan, "Quality and Usability: A New Framework," in *Achieving Software Product Quality*, E. v. Veenendaal and J. McMullan, Eds. Uitgeverij Tutein Nolthenius, Holland, 1997.
- (Cha89) R. N. Charette, *Software Engineering Risk Analysis and Management*: McGraw-Hill, 1989.
- (Cro79) P. B. Crosby, *Quality is free*: McGraw-Hill, 1979.

- (Dem86) W. E. Deming, *Out of the Crisis*: MIT Press, 1986.
- (Dod00) Department of Defense and US Army, "Practical Software and Systems Measurement: A Foundation for Objective Project Management, Version 4.0b," October, 2000, available at <http://www.psmsc.com>
- (Hum89) W. Humphrey, "Managing the Software Process," Massachusetts: Addison Wesley, 1989, Chap. 8, 10, 16.
- (Hya96) L. E. Hyatt and L. Rosenberg, "A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality," presented at 8th Annual Software Technology Conference, Utah, 1996
- (Inc94) D. Ince, *ISO 9001 and Software Quality Assurance*. London: McGraw-Hill, 1994.
- (Jur89) J. M. Juran, *Juran on Leadership for Quality*. New-York: The Free Press, 1989.
- (Kin92) M. R. Kindl, "Software Quality and Testing: What DoD Can Learn from Commercial Practices," U.S. Army Institute for Research in Management Information, Communications and Computer Sciences, Georgia Institute of Technology, Atlanta, Georgia August, 1992
- (NASA97) NASA, "Formal Methods Specification and Analysis Guidebook for the Verification of Software and Computer Systems, Volume II: A Practitioner's Companion," 1997, available at http://eis.jpl.nasa.gov/quality/Formal_Methods/
- (Pal) J. D. Palmer, "Traceability," in *Software Engineering*, M. Dorfman and R. Thayer, Eds., 1997, 266-276.
- (Ros98) L. Rosenberg, "Applying and Interpreting Object-Oriented Metrics," presented at Software Tech. Conf., 1998, available at <http://satc.gsfc.nasa.gov/support/index.html>
- (Sur03) W. Suryn, A. Abran and A. April, "ISO/IEC SQuaRE. The second generation of standards for software product quality," presented at IASTED2003, Marina del Rey, California, 2003
- (Vin90) W. G. Vincenti, *What Engineers Know and How They Know It - Analytical Studies form Aeronautical History*. Baltimore and London: John Hopkins University Press, 1990

附录B 有关的标准

- (FIPS140.1-94) FIPS 140-1, *Security Requirements for Cryptographic Modules*, 1994.
- (IEC61508-98) IEC 61508, "Functional Safety - Safety - related Systems Parts 1, 2, 3," Institution of Electrical Engineers, 1998
- (IEEE610.12-90) IEEE Std 610.12-1990 (R2002), *IEEE Standard Glossary of Software Engineering Terminology*: IEEE, 1990.
- (IEEE730-02) IEEE Std 730-2002, *IEEE Standard for Software Quality Assurance Plans*: IEEE, 2002.
- (IEEE982.1-88) IEEE Std 982.1-1988, *IEEE Standard Dictionary of Measures to Produce Reliable Software*, 1988.
- (IEEE1008-87) IEEE Std 1008-1987 (R2003), *IEEE Standard for Software Unit Testing*: IEEE, 1987.
- (IEEE1012-98) IEEE Std 1012-1998, *Software Verification and Validation*: IEEE, 1998.
- (IEEE1028-97) IEEE Std 1028-1997 (R2002), *IEEE Standard for Software Reviews*: IEEE, 1997.

(IEEE1044-93) IEEE Std 1044-1993 (R2002), *IEEE Standard for the Classification of Software Anomalies*: IEEE, 1993.

(IEEE1059-93) IEEE Std 1059-1993, *IEEE Guide for Software Verification and Validation Plans*: IEEE, 1993.

(IEEE1061-98) IEEE Std 1061-1998, *IEEE Standard for a Software Quality Metrics Methodology*: IEEE, 1998.

(IEEE1228-94) IEEE Std 1228-1994, *Software Safety Plans*: IEEE, 1994.

(IEEE1462-98) IEEE Std 1462-1998//ISO/IEC14102, *Information Technology – Guideline for the Evaluation and Selection of CASE tools*.

(IEEE1465-98) IEEE Std 1465-1998//ISO/IEC12119:1994, *IEEE Standard Adoption of International Standard ISO/IEC12119:1994(E), Information Technology-Software packages-Quality requirements and testing*: IEEE, 1998.

(IEEE12207.0-96) IEEE/EIA 12207.0-1996//ISO/IEC12207:1995, *Industry Implementation of Int. Std. ISO/IEC 12207:95, Standard for Information Technology-Software Life Cycle Processes*, vol. IEEE, 1996.

(ISO9001-00) ISO 9001:2000, *Quality Management Systems-Requirements*: ISO, 1994.

(ISO9126-01) ISO/IEC 9126-1:2001, *Software Engineering-Product Quality-Part 1: Quality Model*: ISO and IEC, 2001.

(ISO14598-98) ISO/IEC 14598:1998, *Software Product Evaluation*: ISO and IEC, 1998.

(ISO15026-98) ISO/IEC 15026:1998, *Information technology -- System and software integrity levels*: ISO and IEC, 1998.

(ISO15504-98) ISO/IEC TR 15504-1998, *Information Technology – Software Process Assessment (parts 1-9)*: ISO and IEC, 1998.

(ISO15939-00) ISO/IEC 15939:2000, *Information Technology – Software Measurement Process*: ISO and IEC, 2000.

(ISO90003-04) ISO/IEC 90003:2004, *Software and Systems Engineering-Guidelines for the Application of ISO9001:2000 to Computer Software*: ISO and IEC, 2004..

////////////////////////////////////

第12章 软件工程相关学科介绍

引言

为确定软件工程的范围,有必要识别出与软件工程有公共边界的学科。本章以字母顺序,描述这些相关学科。当然,相关学科之间也有公共边界。

本章使用了一致认可的来源,来鉴别每个相关学科,可行时,给出学科的资料性的定义,或者给出其知识域列表。

图1给出了这些相关学科的图形表示。

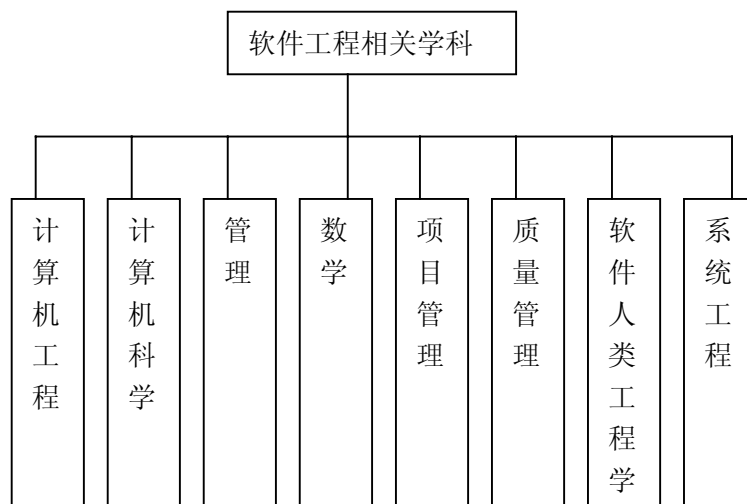


图 1 软件工程相关学科

相关学科及其知识域列表

计算机工程

计算技术课程表2001 (Computing Curricula 2001, CC2001) 项目在计算机工程卷的草案报告(参见:http://www.eng.auburn.edu/ece/CCCE/Iron_Man_Draft_October_2003.pdf)中指出: 计算机工程体现了设计、构造、实现和维护现代计算机系统和计算机控制的设备中的软件和硬件成分的科学和技术。

这份报告鉴别了计算机工程的以下知识域(报告中称为域): 算法与复杂性、计算机体系结构与组织、计算机系统工程、电路与系统、数字逻辑、离散结构、数字信号处理、分布式系统、电子学、嵌入式系统、人机交互、信息管理、智能系统、计算机网络、操作系统、程序设计基础、概率与统计、社会与职业问题、软件工程、测试与验证、VLSI/ASIC设计。

计算机科学

计算技术课程表2001 (Computing Curricula 2001, CC2001) 项目在计算机科学卷的最后报告(参见: <http://www.computer.org/education/cc2001/final/cc2001.pdf>)中, 为计算机科学鉴别出了下列知识域(报告中称为域): 离散结构、程序设计基础、算法与复杂性、体系结构与组织、操作系统、以网络为中心的计算、程序设计语言、人机交互、图形学与可视化计算、智能系统、信息管理、社会与职业问题、软件工程、计算科学与数值方法。

管理

欧洲国家认证团体协会 (European association of national accreditation bodies, EQUAL) 定义的欧洲MBA指南(参见: <http://www.efmd.be/>)指出, MBA应该包括以下方面的教育培训: 会计学、财政学、市场与营销、运作管理、信息系统管理、法律、人力资源管理、经济学、定量分析、商业策略与战略。

数学

我们选择了两个来源, 来鉴别数学的知识域。

加拿大工程认证委员会 (Canadian Engineering Accreditation Board) 的报告“认证

准则与程序”（参见：http://www.ccpe.ca/e/files/report_ceab.pdf）指出，本科生工程课程表中，应该包括下列领域的适当内容：线性代数、微积分、微分方程、概率、统计、数值分析、离散数学。

支持软件工程的更专门的数学主题列表（报告中称为单元和主题）可以在计算技术课程表2001（Computing Curricula 2001, CC2001）项目的软件工程卷的草案报告找到。（参见：<http://sites.computer.org/ccse/volume/FirstDraft.pdf>）

项目管理

项目管理在2000年版的《项目管理知识体系指南》（A Guide to the Project Management Body of Knowledge, PMBOK®6 Guide）中定义，指南由项目管理研究所（Project Management Institute）出版，并被采纳为IEEE1490-2003标准。项目管理定义为：将知识、技能、工具和技术应用到项目活动上，以满足项目需求。PMBOK指南为项目管理鉴别的知识域包括：项目集成管理、项目范围管理、项目时间管理、项目成本管理、项目质量管理、项目人力资源管理、项目通信管理、项目风险管理、项目采购管理。

（PMBOK是美国和其它国家的注册商标）

质量管理

在ISO9000-2000中，质量管理定义为：指导和控制组织关于质量的调协的活动。我们选择的质量管理的3个参考文献是：ISO9000:2000质量管理体系——基础与词汇、ISO9004:2000质量管理体系——需求、ISO9004:2000质量管理体系——行为改进指南。

美国质量协会在其质量工程师认证知识体系中（参见：<http://www.asq.org/cert/types/cqe/bok.html>），为质量管理鉴别了以下知识域（第一级结构分解）：质量工程的管理与领导；质量系统开发、实现和验证；产品和过程质量的计划、控制和保证；可靠性与风险管理、问题解决与质量改进、定量方法。

软件人类工程学

人类工程学领域由ISO的159人类工程技术委员会定义：人类工程学或人类因素是涉及理解人和一个系统中其它元素之间的交互的科学学科，是一个应用理论、数据和方法来进行设计，以优化人类良好的本质与系统的总体性能。（参见：

http://isotc.iso.ch/livelink/livelink.exe/fetch/2000/2122/687806/ISO_TC_159__Ergonomics_.pdf?nodeid=1162319&vernum=0）

人类工程学应用到软件时的知识域列表（注：这个表是为SWEBOK指南2001年版编排的，来自John Hopkins 大学的认知科学系和ACM的SIGHCI为人机交互所列的课程表。后来，这个领域的3位专家精化了这个列表，3位专家有2位来自Université du Québec à Montréal。另一位专家W W McMillan来自Eastern Michigan University。我们请他们指出哪些主题应该由软件工程师了解，被其中两位专家否决的主题被删除）。

（1）认知、认知型人工智能I：推理、机器学习和语法归纳、认知科学的形式化方法：语言、认知科学的形式化方法：推理、认知科学的形式化方法：认知体系结构、认知人工智能II：学习、认知科学基础、语音和文本的信息提取、词法处理、计算语言获取、人机交互本质、人机交互（元）模型。（2）计算机的使用和上下文：人类社会组织与工作、应用领域、人机适合与适应。（3）人类特征：人类信息处理；语言、通信、交互；人类工程学。

（3）计算机系统和界面体系结构：输入输出设备、对话技术、对话类型、计算机图形学。

（4）对话体系结构。（5）开发过程：设计方法、实现技术、评价技术、范例系统与实例学习。

用于软件工程课程表的人机交互设计主题（报告中称谓单元或主题）的更专门列表，可以参见计算技术课程表2001项目（CC2001）的软件工程卷的草案报告。（参见：<http://sites.computer.org/ccse/volume/FirstDraft.pdf>）

系统工程

国际系统工程理事会（International Council on Systems Engineering, INCOSE）（参见：www.incose.org）指出：系统工程是一个能够实现成功系统的交叉学科的途径和手段，它在开发周期的早期就集中于定义客户需要和要求的功能，然后进行设计综合和系统的确认，同时考虑以下全部问题：运行性能、测试、制造、成本和进度、培训和支持、安置。

系统工程将所有的学科和专门小组整合为一个团队，形成一个结构化的过程，过程从概念到产品、再到运行。系统工程同时考虑所有客户的业务和技术需要，目标是提供一个满足用户需要的高质量产品。

国际系统工程理事会正在编写系统工程知识体系的指南。初步的版本包括以下第一级的技能域：（1）业务过程和运营评定（BP0A）；（2）系统/方案/测试体系结构（SSTA）；（3）生命周期成本与成本效益分析（LCC&CBA）；（4）可服务性和后勤保障（S/L）；（5）建模、模拟和分析（MS&A）；（6）管理：风险、配置、基线（Mgt）。

////////////////////////////////////

附录D 根据Bloom的分类学进行的主题分类

引言

Bloom的分类学（参见：B Bloom编，Taxonomy of Educational Objectives: The Classification of Educational Goals, Mackay, 1956）是众所周知并广泛使用的认知教育目标分类。为帮助这个领域中希望使用指南作为设计课程材料、大学课程表、大学程序或认证准则、软件工程过程定义中的职务描述和角色描述、职业发展路径和职业培训程序、以及其它需要的读者，本附录给出了具有4年工作经验的软件工程专业本科生的应该具有的SWEBOK指南的主题的Bloom分类等级。具有4年工作经验的软件工程专业本科生实质上是根据普遍接受的知识的定义（参见SWEBOK指南引言）而确定的SWEBOK指南的“目标”。

由于附录仅仅与被认为是“普遍接受的”知识有关，应该指出，软件工程师必须了解比这个知识目录更多的东西。除了“普遍接受的”知识，具有4年工作经验的软件工程专业本科生必须具有来自相关学科的知识，以及某些专门的知识、高级知识，甚至是研究性知识（参见SWEBOK指南引言）。在确定分类学级别时，我们做了下列假设：

（1）这个评价是针对“最普通的”软件工程师，而不是针对工作在特定小组的软件工程师的，例如软件配置管理小组。很明显，这样一个软件工程师需要这个特定小组的更高级别的分类等级。

（2）具有4年工作经验的软件工程专业本科生仅仅在其职业生涯的开始阶段，分配给他的管理任务比较少，至少不是主要的任务。因此，在建议的评价中，“管理相关的主题”的优先权少。这样，向与软件需求有关的“生命周期早期的主题”的分类学等级就比软件设计、软件构造和软件测试等与技术有关的主题的等级低。

（3）为了使评价可以适应更高级别的软件工程师或专业为某些特殊知识领域的软件工程师，主题的分类学等级没有超过分析，这与软件工程教育知识体系（Software Engineering Education Body of Knowledge, SEEK）中采用的方法一致，在那里，主题的等级没有超过应用。（参见：See Joint Task Force on Computing Curricula - IEEE Computer Society Association for Computing Machinery, Computing Curricula - Software Engineering

Volume - Publid Draft 1 - Computing Curriculum SoftwareEngineering, 2003.
<http://sites.computer.org/ccse/>)。SEEK的目的是为指导开发本科生软件工程课程表，定义软件工程教育知识体系。尽管二者的范围不同，SEEK和SWEBOK密切相关。（参见：P Bourque, F. Robert, J.-M. Lavoie, A. Lee, S. Trudel, T. Lethbridge, “Guide to the Software Engineering Body of Knowledge (SWEBOK) and the Software Engineering Education Body of Knowledge (SEEK) - A Preliminary Mapping”, in *Proc. Tenth Intern. Workshop Software Technology and Engineering Practice Conference (STEP 2002)*, pp. 8-35, 2002)。

Bloom在1956年提出的认知领域分类学包括6个级别，表1给出了这些级别以及与各个级别相关的关键词。（参见：<http://www.nwlink.com/~donclark/hrd/bloom.html>）。

表1 Bloom分类学

Bloom分类学级别	对应的关键词
知识knowledge: 回忆数据	定义、描述、识别、知道、标记、列表、匹配、命名、轮廓、回忆、认识、复现、选择、陈述
理解comprehension: 了解含义，对指令和问题的翻译、添写、解释。用自己的话陈述问题。	理解、转换、辩解、区别、估计、解说、扩展、推广、举例子、推理、翻释、释义、预言、重写、总结
应用application: 在新情形中使用一个概念，或抽象的临场使用。将教室里学到的东西应用到工作中新的情况中。	应用、改变、计算、构造、展示、发现、操纵、修改、操作、预测、准备、生产、联系、演示、解决、使用
分析analysis: 将材料或概念分解为成分，以理解其组成结构，可以区分事实与推论。	分析、结构分解、比较、对比、做图表、析构、区分、辨别、区别、识别、例示、推导、轮廓、联系、选择、分离
综合synthesis: 用分不同的元素构造一个结构或模式，将部分组装为整体，并创造一个新含义或结构	分类、组合、编排、合成、创造、设想、设计、解释、产生、修改、组织、规划、重排、重构、联系、重组、修订、重写、总结、讲述、写作
评价evaluation: 对思想和材料的价值作出判断	赞同、比较、下结论、对比、批评、辩解、描述、区分、评价、解释、释义、判断、联系、总结、支持

表中主题的结构分解并不完全与知识域的结构匹配。这个附录的评价正在准备中，新的意见正在加入。

最后，请记住，这个附录的评价只能被认为是以后进一步发展和确认的建议。

软件需求（K知识、C理解、AP应用、AN分析、E评价、S综合。下同）

主题分解结构	分类学级别
1软件工程基础	
软件需求定义	C
产品和过程需求	C
功能与非功能需求	C
突发性特性	C

定量需求	C
系统需求与软件需求	C
2需求过程	
过程模型	C
过程参与者	C
过程支持与管理	C
过程质量与改进	C
3需求获取	
需求来源	C
获取技术	AP
4需求分析	
需求分类	AP
概念建模	AN
体系结构设计与需求分配	AN
需求协商	AP
5需求规格说明	
系统定义文档	C
系统需求规格说明	C
软件需求规格说明	AP
6需求确认	
需求评审	AP
建立原型	AP
模型确认	C
接收测试	AP
7实际考虑	
需求过程的迭代本质	C
变更管理	AP
需求属性	C
需求追踪	AP
度量需求	AP

软件设计

主题分解结构	分类学级别
1软件设计基础	
一般设计概念	C
软件设计上下文	C
软件设计过程	C
使能技术	AN
2软件设计关键问题	
并发性	AP

事件的控制与处理	AP
组件的分布	AP
错误与异常处理、容错	AP
交互与表现	AP
数据持久性	AP
3软件结构与体系结构	
体系结构与视点	AP
体系结构风格（宏观体系结构模式）	AN
设计模式（微观体系结构模式）	AN
程序与框架族	C
4软件设计质量的分析和评价	
质量属性	C
质量分析与评价技术	AN
度量	C
5软件设计符号	
结构描述（静态）	AP
行为描述（动态）	AP
6软件设计策略与方法	
一般策略	AN
面向功能的（结构化）设计	AP
面向对象设计	AN
数据结构为中心的设计	C
基于组件的设计（CBD）	C
其他方法	C

软件构造

主题分解结构	分类学级别
1软件构造基础	
复杂性最小化	AN
变更的预测	AN
为验证进行构造	AN
构造标准	AP
2构造的管理	
构造的方法	C
构造的计划	AP
构造的度量	AP
3实际考虑	
构造的设计	AN
构造的语言	AP
编码	AN
构造的测试	AP
构造的质量	AN
集成	AP

软件测试

主题分解结构	分类学级别
1软件测试基础	
测试相关的术语	C
关键问题	AP
测试与其它活动的联系	C
2测试级别	
测试对象	AP
测试目标	AP
3测试技术	
基于测试者直觉和经验的测试	AP
基于规格说明的测试	AP
基于代码的测试	AP
基于错误（Fault）的测试	AP
基于使用的测试	AP
基于应用本质的测试	AP
技术的选择与组合	AP
4测试相关的度量	
评价被测试的程序	AN
评价执行的测试	AN
5测试过程	
与管理相关的问题	C
测试活动	AP

软件维护

主题结构分解	分类学级别
1软件维护基础	
定义与术语	C
维护的本质	C
维护的必要性	C
维护成本的大份额性	C
软件的演化	C
维护类别	AP
2软件维护关键问题	
技术问题	
有限的了解	C
测试	AP
后果（impact）分析	AN
可维护性	AN
管理问题	

与组织结构问题的对应 (alignment)	C
雇佣人员 (staffing)	C
过程问题	C
组织结构	C
维护成本估算	
成本估算	AP
参数模型	C
经验	AP
软件维护度量	AP
3维护过程	
维护过程模型	C
维护活动	
特定的活动	AP
支持性活动	AP
4维护技术	
程序理解	AN
再工程	C
逆向工程	C

软件配置管理

主题结构分解	分类学级别
1软件配置管理过程的管理	
软件配置管理的组织结构上下文	C
软件配置管理的约束与指导	C
软件配置管理的规划	
软件配置管理组织机构与责任	AP
软件配置管理资源与进度	AP
工具选择与实现	AP
供应商/子合同控制	C
接口控制	C
软件配置管理计划	C
软件配置管理监管	
软件配置管理度量与结果	AP
软件配置管理过程内部审计	C
2软件配置标识	
识别要控制的配置项	
软件配置	AP
软件配置项	AP
软件配置项之间关系	AP
软件版本	AP
基线	AP
获取软件配置项	AP

软件库	C
3软件配置控制	
软件变更的请求、评价和批准	
软件配置控制委员会	AP
软件变更请求过程	AP
实现软件变更	AP
偏离与放弃（deviation and waiver）	C
4软件配置状态簿记	
软件配置状态信息	C
软件配置状态报告制作	AP
5软件配置审计	
软件功能配置审计	C
软件物理配置审计	C
软件基线的过程内部审计	C
6软件发布管理与交付	
软件建造	AP
软件发布管理	C

软件工程管理

主题分解结构	分类学级别
1启动和范围定义	
需求的确定和协商	AP
可行性分析	AP
需求评审和修订过程	C
2软件项目计划	
过程计划	C
确定可交付对象	AP
工作量、进度和成本估算	AP
资源分配	AP
风险管理	AP
质量管理	AP
计划管理	C
3软件项目实施	
计划的实现	AP
供应商合同管理	C
度量过程的实现	AP
过程的监管	AN
过程的控制	AP
报告编制	AP
4评审与评价	
确定需求的满足程度	AP

性能的评审和评价	AP
5关闭	
确定是否关闭	AP
关闭活动	AP
6软件工程度量	
建立和维持度量工作	C
度量过程计划	C
度量过程的实施	C
评价度量	C

软件工程过程

主题分解结构	分类学级别
1过程的实现与变更	
过程基础结构	
软件工程过程小组	C
经验工厂	C
活动	AP
过程实现和变更模型	K
实际考虑	C
2过程定义	
生命周期模型	AP
软件生命周期过程	C
过程定义符号	C
过程适应	C
自动化	C
3过程评定	
过程评定模型	C
过程评定方法	C
4产品和过程度量	
软件过程度量	AP
软件产品度量	AP
规模度量	AP
结构度量	AP
质量度量	AP
度量结果的质量	AN
软件信息模型	
模型构造	AP
模型实现	AP
度量技术	
解析型技术	AP

基准技术	C
------	---

软件工程工具和方法

主题分解结构	分类学级别
1软件工具	
软件需求工具	AP
软件设计工具	AP
软件构造工具	AP
软件测试工具	AP
软件维护工具	AP
软件工程过程工具	AP
软件质量工具	AP
软件配置管理工具	AP
软件工程管理工具	AP
其它工具问题	AP
2软件工程方法	
启发式方法	AP
形式化方法和符号	C
原型方法	AP
其它方法问题	C

软件质量

主题分解结构	分类学级别
1软件质量基础	
软件工程文化与伦理学	AN
质量的价值和成本	AN
质量模型与特征	
软件过程质量	AN
软件产品质量	AN
质量改进	AP
2软件质量管理过程	
软件质量保证	AP
验证与确认	AP
评审与审计	
检查	AP
同行评审	AP
走查	AP
测试	AP
审计	C
3实际考虑	
质量需求应用	

系统的关键特性	C
可依赖性	C
软件的完整程度	C
缺陷特征	AP
软件质量管理技术	
静态技术	AP
人力密集的技术	AP
分析型技术	AP
动态技术	AP
软件质量度量	AP